zynqMP SOC Linux 开发教程基础篇

2022-08-16 13:19:54 V1.04

ALINX

版权声明

Copyright ©2012-2020 芯驿电子科技（上海）有限公司

公司网址:
http://www.alinx.com

天猫旗舰店:
https://alinx.tmall.com

京东旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:

文档修订记录:

版本	时间	描述
1.01	2020/4/13	初始版本
1.04	2021/12/8	修改 SD 卡启动，增加 boot.scr
1.05	2022/6/15	和视频对应，重新整理；合并应用教程。

我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据论坛上大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQ MPSoC 的开发板！您对我们和我们产品的支持和信任,给我们增添了勇往直前的信心和勇气。”播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望。

本文档为 Linux 开发基础篇，主要介绍 Linux 开发工具安装，定制及应用程序开发。芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明 2
前言 4
目录 5
第一章 使用 Petalinux 定制 Linux 系统 8
1.1 Petalinux 建立工程 8
1.2 设置离线编译 14
1.3 设置本地内核源码以及 u-boot 源码 21
1.4 修改设备树、编译 petalinux 工程 31
1.5 生成 BOOT 文件 33
1.6 运行 Linux 34
1.7 Petalinux 使用流程总结 36
1.8 创建和使用 bsp 包 37
第二章 添加开机自启动脚本及用户文件 40
2.1 准备工作 40
2.2 自启动脚本 40
第三章 程序 hello world 46
3.1 程序代码 46
3.2 编译主机程序 46
3.3 交叉编译 arm 程序 47
第四章 NFS 服务 49
第五章 gpio 控制 led 49
5.1 petalinux 自动生成的设备树 49
5.2 petalinux 用户可修改的设备树 50
5.3 工程创建 50
5.4 程序运行 52
第六章 QT 和 OPENCV 交叉编译开发环境 54
6.1 配置根文件系统 54
6.2 安装 sdk 60
6.3 QT Creator 交叉编译环境设置 61
6.4 创建 QT 测试工程 66
6.5 运行交叉编译版本 QT 程序 71
第七章 使用 Vitis 开发 Linux 程序 73
7.1 使用 Vitis 建立 Linux 应用程序 73
7.2 通过 NFS 共享运行 79
第八章 SD 卡根文件系统 80

8.1 Petalinux 配置 80
8.2 制作 SD 卡文件系统 81
8.2.1 SD 卡修改分区 81
8.2.2 制作启动 linux 的 SD 卡 85
第九章 Vitis 加速基本平台创建 87
9.1 Vivado 硬件平台 87
9.2 petalinux 定制系统 105
9.3 vitis 创建加速平台 117
9.4 创建应用程序测试加速平台 117
9.5 运行程序测试加速 129
第十章 从 QSPI Flash 启动 Linux 130
第十一章 从 QFlash 启动内核并从 EMMC 挂载根文件系统 178
11.1 创建并配置 petalinux 工程 178
11.2 设置 EMMC 分区制作根文件系统 186
11.3 Qflash+EMMC 启动测试 188
第十二章 Linux 下的 NVMe SSD 操作 137
12.1 petalinux 配置 137
12.2 设置分区 141
12.3 读写测速 143
第十三章 可远程调试的 Hello World 144
13.1 GDB 简介 144
13.2 QtCreator 简介 144
13.3 实验目标 144
13.4 Qt Creator 工程 145
13.5 获取板卡的 IP 地址 145
13.6 QtCreator 配置 GDB 连接板卡 145
13.7 运行 Hello World 147
13.8 运行参数设置 149
第十四章 OpenCV 之边缘检测 151
14.1 OpenCV 简介 151
14.2 边缘检测简介 151
14.3 实验目标 152
14.4 软件流程图 152
14.5 运行准备 152
14.6 程序运行 154
14.7 代码解析 155
第十五章 OpenCV+Qt 之人脸检测 158
15.1 OpenCV 的级联分类器 158

15.2 积分图 159
15.3 选取特征 160
15.4 Qt 简介 160
15.5 实验目标 161
15.6 软件流程图 161
15.7 运行准备 161
15.8 程序运行 161
15.9 代码解析 162
第十六章 GStreamer 的摄像头显示 164
16.1 GStreamer 简介 164
16.2 GStreamer 基本概念 165
16.3 实验目标 166
16.4 Gstreamer 常用工具 166
16.5 gst-launch-1.0 摄像头显示 167
16.6 编程运行程序 168
16.7 代码解析 168
第十七章 Qt+DRM+Gstreamer 的摄像头显示 169
17.1 DRM 简介 169
17.2 实验介绍 170
17.3 实验目标 170
17.4 运行程序 170
17.5 代码分析 171
第十八章 Qt+GPU 的摄像头显示 172
18.1 Mali GPU 简介 172
18.2 MPSoc GPU 简介 172
18.3 V4L2 简介 173
18.4 实验目标 174
18.5 运行程序 174
18.6 代码分析 175
第十九章 linux 寄存器操作 176
19.1 linux 内存映射 176
19.2 实验简介 176
19.3 运行程序 176
19.4 代码分析 177

第一章 使用 Petalinux 定制 Linux 系统

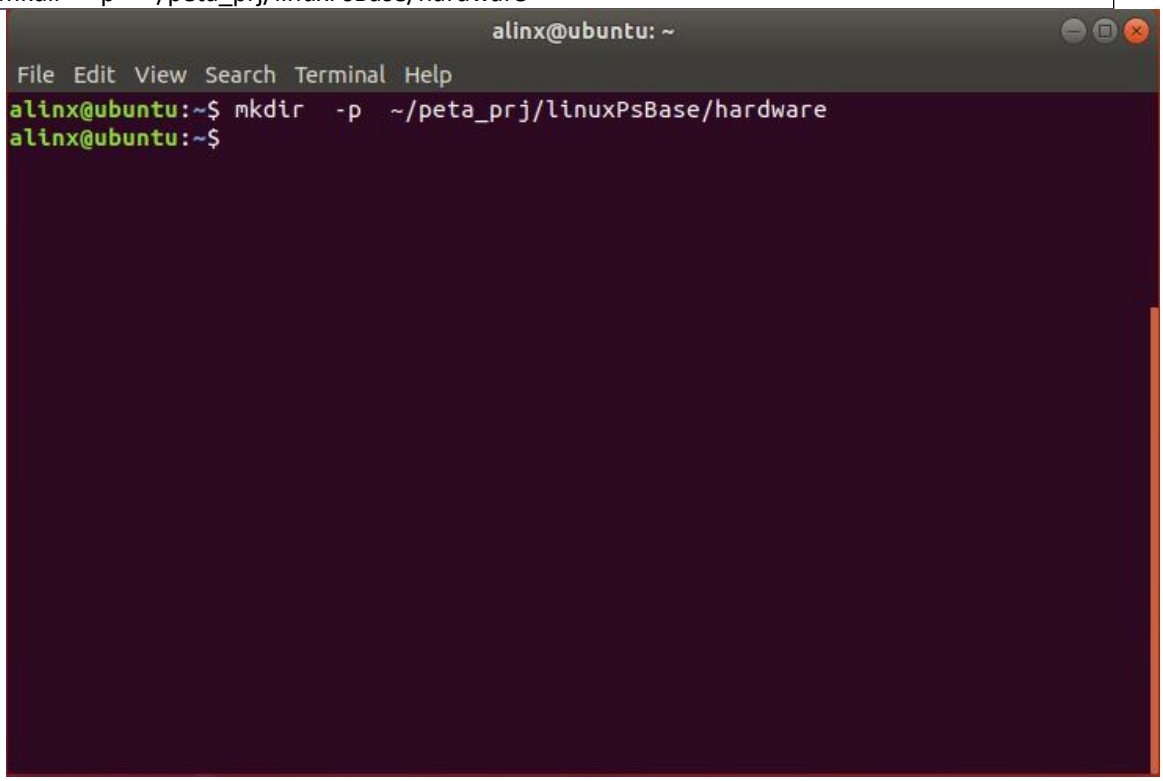
特别提醒，本教程所使用的环境是按照教程“Xilinx 开发环境安装教程”搭建的，请先按该教程搭建开发环境。（尽量升级 VM 到最新版本，Ubuntu16 与 vivado2017 版本兼容，Ubuntu18 与 vivado2020 兼容，请尽量使用 Ubuntu18，本教程使用 Ubuntu16，步骤一样）

本章教程参考 Xilinx 官方的教程“ug1144-petalinux-tools-reference-guide”，使用 Petalinux 建立一个最基本的 Linux 系统。

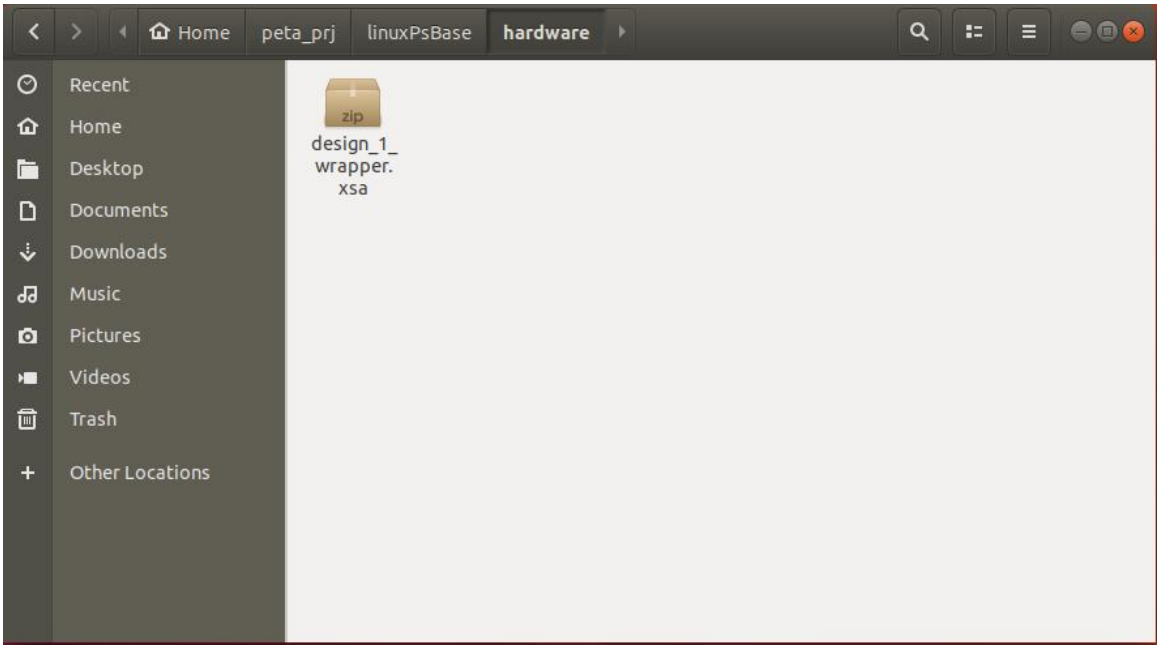
1.1 Petalinux 建立工程

1. 在用户目录下，创建一个工作目录。“~”即表示用户目录，例如我们的用户名是“alinx”时，“~”就等价于“/home/alinx”路径。peta_prj 路径用于存放所有的 petalinux 工程，linuxPsBase 路径是这一章用到的 petalinux 工程存放的路径，hardware 路径用于存放这一章 petalinux 工程需要用到的硬件描述文件。

```
mkdir   -p   ~/peta_prj/linuxPsBase/hardware
```



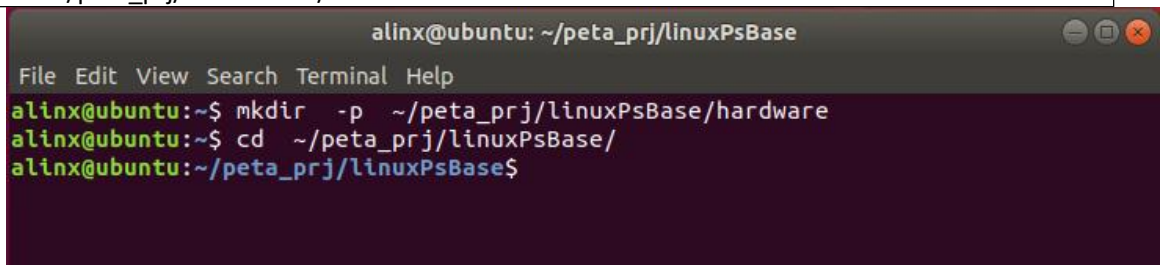
2. 将 vivado 工程导出的后缀为 xsa 的文件(也就是步骤 1 中说的硬件描述文件)拷贝到刚才创建的 hardware 文件夹下(vivado 工程在基础教程源码路径对应章节的文件夹中)。



注：这里使用的是本教程源码包对应章节路径中的 vivado 工程导出的 xsa 文件，也可以使用”Vitis 应用教程”源码包中的 ps_hello 工程。

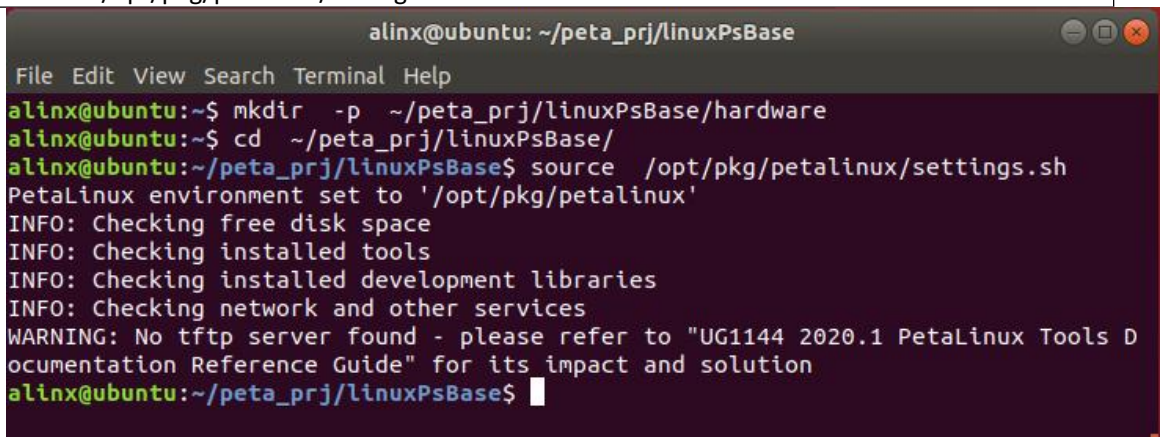
3. 进入存放 petalinux 工程的路径(步骤 1 中的)。

```
cd   ~/peta_prj/linuxPsBase/
```



4. 设置 petalinux 环境变量，运行下面命令 (环境变量只在当前打开的终端中有效，后面和 petalinux 相关的命令都要在当前终端中输入，如果关闭了当前终端则需要重新运行这条命令) 。

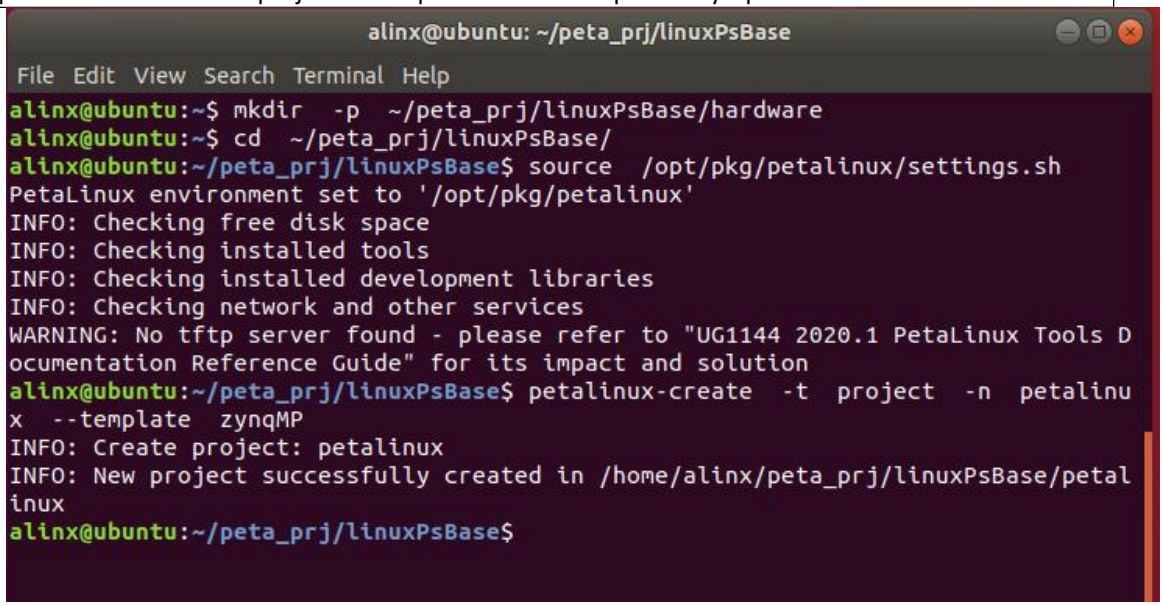
```
source   /opt/pkg/petalinux/settings.sh
```



5. 使用“petalinux-create”命令创建 petalinux 工程。“-t”参数是指需要创建的类型，这里我

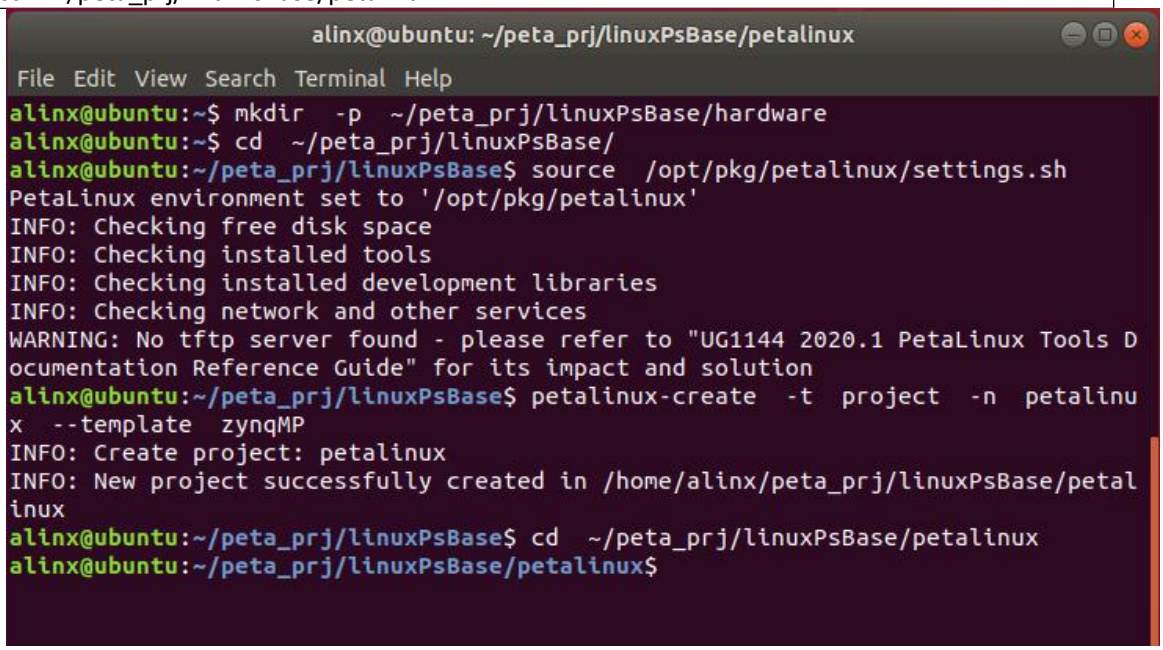
们是创建工程所以后面跟“project”， “-n”指工程名称为“petalinux”， “--template”参数是指使用“zynqMP”模板来创建这个工程。

```
petalinux-create  -t  project  -n  petalinux  --template  zynqMP
```



6. 使用下面的命令进入 petalinux 工程目录。

```
cd  ~/peta_prj/linuxPsBase/petalinux
```

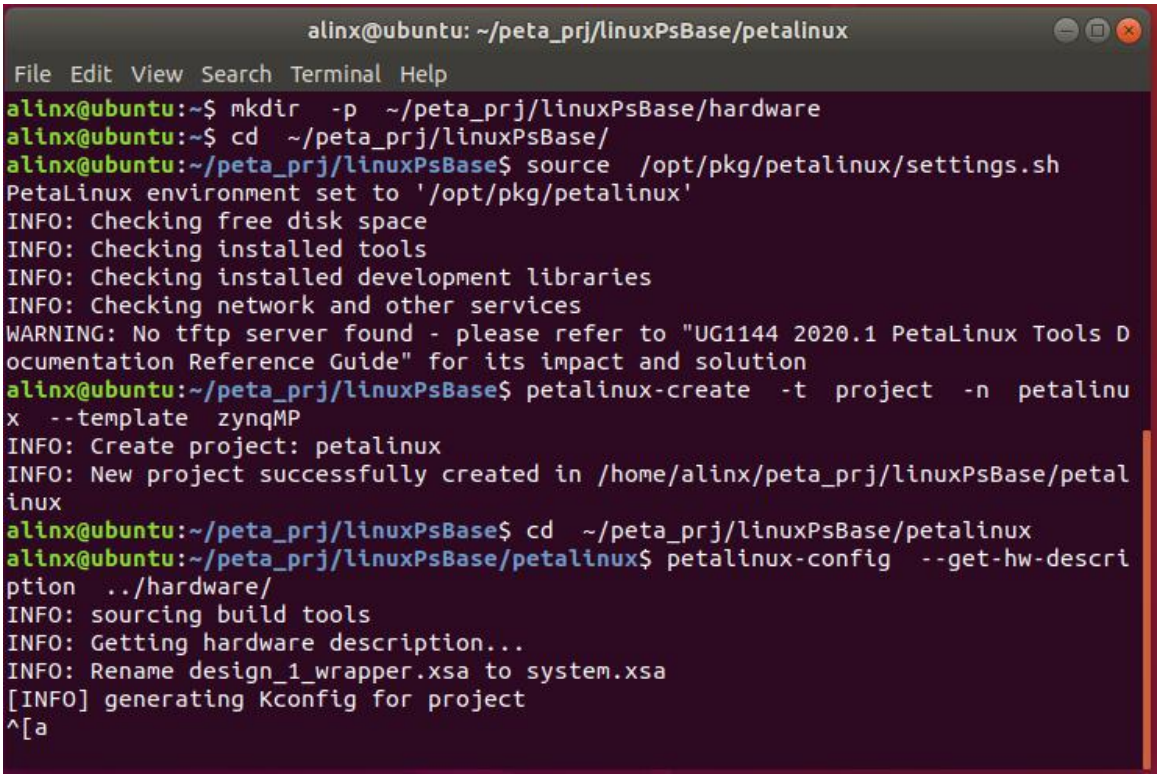


7. 使用“petalinux-config”命令加上“--get-hw-description”参数配置 petalinux 工程的硬件信息，“--get-hw-description”参数后面我们硬件描述文件(xsa 文件)所在的路径，也就是步骤 1 中创建的路径，“../hardware/”中的两个点是指代当前工作路径的上级路径，在当前步骤中是和“~/peta_prj/linuxPsBase/hardware”等价的。注意：硬件信息目录(当前就是“../hardware/”)里必须有且只有一个 xsa 文件。

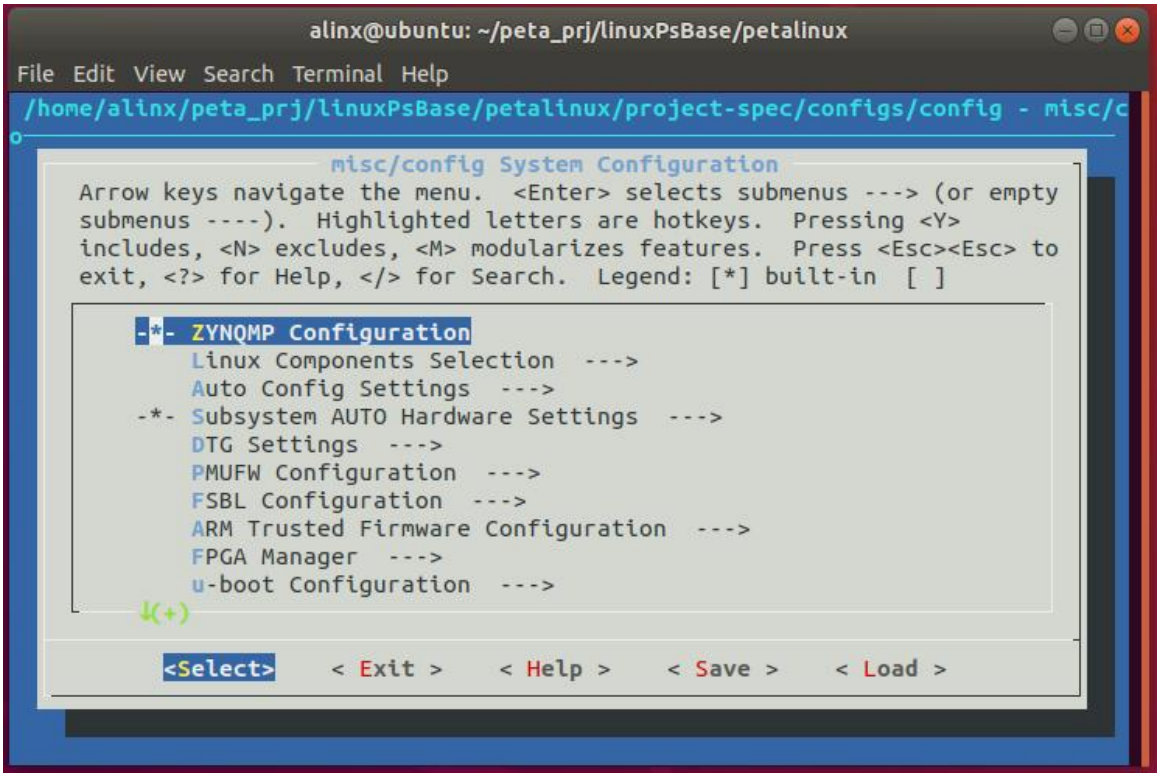
```
petalinux-config  --get-hw-description  ../hardware/
```

如果出现了这个错误，在执行这条命令前，将 Ubuntu 界面全屏，并将终端命令框全屏化。

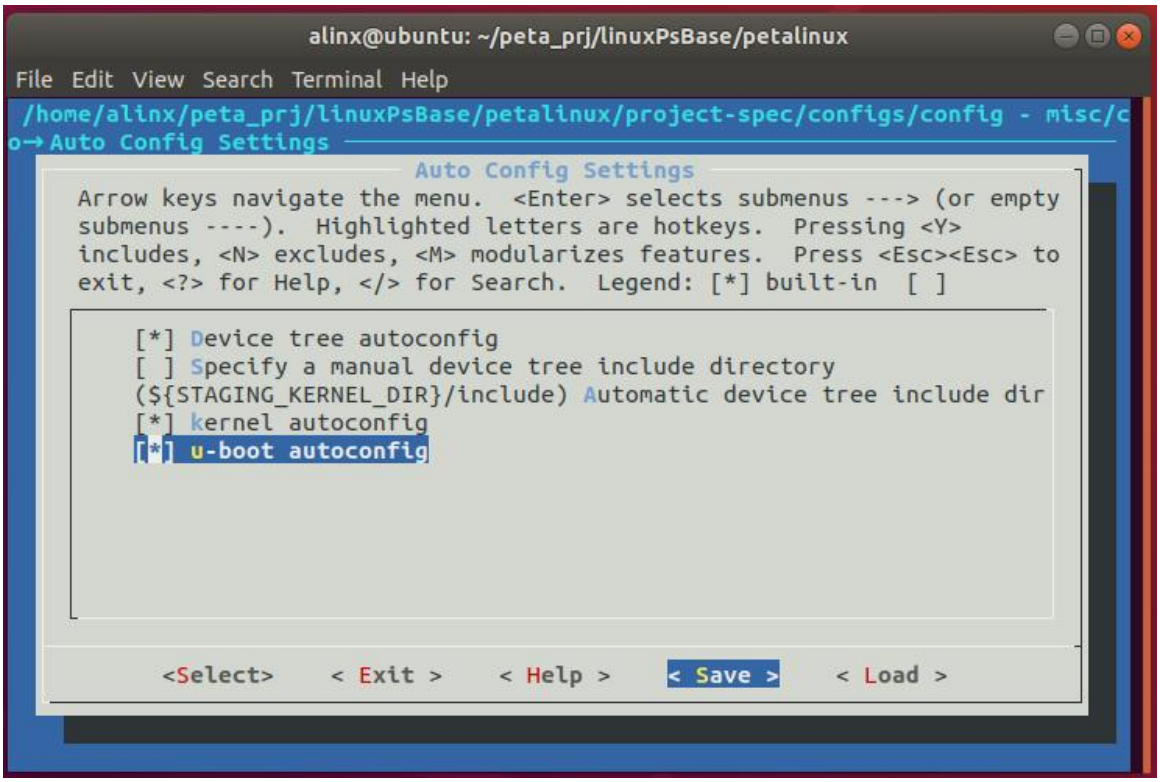
```
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$ petalinux-config --get-hw-de
scription  ../hardware/
INFO: sourcing build tools
INFO: Getting hardware description...
INFO: Rename design_1_wrapper.xsa to system.xsa
[INFO] menuconfig project
ERROR: Failed to menu config project component
ERROR: Failed to config project.
ERROR: Get hw description Failed!.
```



这条命令执行完成后，会变成如下图所示的配置界面：



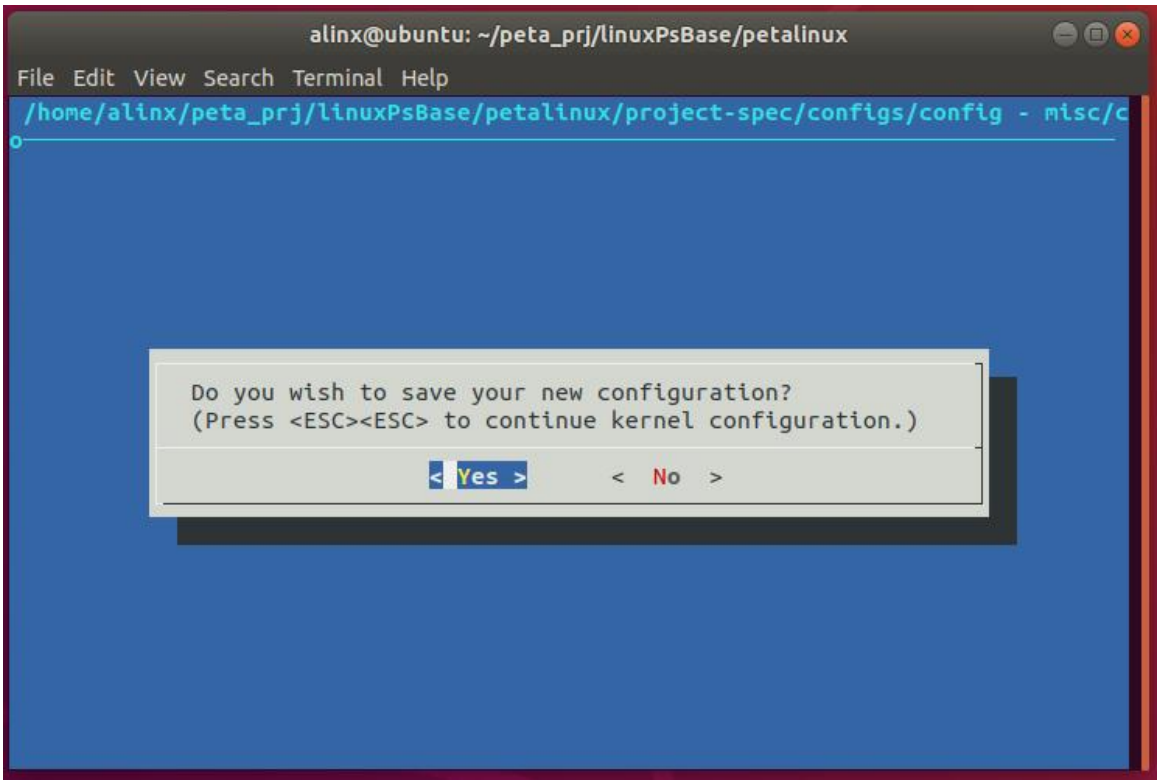
8. 在配置界面中，使用键盘上的“上下”方向键把蓝色的条移动到“Auto Config Settings”选项上，按回车进入该选项，同样使用方向建和回车键选择当前界面中的 “Device tree autoconfig”、“kernel autoconfig”、“u-boot autoconfig”。然后使用键盘方向键的“左右键“选择下方菜单中的“Save”，接着多次选择“Exit”选项直到退出配置界面。



注意：本教程沿袭了旧版 petalinux 的使用习惯配置了“kernel autoconfig”、“u-boot

autoconfig”这两个选项，但这两个选项不选也可以。如果工程中配置了这两个选项且编译出错，也可以尝试把这两个选项去掉再编译。

9. 如果上一步没有选择“Save”，在退出配置界面时，会弹出是否保存配置的界面，选择<Yes>即可。



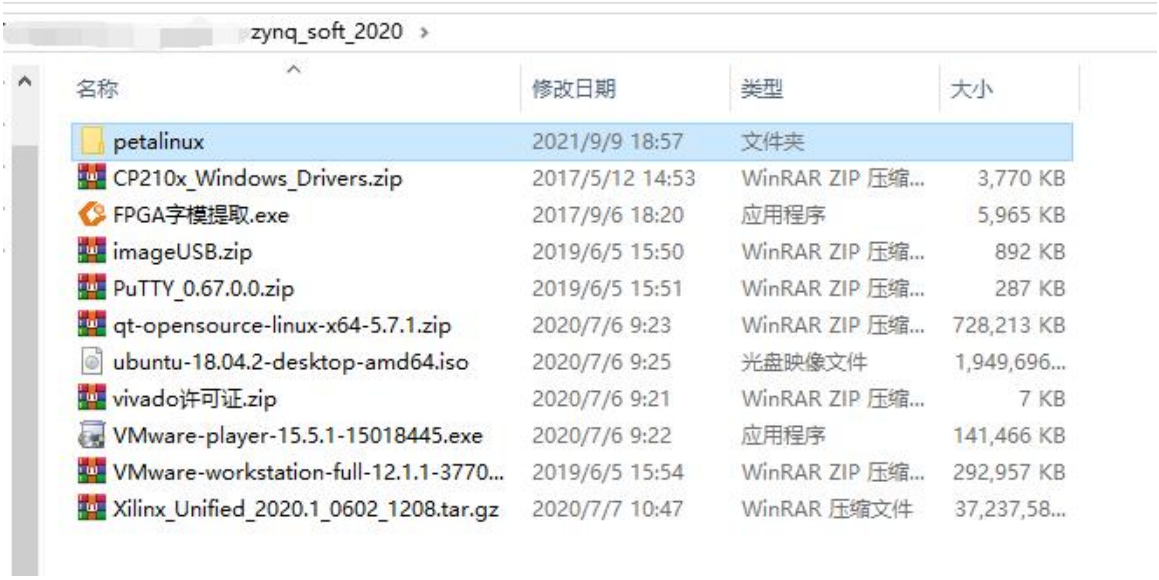
10. 退出配置界面后，等待配置完成。

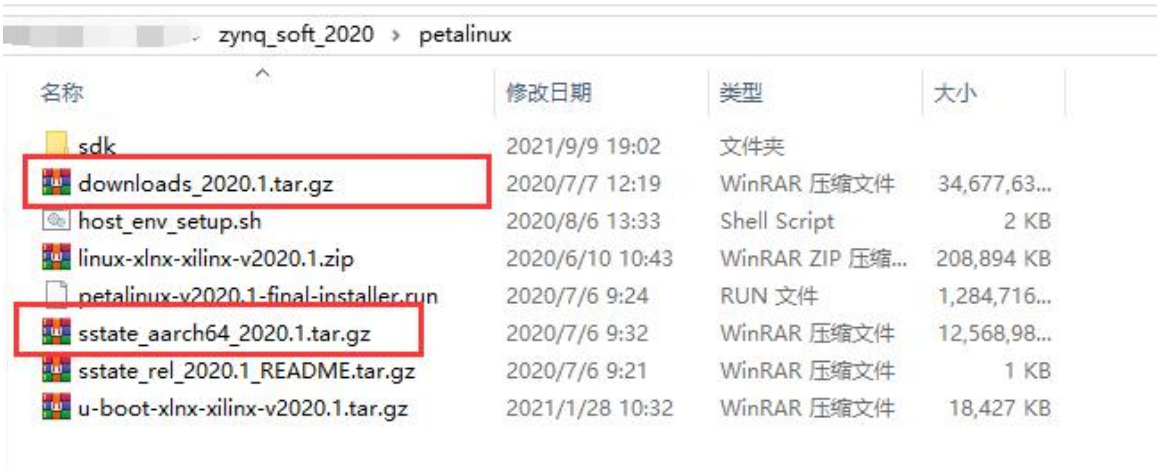
```
alinx@ubuntu: ~/peta_prj/linuxPsBase/petalinux
File  Edit  View  Search  Terminal  Help
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$ petalinux-config  --get-hw-descri
ption  ../hardware/
INFO: sourcing build tools
INFO: Getting hardware description...
INFO: Rename design_1_wrapper.xsa to system.xsa
[INFO] generating Kconfig for project
[INFO] menuconfig project
configuration written to /home/alinx/peta_prj/linuxPsBase/petalinux/project-spec
/configs/config

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

[INFO] extracting yocto SDK to components/yocto
[INFO] sourcing build environment
[INFO] generating u-boot configuration files, This will be deprecated in upcomin
g releases
[INFO] generating kernel configuration files, This will be deprecated in upcomin
g releases
[INFO] generating kconfig for Rootfs
[INFO] silentconfig rootfs
[INFO] generating plnxtool conf
[INFO] generating user layers
[INFO] generating workspace directory
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$
```

1.2 设置离线编译

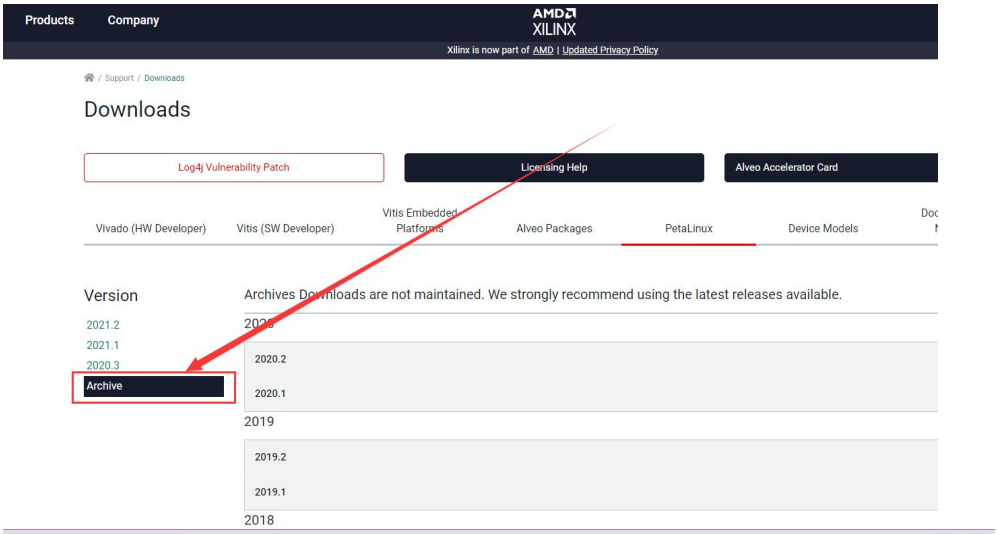
Petalinux 编译工程默认需要联网下载一些资源，如果网络不佳会编译非常缓慢甚至编译失败。为了解决这个问题，我们可以预先下载两个离线资源包，然后设置 petalinux 离线编译。

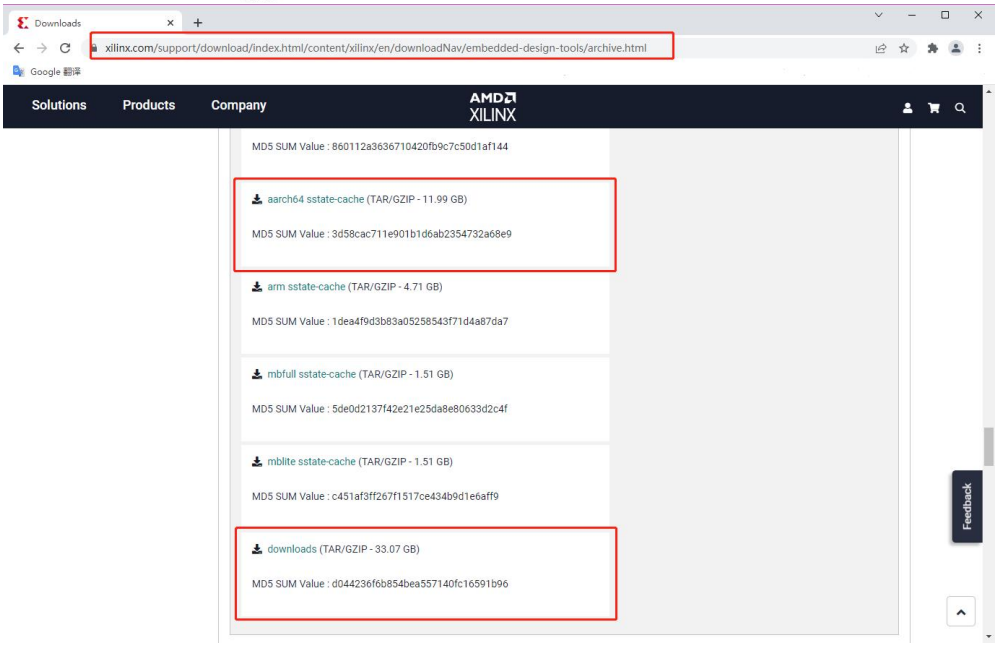
这两个资源包在我们提供的资源包中也包含了，可以直接使用。所在路径如下图：



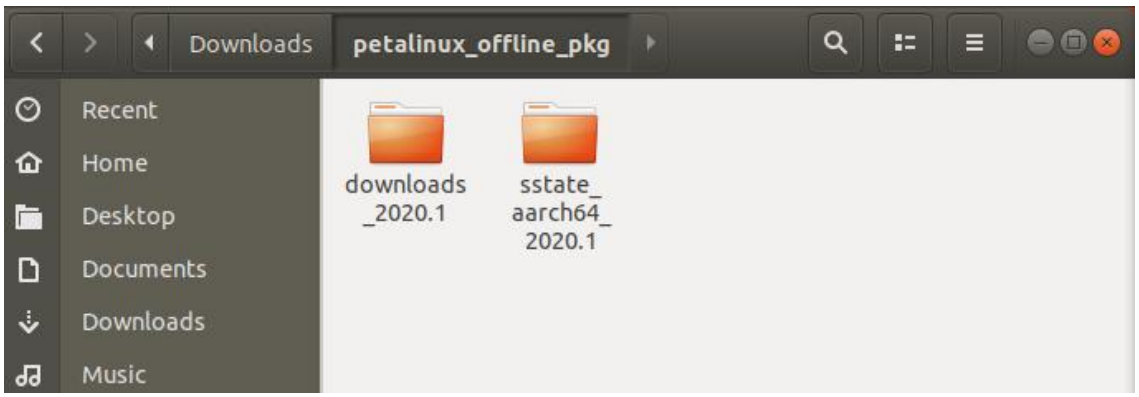


这两个离线资源包也可以从 xilinx 官网获取：https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-design-tools/archive.html 下载下图中红框标出来的两个文件。注意，一定要下载和 petalinux 版本对应的文件，我们使用的 petalinux 版本是 2020.1，就需要找到 2020.1 标签中的这两个文件下载。



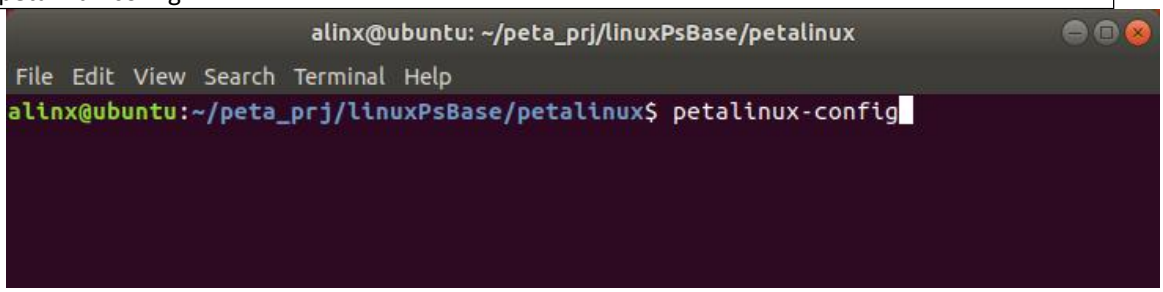


1. 把资源包拷贝到虚拟机中并解压。这里我解压到/home/Downloads/petalinux_offline_pkg 路径中，如下图：

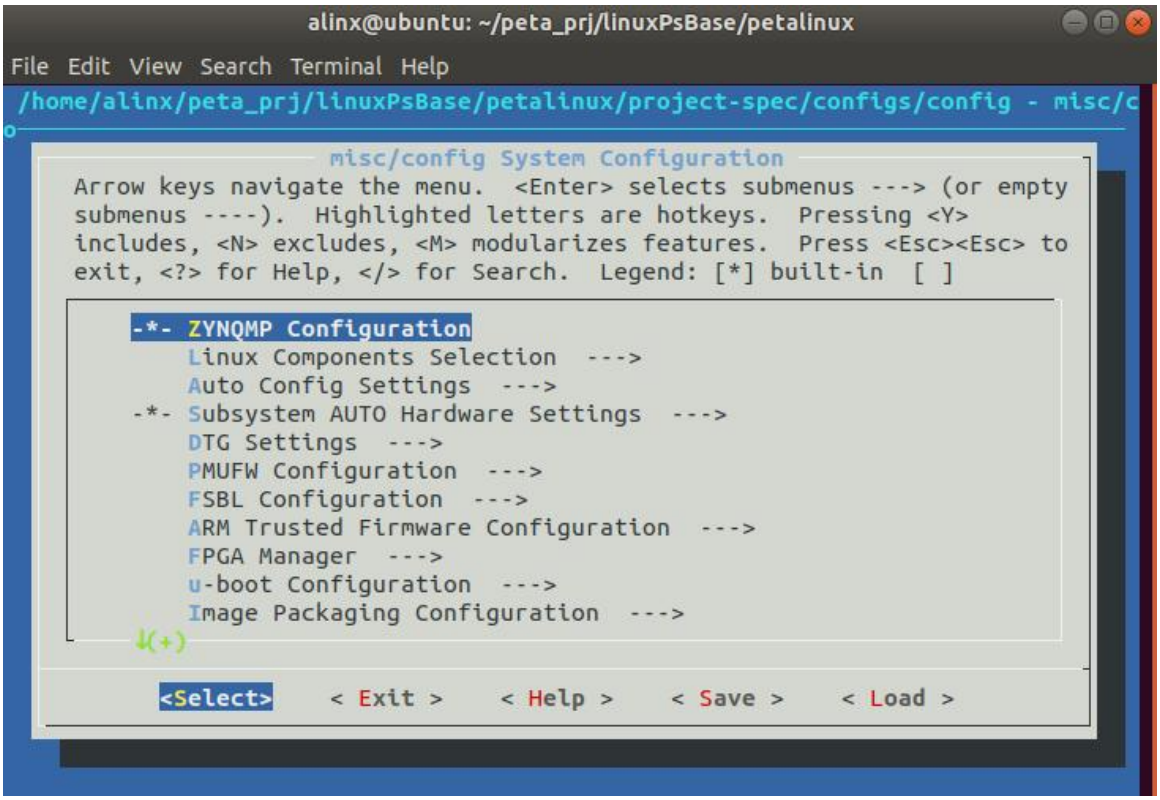


2. 在 petalinux 工程中打开终端，输入命令（如果出现错误，试试将 Ubuntu 和命令框全屏化）

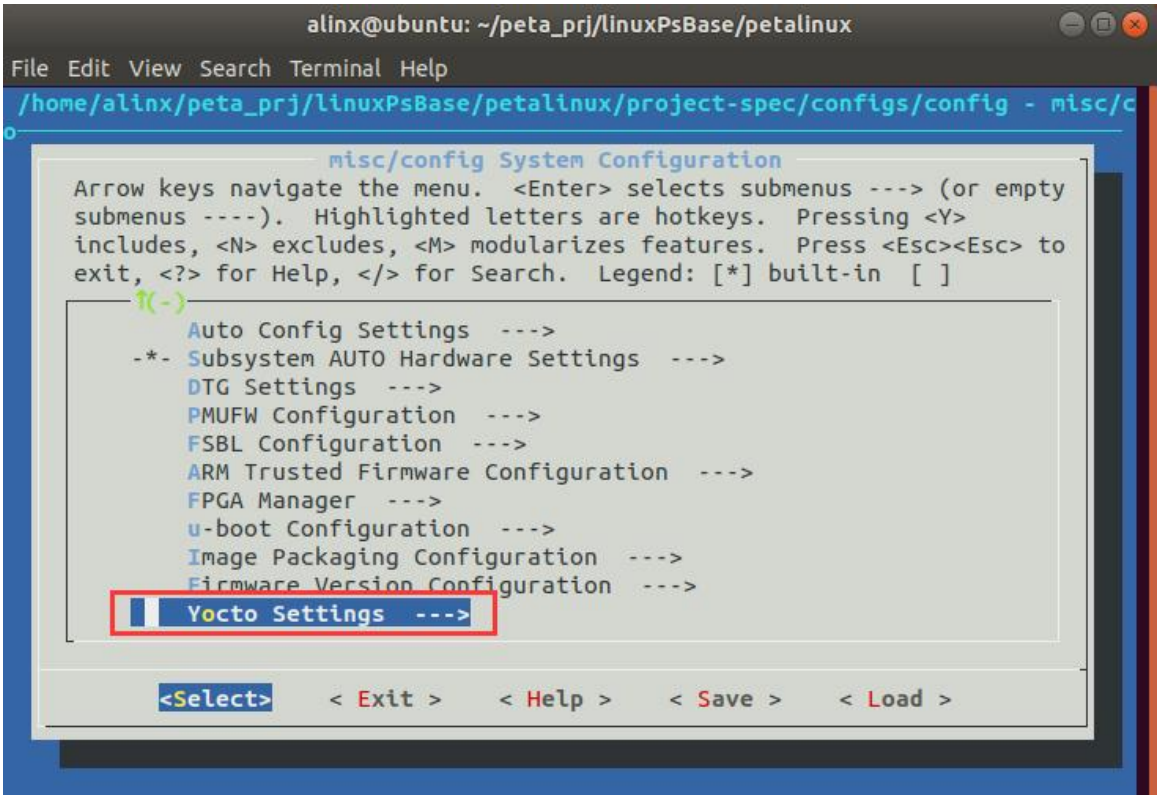
```
petalinux-config
```



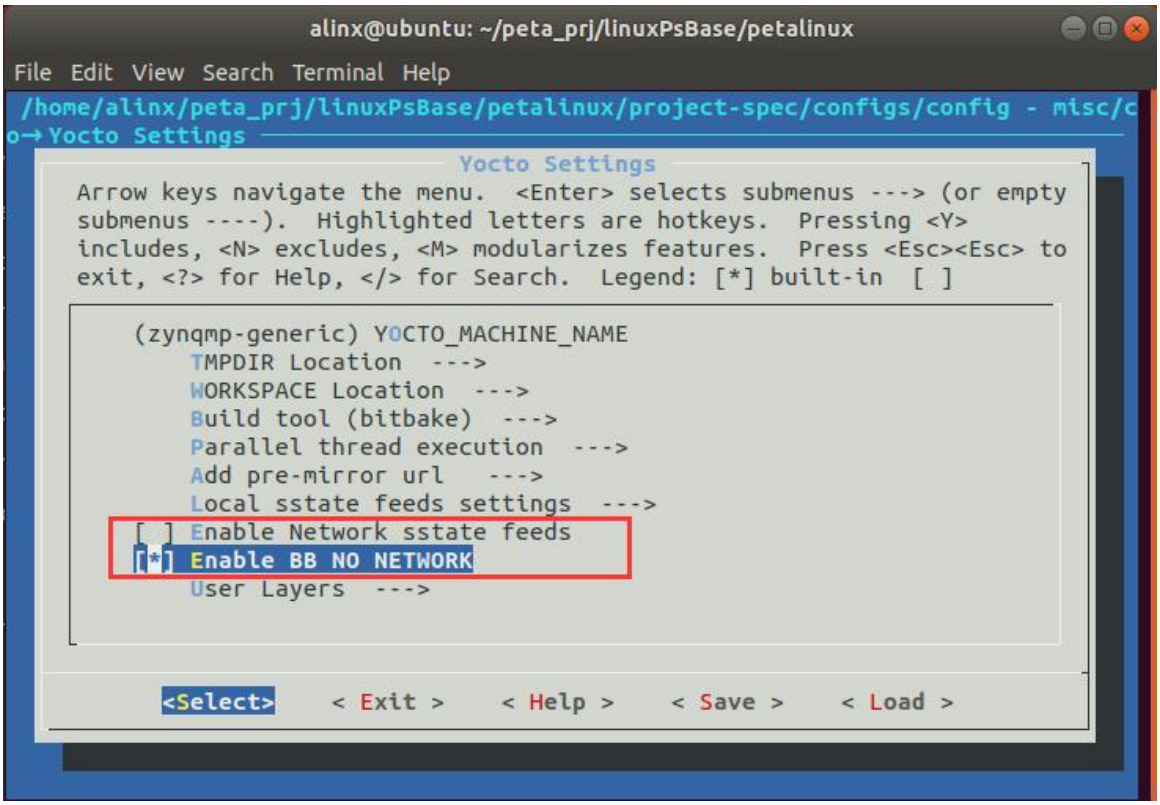
然后会弹出如下配置界面：



3. 选择最下面的“Yocto Settings ---->”选项。

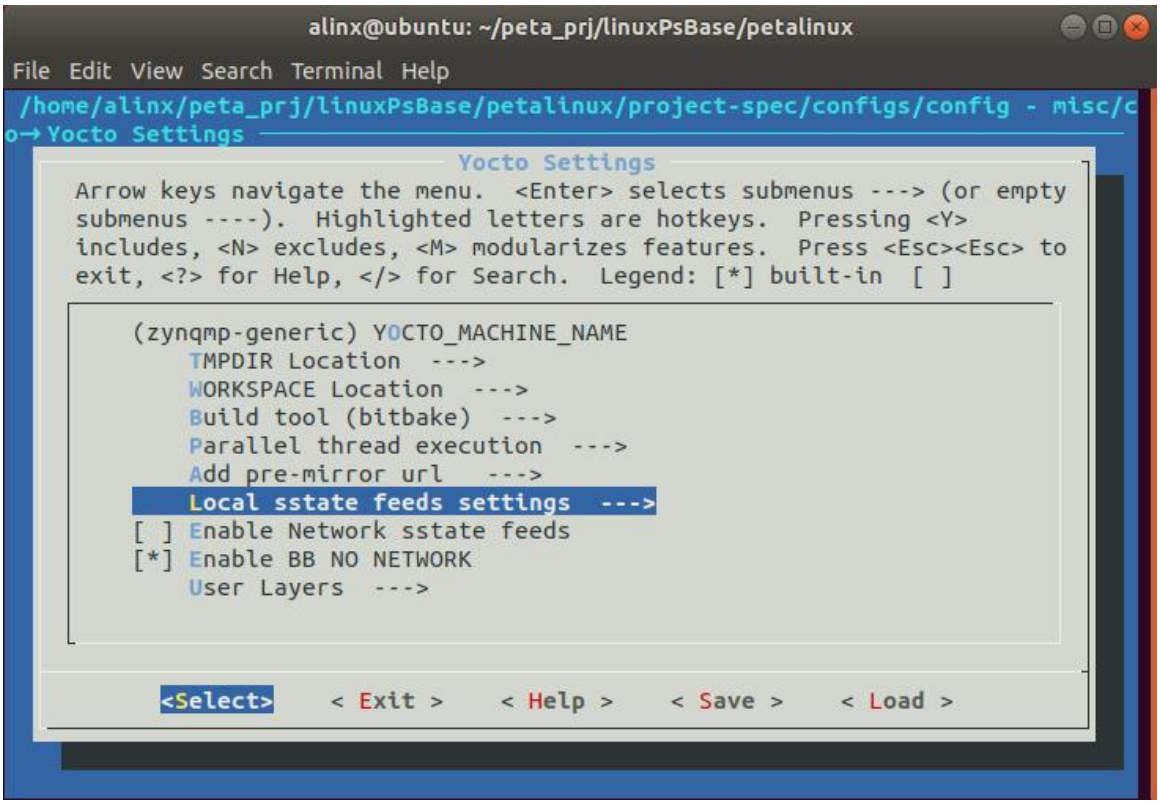


4. 在“Yocto Settings ---->”界面中，取消选择“Enable Network sstate feeds”，选择“”选项，修改成如下图的状态：

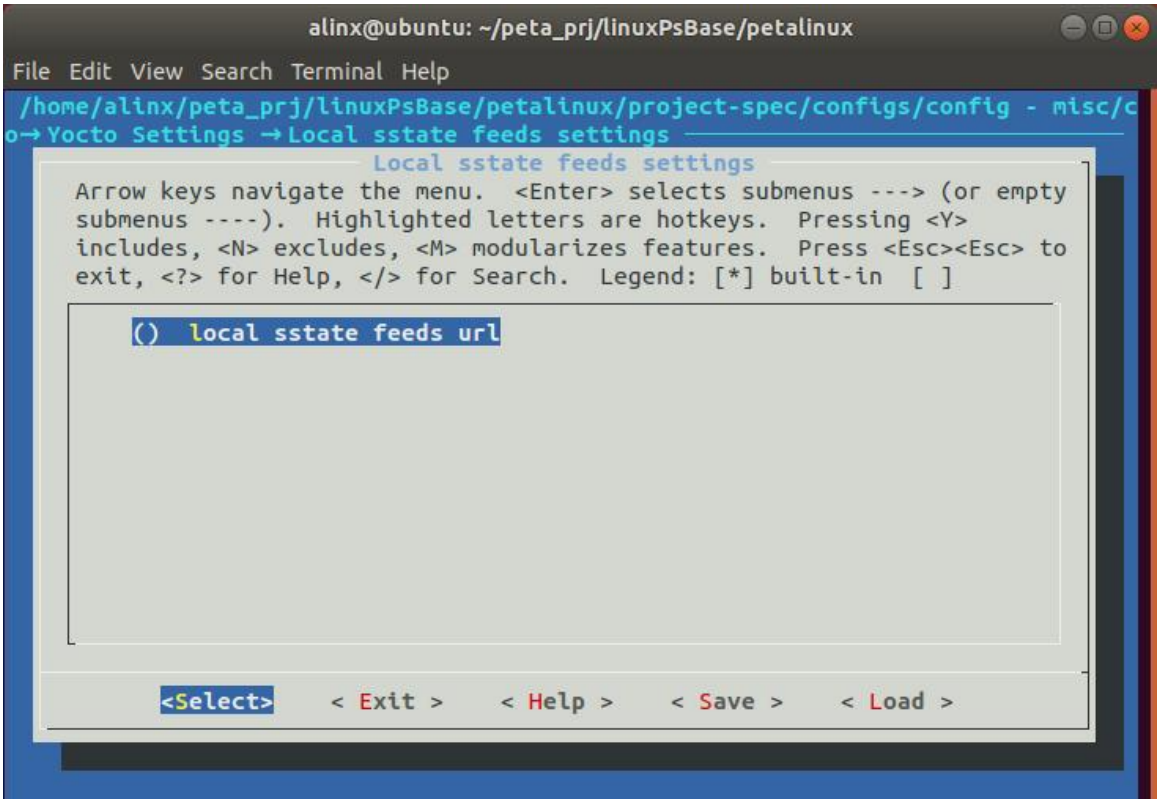


注意，如果之后编译报错提示“Network access disabled through BB_NO_NETWORK”，就把“Enable BB NO NETWOTRK”取消选中再尝试编译。

5. 设置“sstate_aarch64”包的路径，选择“Local sstate feeds settings ---->”选项

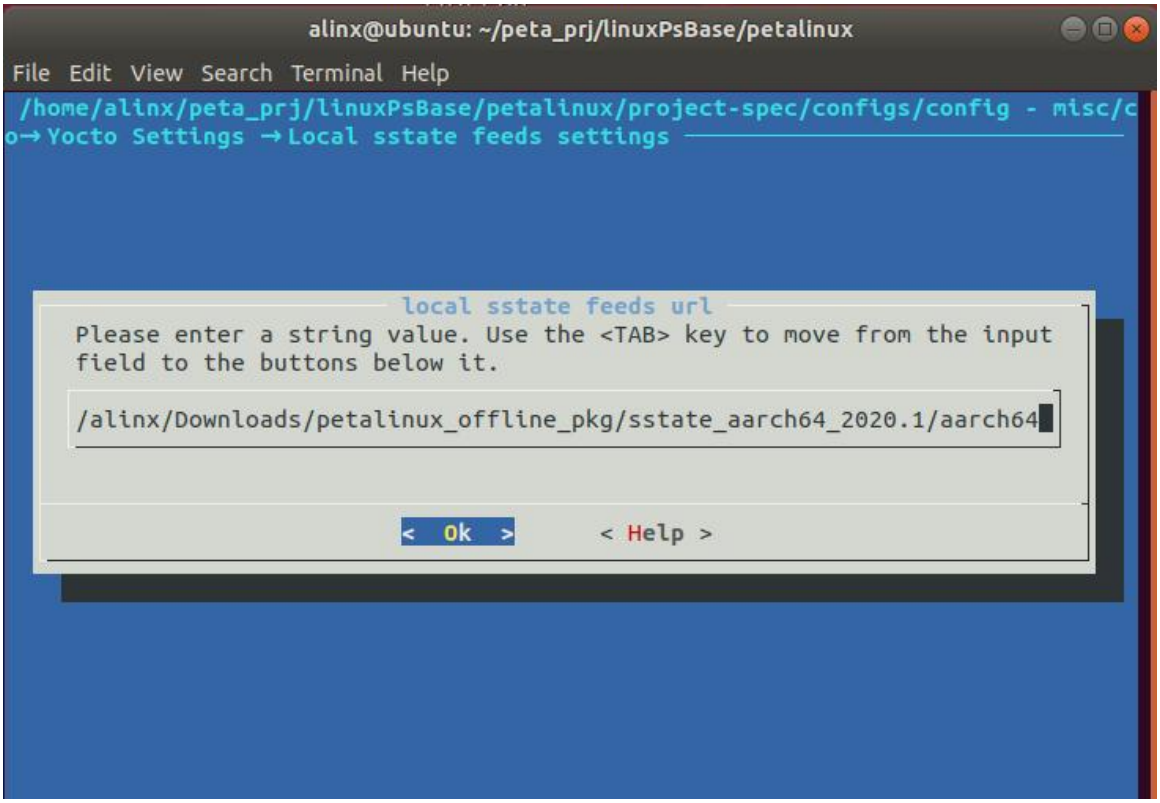


在之后的界面中直接按回车，



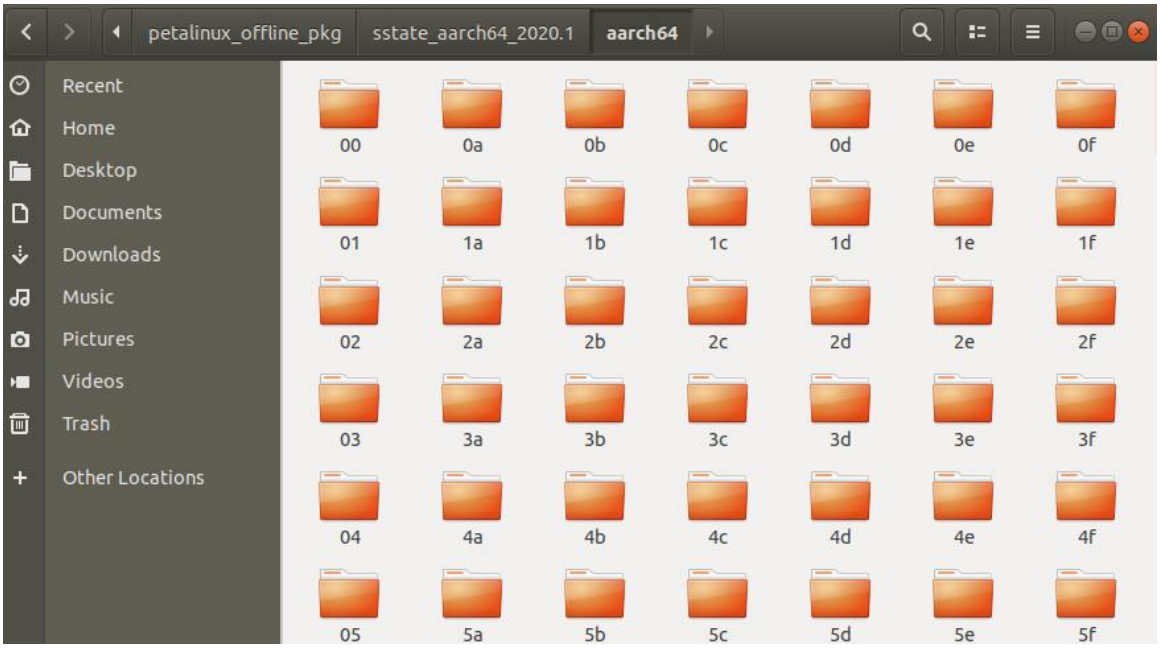
然后在弹窗中输入我们解压后的“sstate_aarch64”包的路径

/home/alinx/Downloads/petalinux_offline_pkg/sstate_aarch64_2020.1/aarch64

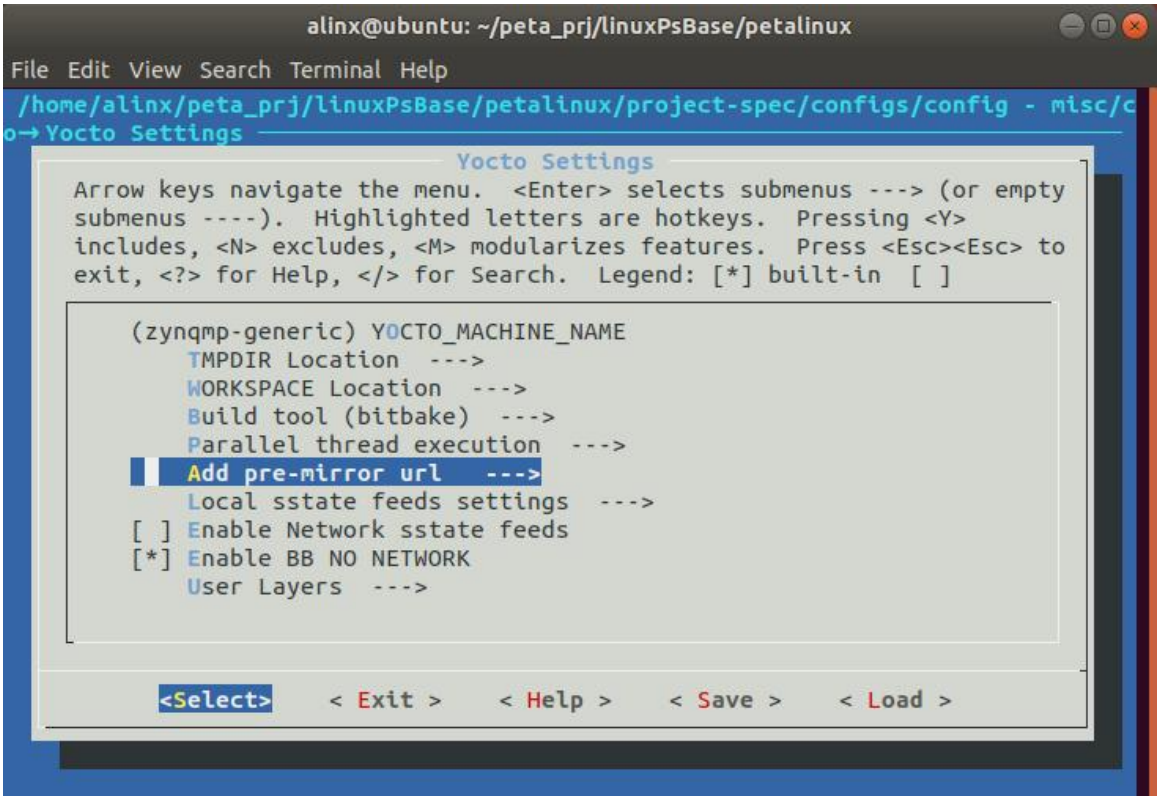


注意：图片中的路径没有显示全，设置好之后按回车键选择< ok >选项。

路径中的内容如下图：



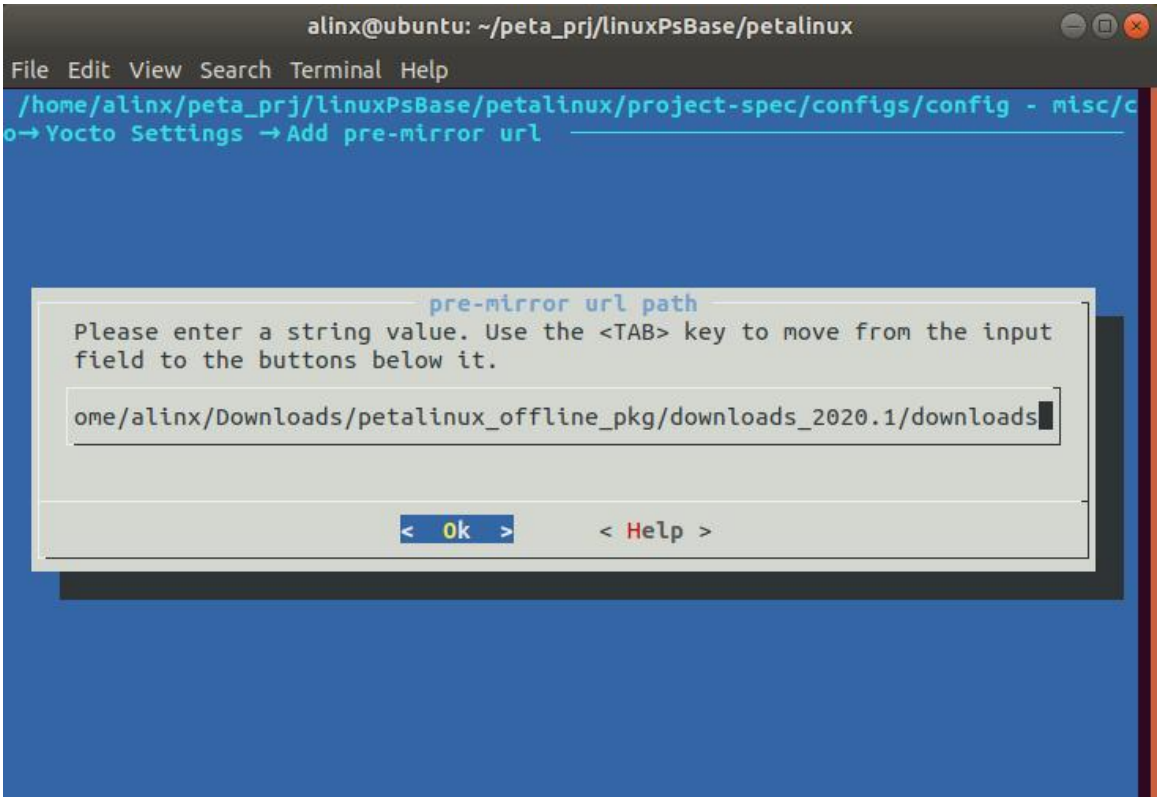
6. 设置“downloads”包的路径，先选择< Exit >选项返回到下图的界面，然后选择“Add pre-mirror url --->”选项



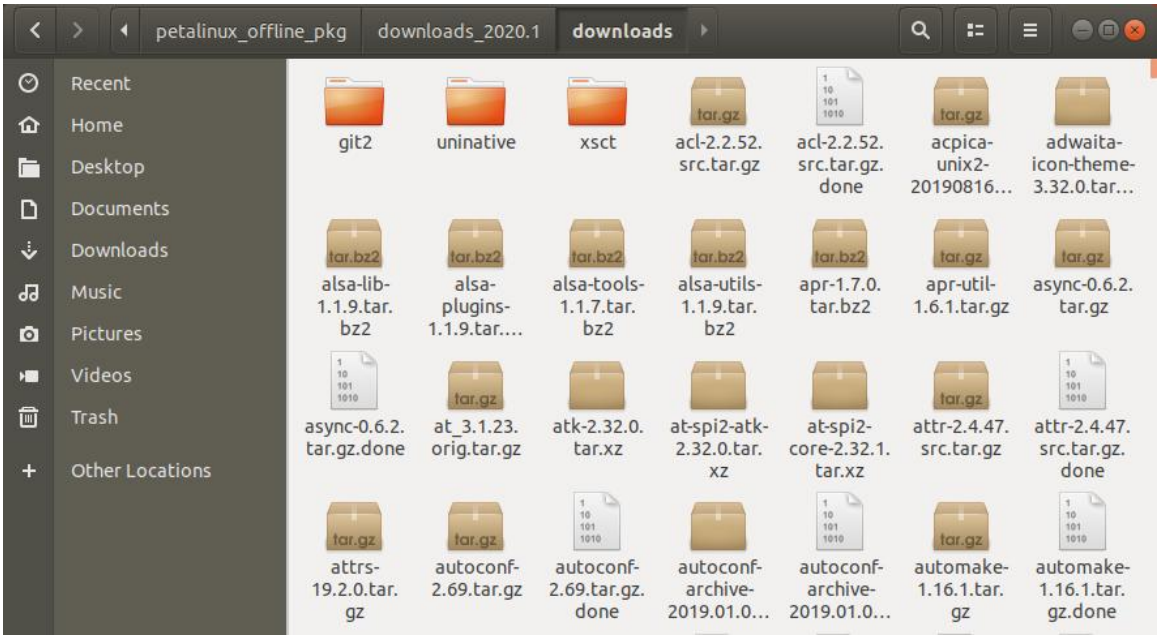
修改这个选项里面原有的路径为：

file:///home/alinx/Downloads/petalinux_offline_pkg/downloads_2020.1/downloads

注意路径以及路径格式，然后选择< ok >。



路径中的内容如下图：

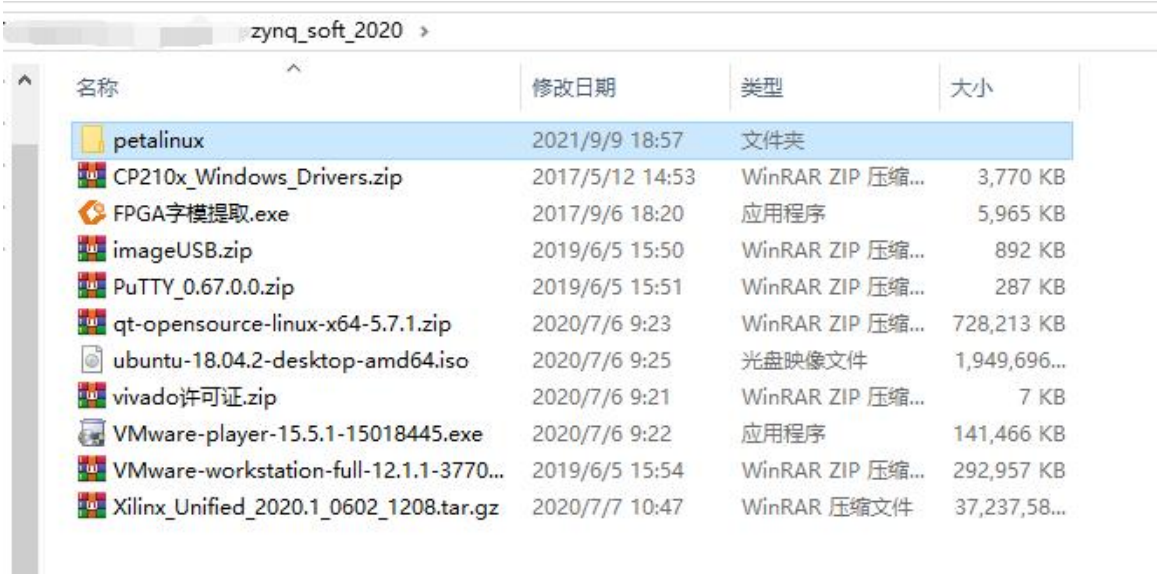


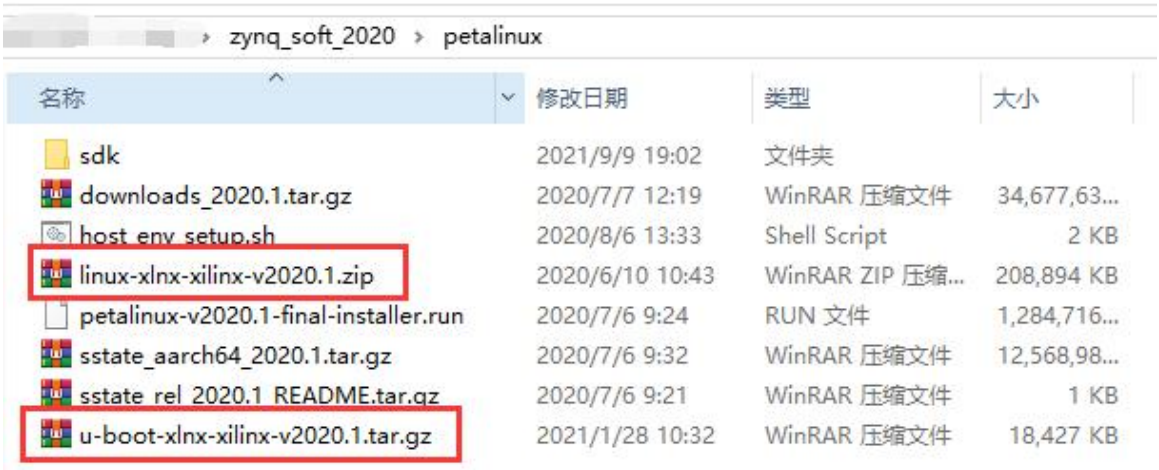
7. 选择< Save >选项，然后选择< Exit >选项退出配置界面即可。

1.3 设置本地内核源码以及 u-boot 源码

如果有修改内核源码以及 u-boot 源码的需要，我们可以设置使用本地的内核源码和 u-boot 源码。

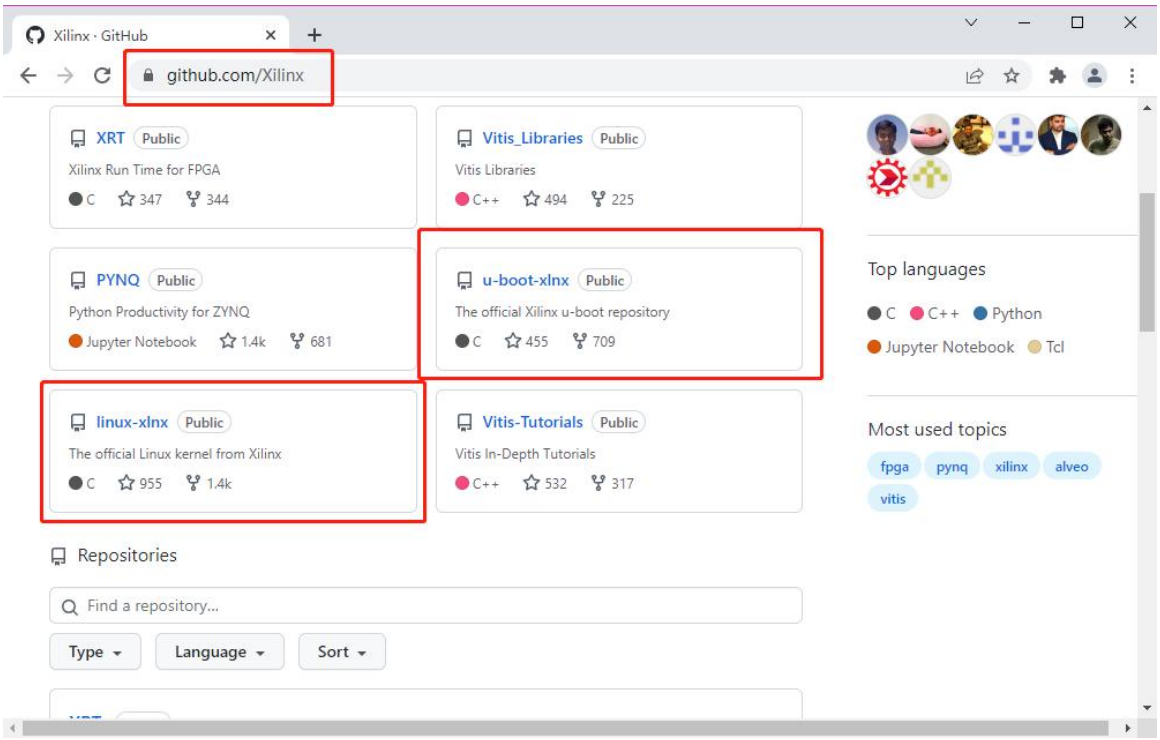
我们提供的资源包中也包含了 xilinx 的内核源码和 u-boot 源码，可以直接使用。所在路径如下图：



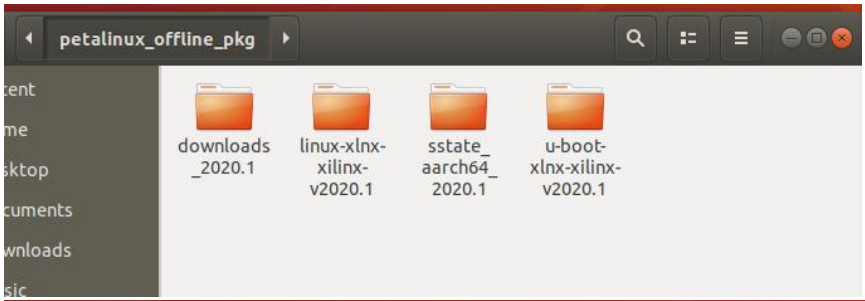


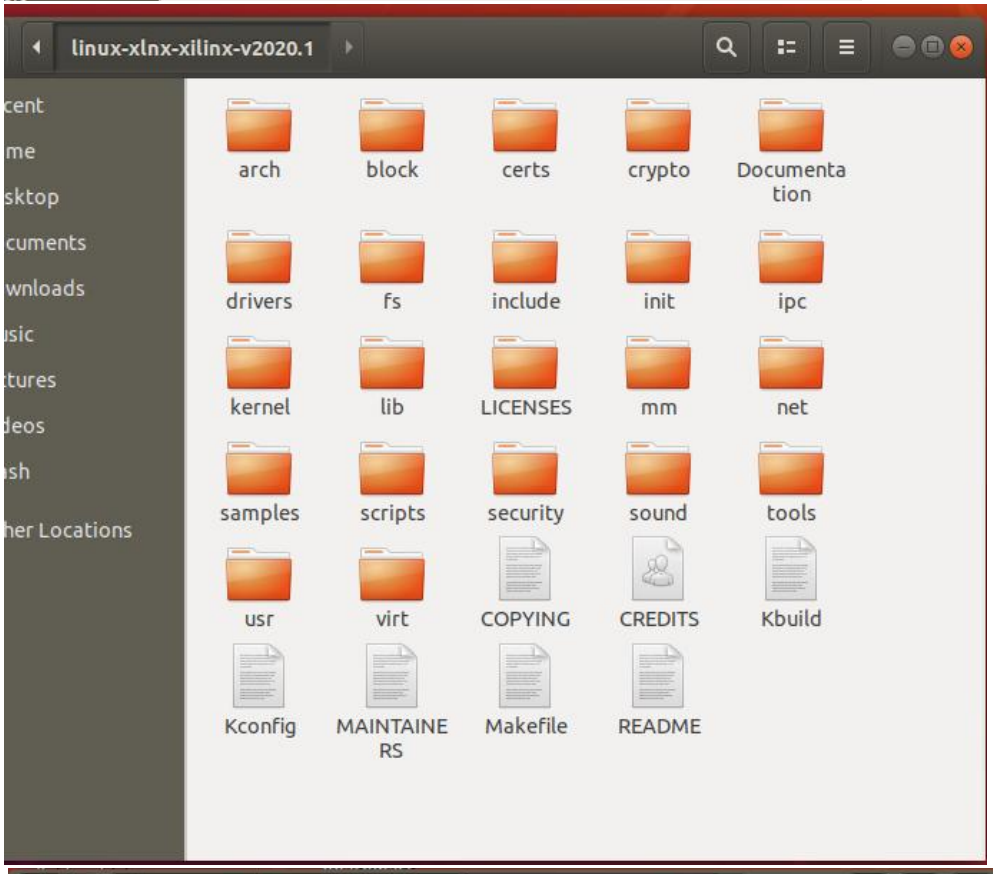
内核源码和 u-boot 源码由 xilinx 提供移植好的版本，可以到 xilinx 的 github 主页下载：https://github.com/Xilinx。

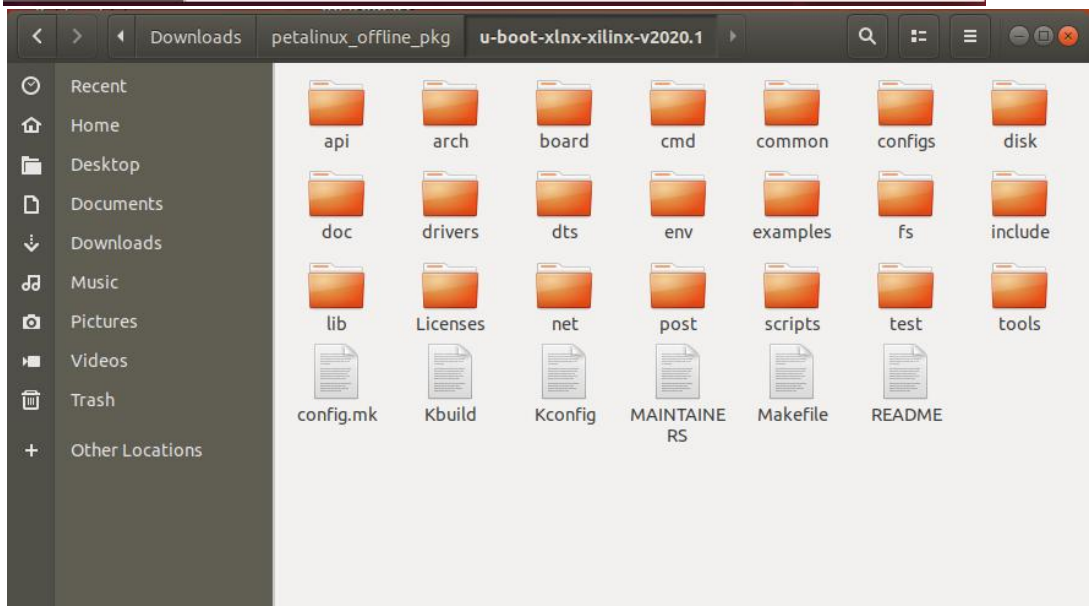
注意：①一定要使用和 petalinux 对应版本；②如果多个 petalinux 工程都需要使用本地源码，那么每一个 petalinux 工程都需要单独对应内核源码和 u-boot 源码，否则其中一个 petalinux 工程修改了源码，就会影响到其他的工程。



1. 把内核源码和 u-boot 源码拷贝到虚拟机中并解压。这里我解压到 /home/Downloads/petalinux_offline_pkg 路径中，如下图：

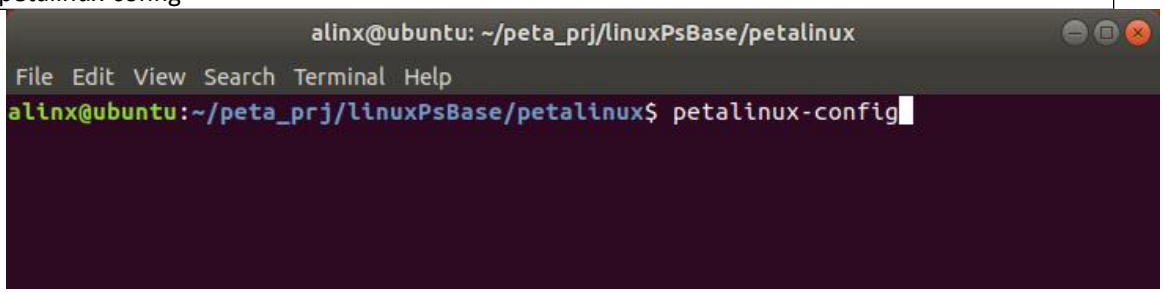

petalinux_offline_pkg
downloads_2020.1
linux-xlnx-xilinx-v2020.1
sstate_aarch64_2020.1
u-boot-xlnx-xilinx-v2020.1


linux-xlnx-xilinx-v2020.1
arch
block
certs
crypto
Documentation
drivers
fs
include
init
ipc
kernel
lib
LICENSES
mm
net
samples
scripts
security
sound
tools
usr
virt
COPYING
CREDITS
Kbuild
Kconfig
MAINTAINERS
Makefile
README

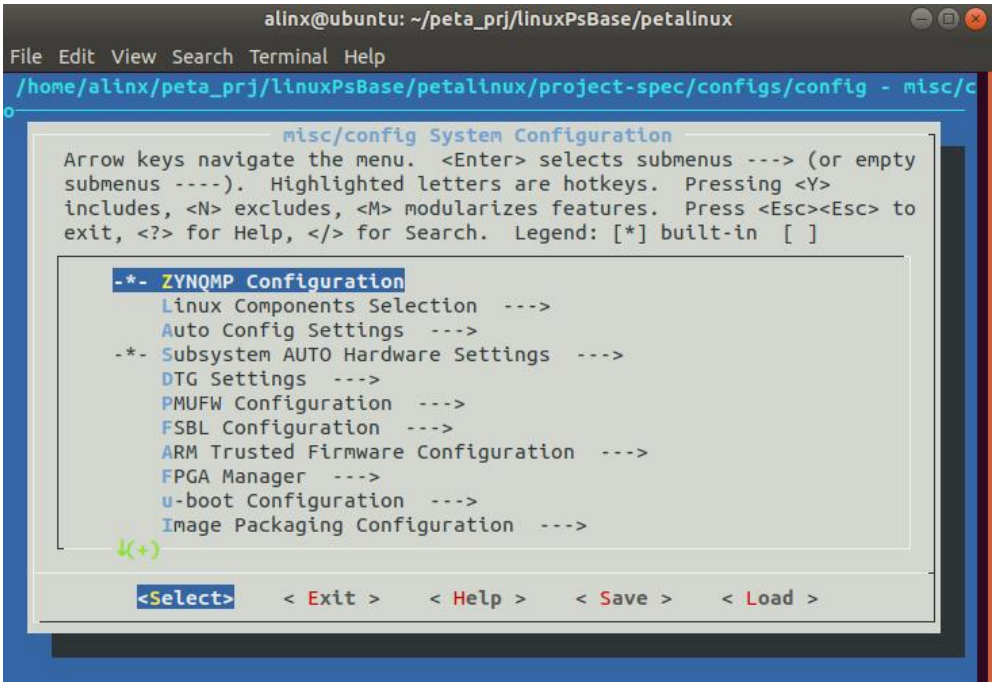

Downloads
petalinux_offline_pkg
u-boot-xlnx-xilinx-v2020.1
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Other Locations
api
arch
board
cmd
common
configs
disk
doc
drivers
dts
env
examples
fs
include
lib
Licenses
net
post
scripts
test
tools
config.mk
Kbuild
Kconfig
MAINTAINERS
Makefile
README

2. 在 petalinux 工程中打开终端，输入命令

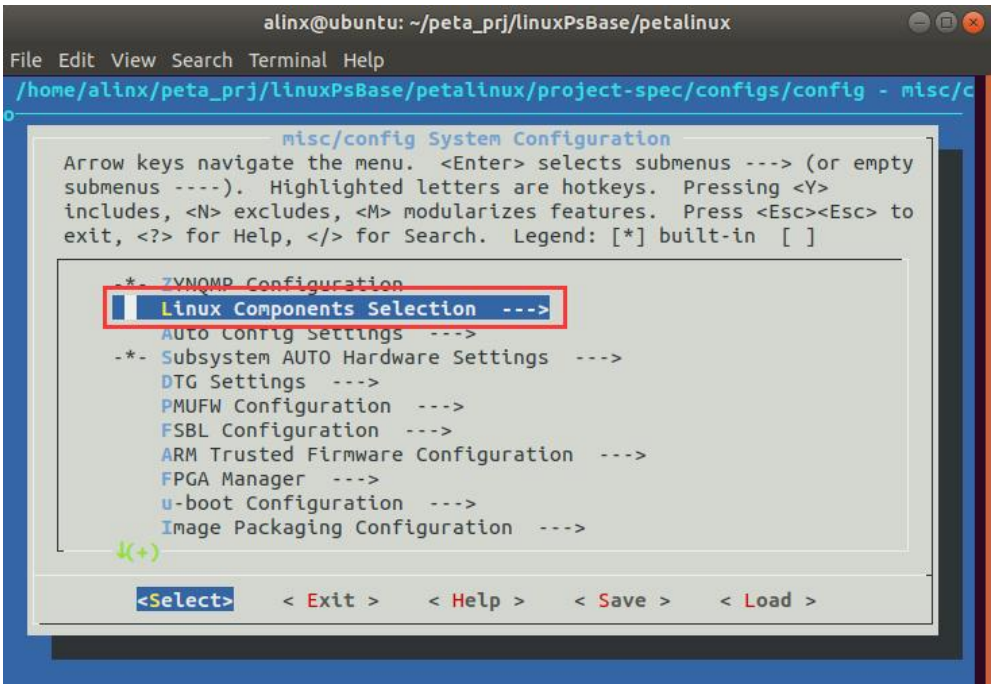
```
petalinux-config
```



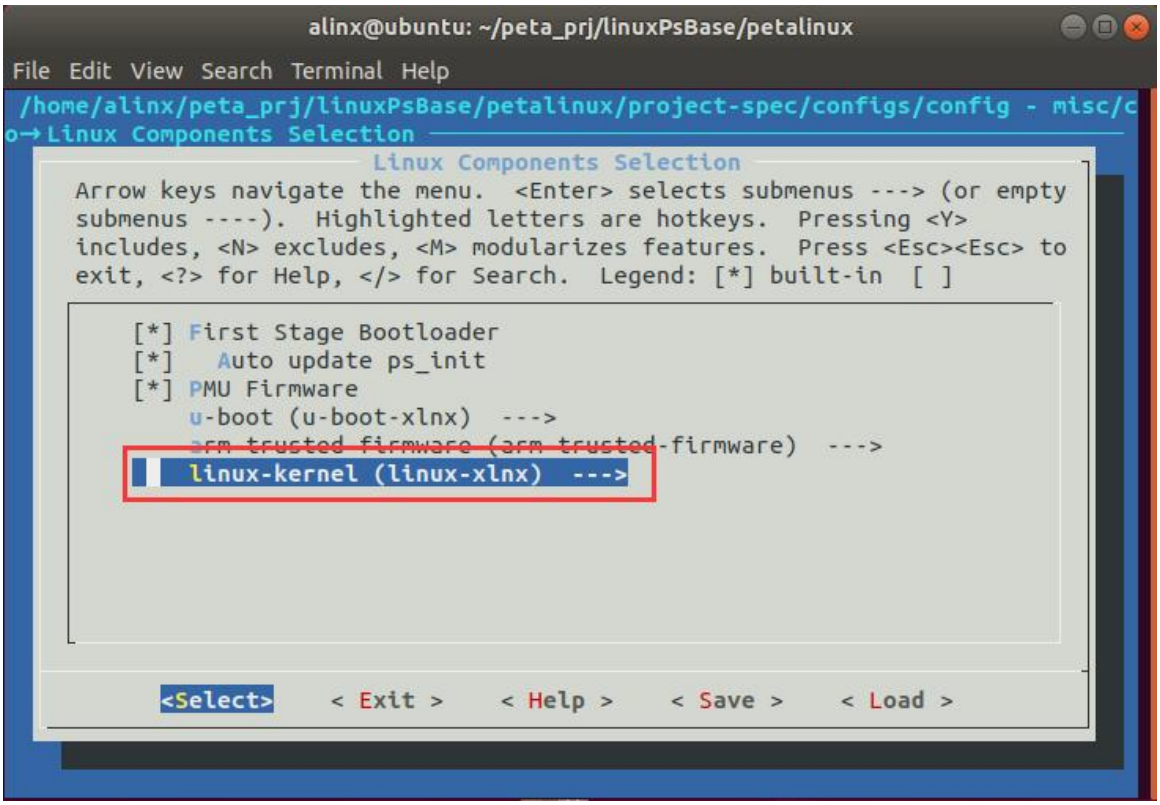
然后会弹出如下配置界面：



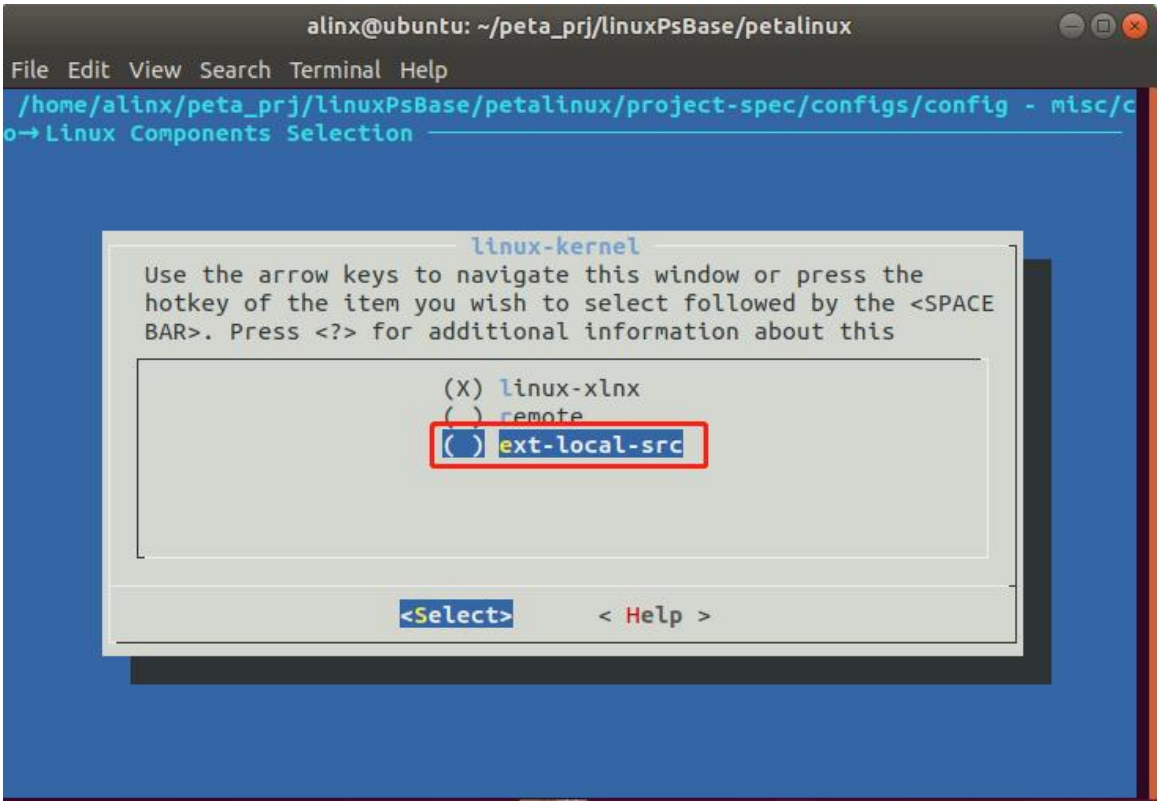
3. 设置内核源码的路径，选择“Linux Components Selection --->”选项，



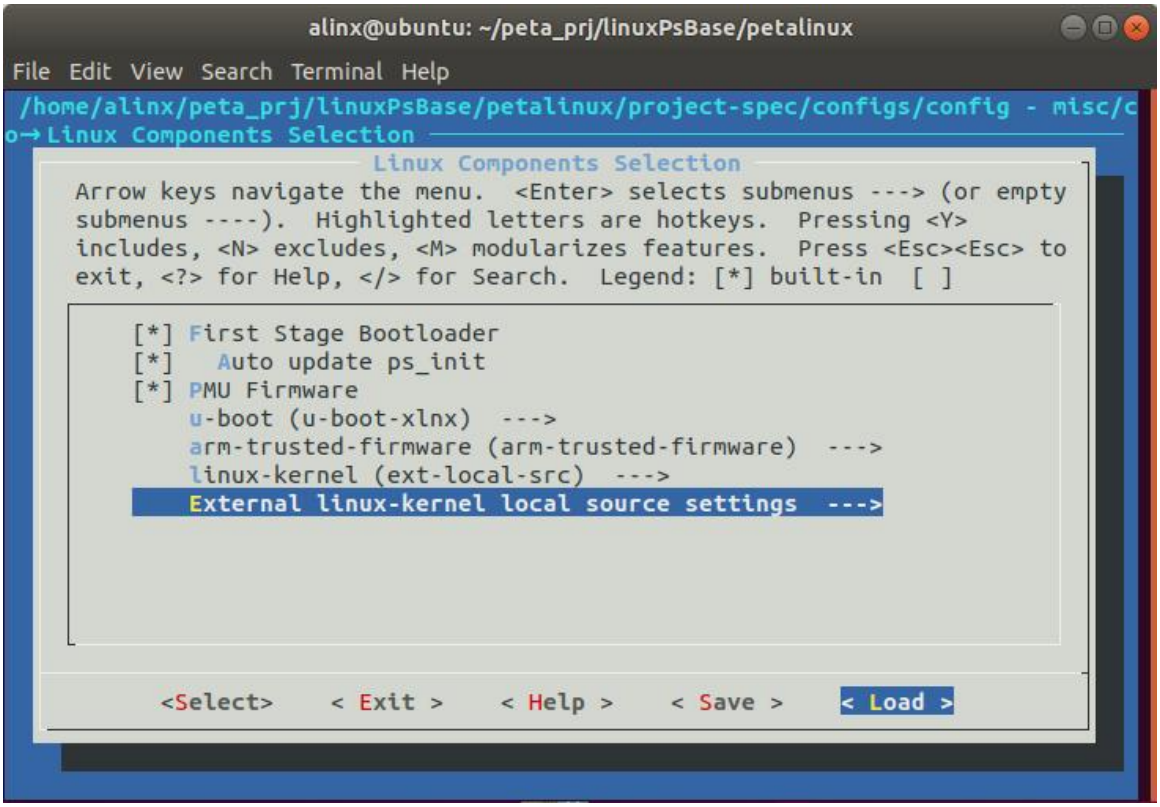
然后选择最下面的“linux-kernel (linux-xlnx) --->”选项，



接下来的界面中选择最下面的“( ) ext-local-src”选项，

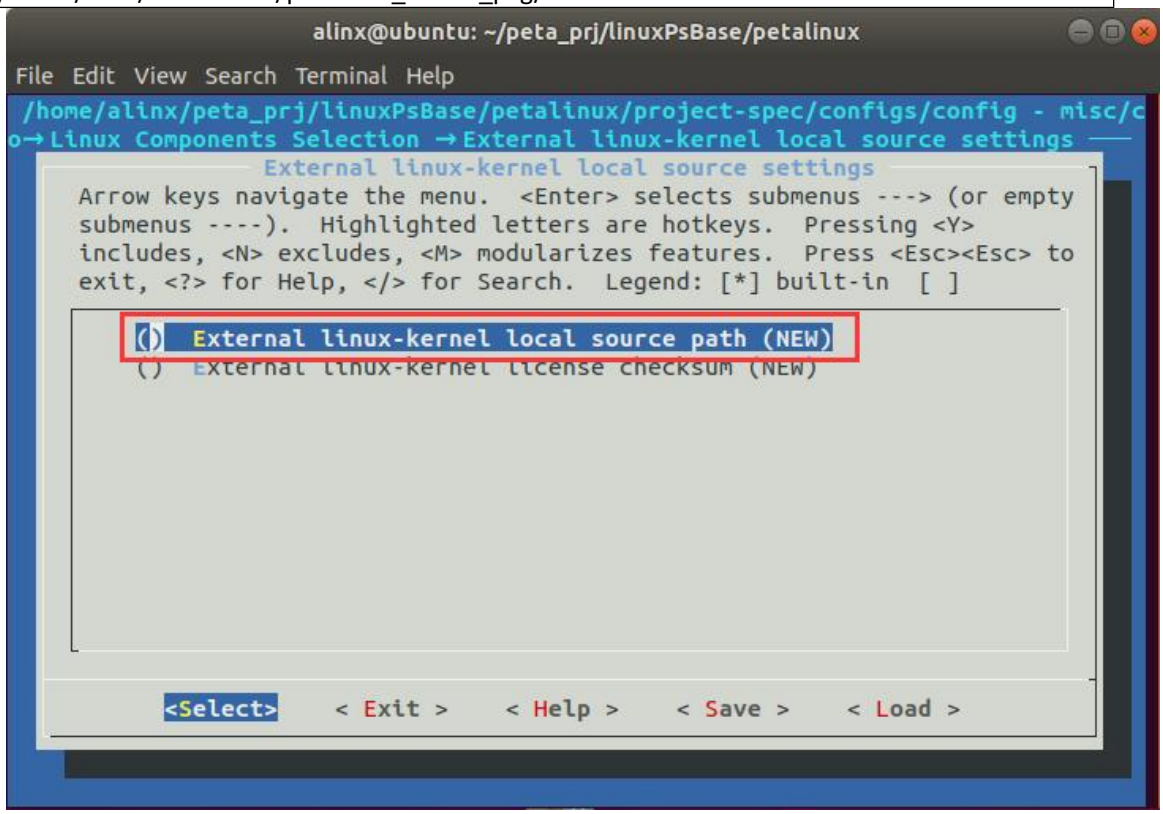


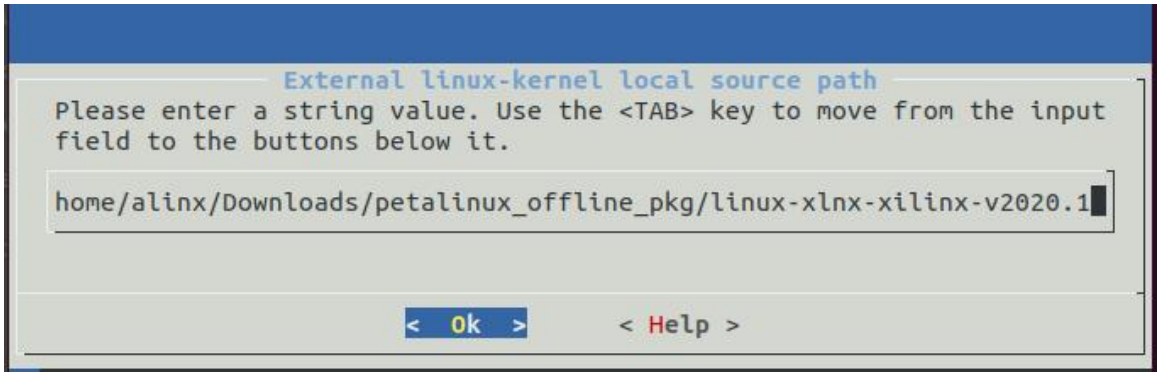
然后刚才的界面中最下面会多出一行选项，按回车选择他，

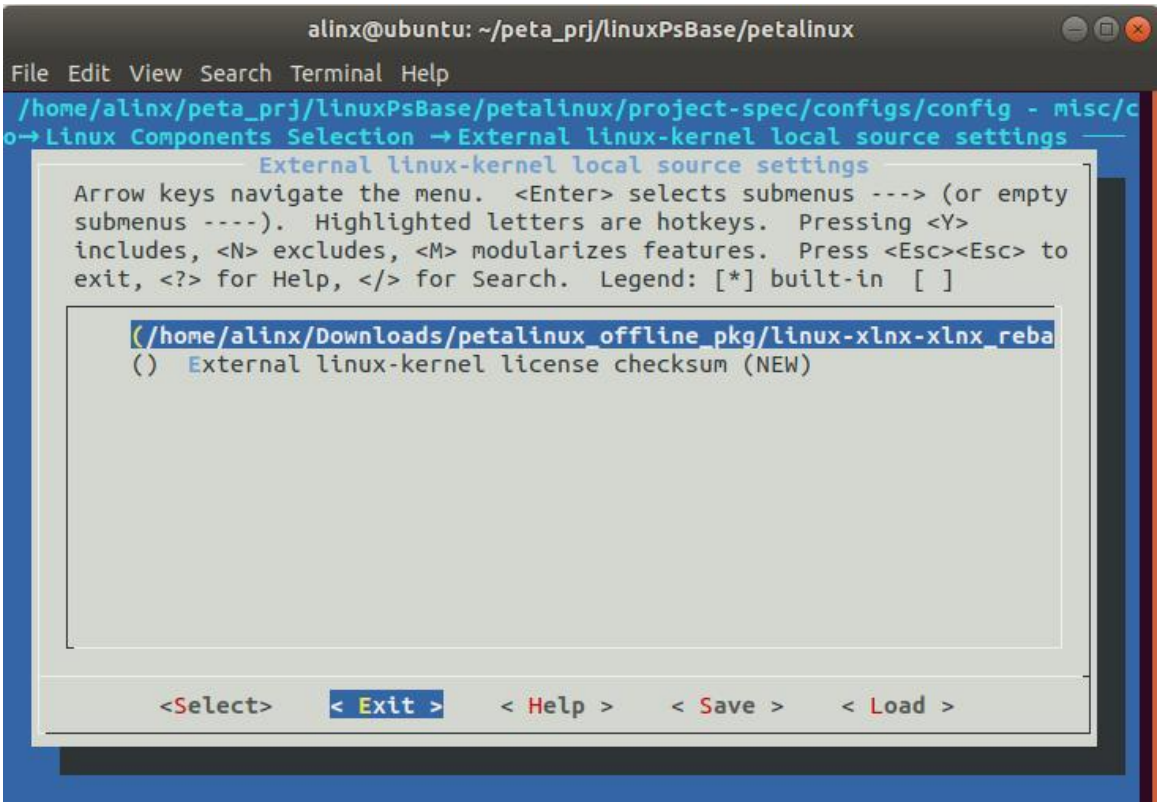


接下来就可以设置内核源码路径了，选择“”选项，输入内核源码路径

/home/alinx/Downloads/petalinux_offline_pkg/linux-xlnx-xilinx-v2020.1

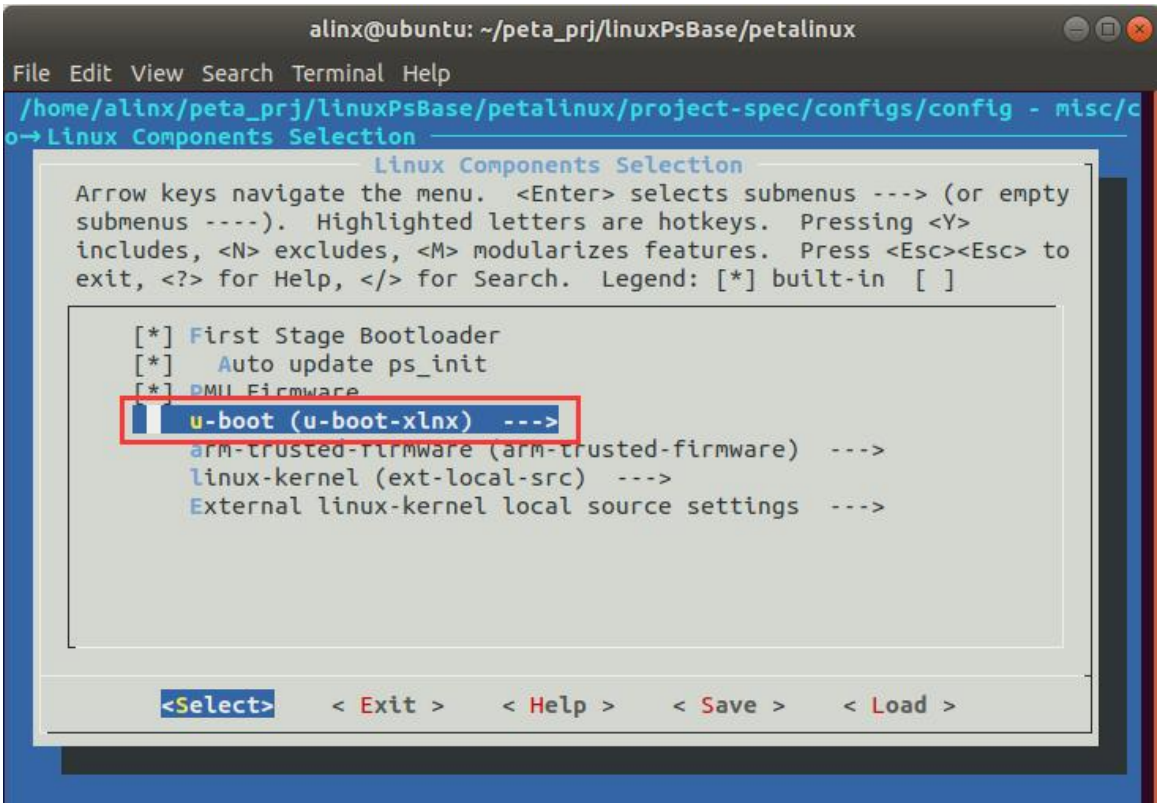




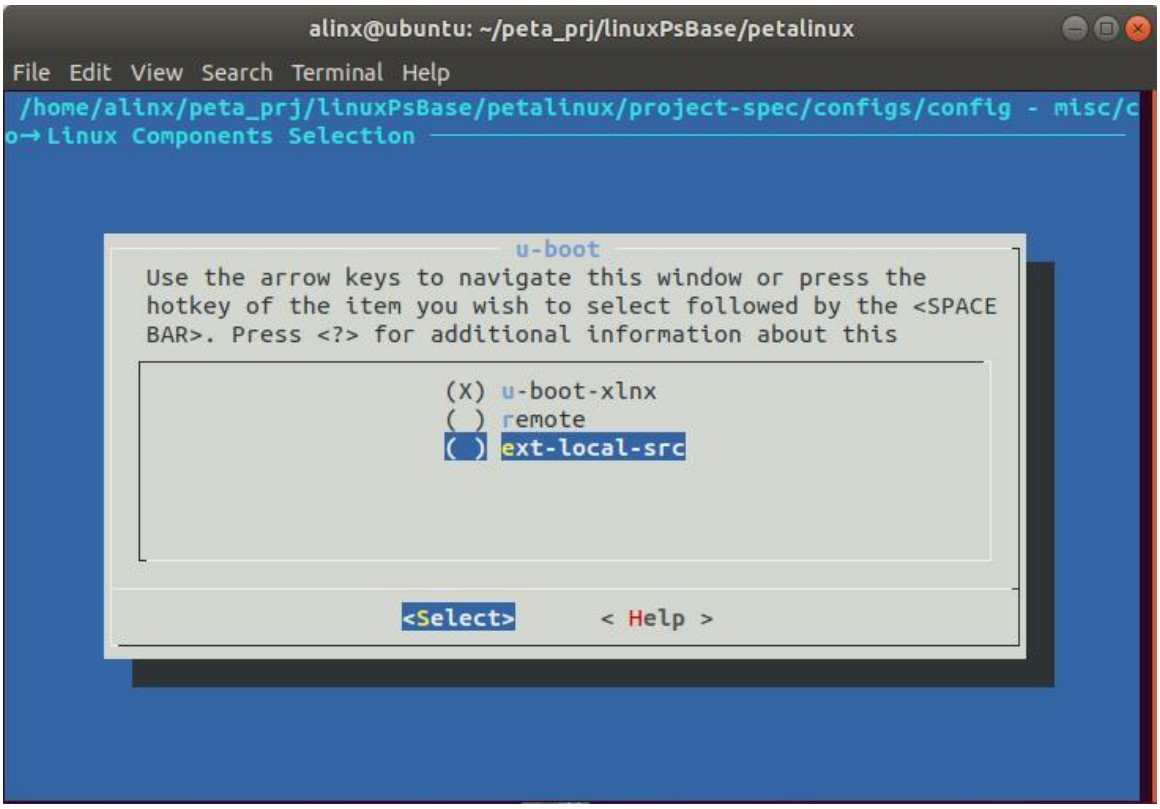


然后选择< Save >保存一下（如果不需要使用 u-boot 源码，选择< Exit >选项退出配置界面即可）。

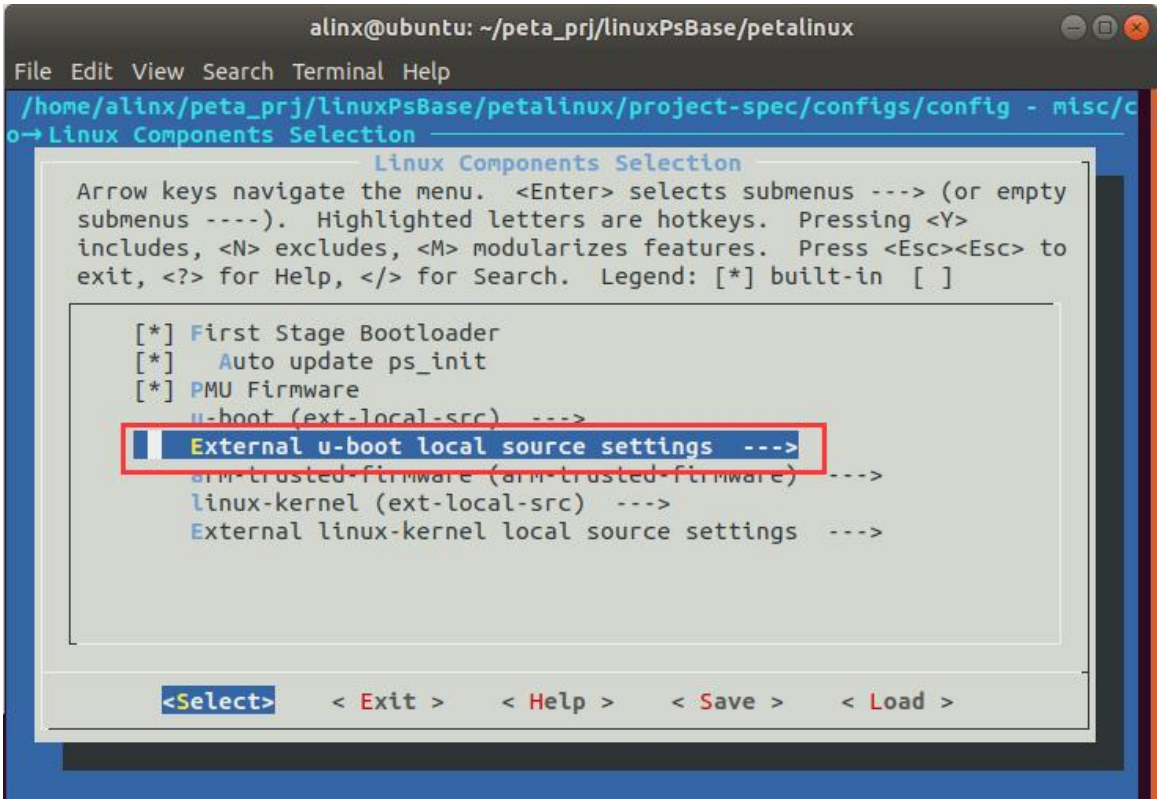
4. 设置 u-boot 源码的路径，接着上面的步骤选择< Exit >选项回到下图的界面，选择“u-boot (u-boot-xlnx) --->”选项，



选择“( ) ext-local-src”选项，

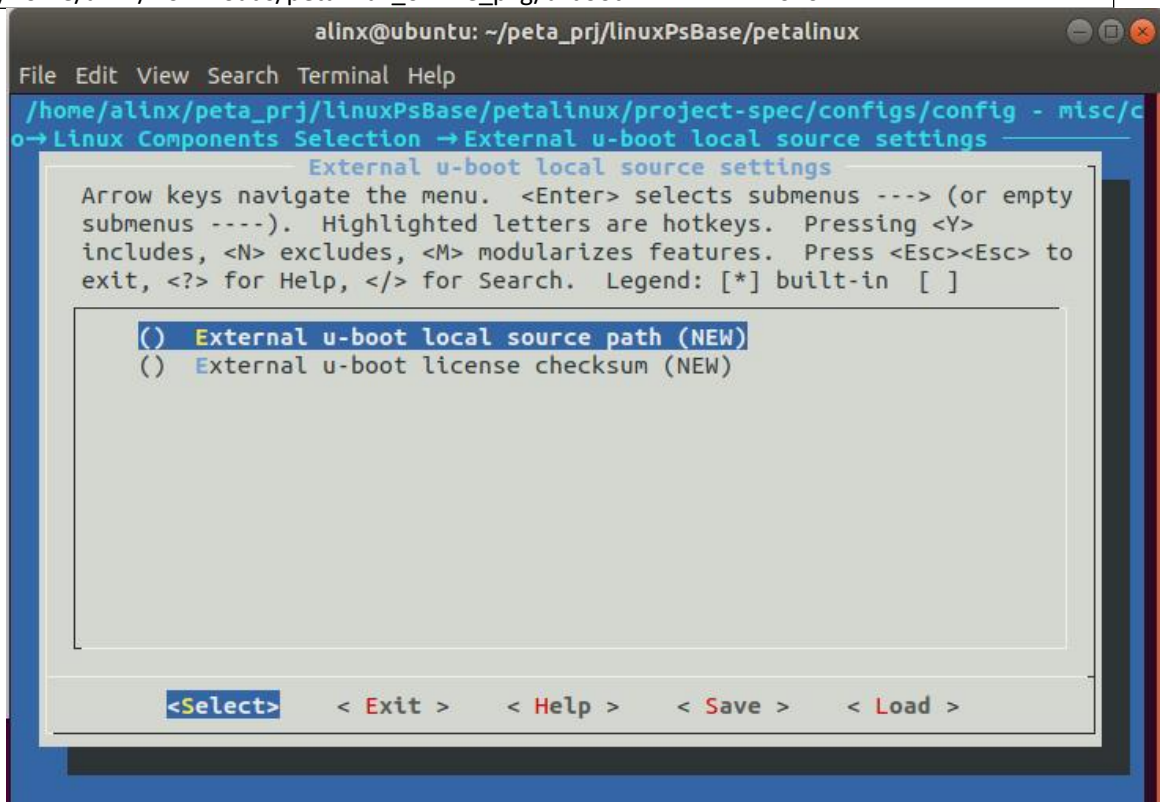


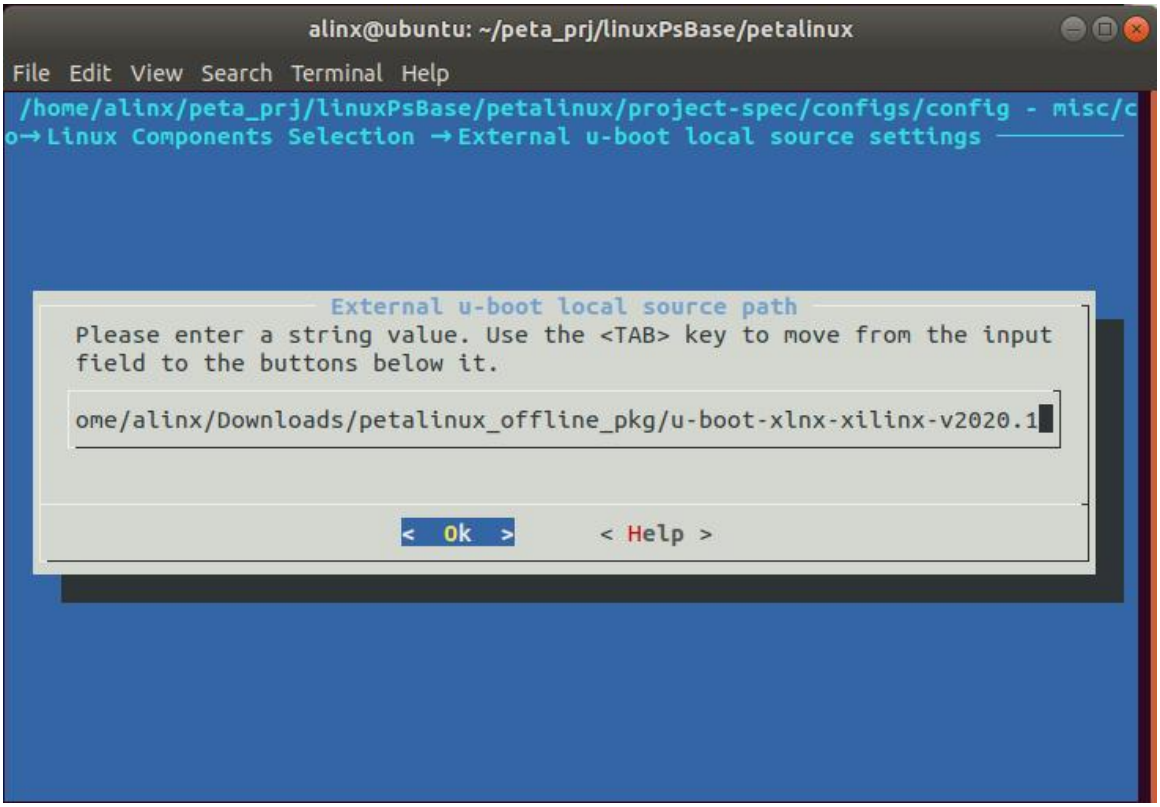
然后选择多出来的这一行“External u-boot local source settings --->”选项，

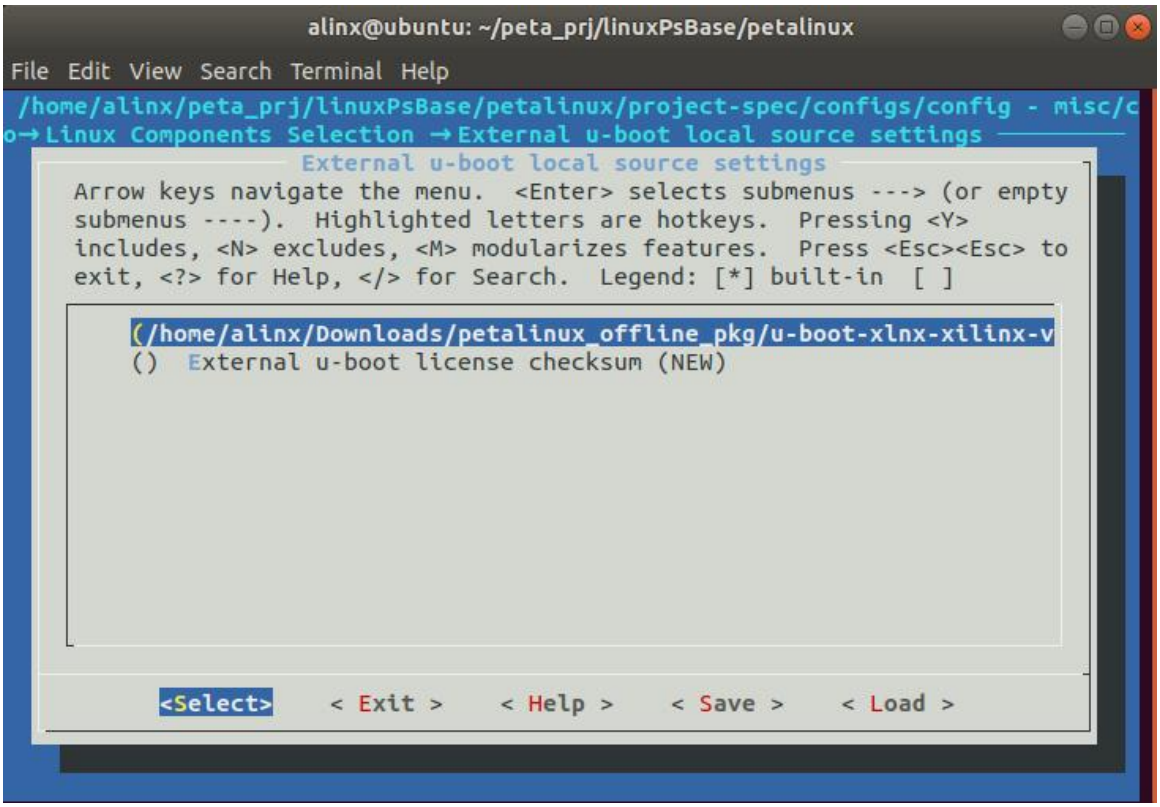


选择“() External u-boot local source path”选项设置 u-boot 源码路径，

/home/alinx/Downloads/petalinux_offline_pkg/u-boot-xlnx-xilinx-v2020.1

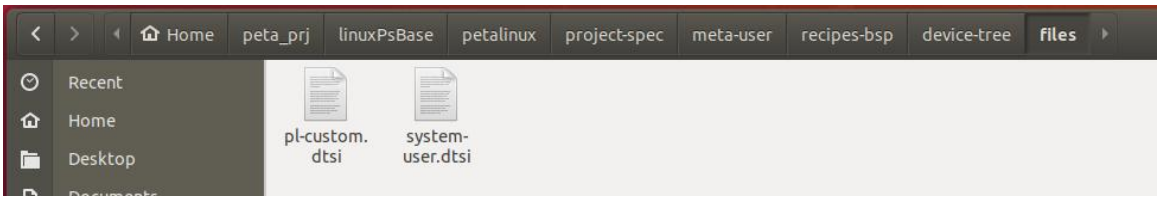






然后选择< Save >选项再选择< Exit >选项退出配置界面即可。

1.4 修改设备树、编译 petalinux 工程



1. 上图文件夹的路径中，“Home/peta_prj_linuxPsBase”是我们使用 mkdir 命令创建的路径，从“petalinux”开始往后的路径以及路径中的文件是由 petalinux-create 命令生成的。我们进入到/home/alinx/peta_prj/linuxPsBase/petalinux/project-spec/meta-user/recipes-bsp/device-tree/files 路径中，这里的两个 dtsi 后缀的文件，都是 petalinux 帮我们生成的自定义设备树文件，我们可以按照自己的需求来编辑这两个设备树这两个文件编辑两个或者只其中任意一个都可以，在本教程中我们选择编辑 system-user.dtsi 文件，双击打开这个文件把他修改为如下内容：

```
/include/ "system-conf.dtsi"
/ {
};

/* SD */
&sdhci1 {
	disable-wp;
	no-1-8-v;
};

/* USB */
&dwc3_0 {
	status = "okay";
	dr_mode = "host";
};
```

2. 输入下面的命令，编译 petalinux 工程：

```
petalinux-build
```

注：如果没有设置离线编译，该步骤必须保证 ubuntu 系统可以连接网络，petalinux 需要从 github 上下载一些源代码。

3. 编译完成后终端窗口状态如下图所示：

```
alinx@ubuntu:~/peta_prj/linxPsBase/petalinux$ petalinux-build
[INFO] building project
[INFO] sourcing bitbake
INFO: bitbake petalinux-user-image
Loading cache: 100% |################################################################################| Time: 0:00:00
Loaded 3460 entries from dependency cache.
Parsing recipes: 100% |##############################################################################| Time: 0:00:03
Parsing of 2569 .bb files complete (2534 cached, 35 parsed). 3461 targets, 137 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |###########################################################################| Time: 0:00:08
Checking sstate mirror object availability: 100% |###################################################| Time: 0:00:24
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
NOTE: Tasks Summary: Attempted 3132 tasks of which 3132 didn't need to be rerun and all succeeded.
INFO: Copying Images from deploy to images
NOTE: Failed to copy built images to tftp dir:  /tftpboot
[INFO] successfully built project
alinx@ubuntu:~/peta_prj/linxPsBase/petalinux$
```

如果使用离线编译，编译成功后可能会有一些警告，但不会影响我们使用。

```
alinx@ubuntu: ~/peta_prj/linuxPsBase/petalinux
File  Edit  View  Search  Terminal  Help
te)
NOTE: Executing Tasks
NOTE: Setscene tasks completed
WARNING: arm-trusted-firmware-2.0-xilinx-v2020.1+gitAUTOINC+5918e656ef-r0 do_pac
kage: arm-trusted-firmware: NOT adding alternative provide /boot/arm-trusted-fir
mware.elf: /boot/arm-trusted-firmware-999.elf does not exist
NOTE: linux-xlnx: compiling from external source tree /home/alinx/Downloads/peta
linux_offline_pkg/linux-xlnx-xlnx_rebase_v5.4_2020.1
WARNING: device-tree-xilinx-v2020.1+gitAUTOINC+bc84458333-r0 do_package: device-
tree: NOT adding alternative provide /boot/device-tree.dtb: /boot/device-tree-99
9.dtb does not exist
WARNING: pmu-firmware-2020.1+gitAUTOINC+6cbb920f4d-r0 do_package: pmu-firmware:
NOT adding alternative provide /boot/pmu-firmware.elf: /boot/pmu-firmware-999.el
f does not exist
NOTE: u-boot-xlnx: compiling from external source tree /home/alinx/Downloads/pet
alinux_offline_pkg/u-boot-xlnx-xilinx-v2020.1
WARNING: fsbl-2020.1+gitAUTOINC+6cbb920f4d-r0 do_package: fsbl: NOT adding alter
native provide /boot/fsbl.elf: /boot/fsbl-999.elf does not exist
NOTE: Tasks Summary: Attempted 3608 tasks of which 2652 didn't need to be rerun
and all succeeded.

Summary: There were 4 WARNING messages shown.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] successfully built project
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$
```

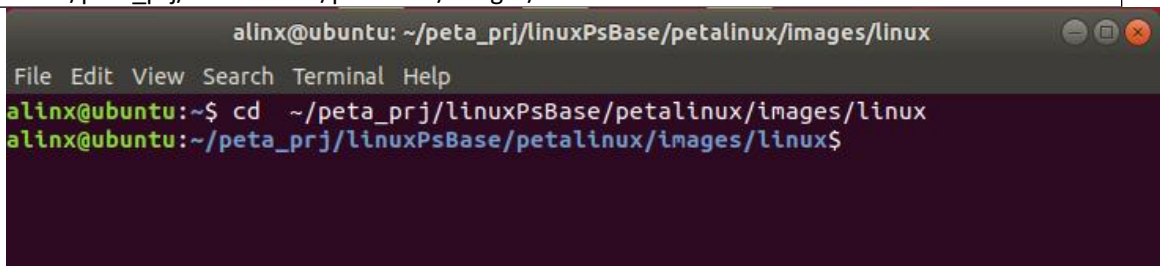
注意：如果出现了这个错误，仔细检查 1.3 解压的源码和你设置的路径对不对得上。

```
WARNING: linux-xlnx-5.4+git999-r0 do_populate_lic: Could not copy license f
ile /home/alinx/Downloads/petalinux_offline_pkg/linux-xlnx-xilinx_v2020.1/C
OPYING to /home/alinx/peta_prj/linuxPsBase/petalinux/build/tmp/work/zynqmp_
generic-xilinx-linux/linux-xlnx/5.4+git999-r0/license-destdir/linux-xlnx/CO
PYING: [Errno 2] No such file or directory: '/home/alinx/Downloads/petalinu
x_offline_pkg/linux-xlnx-xilinx_v2020.1/COPYING'
ERROR: linux-xlnx-5.4+git999-r0 do_populate_lic: QA Issue: linux-xlnx: LIC_
FILES_CHKSUM points to an invalid file: /home/alinx/Downloads/petalinux_off
line_pkg/linux-xlnx-xilinx_v2020.1/COPYING [license-checksum]
ERROR: linux-xlnx-5.4+git999-r0 do_populate_lic: Fatal QA errors found, fa
iling task.
ERROR: Logfile of failure stored in: /home/alinx/peta_prj/linuxPsBase/petal
inux/build/tmp/work/zynqmp_generic-xilinx-linux/linux-xlnx/5.4+git999-r0/te
mp/log.do_populate_lic.27098
ERROR: Task (/home/alinx/peta_prj/linuxPsBase/petalinux/components/yocto/la
yers/meta-xilinx/meta-xilinx-bsp/recipes-kernel/linux/linux-xlnx_2020.1.bb:
do_populate_lic) failed with exit code '1'
```

1.5 生成 BOOT 文件

1. 打开一个终端，使用下面的命令，进入上面 petalinux 编译之后的结果目录中。

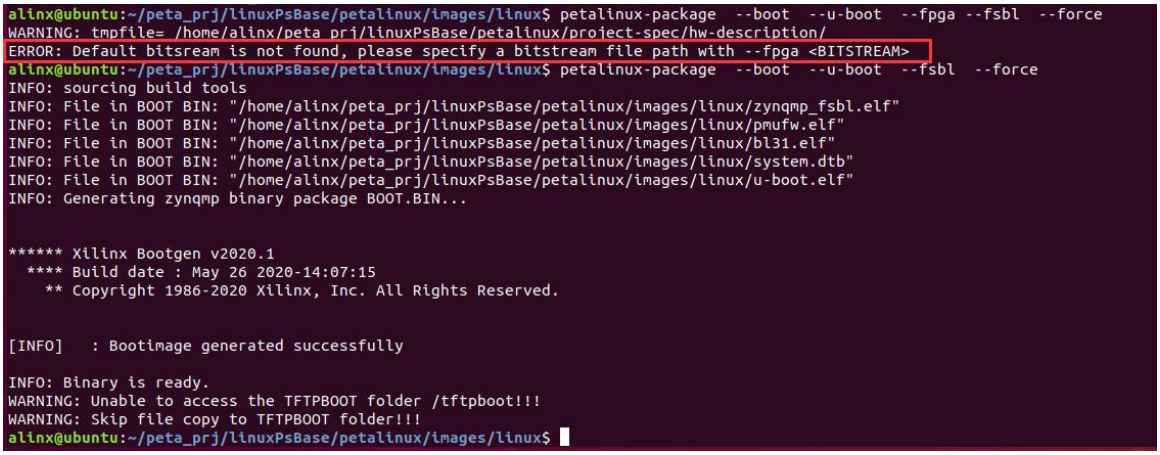
```
cd  ~/peta_prj/linuxPsBase/petalinux/images/linux
```



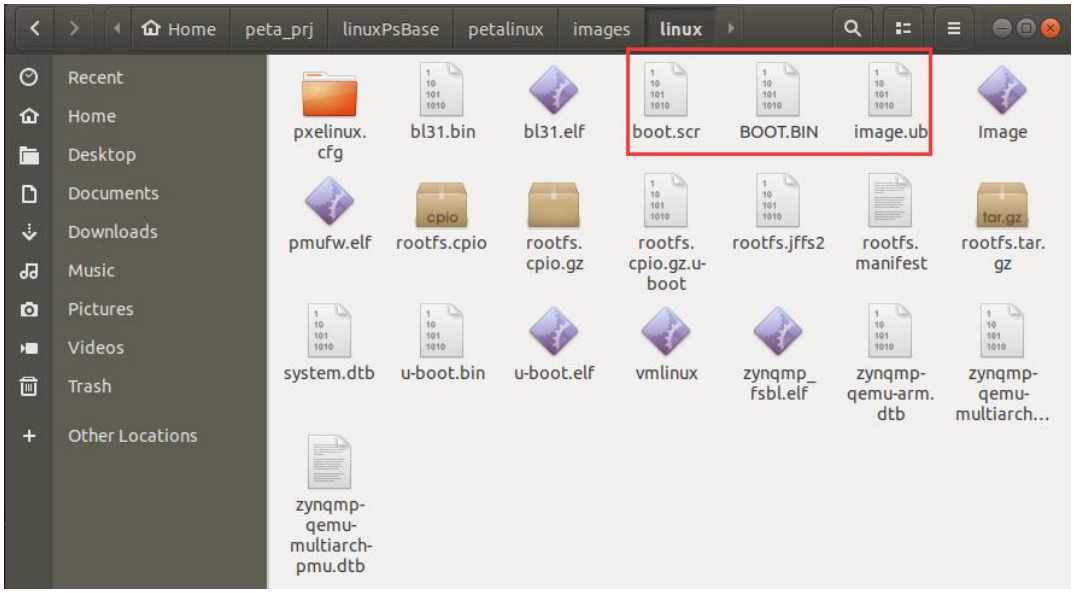
2. 使用“petalinux-package –boot”命令打包 BOOT.bin 文件。“--u-boot”参数会把 u-boot 依赖项全部合成到 BOOT.bin 中，“--fpga”指合成 bit 文件到 BOOT.bin 中(注意：如果你的 xsa 文件中不包含 bitstream，就不要加这个参数)，“--fsbl”指合成 fsbl 文件到 BOOT.bin 中，“--force”为强制合成，覆盖以前合成的 BOOT.BIN。注意这条命令里的参数都是 2 个短横线。

```
petalinux-package  --boot  --u-boot  --fpga  --fsbl  --force
```

如果 xsa 文件中不包含 bitstream，但是命令中却包含了“--fpga”参数，就会出现下图红框中的错误。去掉“--fpga”参数即可打包成功，如下图。

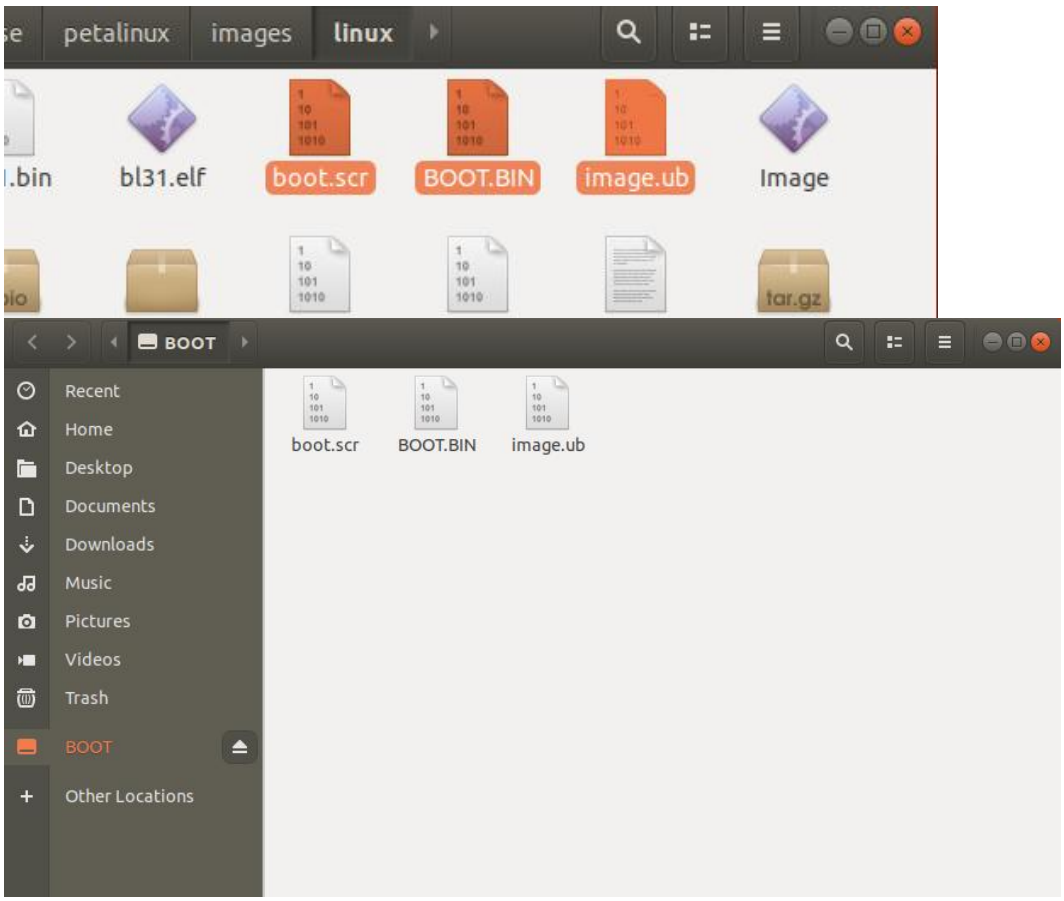


3. 进入到 petalinux 工程编译结果目录中，可以看到红框中的三个文件，其中 BOOT.bin 文件是我们刚才生成的，其他两个文件是“petalinux-build”命令生成的。

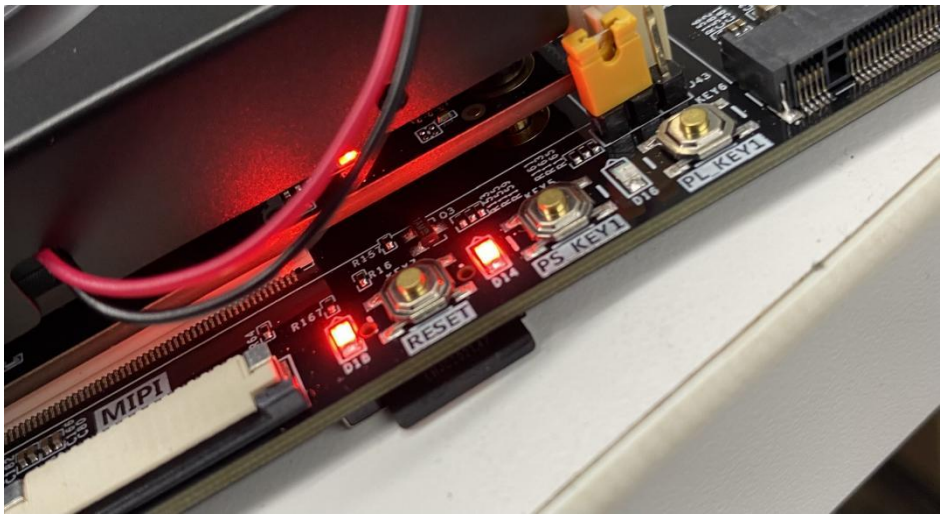


1.6 运行 Linux

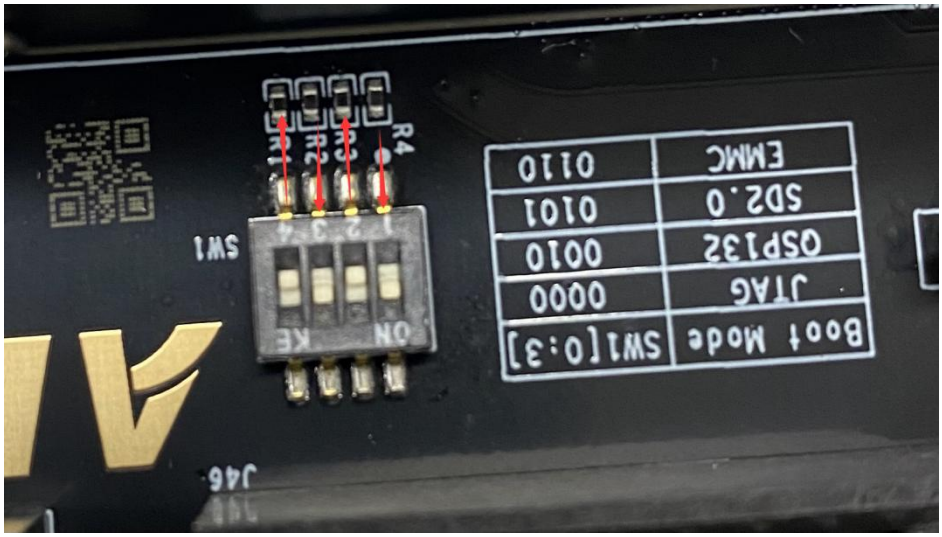
1. 把 SD 卡格式化为 FAT32 格式。
2. 将之前得到的 BOOT.bin、boot.scr 及 image.ub 三个文件拷贝到上一步格式化的 SD 卡中，然后弹出 SD 卡。



3. 将 SD 卡插入板卡。

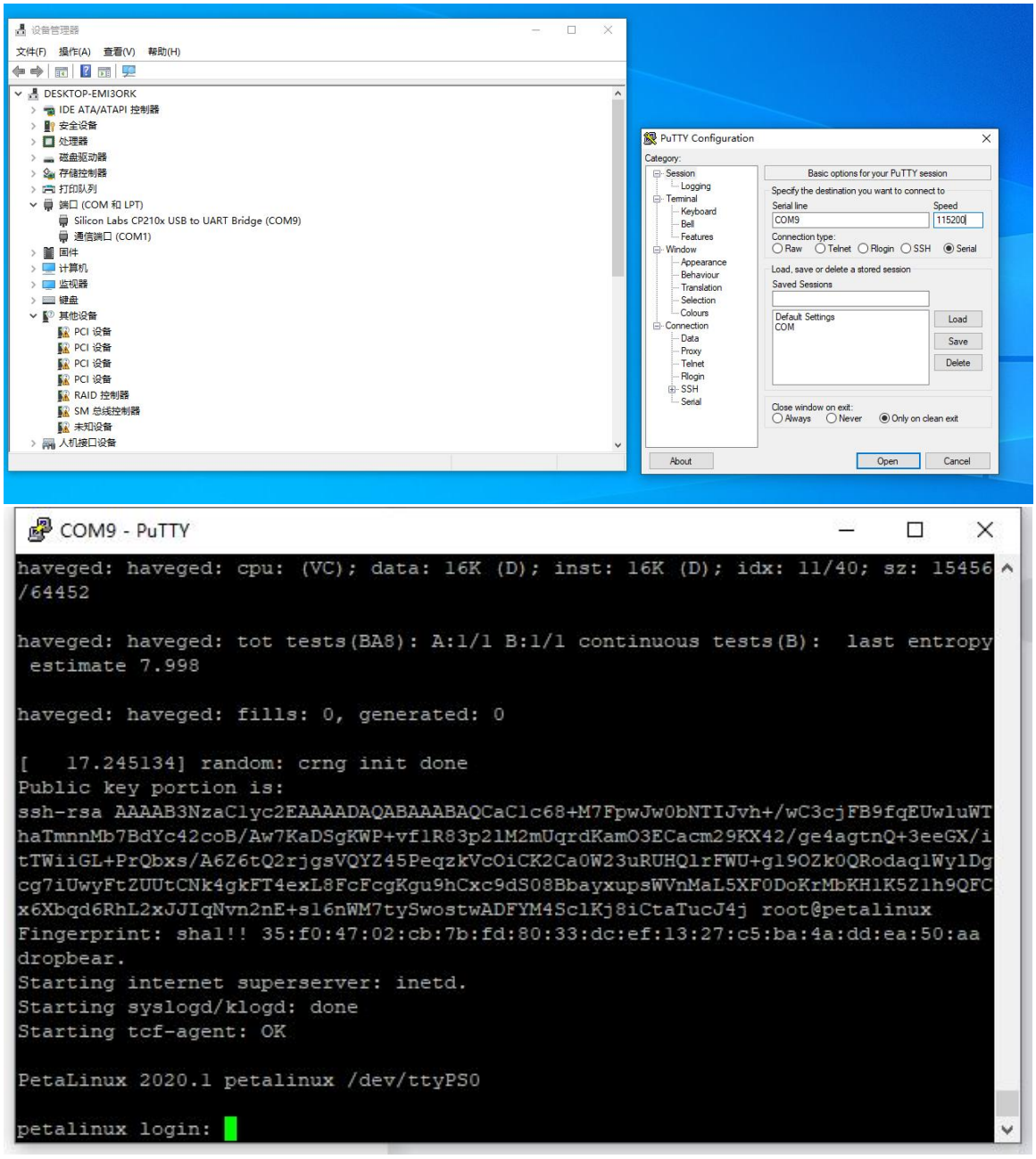


4. 把板卡设置为 SD 卡启动模式。

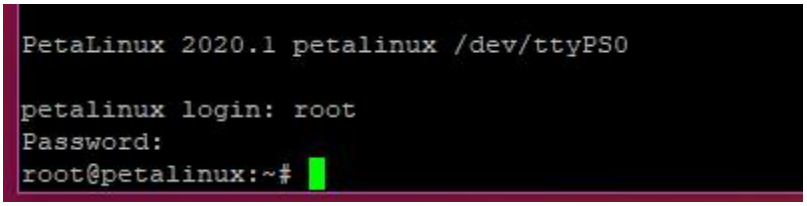


5. 连接串口线到开发板的“PS_UART”口上，打开串口调试工具，将板卡上电。

看自己的串口号是多少，然后以图设置



6. 在串口工具中输入用户名 root，密码 root 即可登录板载的 linux 系统（因为 petalinux 制作的根文件默认为 ram 类型，所以文件系统不能保存文件和配置即重启后回恢复初始状态）。



1.7 Petalinux 使用流程总结

1. Petalinux 工作环境：
   - ✧ Petalinux 软件安装

- ✧ "/bin/sh" 需要是 bash，系统默认可能是 dash，需要修改
- ✧ 不能在共享文件夹创建 petalinux 工程
- ✧ 使用 petalinux 命令前需要设置 petalinux 环境变量：source <path-to-installed-PetaLinux>/settings.sh

2. 创建 Petalinux 工程：
   - ✧ petalinux-create --type project --template <CPU_TYPE> --name <PROJECT_NAME>
     - ➢ <CPU_TYPE>可以是 zynqMP、 zynq、 microblaze
     - ➢ <PROJECT_NAME>就是工程名称
3. 导入硬件信息：
   - ✧ 在 petalinux 工程目录中打开终端
   - ✧ 使用 petalinux-config --get-hw-description <HARDWARE>
     - ➢ <HARDWARE>是含有 xsa 文件的目录
4. 配置 petalinux 工程：
   - ✧ petalinux-config 配置整个 petalinux 工程，如果需要配置 uboot、kernel、rootfs 等，分别使用下面的命令
     - ➢ petalinux-config -c u-boot
     - ➢ petalinux-config -c kernel
     - ➢ petalinux-config -c rootfs
5. 编译 petalinux 工程：
   - ✧ petalinux-build
6. 手动修改 petalinux 配置（这些配置文件可以直接复制到其他工程中使用)：
   - ✧ 在 “<plnx-proj-root>/project-spec/meta-user”目录保存着各种可以手动配置的文件
     - ➢ “<plnx-proj-root>/project-spec/meta-user/recipes-bsp/u-boot/files/platform-top.h”文件用于修改 u-boot 配置
     - ➢ “<plnx-proj-root>/project-spec/meta-user/recipes-bsp/device-tree/files/”路径中的“system-user.dtsi”和“pl-custom.dtsi”文件用于自定义设备树
     - ➢ “<plnx-proj-root>/project-spec/meta-user/recipes-kernel/linux/linux-xlnx/”路径中保存了内核的配置
     - ➢ “<plnx-proj-root>/project-spec/meta-user/project-spec/meta-user/recipes-apps/”路径中保存着 app
7. 清理 petalinux：
   - ✧ 如果 petalinux 编译出现各种问题，特别是我们修改一些配置后无法正常编译或者一直卡死，可以尝试使用命令“petalinux-build -x mrproper –f”来清理以前的编译结果。

1.8 创建和使用 bsp 包

使用 petalinux 的 bsp 包，我们可以方便快速的提供我们验证好的 petalinux 工程给他人使用，

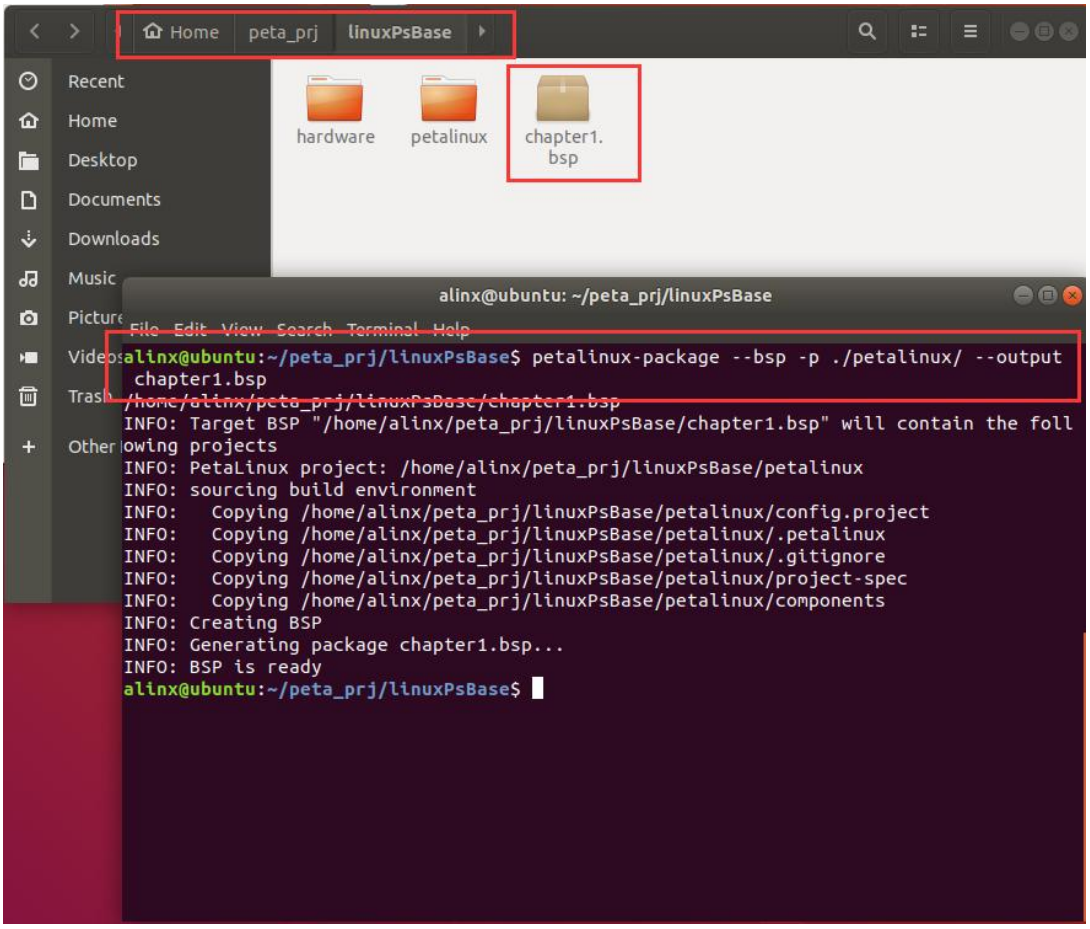
或者是使用他人验证好的 petalinux 工程。只需要掌握创建和使用 bsp 包的两条命令即可。

1. 创建 bsp 包

使用下面的命令即可创建 petalinux 工程的 bsp 包到当前打开终端的路径中：

```
petalinux-package --bsp -p ./petalinux/ --output chapter1.bsp
```

- -p 参数后面的路径是 petalinux 工程的一级路径
- --output 参数后面跟的是 bsp 包的名称

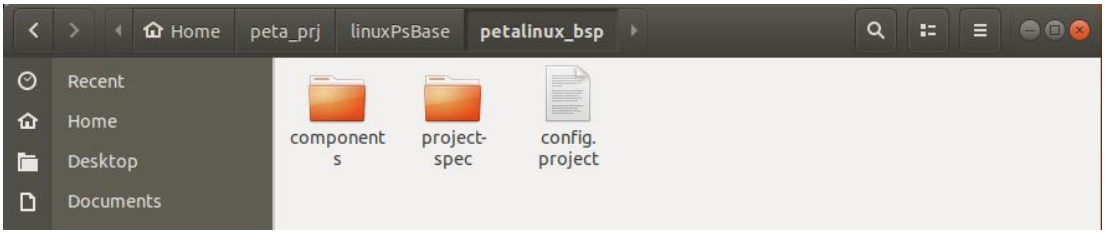


2. 使用 bsp 包

使用下面的命令即可用 bsp 包来创建 petalinux 工程到当前打开终端的路径中：

```
petalinux-create -t project -n petalinux_bsp -s ./chapter1.bsp
```

- -n 参数后面跟的是 petalinux 工程的名称
- -s 参数后面跟的是我们使用的 bsp 包的路径



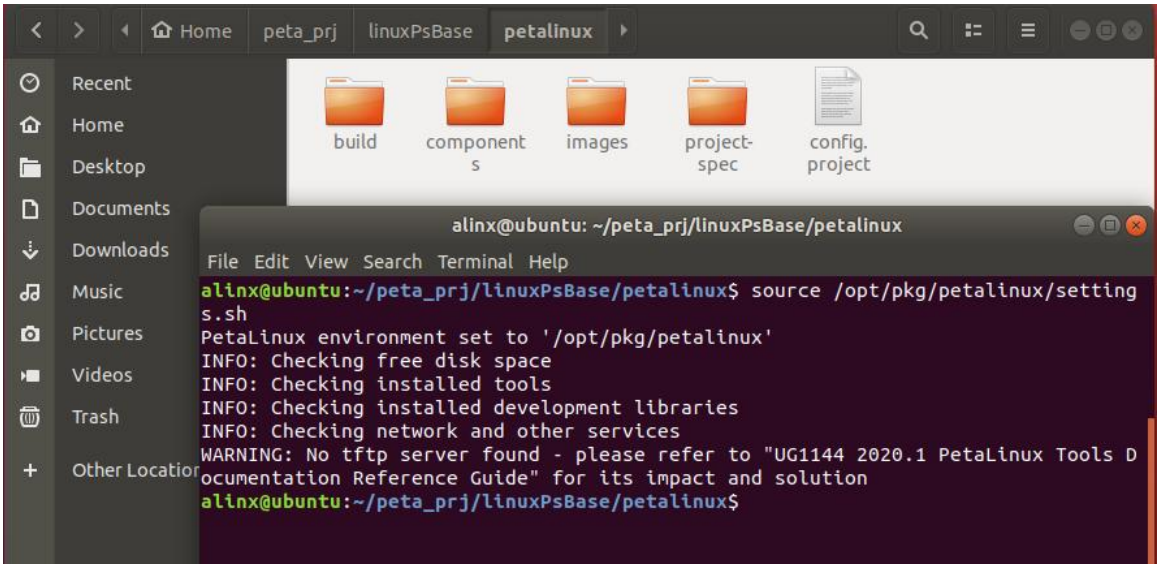
注：如果制作 bsp 包的工程可以编译成功，那使用 bsp 包制作的工程也可以直接编译。

第二章 添加开机自启动脚本及用户文件

使用 petalinux 来创建 Linux，默认根文件系统为 ramdisk 类型，这种类型的文件系统为只读类型，对该文件系统的任何修改，在系统重启后又会丢失。本章讲述如何在这种文件系统中添加用户的应用和数据文件以及开机自动启动用户应用程序的方法。

2.1 准备工作

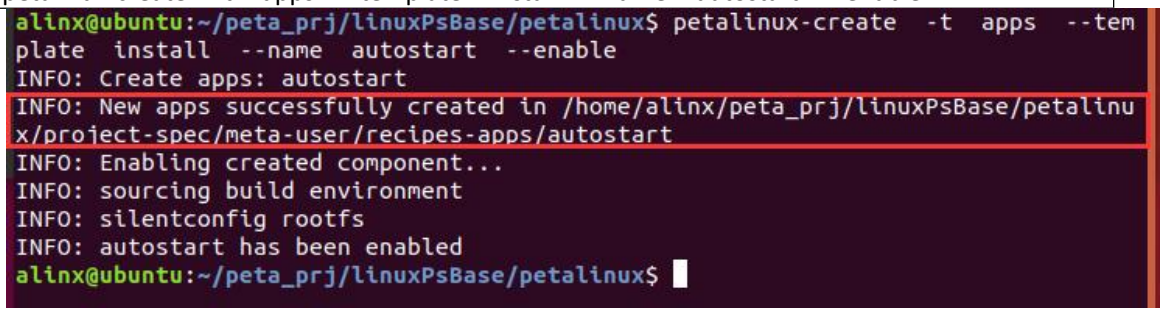
使用第一章制作的 petalinux 工程，在工程路径中打开一个终端，设置好环境变量。



2.2 自启动脚本

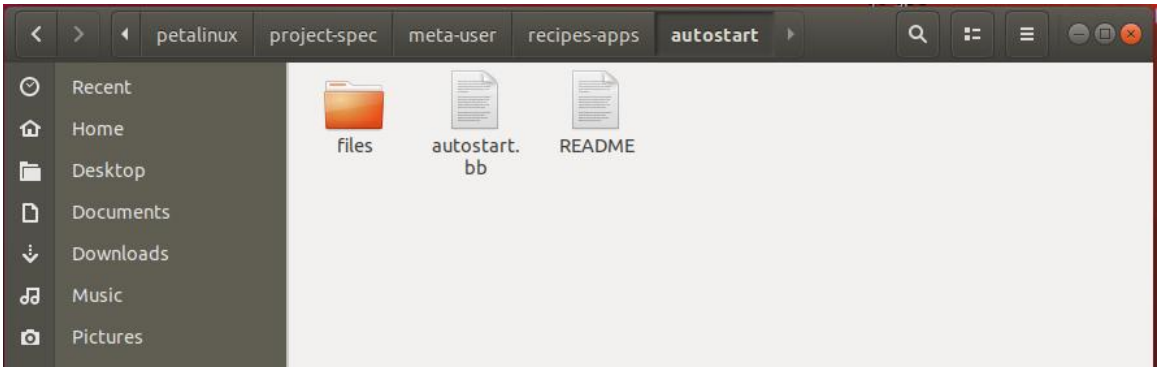
1. 输入下面的命令，用“install”模板，创建一个名为“autostart”的应用程序：

```
petalinux-create -t apps --template install --name autostart --enable
```



创建成功后，会提示我们文件所在路径，如上图红框中所示。

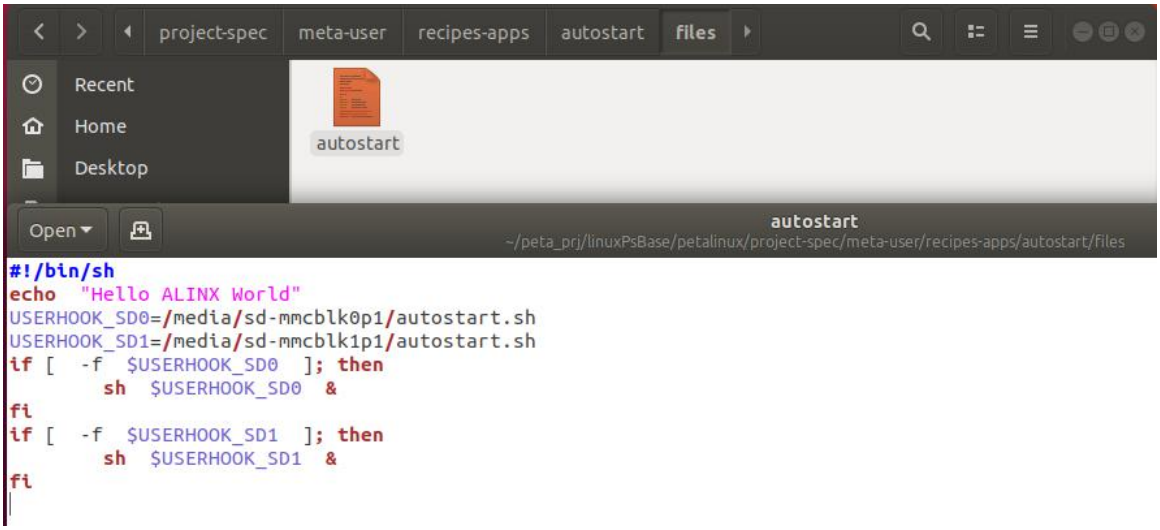
2. 我们进入刚才创建的程序所在的路径，内容如下图所示：



3. 打开 **files 文件夹下（不是上图的.bb 文件）**的 autostart 文件，这是一个脚本文件，原来的内容是打印字符串“Hello Petalinux World”，我们将它改为下面的内容。

```
#!/bin/sh
echo  "Hello ALINX World"
USERHOOK_SD0=/media/sd-mmcblk0p1/autostart.sh
USERHOOK_SD1=/media/sd-mmcblk1p1/autostart.sh
if [  -f  $USERHOOK_SD0  ]; then
     sh  $USERHOOK_SD0  &
fi
if [  -f  $USERHOOK_SD1  ]; then
     sh  $USERHOOK_SD1  &
fi
```

先打印“Hello ALINX World”。然后判断 SD 或 EMMC 第一分区中是否存在“autostart.sh”脚本，如果存在就运行。



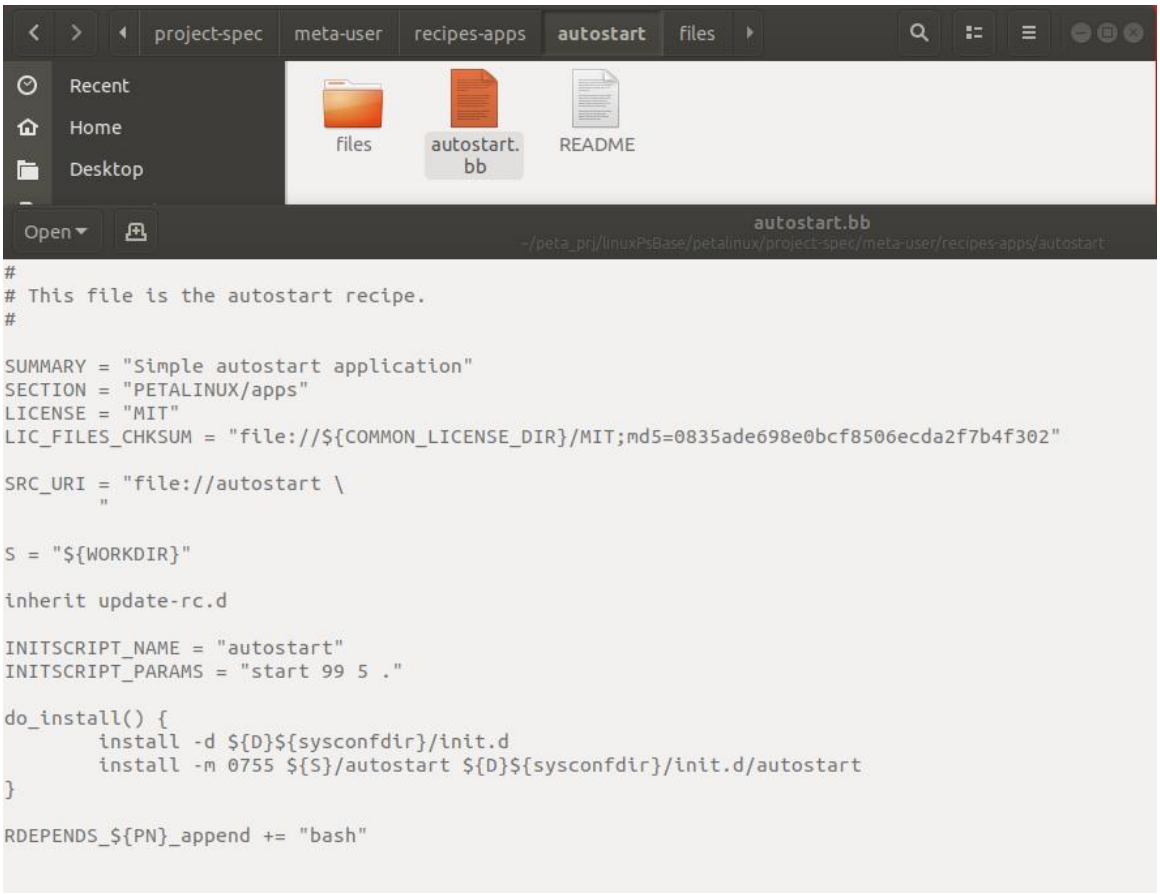
4. 回到上一级目录，编辑文件 autostart.bb 为如下内容：

```
#
# This file is the autostart recipe.
#

SUMMARY = "Simple autostart application"
SECTION = "PETALINUX/apps"
LICENSE = "MIT"
LIC_FILES_CHKSUM =
"file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
```

```
SRC_URI = "file://autostart \
      "

S = "${WORKDIR}"

inherit update-rc.d

INITSCRIPT_NAME = "autostart"
INITSCRIPT_PARAMS = "start 99 5 ."

do_install() {
      install -d ${D}${sysconfdir}/init.d
      install -m 0755 ${S}/autostart ${D}${sysconfdir}/init.d/autostart
}

RDEPENDS_${PN}_append += "bash"
```

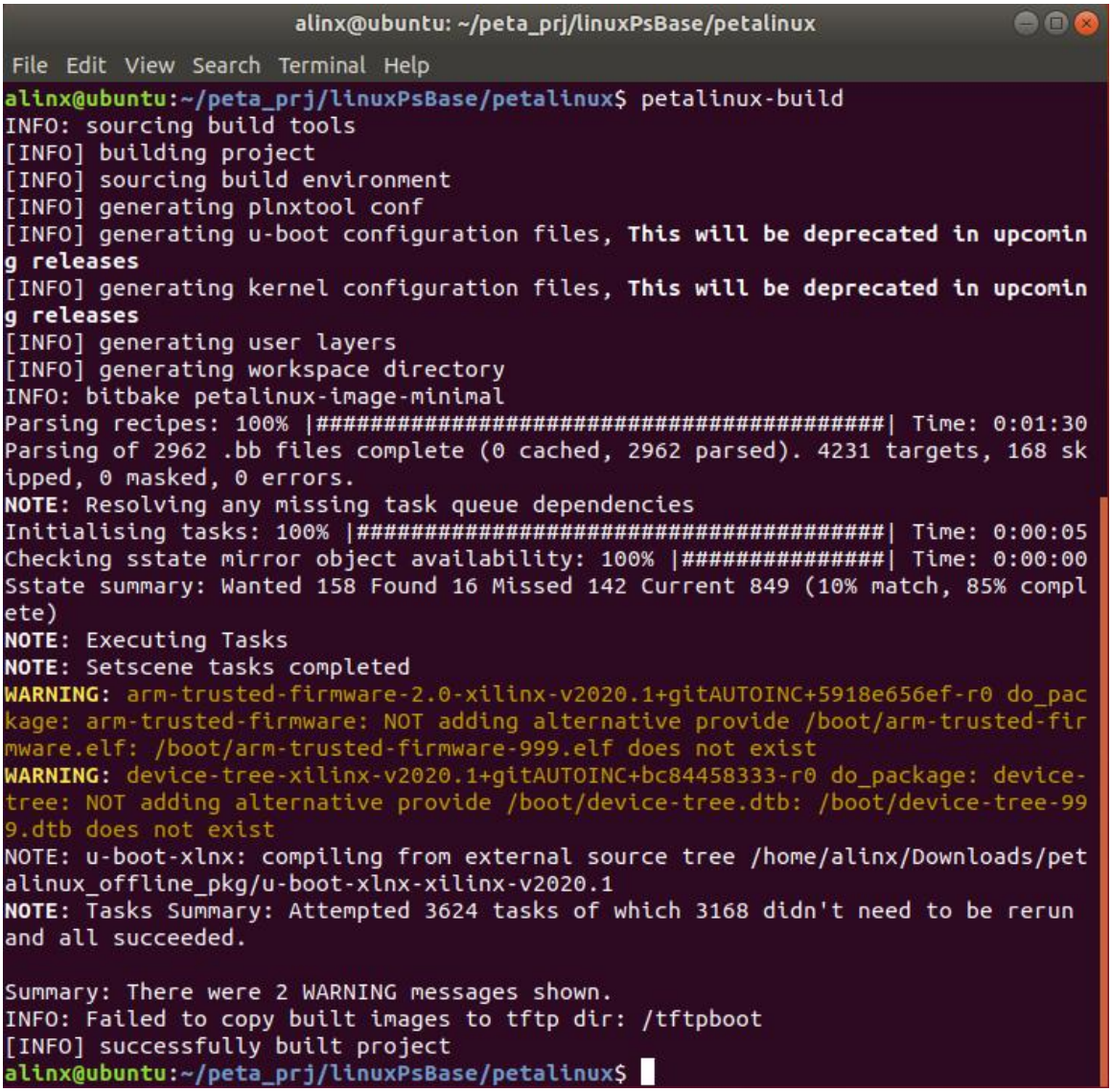
主要功能是复制 autostart 到根文件系统 init.d 目录，然后添加 0755 权限。

RDEPENDS_${PN}_append += "bash"指这个要依赖 bash。



5. 在终端中输入命令重新编译 petalinux 工程：

```
petalinux-build
```



使用下面命令打包 BOOT.bin：

```
petalinux-package  --boot  --u-boot  --fsbl  --force
```

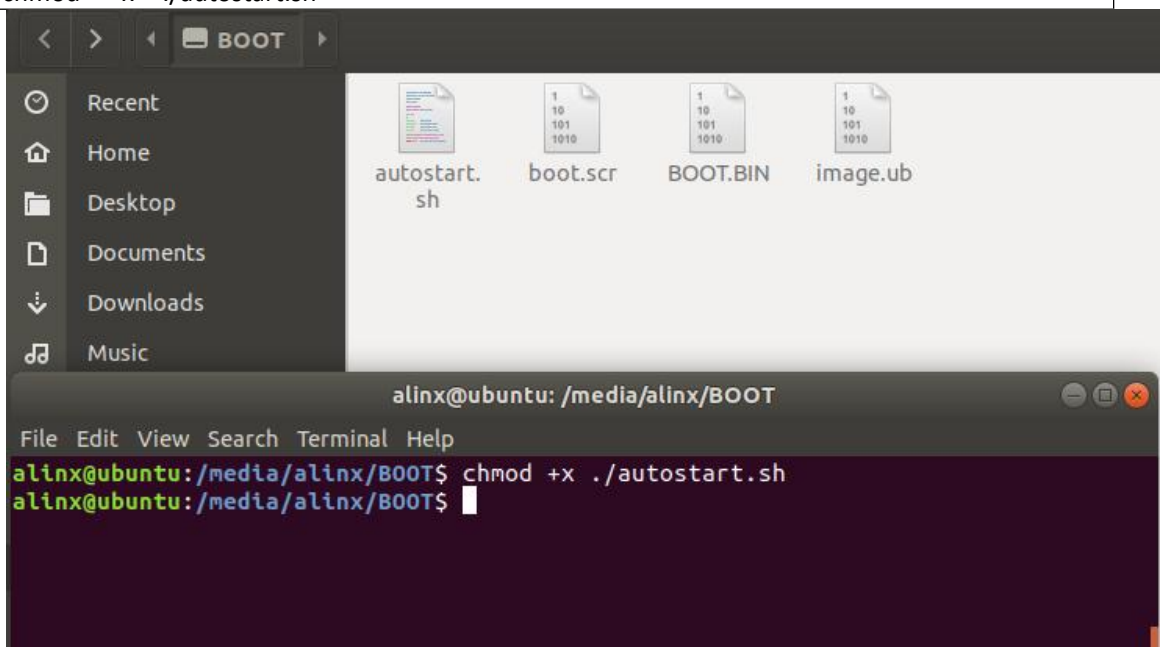
```
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$ petalinux-package  --boot  --u-bo
ot  --fsbl  --force
INFO: sourcing build tools
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxPsBase/petalinux/images/linux
/zynqmp_fsbl.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxPsBase/petalinux/images/linux
/pmufw.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxPsBase/petalinux/images/linux
/bl31.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxPsBase/petalinux/images/linux
/system.dtb"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxPsBase/petalinux/images/linux
/u-boot.elf"
INFO: Generating zynqmp binary package BOOT.BIN...


****** Xilinx Bootgen v2020.1
  **** Build date : May 26 2020-14:07:15
    ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.


[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$
```

6. 把 BOOT.bin、boot.scr 及 image.ub 三个文件拷贝到格式化成 FAT 格式的 SD 卡中。同时在 SD 卡中添加一个“autostart.sh”文件，编辑内容如下：

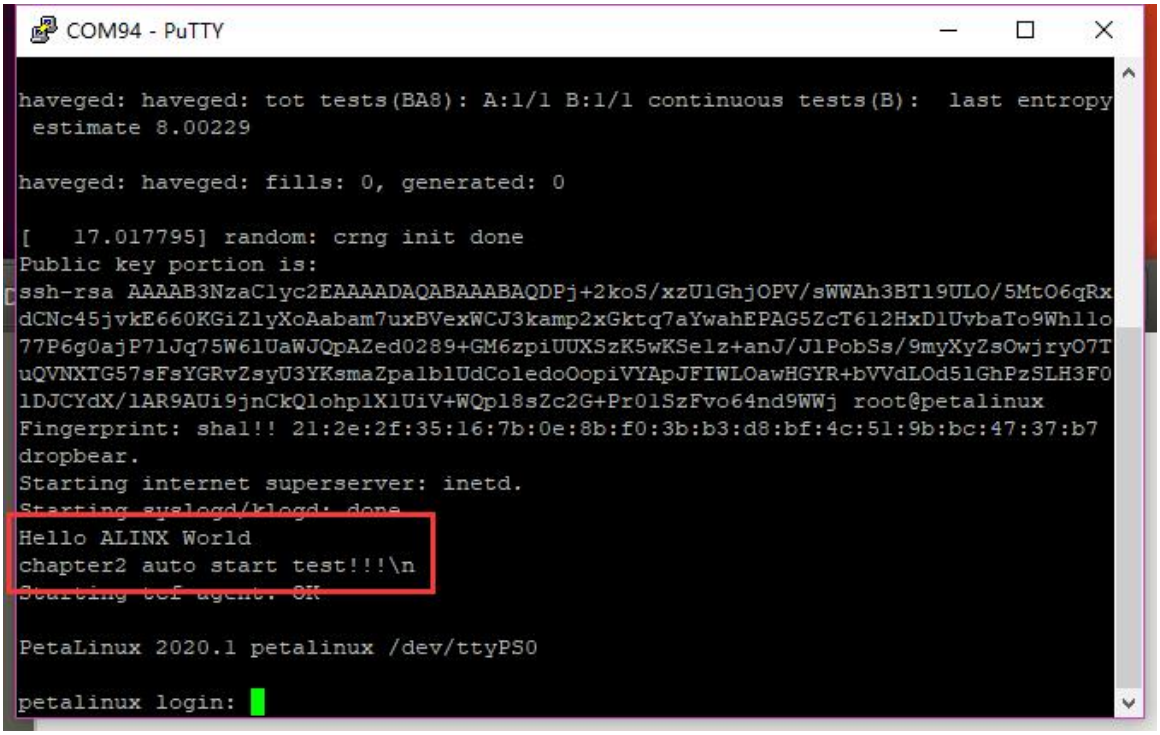
```
#!/bin/sh
echo   "chapter2 auto start test!!!\n"
```

然后在 SD 卡文件夹中打开一个终端，使用下面的命令修改“autostart.sh”文件的权限：

```
chmod   +x   ./autostart.sh
```



7. 弹出 SD 卡，连接好串口，使用这张 SD 卡启动开发板。



可以看到系统启动后先是打印了“Hello ALINX World”，然后输出了我们在 SD 卡中添加的脚本中打印的内容。我们可以修改 SD 卡添加的脚本，添加自定义内容，这些自定义内容就不会被系统重启还原了。

第三章 程序 hello world

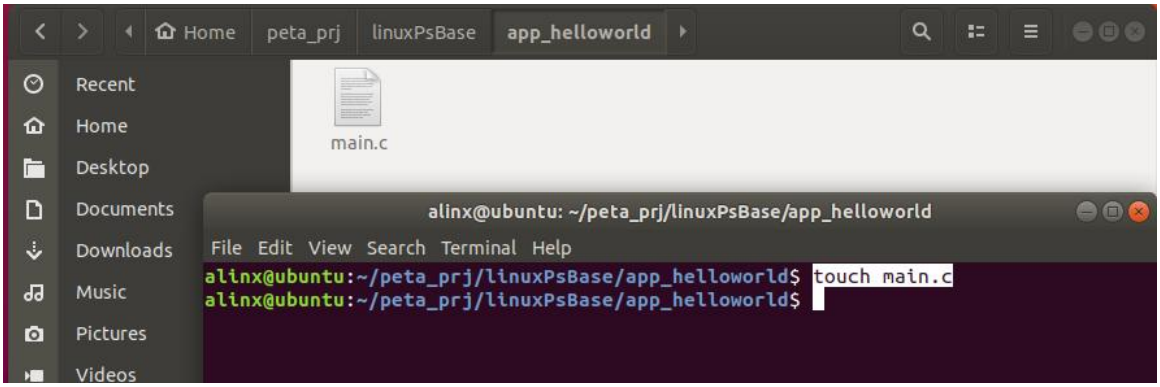
3.1 程序代码

1. 在“/home/alinx/peta_prj/linuxPsBase”路径中创建一个目录“app_helloworld”来存放本章的应用程序。如下图：

2. 在 app_helloworld 目录中打开中端，使用下面的命令

```
touch    main.c
```

创建一个 main.c 文档。如下图：



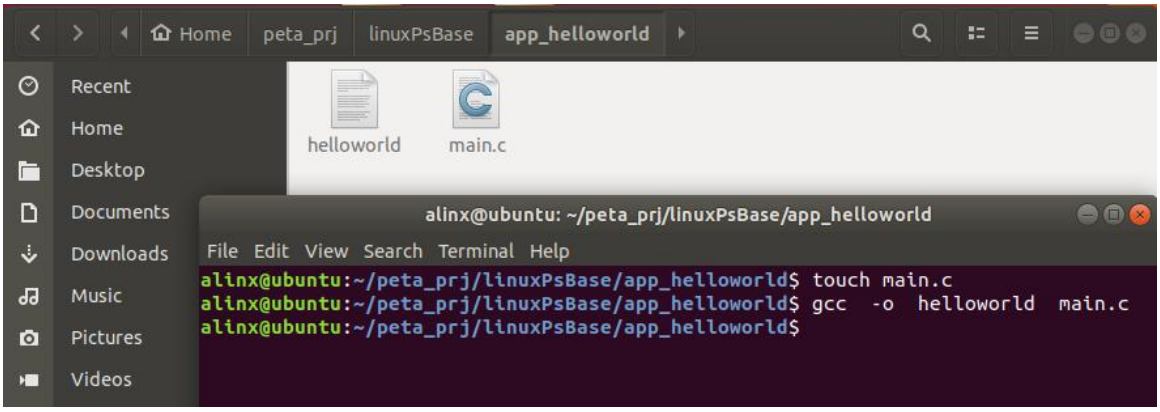
3. 编辑 main.c 文档，输入如下内容并保存。

```
#include    <stdio.h>

int    main(void)
{
       printf("hello world\r\n");
       return    0;
}
```

3.2 编译主机程序

7. 在终端中使用下面的命令编译程序，“/home/alinx/peta_prj/linuxPsBase/app_helloworld”路径中会生成一个名为“helloworld”的可执行文件。

```
gcc    -o    helloworld    main.c
```



8. 在终端中输入下面的命令，直接运行程序，可以看到打印信息“hello world”，说明程序已经在电脑上正确执行。

```
./helloworld
```

3.3 交叉编译 arm 程序

刚才我们使用“gcc”编译器来编译程序在 pc 机上运行。但程序最终是需要运行在板卡上的，我们需要使用 arm 的交叉编译器编译程序才能在板卡上运行。这里，我们先使用 Vivado 或 Vitis 安装后附带的编译器。

1. 运行下面命令，将 Vivado 的 arm 编译器加入环境变量。

```
source /tools/Xilinx/Vivado/2020.1/settings64.sh
```

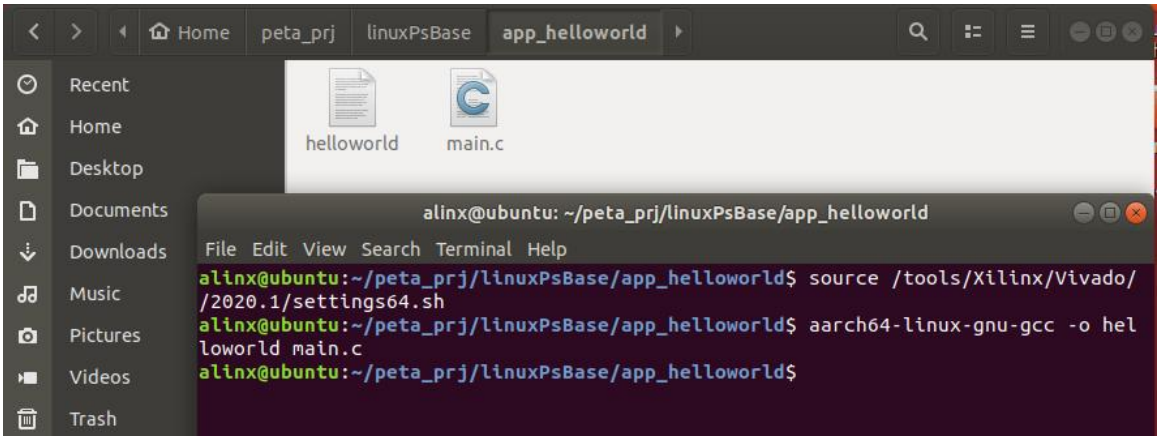
若使用 Vitis 的 arm 编译器则使用下面的命令设置环境变量。

```
source /tools/Xilinx/Vitis/2020.1/settings64.sh
```

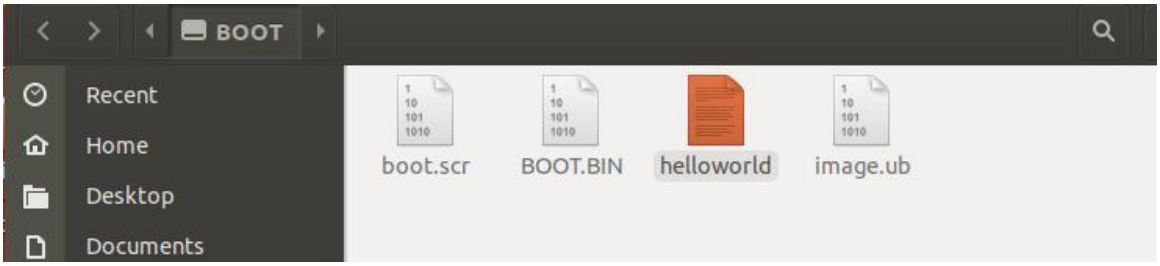
注意：环境变量仅在当前终端有效，后面命令要在当前终端完成。

2. 使用下面的命令交叉编译程序，生成板卡上可执行程序，名字仍然是“helloworld”。

```
aarch64-linux-gnu-gcc -o helloworld main.c
```



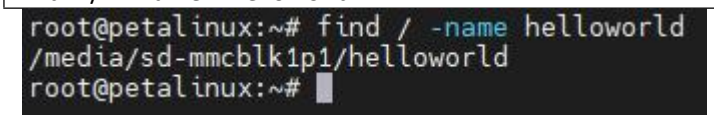
3. 使用第一章中 petalinux 定制的 linux 系统启动开发板，并将可执行程序 helloworld 一同拷贝到 SD 卡中。



4. 连接好串口，将板卡设为从 SD 卡起动然后上电。
5. 输入账号密码，进入 linux 系统。

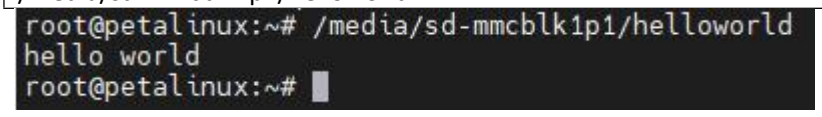
6. 使用下面的命令搜索一下“helloworld”程序所在的路径。

```
find / -name helloworld
```



7. 运行程序，可以看到打印信息“hello world”，说明程序已经在板卡上正确执行。

```
/media/sd-mmcblk1p1/helloworld
```



第四章 NFS 服务

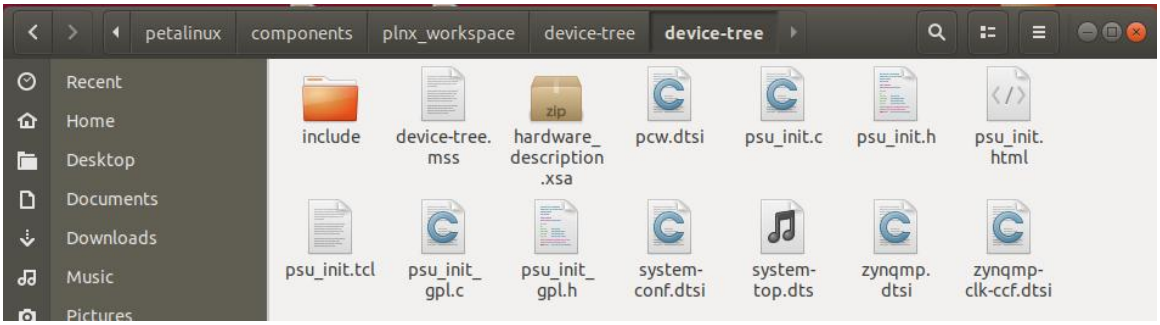
NFS 环境搭建参考教程“Xilinx 开发环境安装教程”。

在各种网络环境下的挂载 NFS 共享路径的方法参考视频教程“NFS 服务”。

第五章 gpio 控制 led

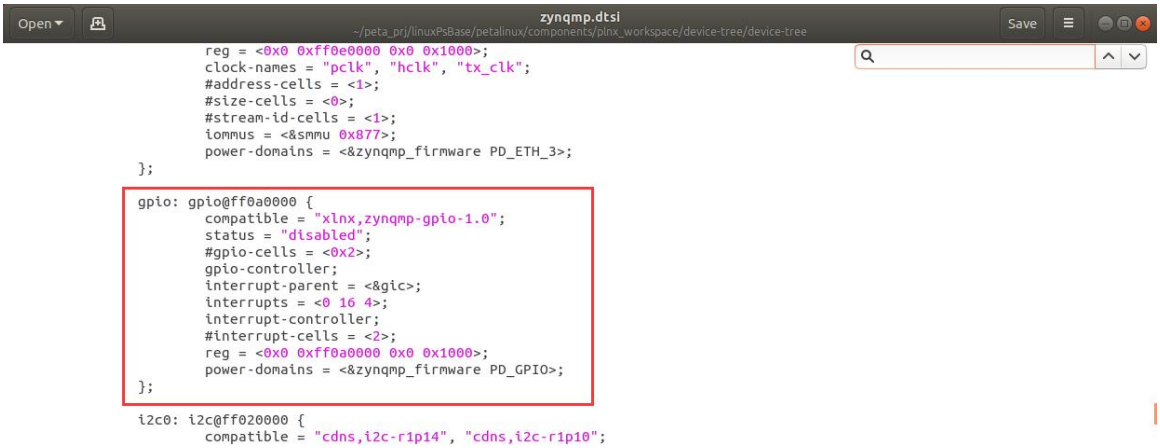
5.1 petalinux 自动生成的设备树

在 linux 内核代码里，已经包含了很多驱动，我们需要将这些驱动，与我们板卡上的硬件关联起来，才能在系统下生成我们可以使用的设备。我们可以使用设备树来关联我们的设备与驱动。使用 petalinux 工具定制 linux 系统的话，在“petalinux-bulid”编译之后 petalinux 会自动生成一些设备树，他们在 petalinux 工程路径下的“”路径中，如下图：



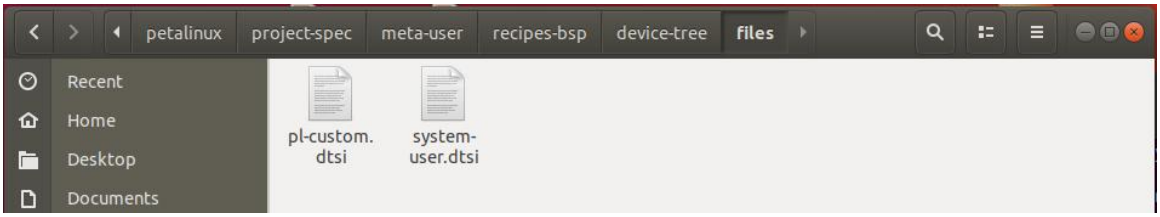
注意：这里的文件都是修改无效的。

其中“zynqmp.dtsi”是 ps 端的设备树，“pl.dtsi”是 pl 端的设备树(因为这里使用的 xsa 硬件描述文件中没有 bitstream，也就是 vivado 工程中没有设置 pl 端的内容，所以没有生成“pl.dtsi”文件)。打开 zynqmp.dtsi 文件，在这个文件中可以找到 ps 端的 gpio 设备树节点，如下图：



5.2 petalinux 用户可修改的设备树

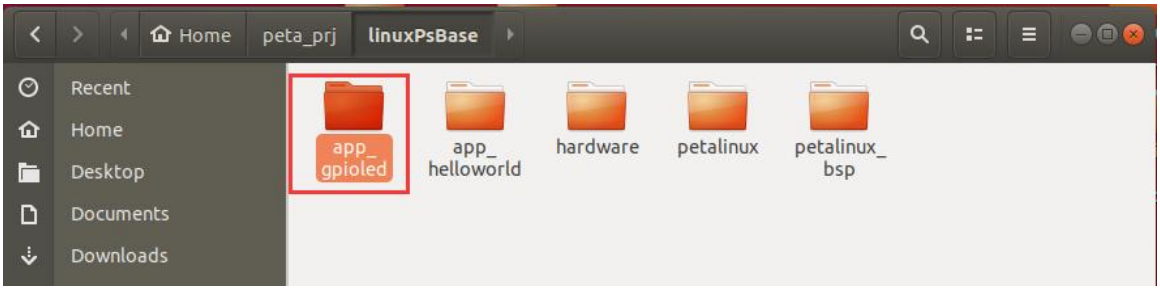
如果需要修改设备树，可以编辑 petalinux 工程路径中“project-spec/meta-user/recipes-bsp/device-tree/files”路径中的“system-user.dtsi”或者“pl-custom.dtsi”这两个文件。



这一章我们使用 gpio，因为 ps 端的 gpio 设备树节点已经由 petalinux 正确的生成，所以我们不需要在这里作任何修改。

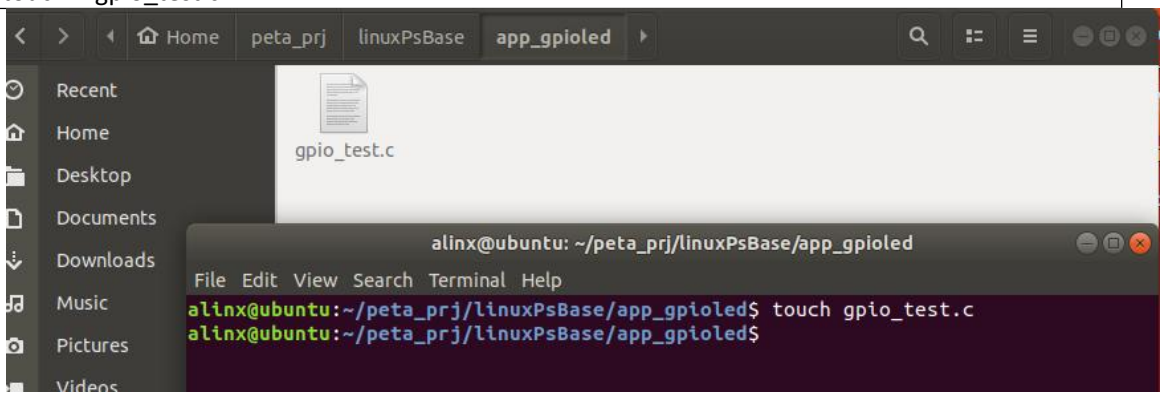
5.3 工程创建

1. 在“/home/alinx/peta_prj/linuxPsBase”路径中创建一个目录“app_gpioled”来存放本章的应用程序。如下图：



2. 在“/home/alinx/peta_prj/linuxPsBase/app_gpioled”路径中打开终端，使用下面的命令创建一个名为“gpio_test.c”的文件：

```
touch   gpio_test.c
```



编辑这个文件，添加下面的代码到文件中：

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<fcntl.h>
#include<stdbool.h>

#define PS_LED_BASE     338
#define PS_LED_INDEX        40
```

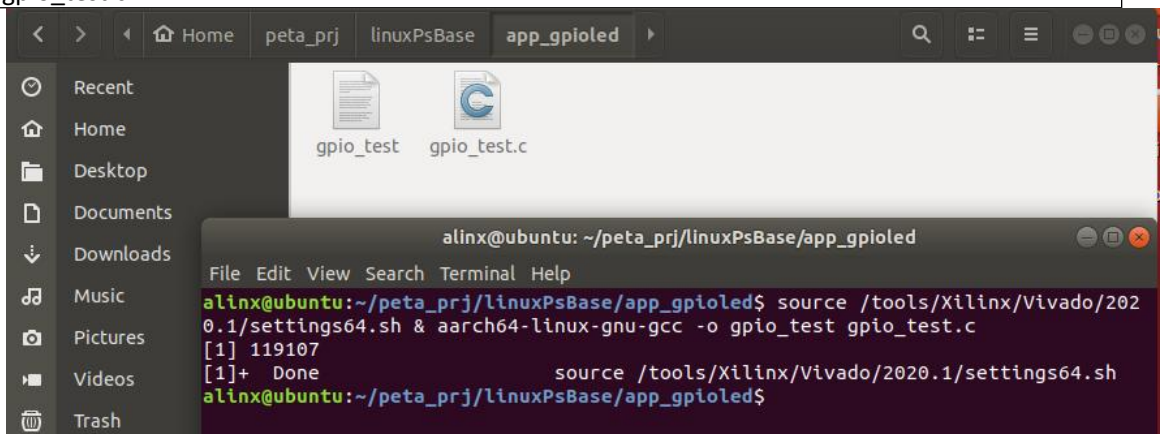
```
#define PS_LED_ADDR     (PS_LED_BASE+PS_LED_INDEX)

void led_ctl_init(void)
{
    int lGpioFd;
    char lCache[100];

    lGpioFd = open("/sys/class/gpio/export", O_WRONLY);
    if(lGpioFd < 0)
    {
        printf("open export fail\n");
        return;
    }
    int len = sprintf(lCache, "%d", PS_LED_ADDR);
    write(lGpioFd, lCache, len+1);
    close(lGpioFd);

    sprintf(lCache, "/sys/class/gpio/gpio%d/direction", PS_LED_ADDR);
    lGpioFd = open(lCache, O_RDWR);
    if (lGpioFd < 0)
    {
    printf("open direction fail\n");
        return;
    }
    write(lGpioFd, "out", 4);
    close(lGpioFd);
}

void led_ctl_on(bool isON)
{
    int lGpioFd;
    char lCache[100];

    sprintf(lCache, "/sys/class/gpio/gpio%d/value", PS_LED_ADDR);
    lGpioFd = open(lCache, O_RDWR);
    if (lGpioFd < 0)
    {
    printf("open value fail\n");
        return;
    }
    if(isON)
    {
        write(lGpioFd, "1", 2);
    }
    else
    {
        write(lGpioFd, "0", 2);
    }
    close(lGpioFd);
}

int main()
{
    led_ctl_init();
    printf("================start led ctrl=============\n");

    while(1)
    {
        led_ctl_on(true);
```

```
        printf("on\r\n");
        usleep(500000);

        led_ctl_on(false);
        printf("off\r\n");
        usleep(500000);
    }

    return 0;
}
```

代码中“led_ctl_init()”函数主要功能为导出 gpio 管脚，并将其初始化为输出管脚。“led_ctl_on()”函数控制管脚的高低电平，实现 led 灯的亮灭。

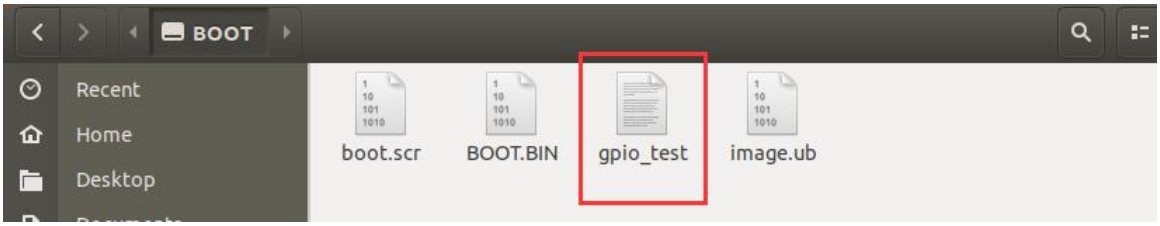
3. 用下面的命令设置环境变量，并编译代码：

```
source /tools/Xilinx/Vivado/2020.1/settings64.sh & aarch64-linux-gnu-gcc -o gpio_test gpio_test.c
```



5.4 程序运行

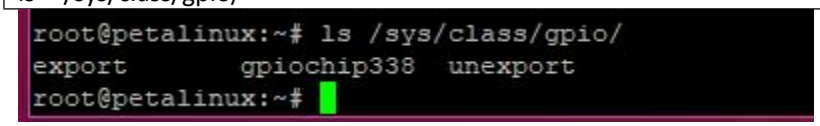
1. 使用第一章中 petalinux 定制的 linux 系统启动开发板，并将可执行程序 gpio_test 一同拷贝到 SD 卡中。



2. 用这个 SD 卡启动开发板，输入用户名密码进入系统。

3. 查看 gpio 相关的设备节点如下：

```
ls /sys/class/gpio/
```



这里的 gpiochip338 就是 ps 端的 gpio 控制器，338 也就是 gpio 编号的起始编号，这个起始值是由 vivado 工程决定的。

4. 使用下面的命令可以查看管脚标签信息以判断 gpio 控制器控制的 gpio 组。

```
cat   /sys/class/gpio/gpiochip338/label
```

```
root@petalinux:~# cat /sys/class/gpio/gpiochip338/label
zynqmp_gpio
root@petalinux:~#
```

5. 通过原理图可以看到（5EV_E 的）PS_LED 连接的管脚为 MIO40。

6. MIO40 也就是说这个管脚在这个 gpio 组中的偏移值 40，那么我们需要操作的编号就是 338+40，也就对应代码中的宏定义：

```
#define PS_LED_BASE     338
#define PS_LED_INDEX    40
#define PS_LED_ADDR     (PS_LED_BASE+PS_LED_INDEX)
```

7. 使用下面的命令搜索一下“gpio_test”程序所在的路径。

```
find   /   -name   gpio_test
```

```
root@petalinux:~# find / -name gpio_test
/media/sd-mmcblk1p1/gpio_test
```

8. 运行“gpio_test”程序，终端开始打印信息，PS_LED 也会开始闪烁。

```
root@petalinux:~# /media/sd-mmcblk1p1/gpio_test
=================start led ctrl==============
on
off
on
off
on
off
on
off
on
off
```

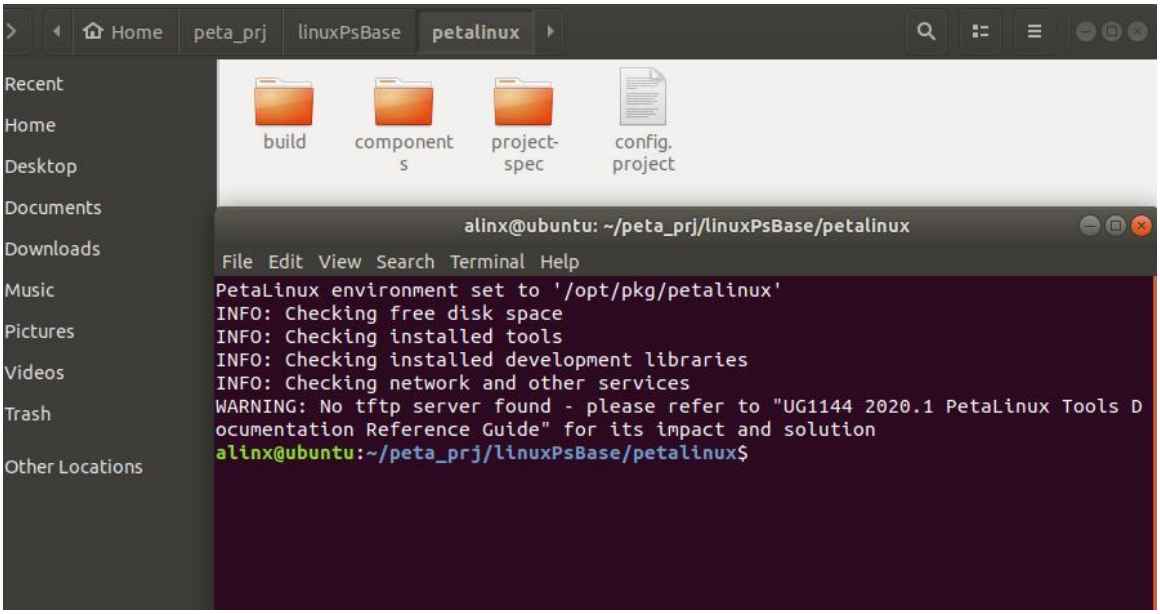
第六章 QT 和 OPENCV 交叉编译开发环境

Qt 是跨平台 C++图形用户界面应用程序开发框架，可以简单的将它理解为用于图形开发的库。如果我们使用 Qt 库为板卡开发带界面的应用程序，则需要交叉编译 Qt 库。OpenCV 是视频处理中有名的库，很多应用程序需要使用 OpenCV 库。

以往 QT 和 OpenCV 库都需要自己下载代码编译，非常麻烦。本章讲解如何使用 Petalinux 配置 OpenCV 和 QT 的开发环境，这也体现出 Petalinux 开发环境的优势，只需简单的配置就可以解决很复杂的问题。

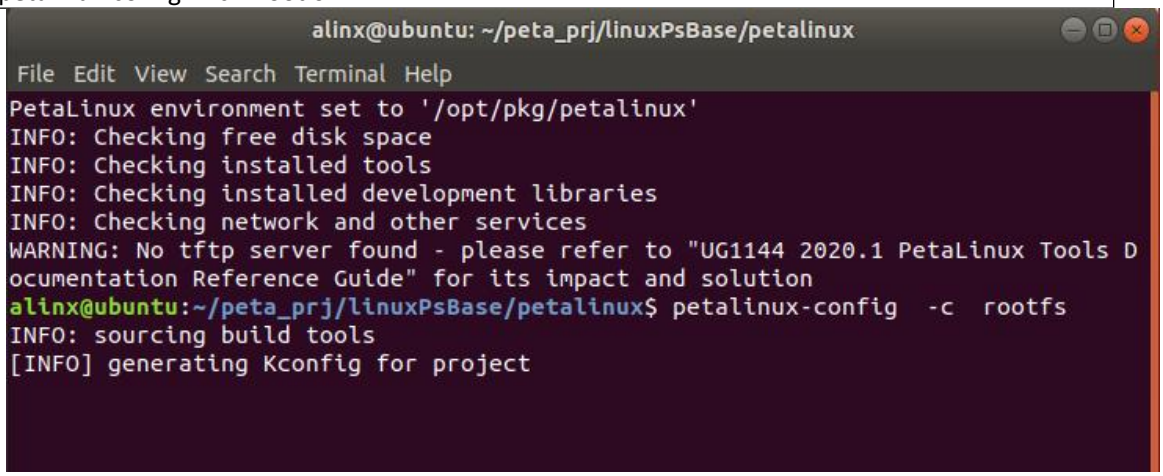
6.1 配置根文件系统

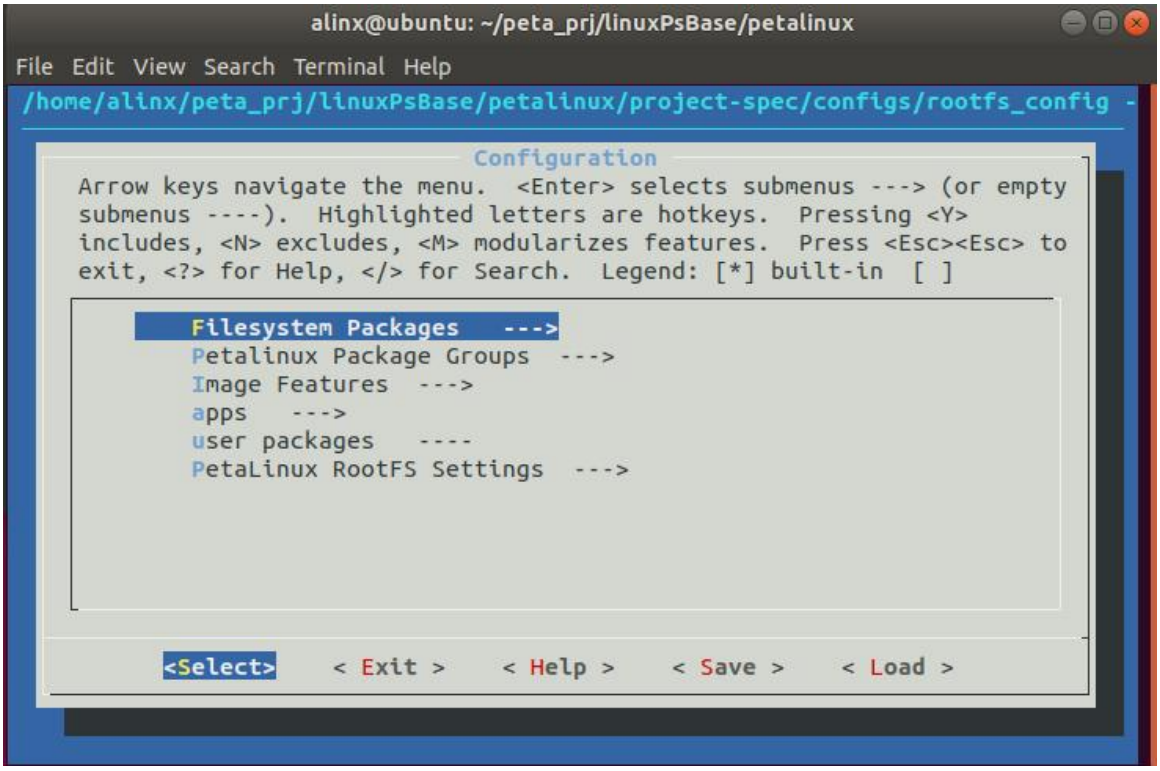
1. 在本教程第一章或第二章制作或使用的 petalinux 工程中打开一个终端，设置好 petalinux 环境变量。



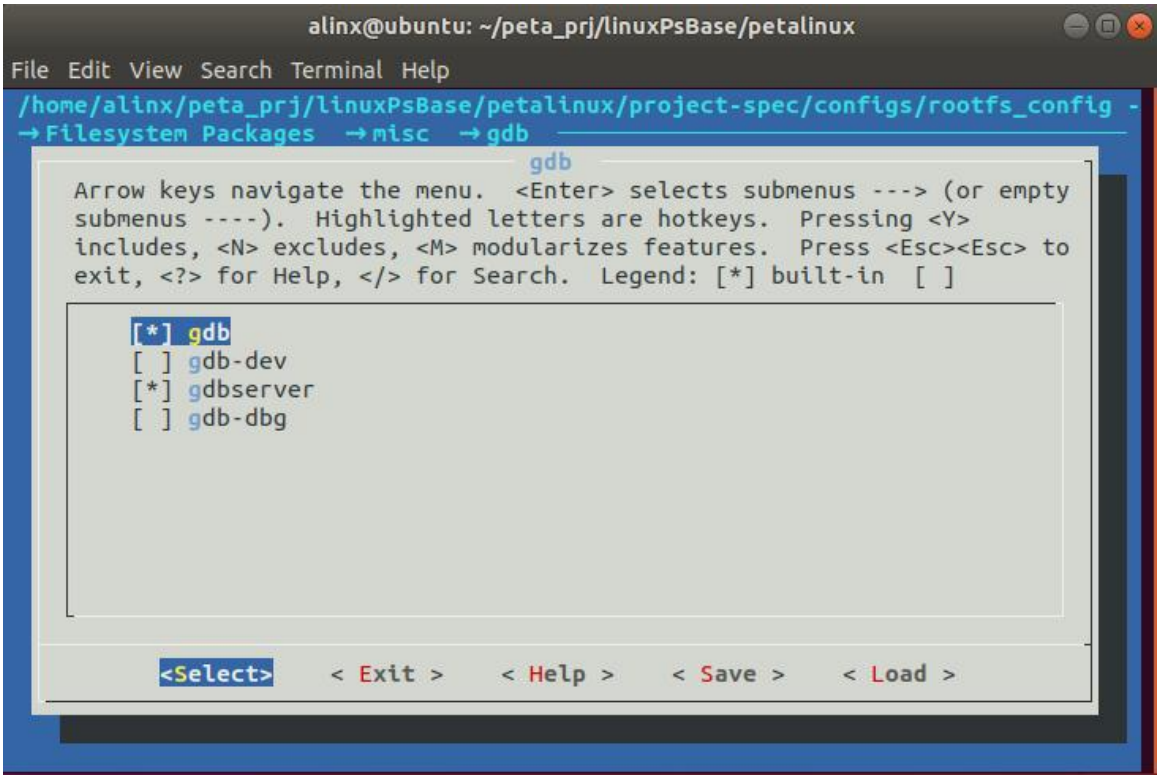
2. 运行 petalinux 命令配置 rootfs

```
petalinux-config   -c   rootfs
```

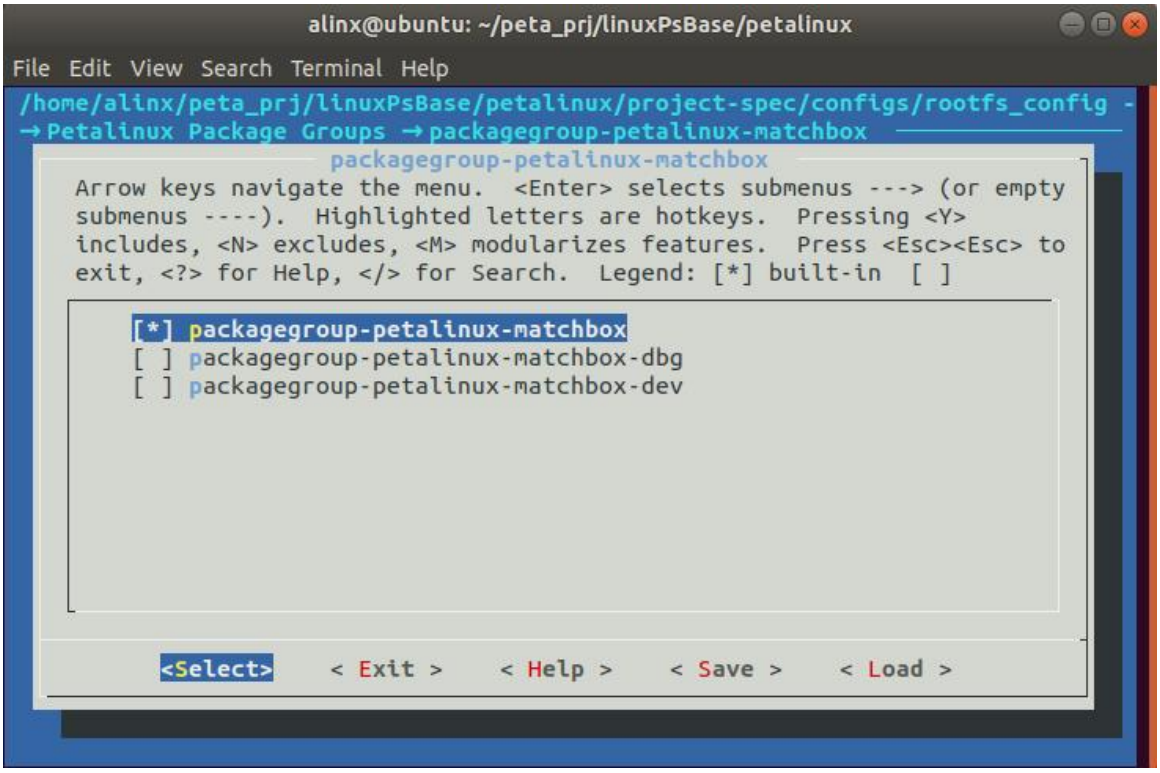




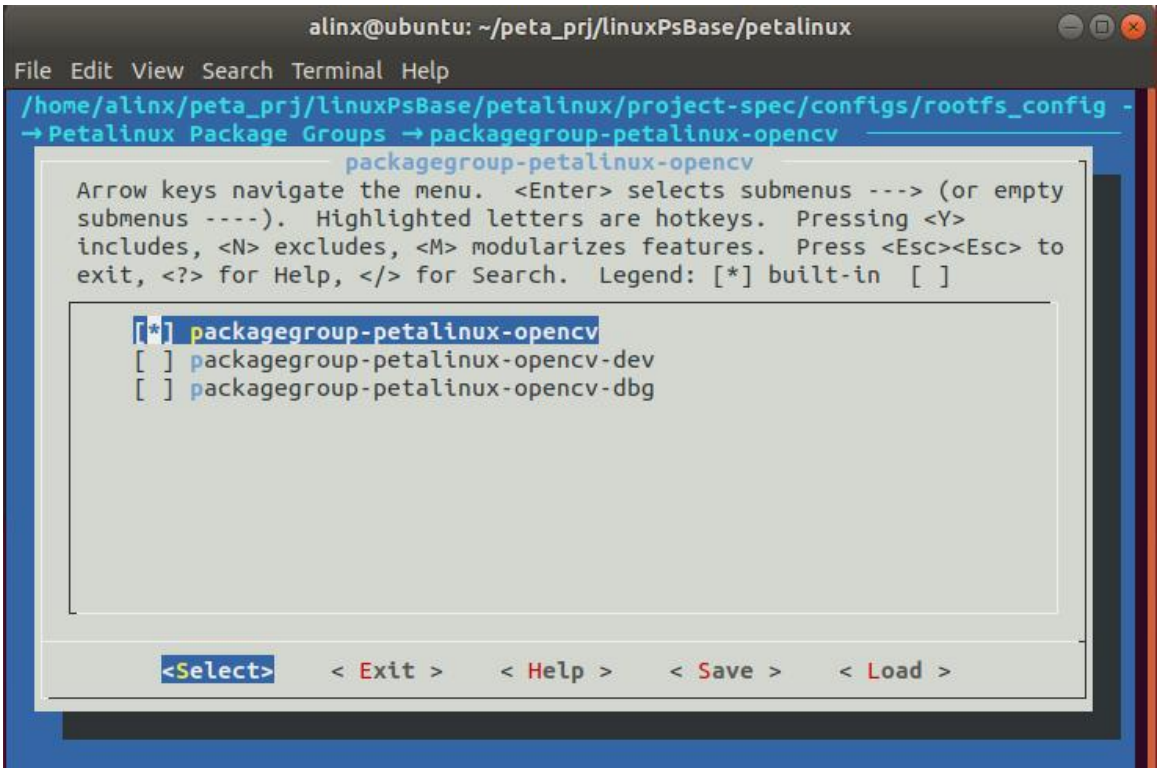
3. 勾选 Filesystem Packages → misc → gdb 和 gdbserver，gdb 可以用来调试应用程序，gdbserver 可以通过网络远程调试应用程序。



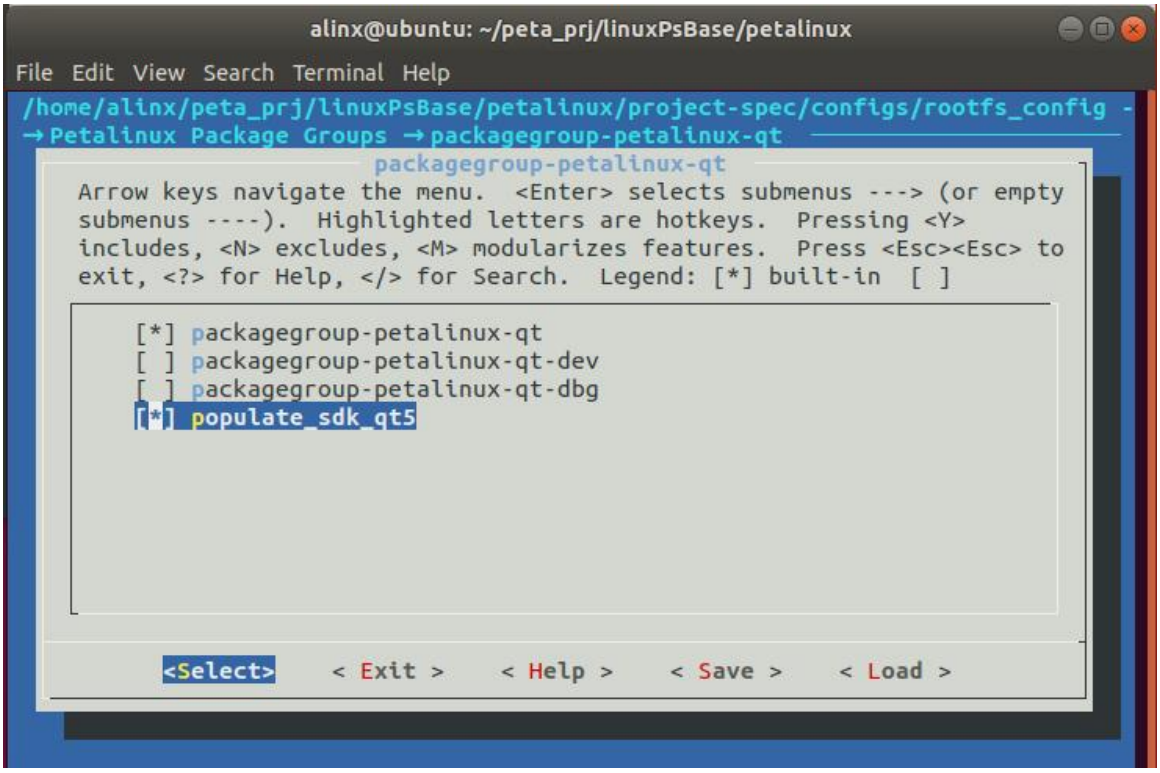
4. 勾选 Petalinux Package Groups → packagegroup-petalinux-matchbox，配置 matchbox 桌面，matchbox 是一种简易的 Linux 桌面管理系统。



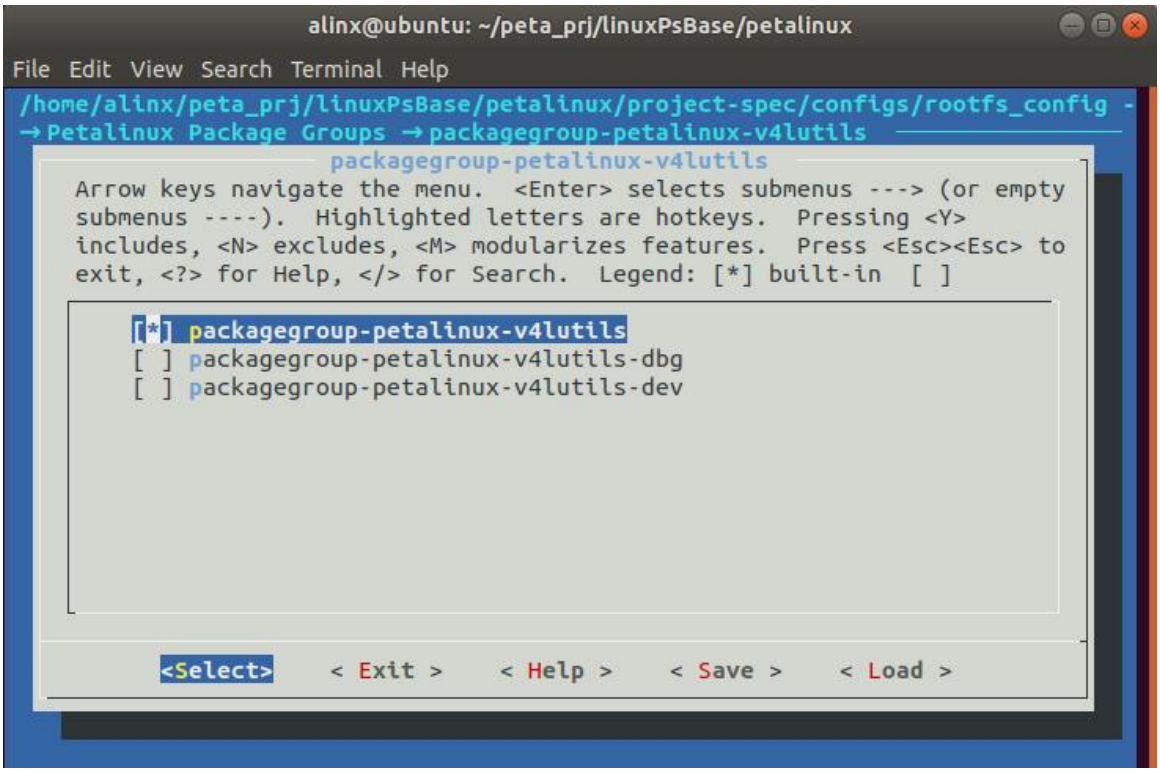
5. 勾选 Petalinux Package Groups → packagegroup-petalinux-opencv，配置 opencv 库，后面的 dev 和 dbg 版本这里没有勾选，不影响运行，勾选以后文件系统体积会特别大。



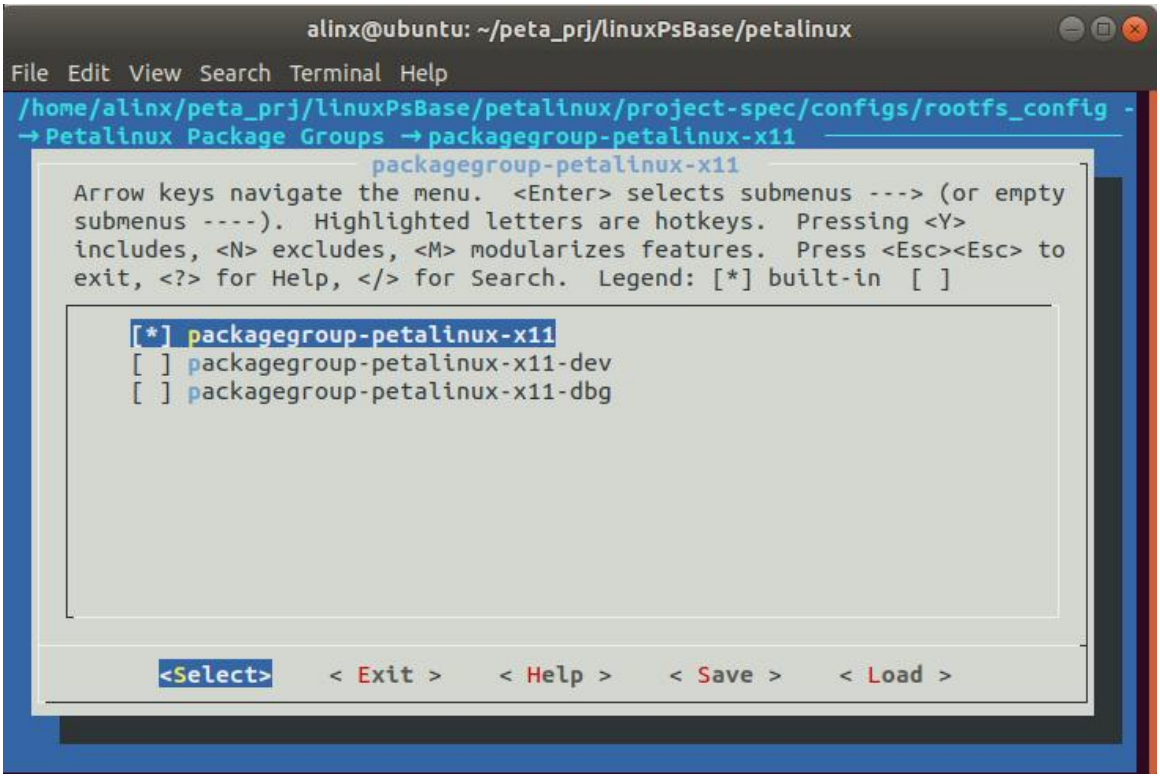
6. 勾选 Petalinux Package Groups → packagegroup-petalinux-qt，配置 qt 库



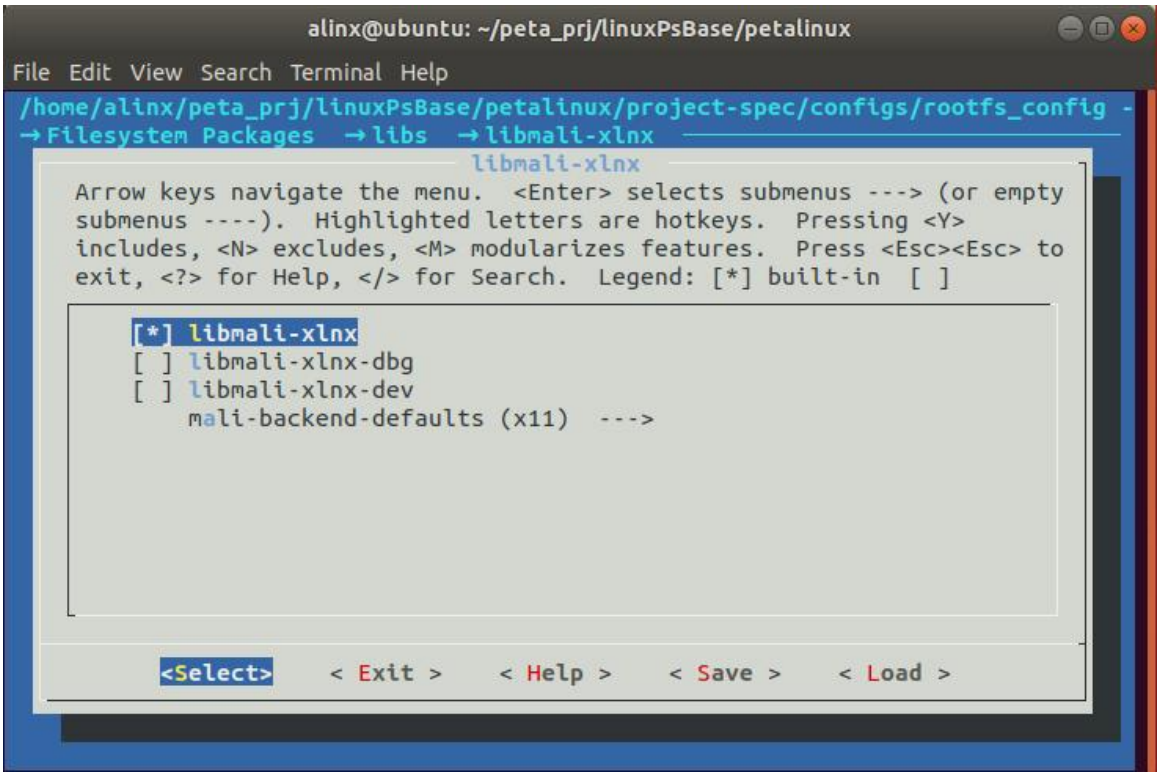
7. 勾选 Petalinux Package Groups → packagegroup-petalinux-v4lutils，配置 v4l 工具，这个主要是一个 Linux 下的视频配置小工具，视频开发可能会用到。



8. 勾选 Petalinux Package Groups → packagegroup-petalinux-x11 配置 x11 库



9. 勾选 Filesystem Packages → libs → libmali-xlnx 配置 GPU 库



10. 选择< Save >保存配置，然后选择< Exit >选项退出配置界面，编译 petalinux 工程。
petalinux-build
如果出现了这个错误：

就输入下列命令：

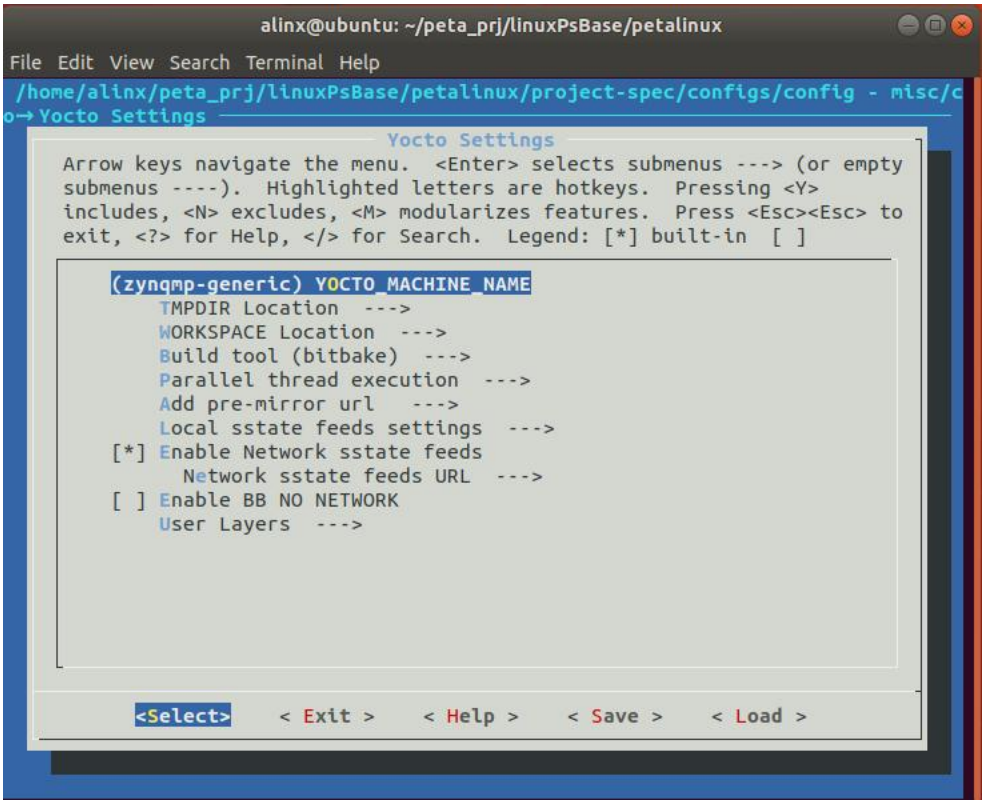
```
alinx@ubuntu:~/peta_prj/linuxPsBase/petalinux$ petalinux-build -x mrproper -f
```

然后再进行 build 编译。

11. 打包 BOOT.bin。

```
petalinux-package --boot --u-boot --fsbl --force
```

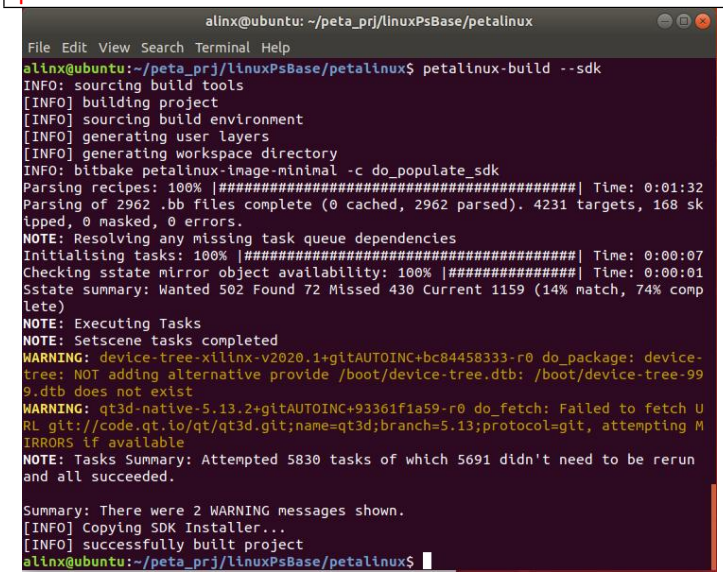
12. 构建 SDK 之前需要先设置允许 petalinux 工程联网，同时也要保证虚拟机能够连接到外网，使用“petalinux-config”命令，进入到选项“”界面中，配置成如下图所示的状态：



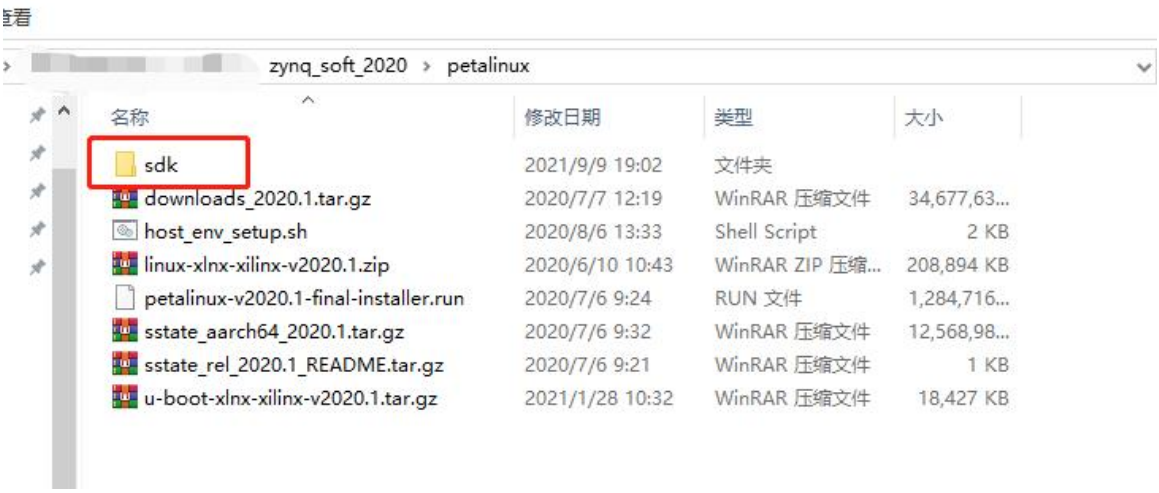
这个“允许在线编译”的设置在构建完 SDK 后就可以恢复之前离线编译的状态了。

13. 使用下面的命令构建 SDK。

```
petalinux-build --sdk
```

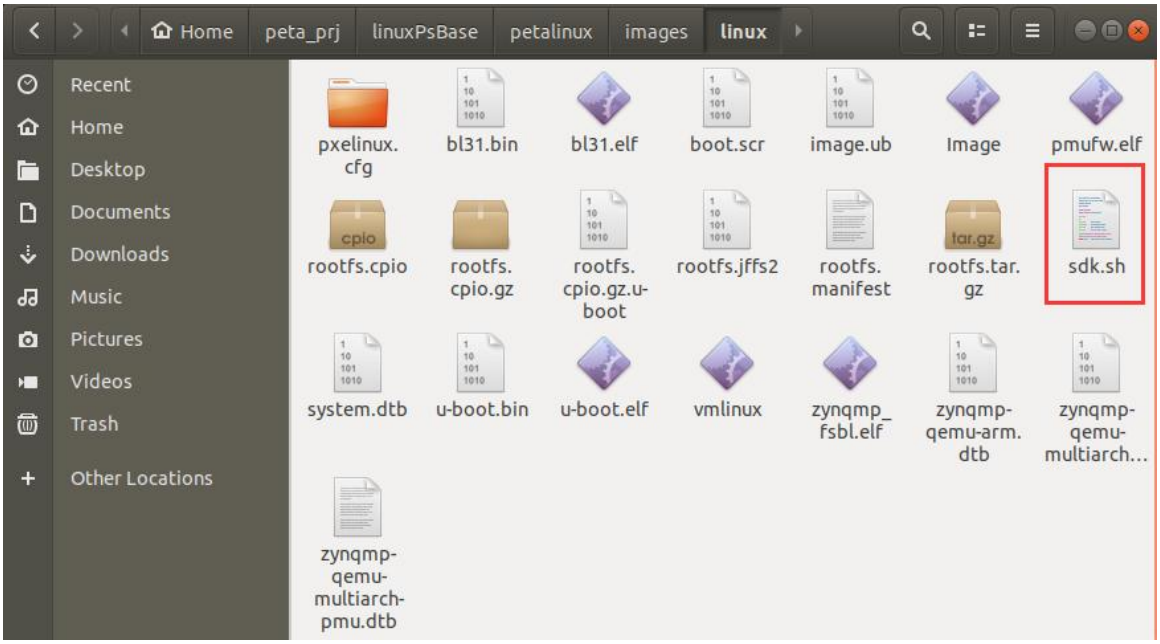


如果网络状态不佳，也可以使用我们编译好的 sdk.sh。路径如下图。



6.2 安装 sdk

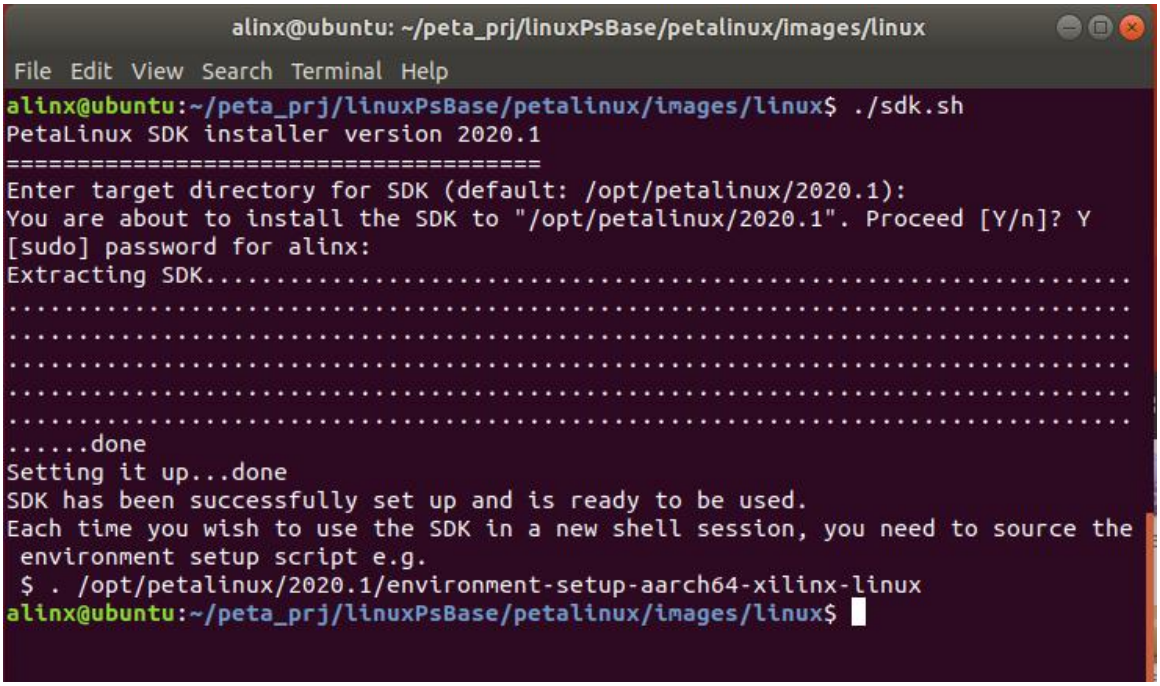
1. 在 petalinux 工程路径的“images/linux”路径中可以找到生成的 sdk.sh 文件，这个文件比较大，里面有交叉编译器、根文件系统、各种库和头文件，非常有用，如果只做 Linux 应用开发，只需要一个 sdk.sh 文件即可，可以脱离 Petalinux 和 Vitis。



2. 运行安装 sdk，提示我们默认安装在/opt/petalinux/2020.1，按回车，然后输入 Y，安装到默认目录，安装完成后，根据提示使用命令

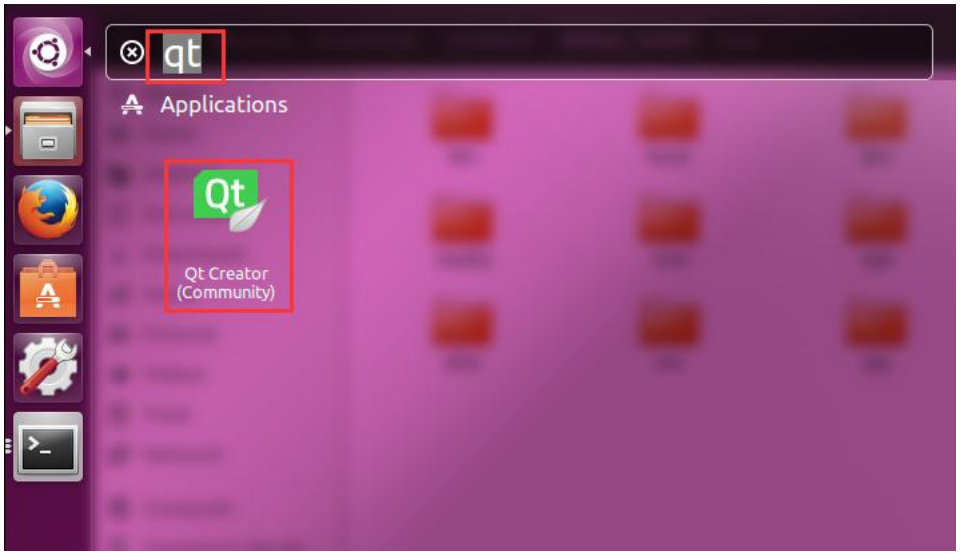
```
source    /opt/petalinux/2020.1/environment-setup-aarch64-xilinx-linux
```

就可以使用 SDK 了。

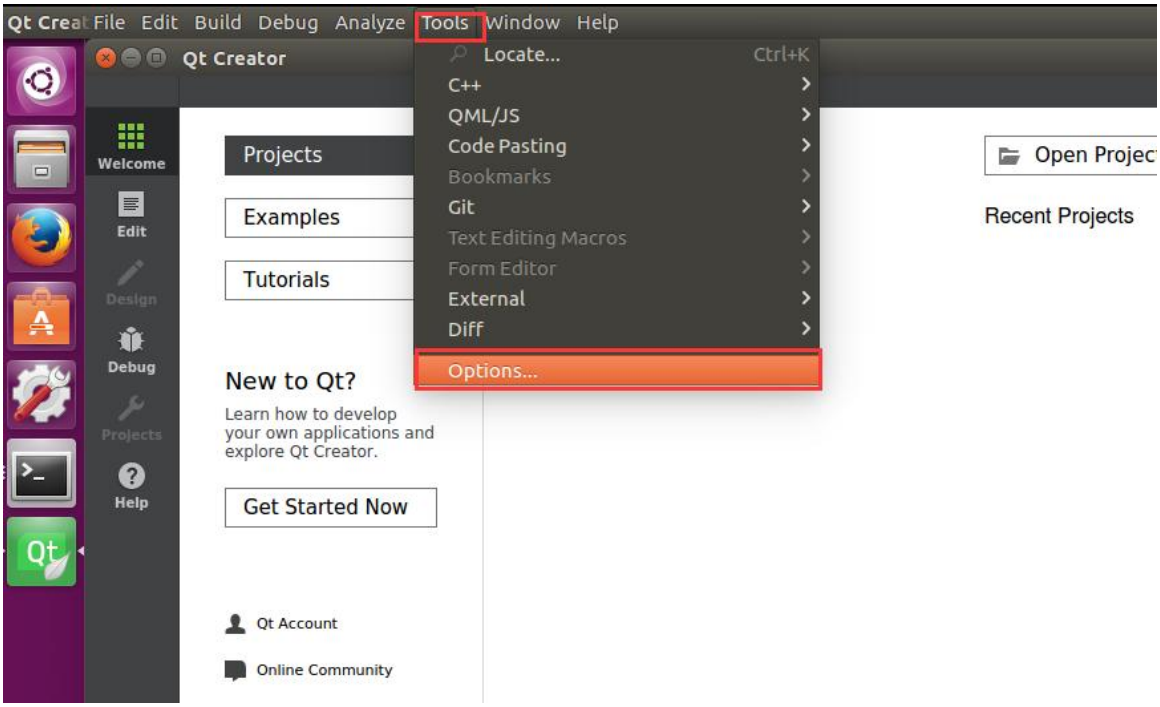


6.3 QT Creator 交叉编译环境设置

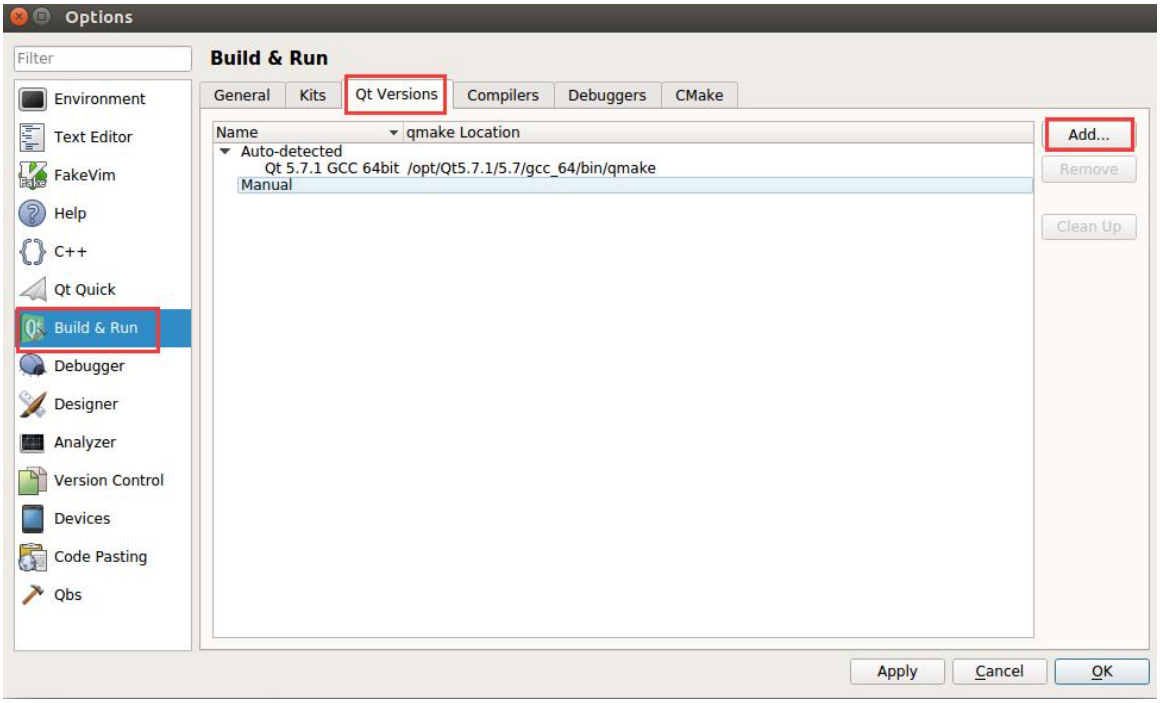
1. 运行 QT Creator，找不到应用程序快捷方式，可以搜索一下。



2. 点击“Tools -> Options”。

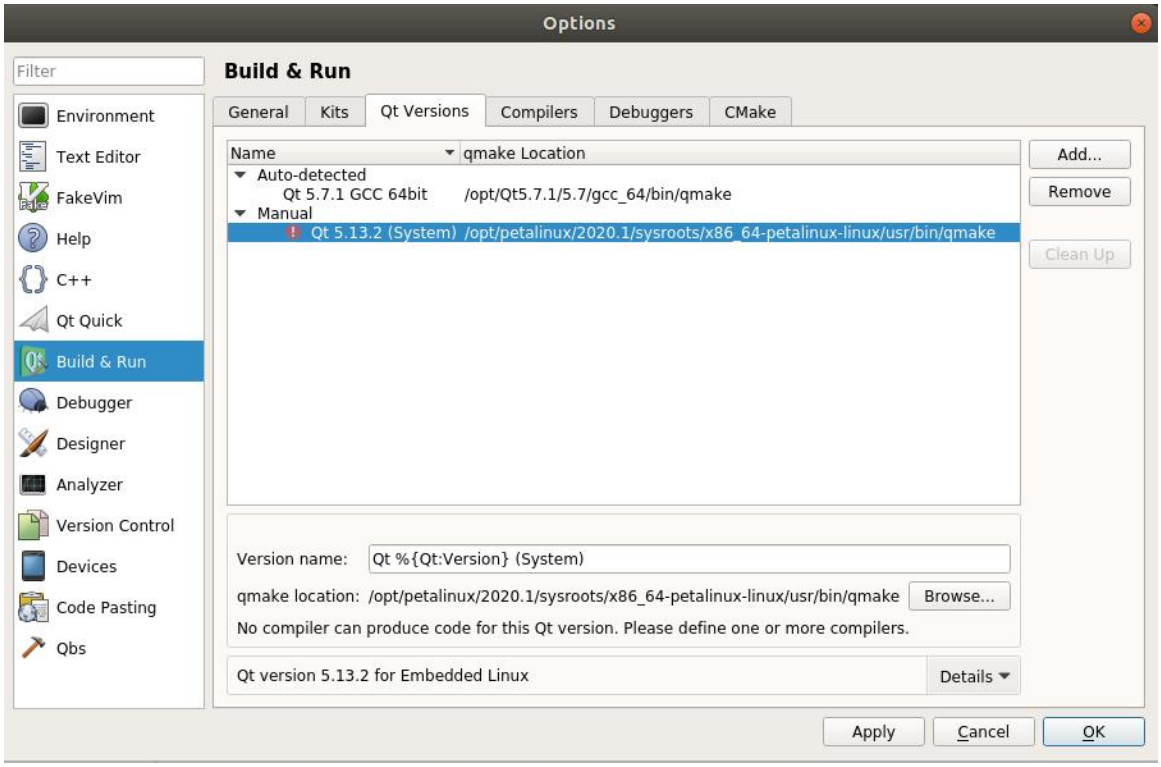


3. 在“Build & Run”选项的“Qt Versions”页点击“Add..”。

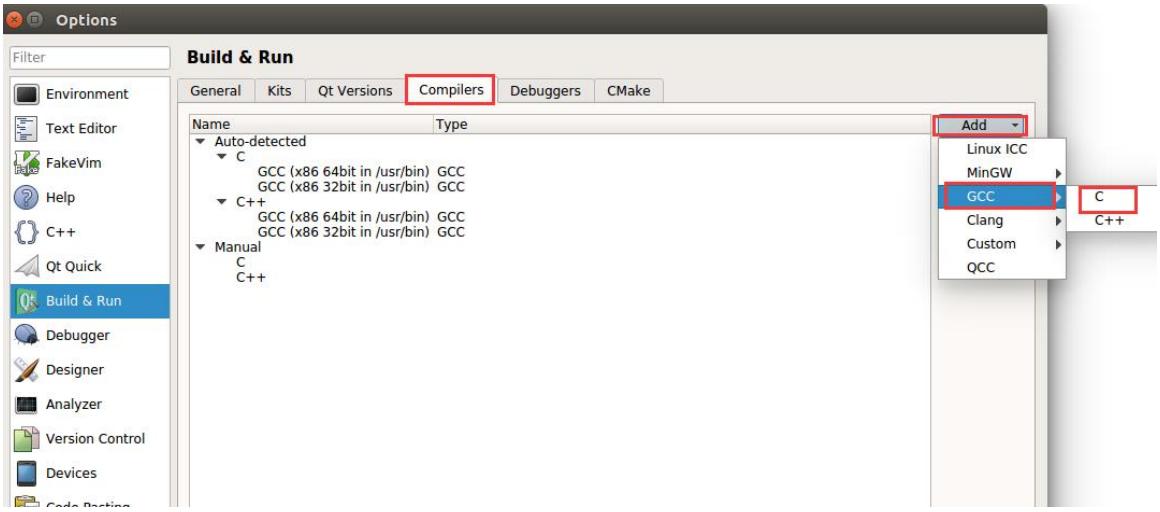


4. 选择前面 sdk 编译生成的 qmake 文件，路径是“/opt/petalinux/2020.1/sysroots/x86_64-petalinux-linux/usr/bin/qmake”。

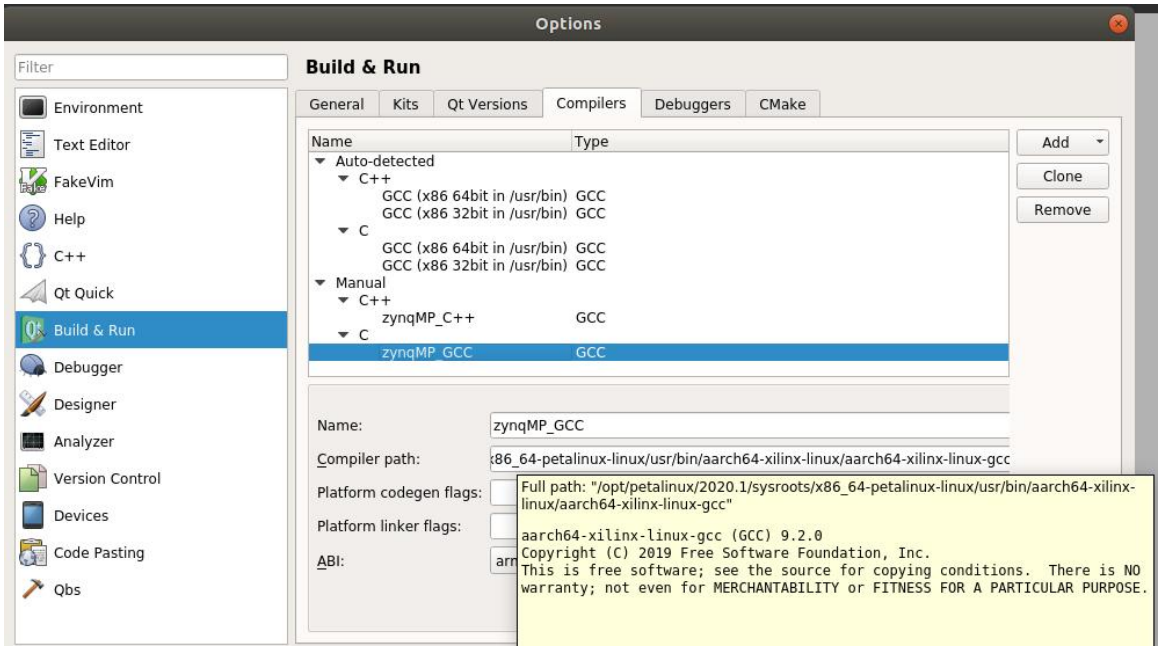
5. 添加 qmake 以后，保持默认 name，不修改。



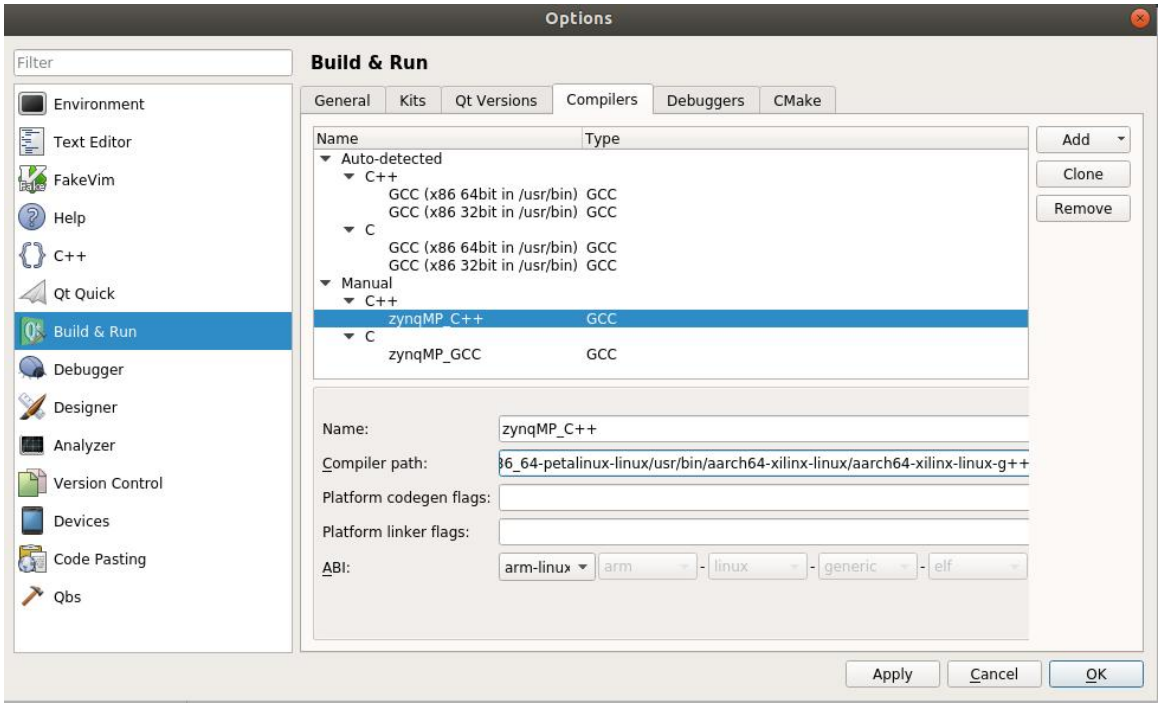
6. 在“Compilers”页面，点击“Add..”，选择“GCC -> C”。



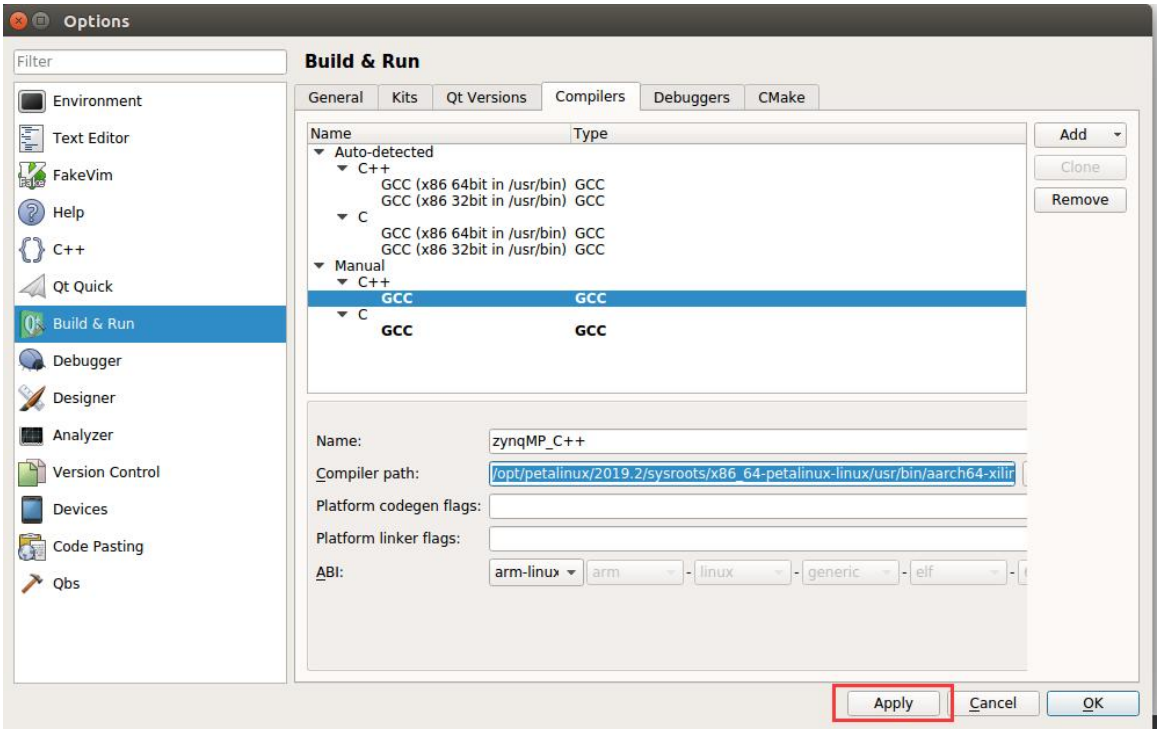
7. “Name”修改为“zynqMP_GCC”，“Compiler path”选择“/opt/petalinux/2020.1/sysroots/x86_64-petalinux-linux/usr/bin/aarch64-xilinx-linux/aarch64-xilinx-linux-gcc”。



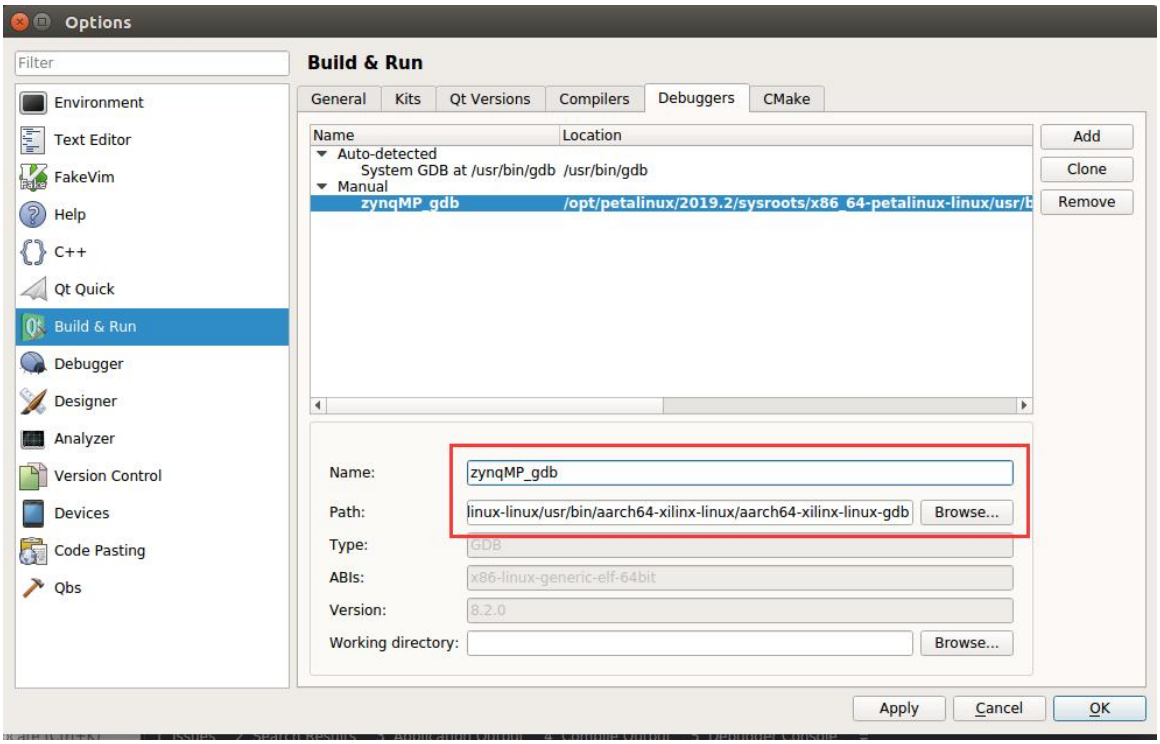
8. 在“Compilers”页面，点击“Add..”，选择“GCC -> C++”，“Name”修改为“zynqMP_C++”，路径选择“/opt/petalinux/2020.1/sysroots/x86_64-petalinux-linux/usr/bin/aarch64-xilinx-linux/aarch64-xilinx-linux-g++”。



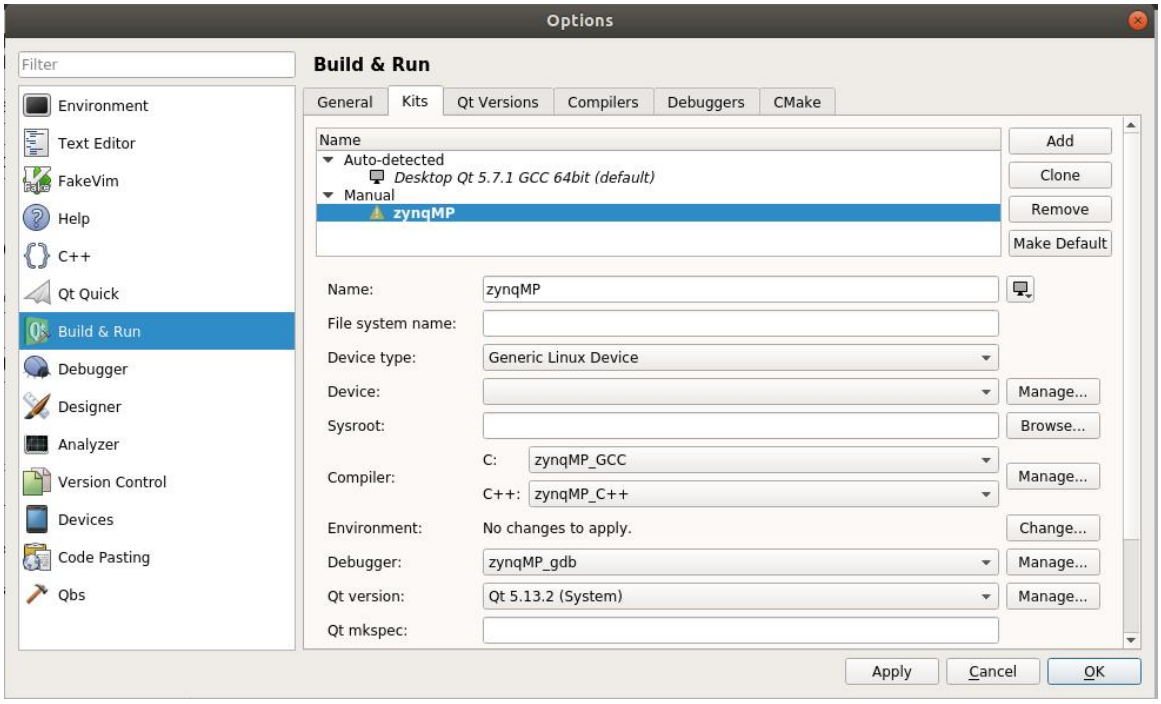
9. 再点击“Apply”，更新信息。



10. 选择“Debuggers”，添加一个，点击“Add”添加一个调试器，名称修改为”zynqMP_gdb”，路径选择”/opt/petalinux/2020.1/sysroots/x86_64-petalinux-linux/usr/bin/aarch64-xilinx-linux/aarch64-xilinx-linux-gdb”，点击”Apply”。



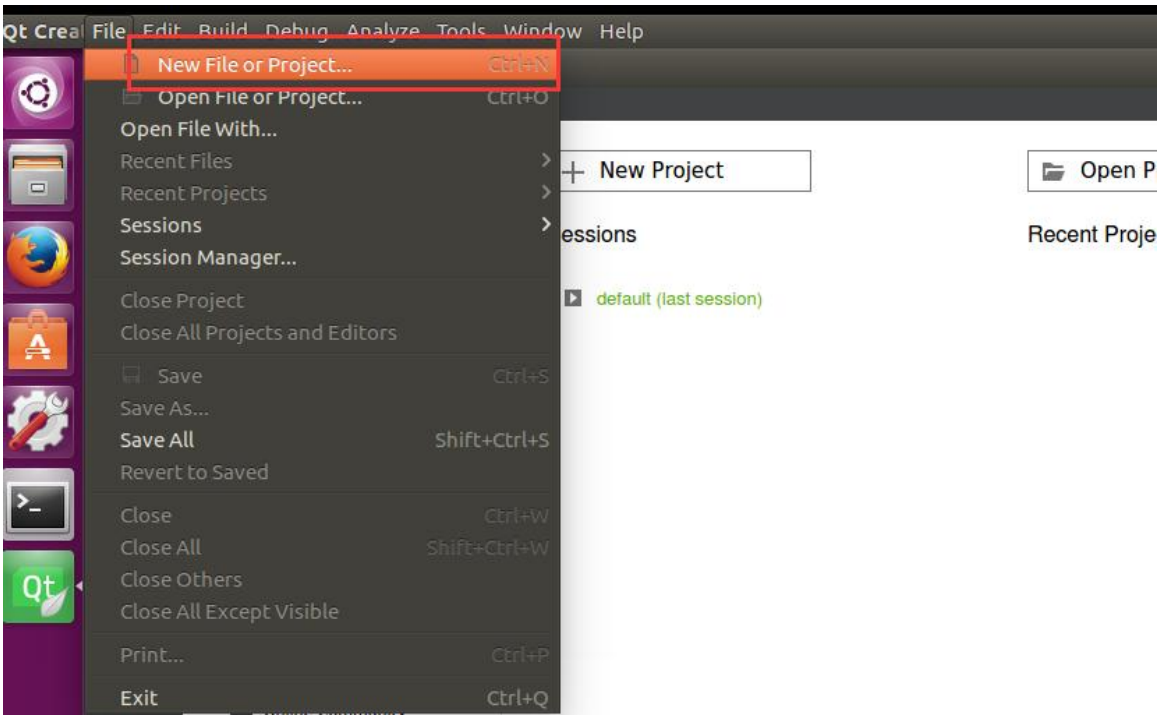
11. 在”Kits”页，点击”Add”添加一个新的”Kit”，”Name”修改为”zynqMP”，”Device type”选择”Generic Linux Device”，”Compiler C”：选择”zynqMP_GCC”，”Compiler C++”选择”zynqMP_C++”，”Debugger”选择”zynqMP_gdb”，”Qt version”选择”Qt 5.13.2(System)”。



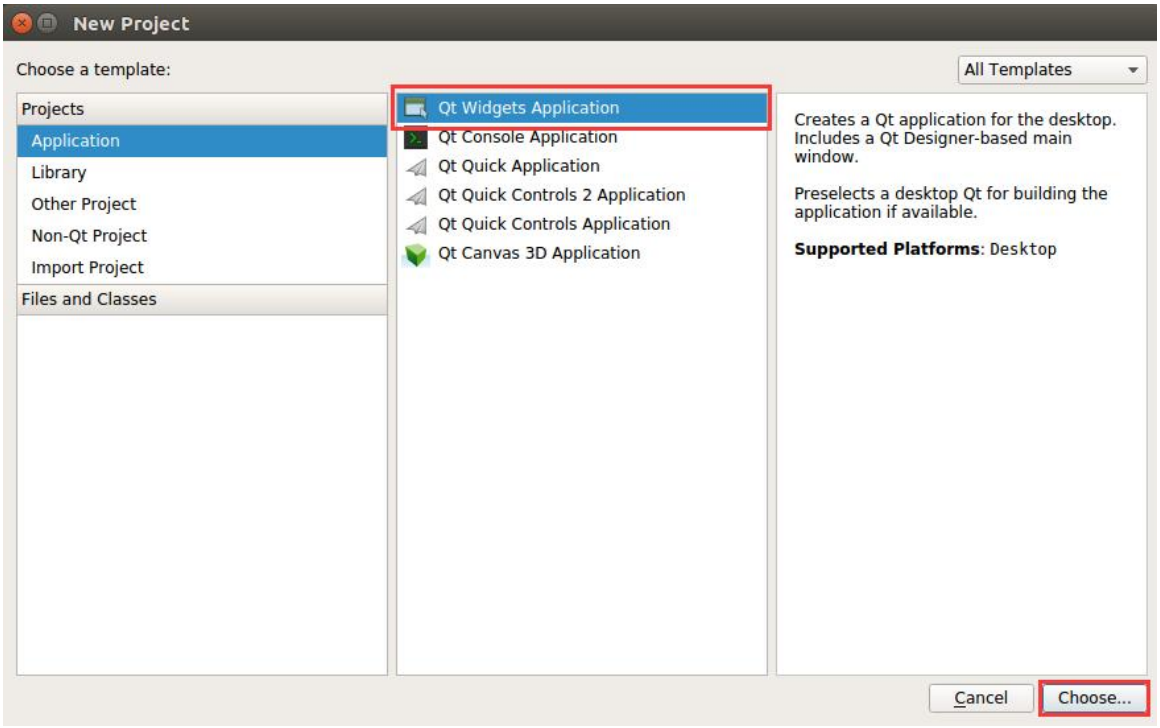
12. 点击 OK 配置 Kits 完成。

6.4 创建 QT 测试工程

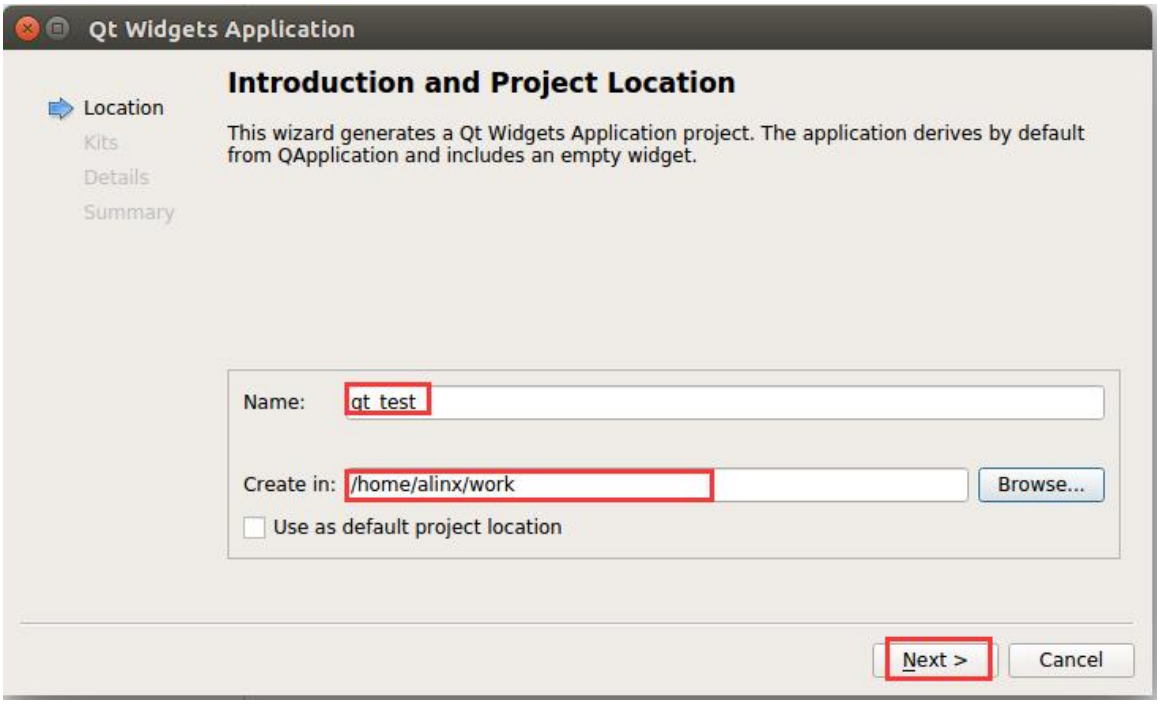
1. 点击 File -> New File or Project。



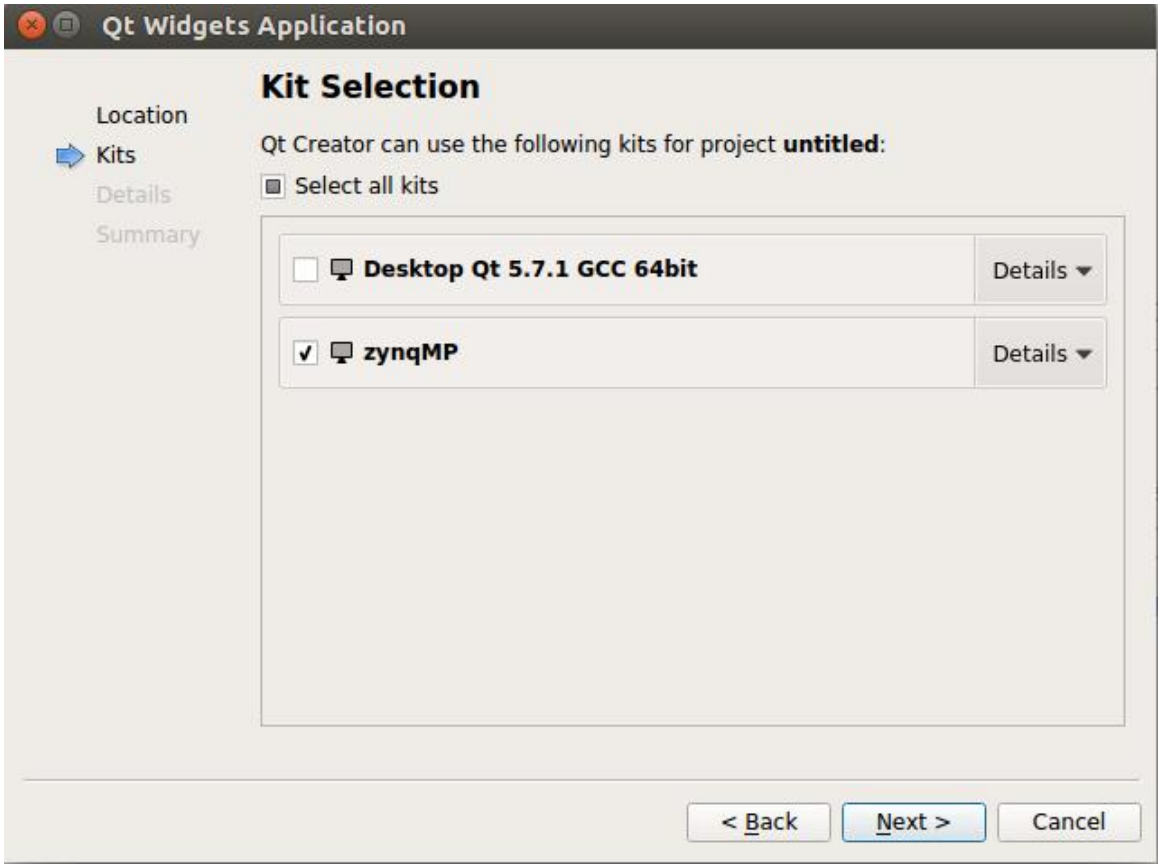
2. 工程模板选择“Application” “Qt Widgets Application”，然后点击“Choose...”。



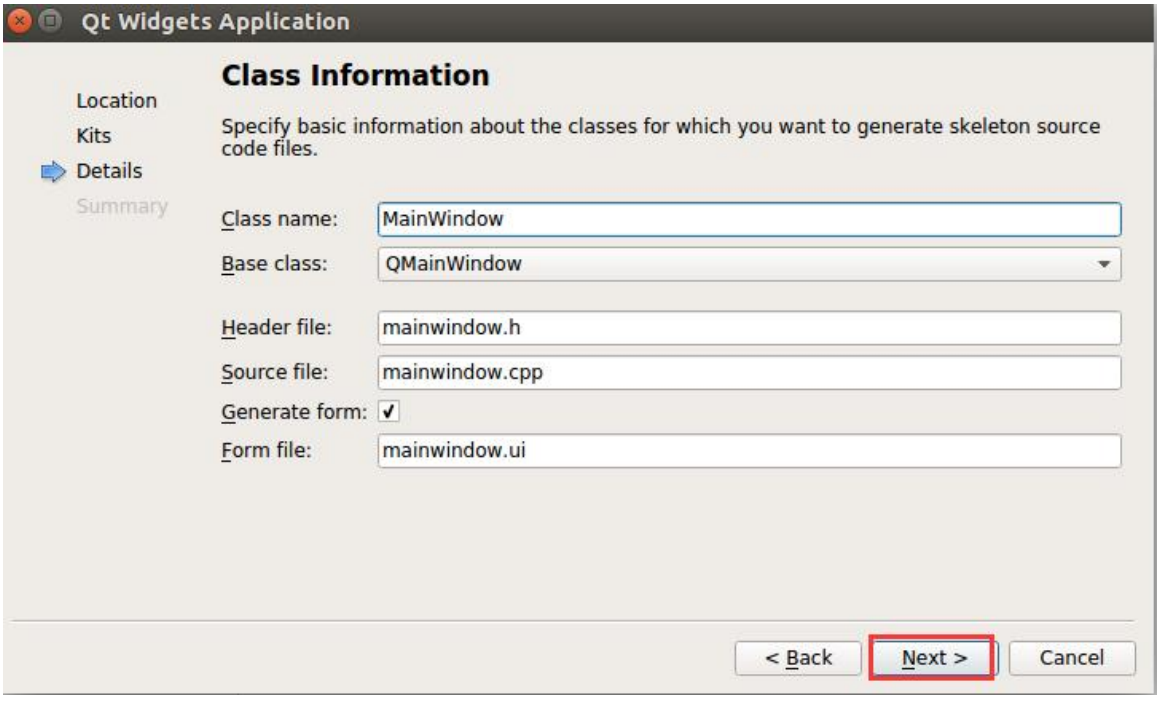
3. Name 修改为 qt_test，路径本实验选择“/home/alinx/work”。



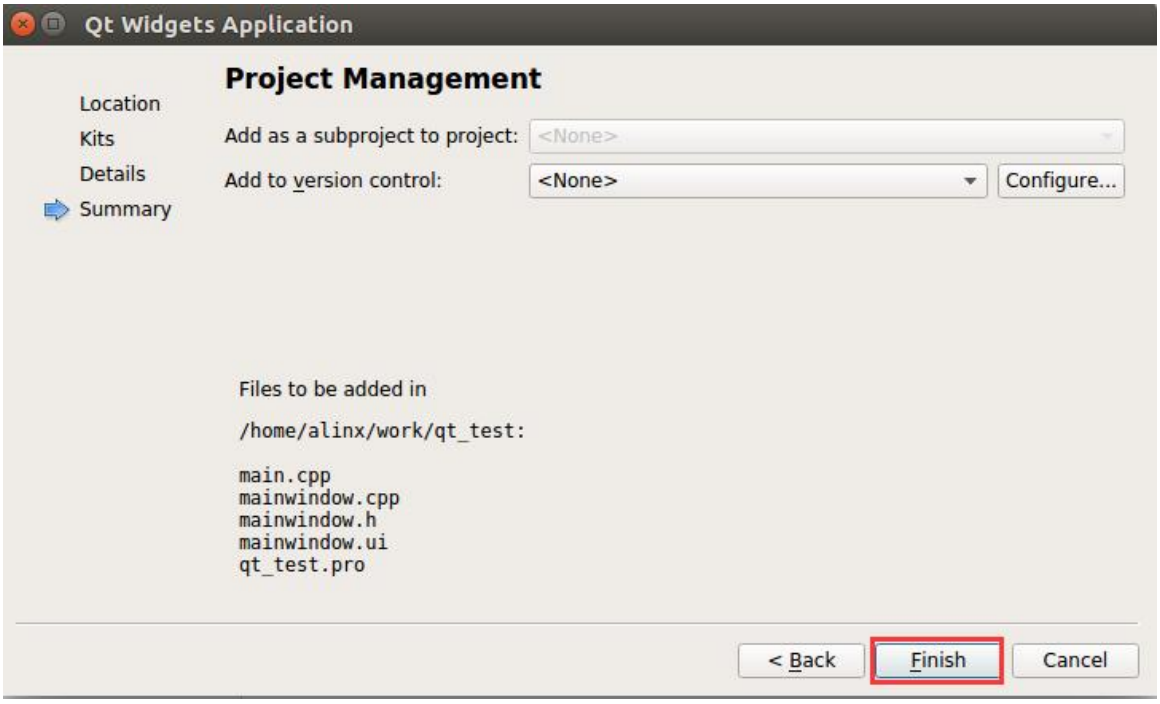
4. kit 选择，这里选择 zynqMP 版本。



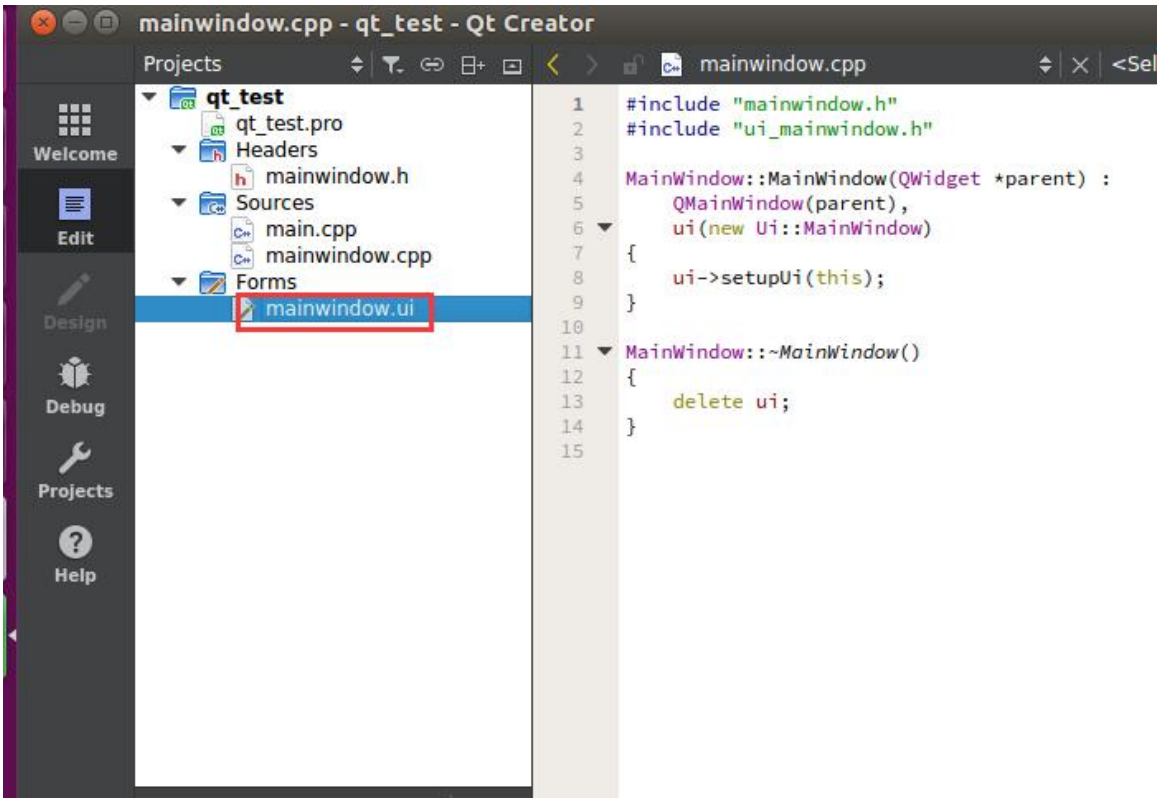
5. 选择 Next 继续。



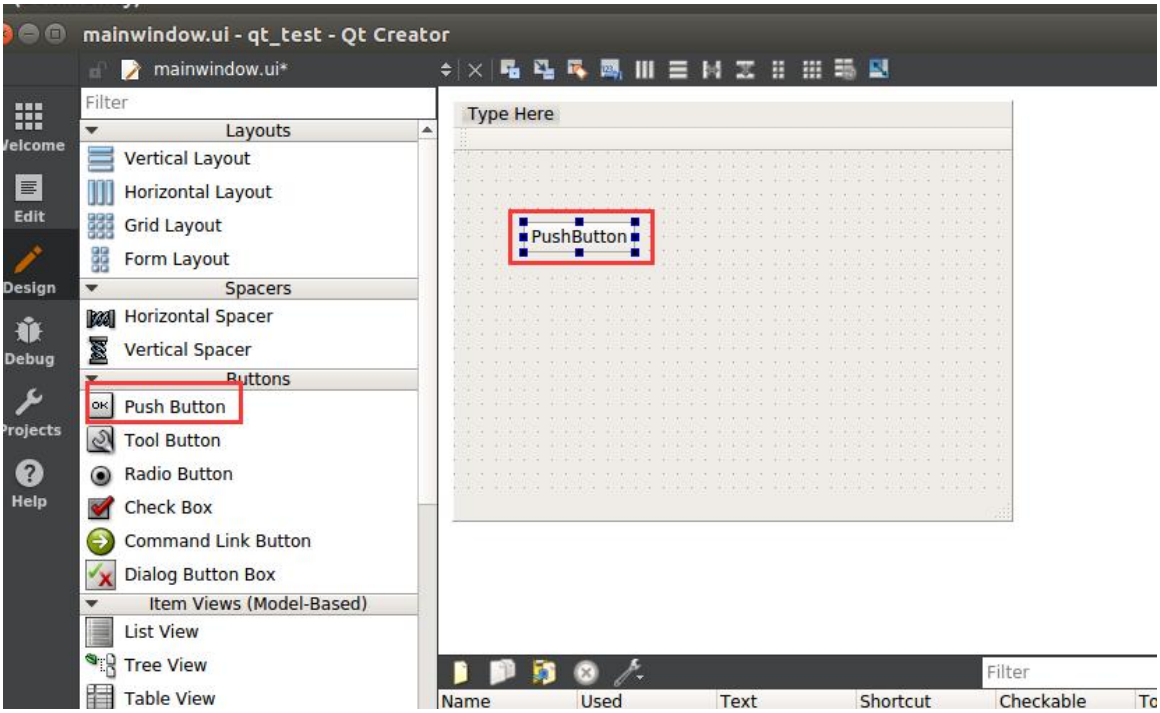
6. 选择 Finish 结束工程创建向导。



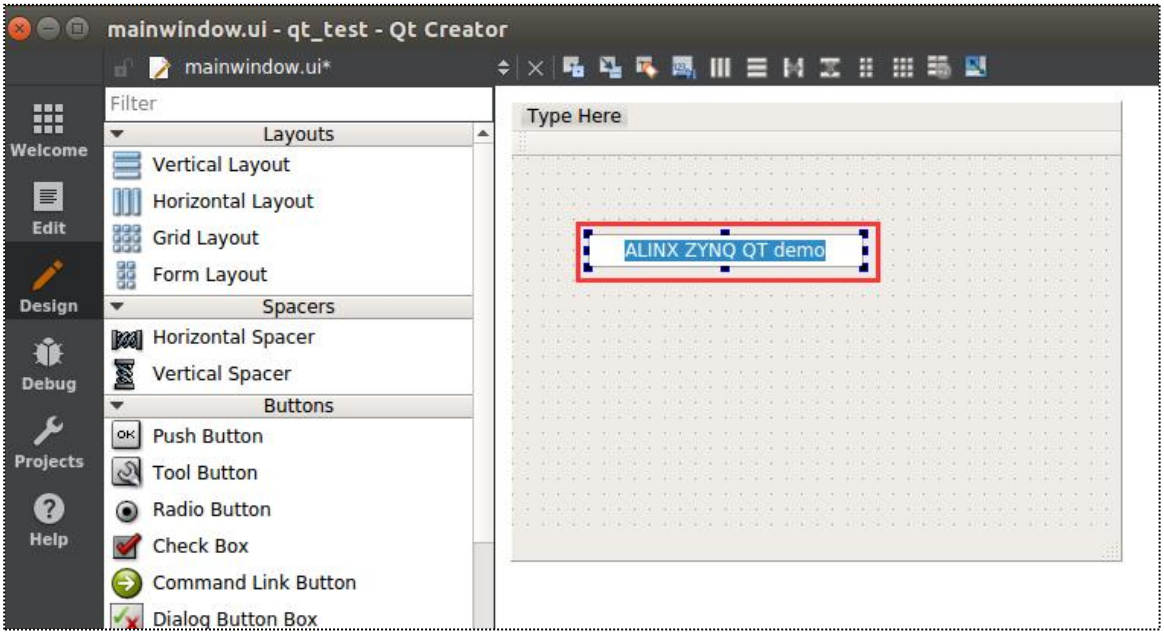
7. 双击 mainwindow.ui 文件。



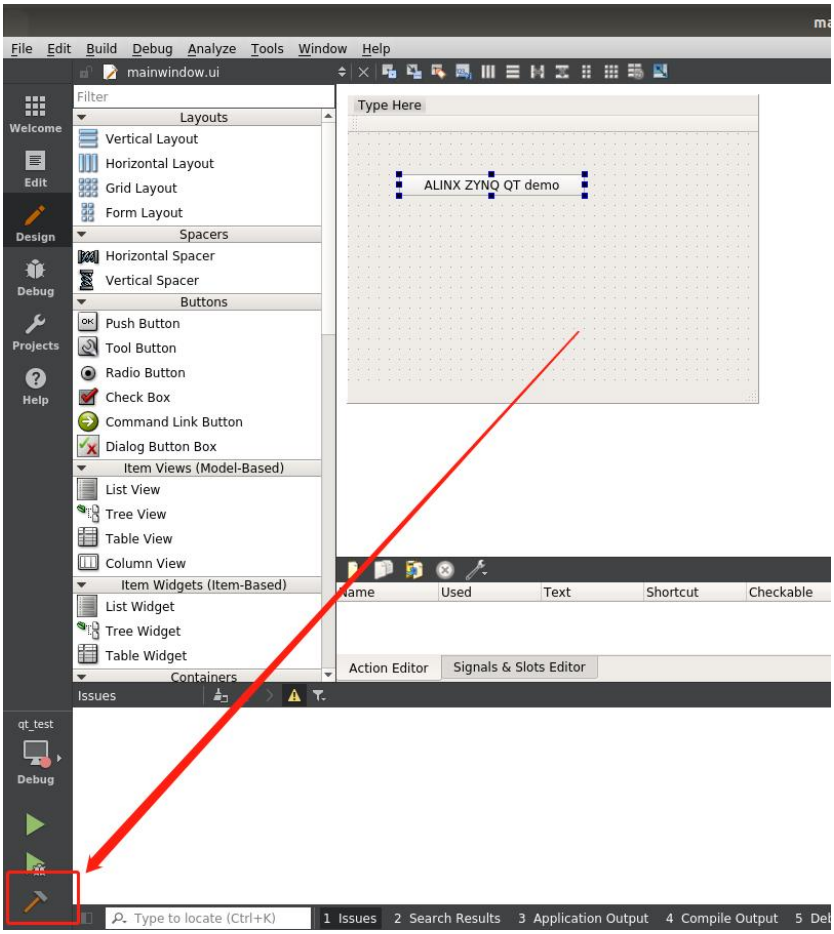
8. 拖拽一个 Push Button 到主界面中。



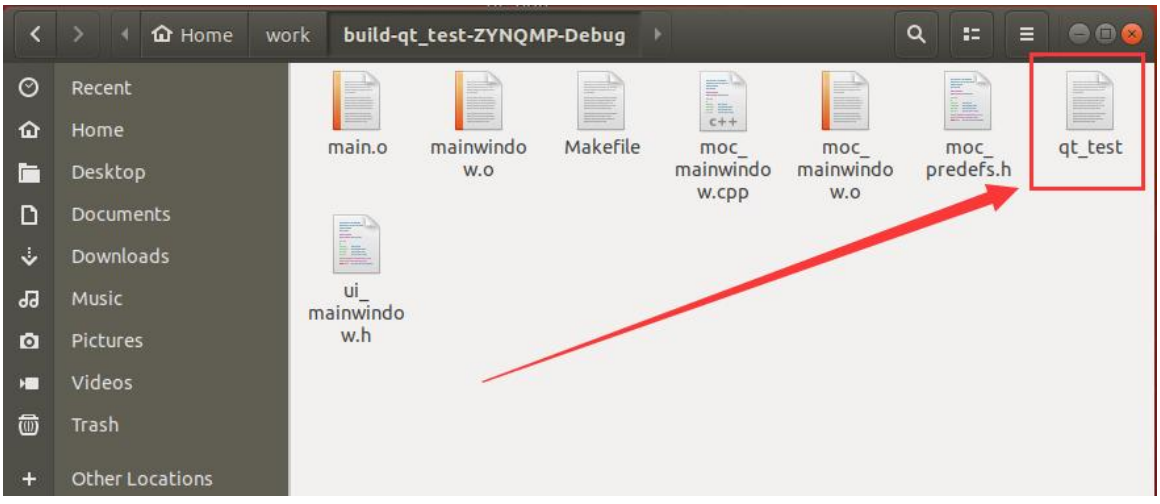
9. 双击文字部分，可以修改按钮文字，拖动边框上蓝色的点可以拉长。



10. 点击左下角点击锤子图标，编译程序。



11. 在 build-qt_test-ZYNQMP-Debug 目录可以看到生成了一个 qt_test 的文件，这个文件要在板卡上运行。



6.5 运行交叉编译版本 QT 程序

1. 使用“6.1”节中得到的 BOOT.bin、boot.scr 及 image.ub 拷贝到 SD 卡中，将可执行程序 qt_test 也一同拷贝到 SD 卡中。

2. 连接串口和 DP 线启动板卡并输入用户名密码进入系统。

```
PetaLinux 2020.1 petalinux /dev/ttyPS0

petalinux login: root
Password:
root@petalinux:~#
```

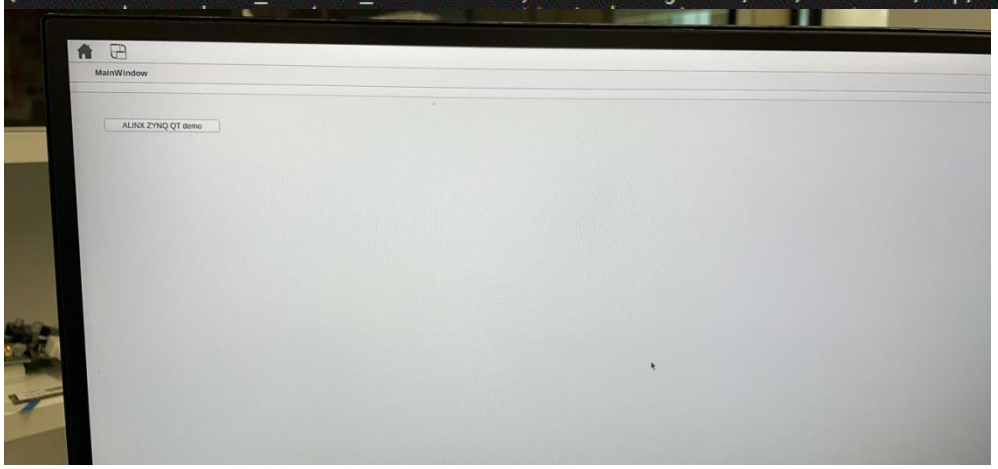
显示器会显示一个 matchbox 的简易桌面。

3. 在串口终端中输入下面的命令设置 Qt 库运行环境变量，（如果没有配置 X11，环境变量使用 export QT_QPA_PLATFORM=linuxfb）。

```
export DISPLAY=:0.0
root@petalinux:~# export DISPLAY=:0.0
```

4. 运行 qt_test 程序，可以看到显示其中显示了 QT 程序。

```
/media/sd-mmcblk1p1/qt_test
root@petalinux:~# /media/sd-mmcblk1p1/qt_test
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/var/volatile/tmp/runtime-root'
```



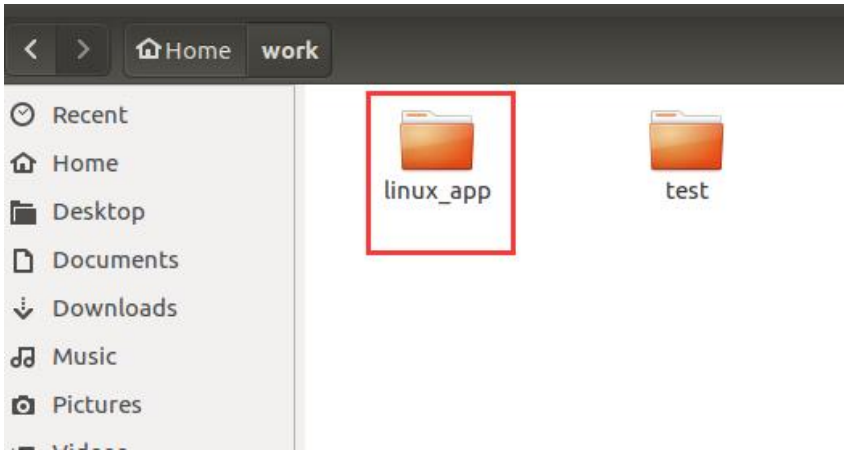
5. 在串口终端中按键盘上 Ctrl + C 组合可以结束程序运行。

第七章 使用 Vitis 开发 Linux 程序

本章使用 ubuntu 环境的 Vitis 软件做一个 Linux 应用程序，然后在开发板上运行。本实验需要使用“QT 和 OPENCV 交叉编译开发环境”一章中做好的 Linux 运行环境。

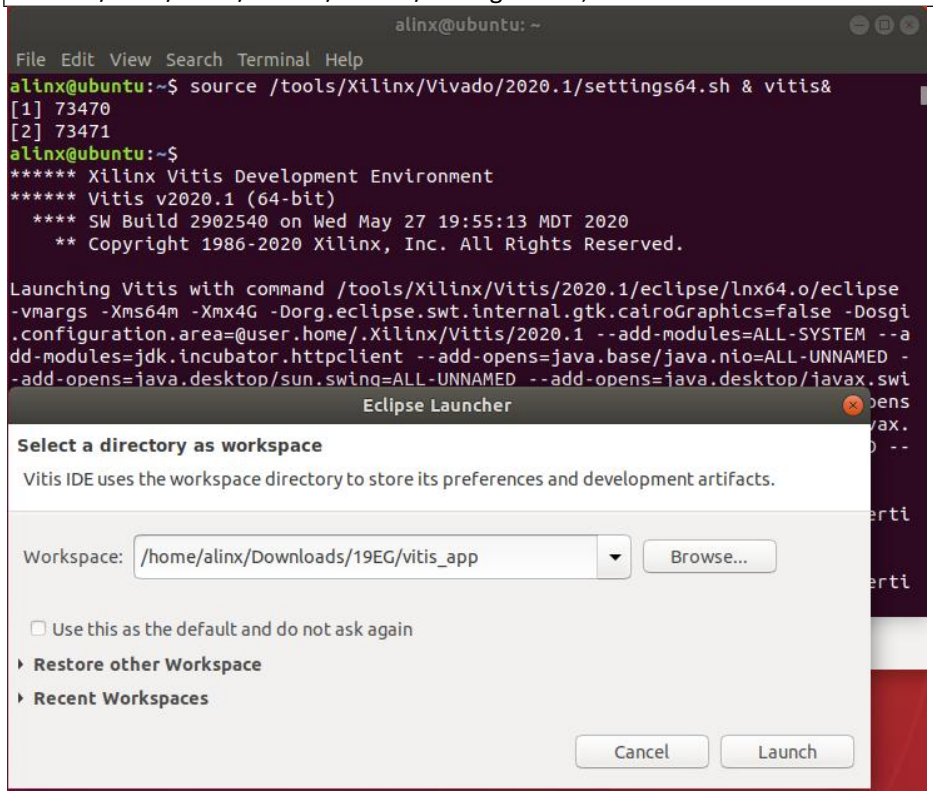
7.1 使用 Vitis 建立 Linux 应用程序

1. 在/home/alinx/work 建立目录 linux_app，用于 SDK 的工作空间

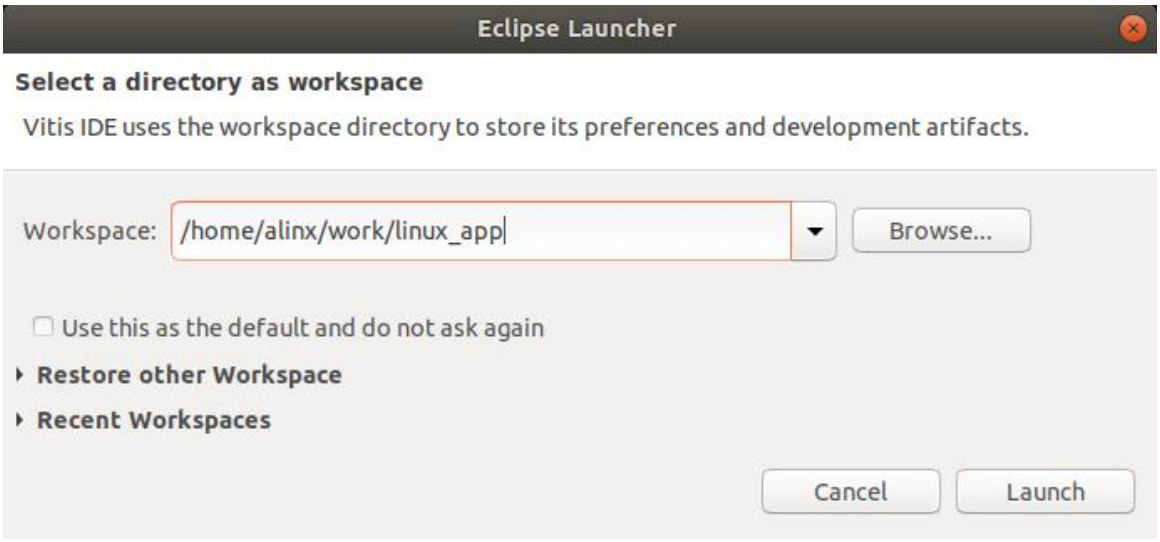


2. 在任意目录打开一个终端，用下面的命令设置 Vivado 环境变量，运行 vitis。

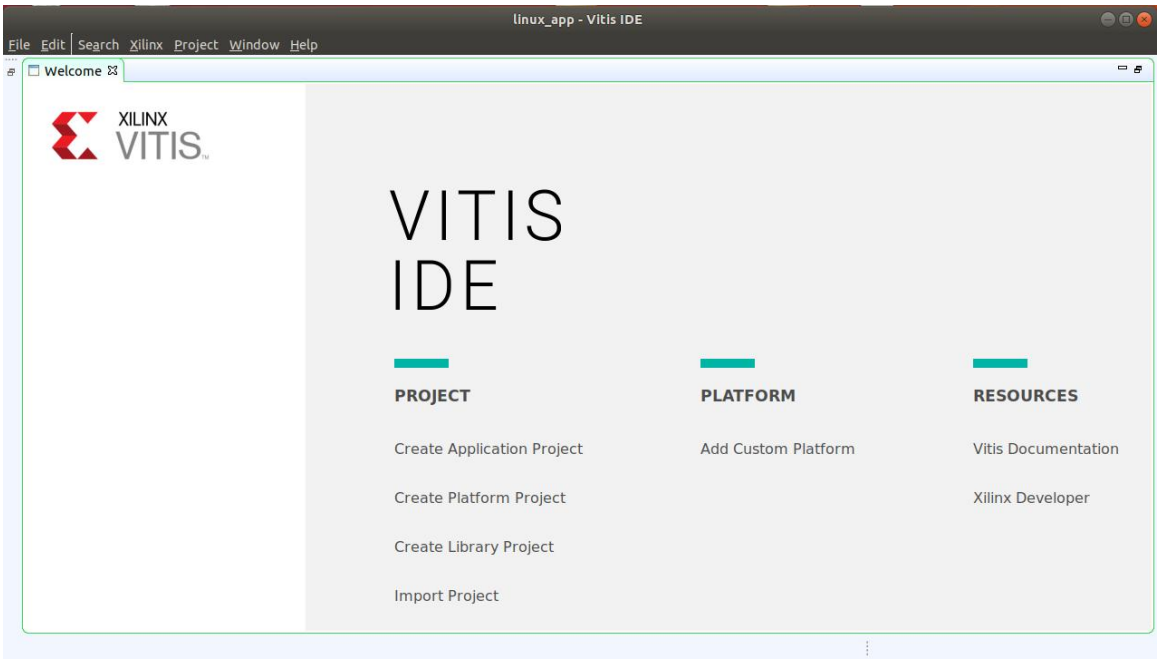
```
source /tools/Xilinx/Vivado/2020.1/settings64.sh ; vitis&
```



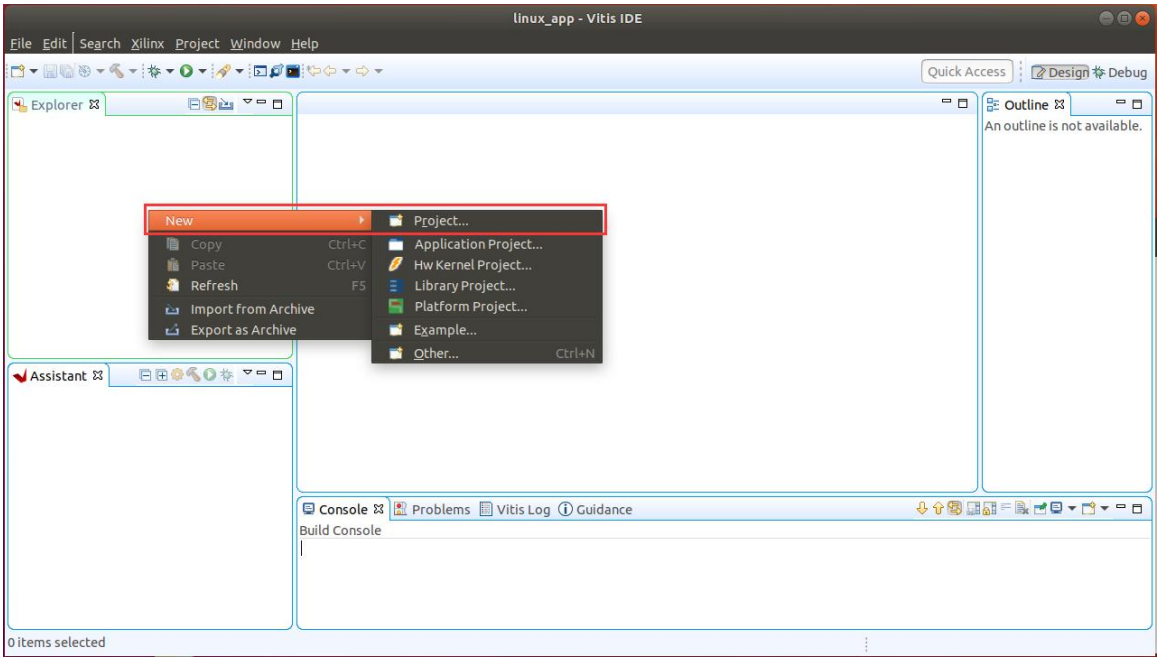
3. “workspace”输入“/home/alinx/work/linux_app”，然后点击右下角的“launch”按钮。



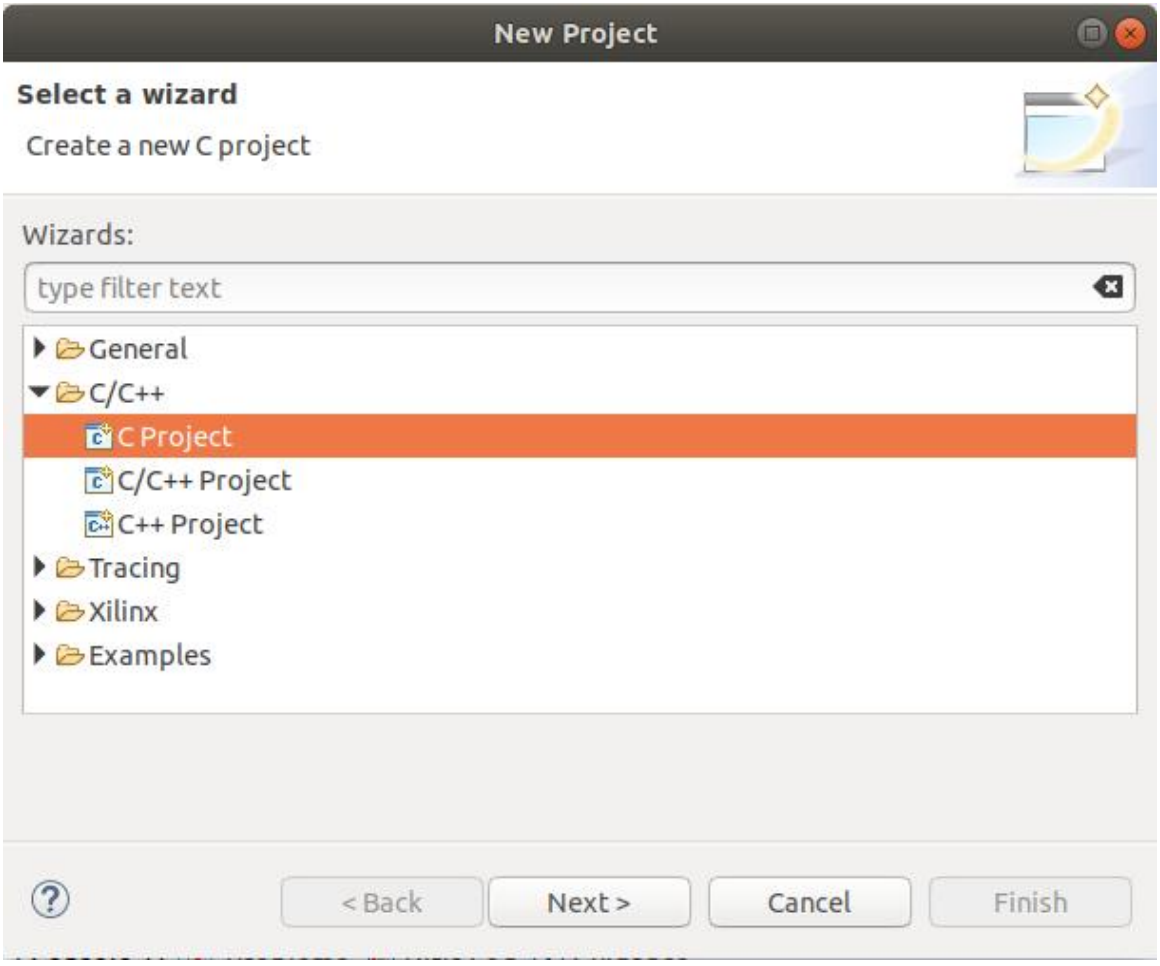
4. 点击“Welcome”旁边的 X，关闭欢迎界面



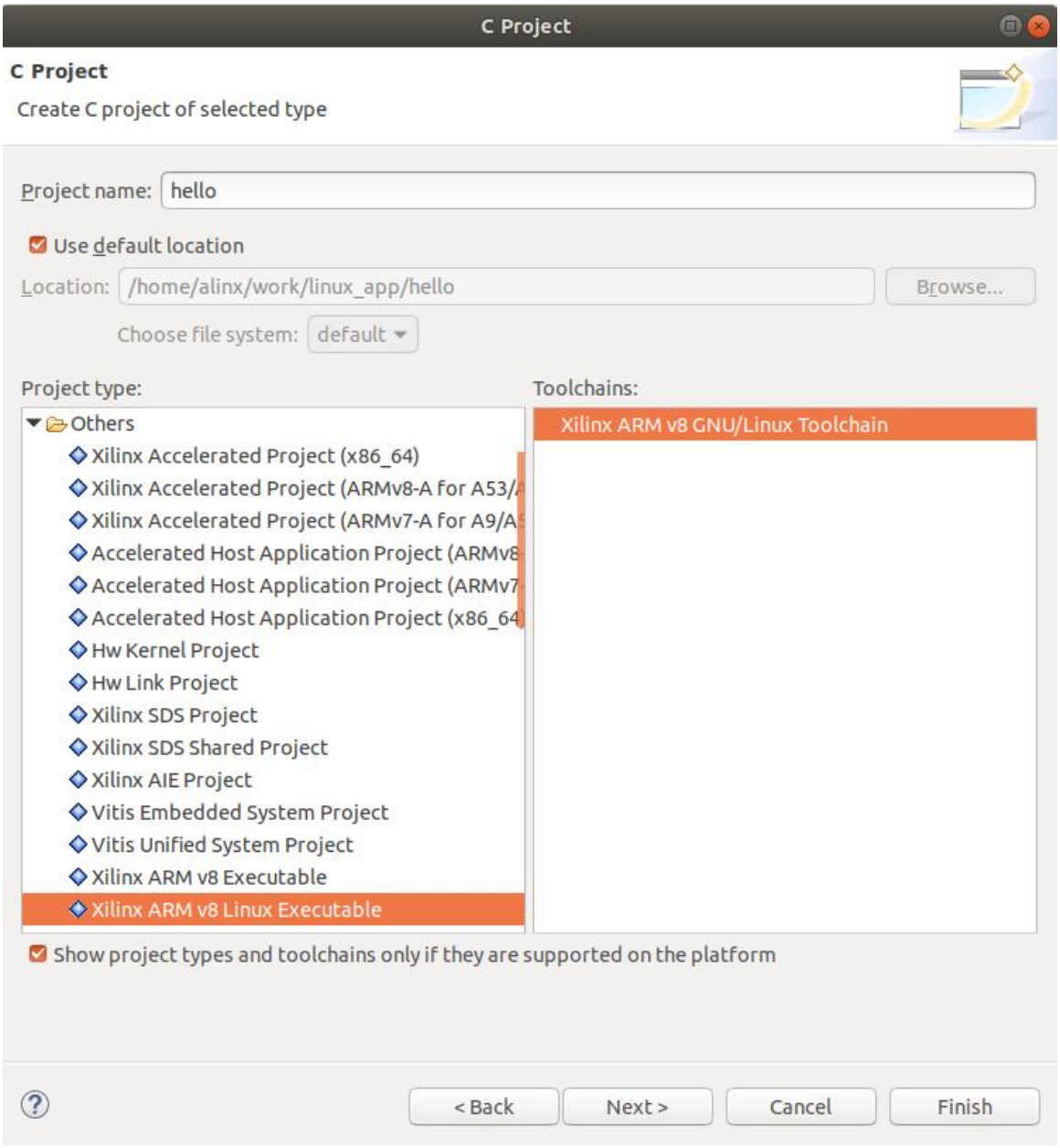
5. Explorer 空白处点击右键“New-> Project…”



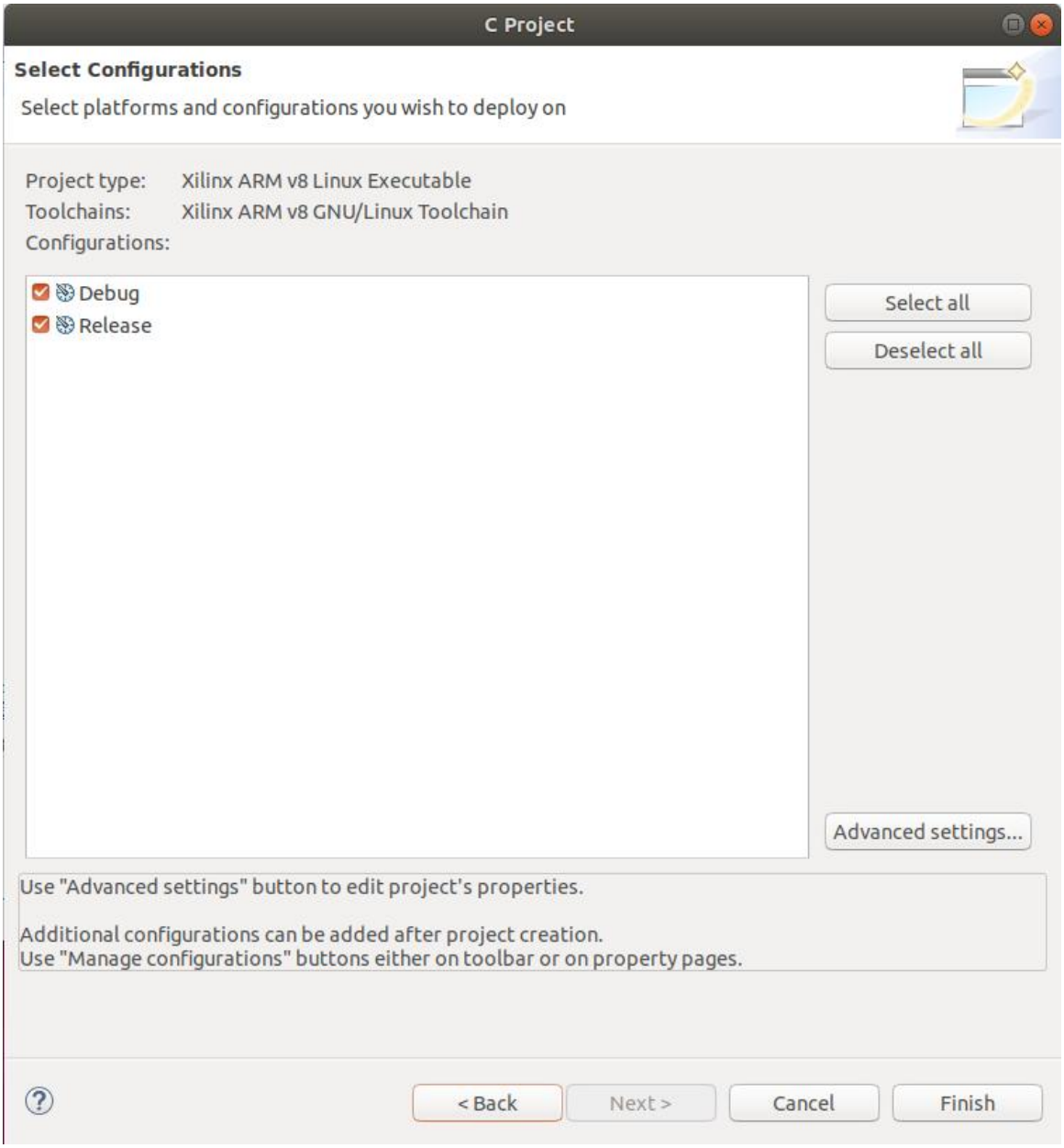
6. 选择“C Project”，就是一个 C 语言的一个工程，点击“Next”



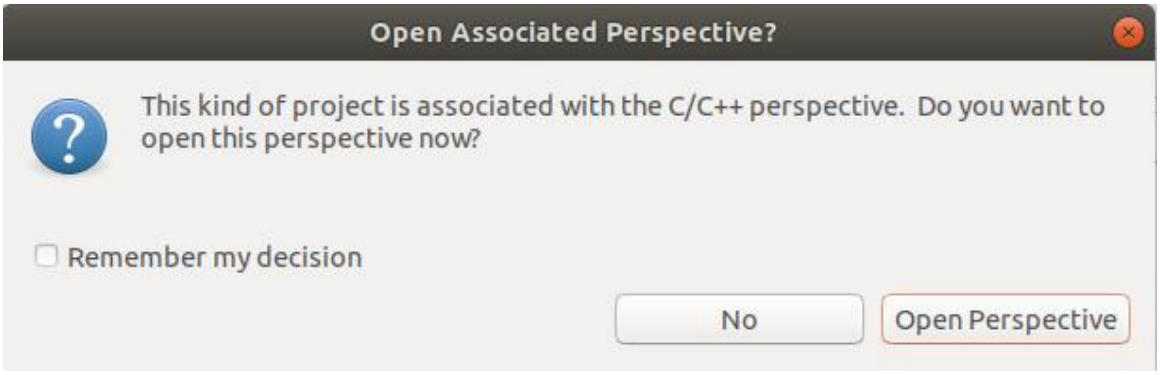
7. 工程名填写“hello”，工程类型选择 Others 文件夹里面的“Xilinx ARM v8 Linux Executable”，然后点击“Finish”



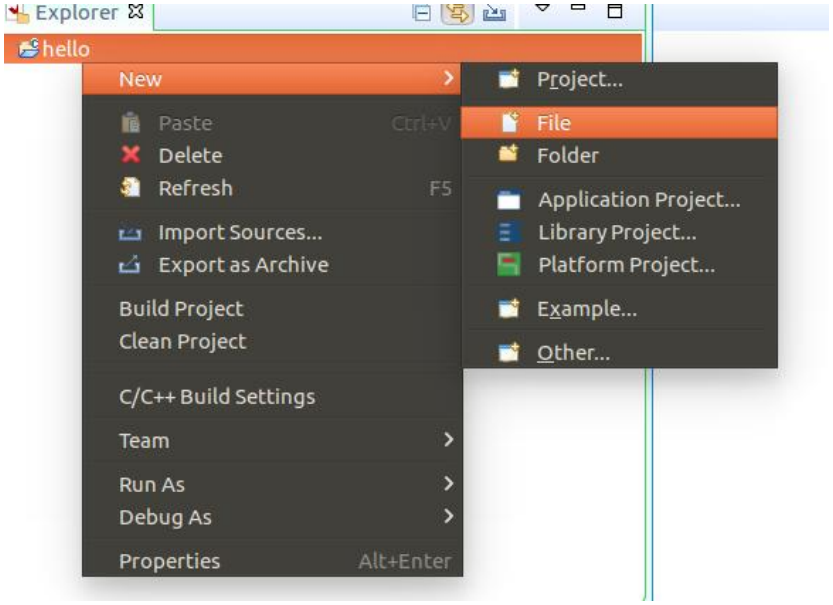
8. 这个界面直接点击“Finish”按钮。



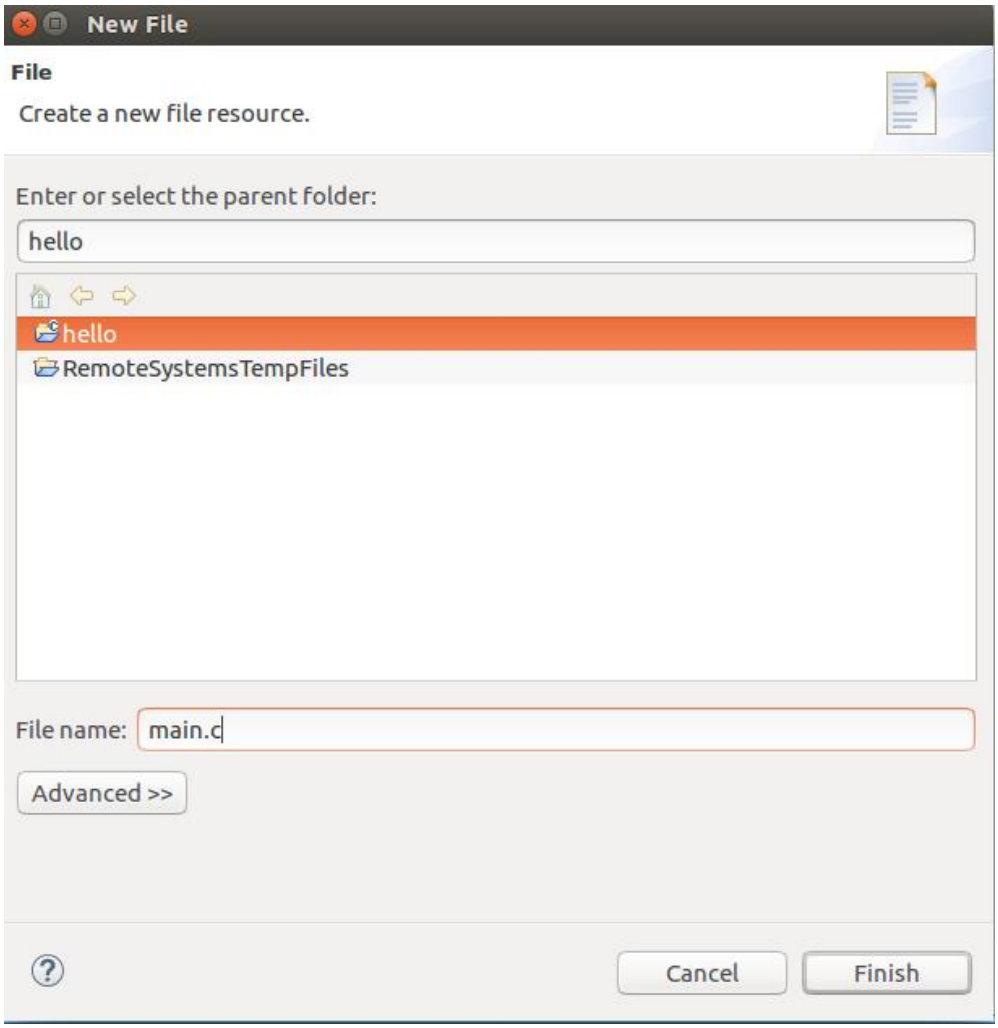
9. 如果弹出下面的信息，点“No”



10. 在“Explorer”窗口中右击然后选择“New > File”



弹出的窗口中，“File name：”后面填写文件名，这里命名为“main.c”，然后点“Finish”。

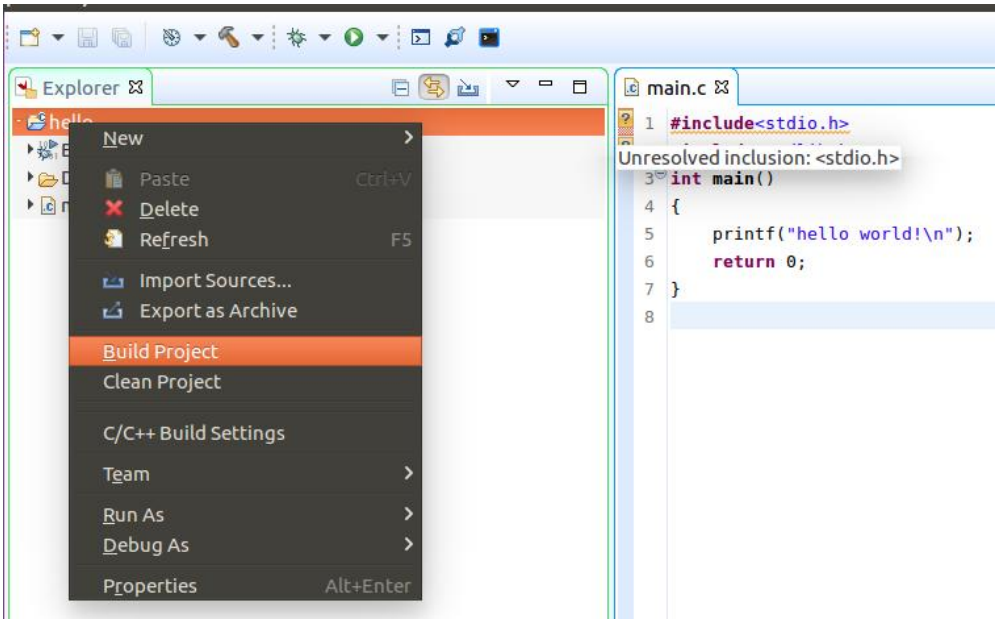


11. 编辑 main.c 文件，写一个简单的打印

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
```

```
    printf("hello world!\n");
    return 0;
}
```

12. 在“hello”目录上右击，然后选择“Build Project”编译工程。



7.2 通过 NFS 共享运行

1. 使用“QT 和 OPENCV 交叉编译开发环境”一章中制作的 SD 卡启动系统，开发板插上串口、连接网线从路由器到开发板的 PS_ETH 网口（需要路由器支持自动获取 IP），然后上电。挂载 Linux 主机 NFS，其中 192.168.1.6 是主机地址需要根据实际修改，/home/alinx/work 为 NFS 共享目录，/mnt 为开发板挂载路径。这里要求主机和开发板在一个网段（红色部分特别注意要和自己的实际情况保持一致）。

```
mount -t nfs -o nolock 192.168.1.6:/home/alinx/work /mnt
```

```
root@petalinux:~# mount -t nfs -o nolock 192.168.1.6:/home/alinx/work /mnt/
```

2. 进入“/mnt/linux_app/hello/Debug”目录，运行 hello.elf，我们可以看到打印出 Hello World。

```
cd /mnt/linux_app/hello/Debug
./hello.elf
```

```
root@petalinux:/mnt/linux_app/hello/Debug# ./hello.elf
hello world!
root@petalinux:/mnt/linux_app/hello/Debug#
```

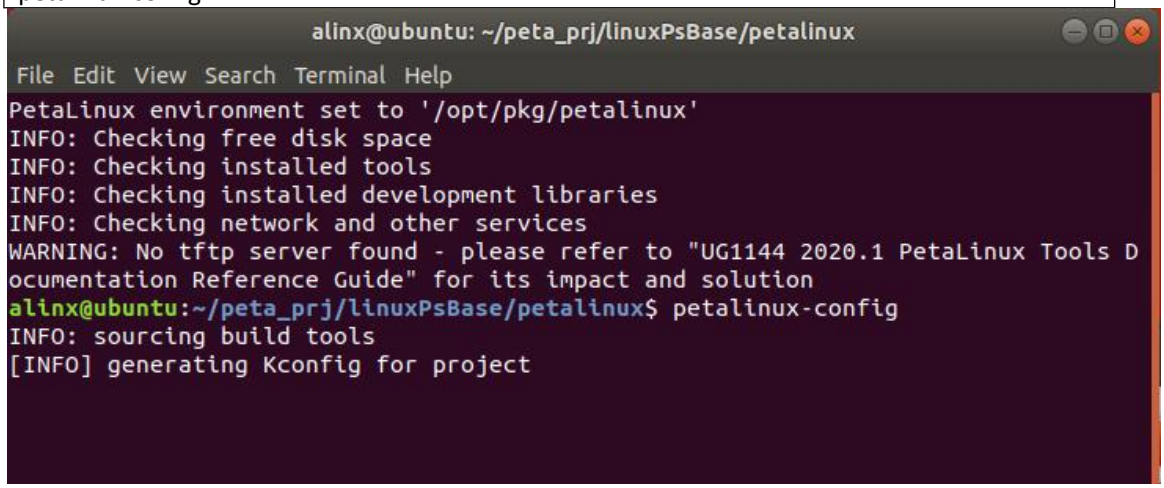
第八章 SD 卡根文件系统

Petalinux 默认的根文件系统类型是 INITRAMFS，不能保存文件和配置。本章介绍如何使用 sd 挂载根文件系统，制作一个可以保存文件和配置的板载 linux 系统。

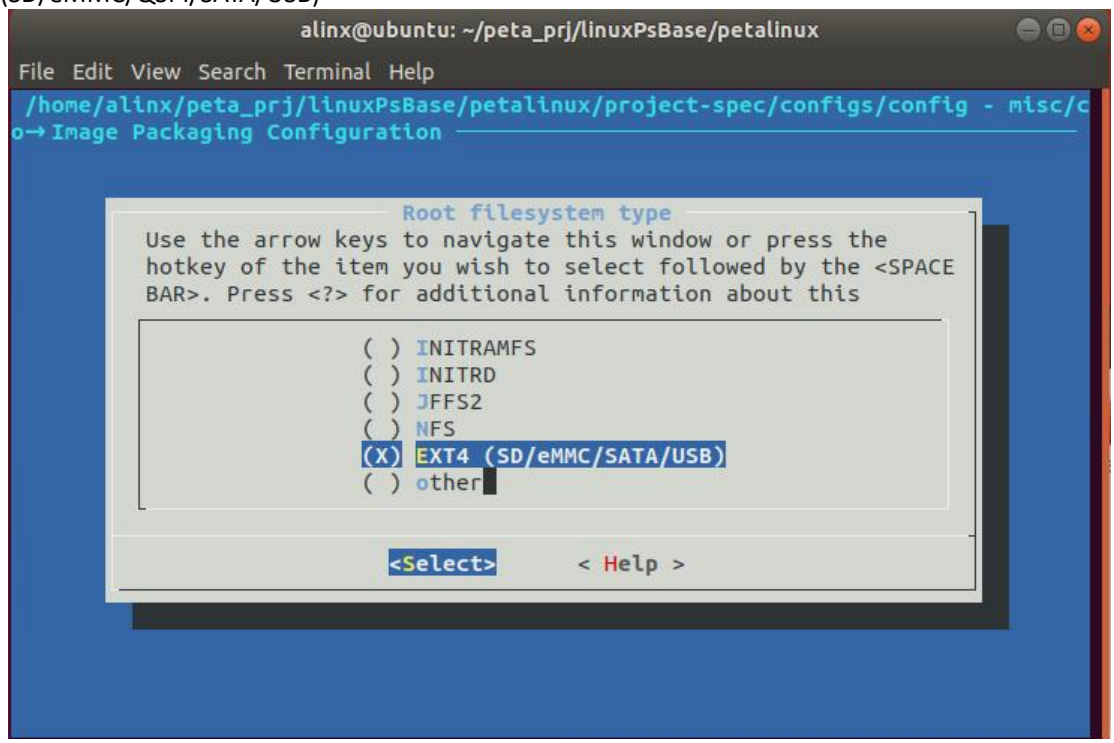
8.1 Petalinux 配置

1. 在“QT 和 OPENCV 交叉编译开发环境”一章制作的 petalinux 工程中，打卡一个终端设置好环境变量，然后运行下面的命令配置 Petalinux。如果不在前面教程的基础上做，而是自建 petalinux 工程的话，尤其需要注意设备树一定要添加第一章 1.4 节中的设备树节点。

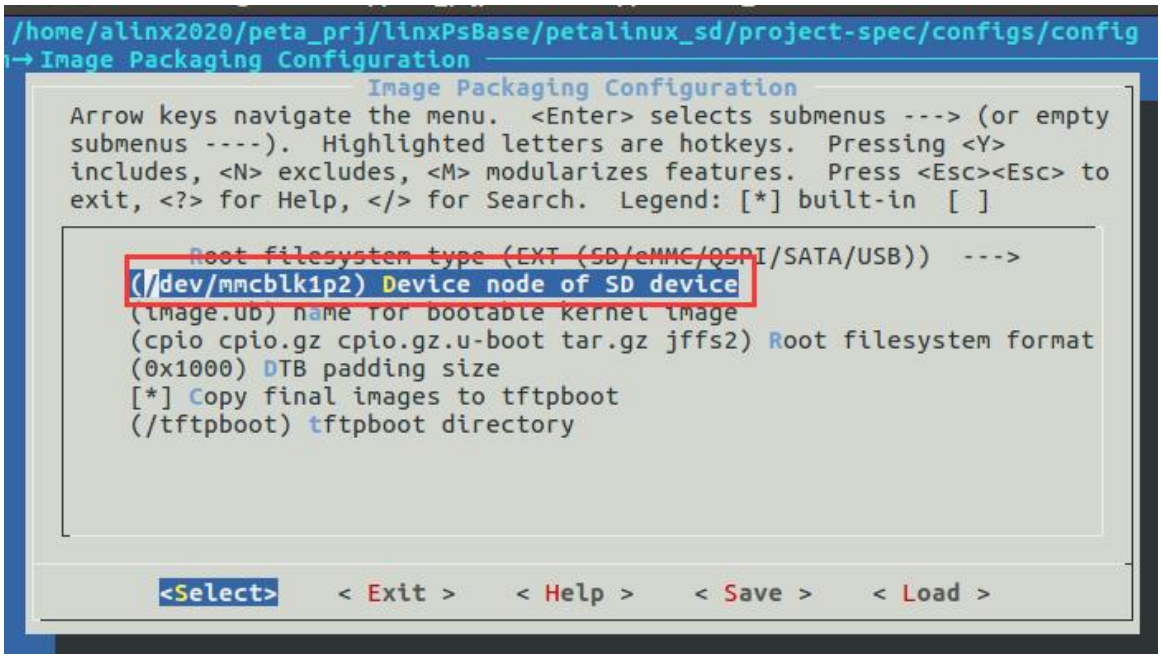
```
petalinux-config
```



2. 进入“Image Packaging Configuration ---> Root filesystem type”选项，选择 EXT (SD/eMMC/QSPI/SATA/USB)



3. 选择“Device node of SD device”选项设置根文件系统路径，这个至关重要，如果填错无法启动 Linux。mmcblk 代表 sd 卡和 emmc，这里有个默认值是“mmcblk0p2”，代表第一个 mmc 设备的第二分区，如果按照教程使用名为“ps_hello”的 vivado 工程，那我们的 SD 卡就是第二个 mmc 设备，这里填写/dev/mmcblk1p2。



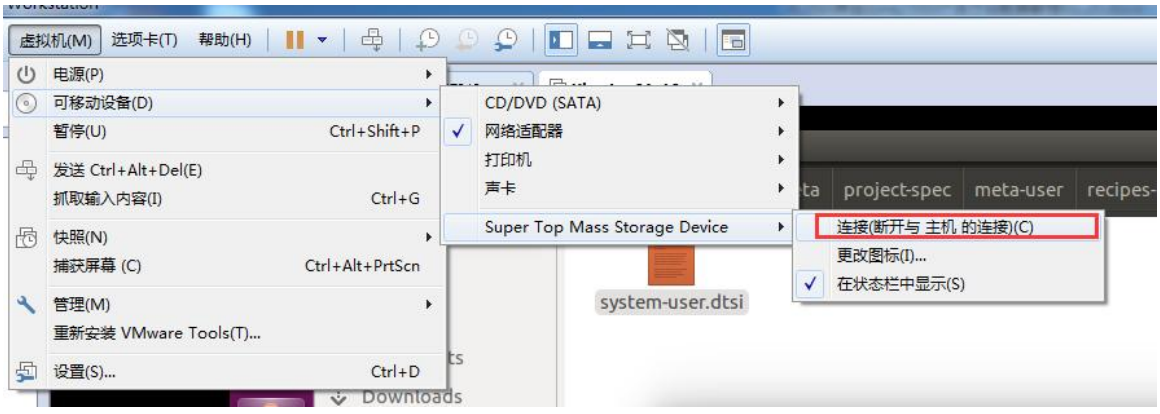
4. （注意，在“QT 和 OPENCV 交叉编译开发环境”一章中为了构建 sdk，这个 petalinux 工程设置了允许在线编译，有需要的话记得恢复成离线编译的设置。）然后保存设置退出配置界面，编译 petalinux 系统，打包 BOOT.bin。

8.2 制作 SD 卡文件系统

制作 SD 卡文件系统会导致 SD 卡里内容丢失，请先做好备份。

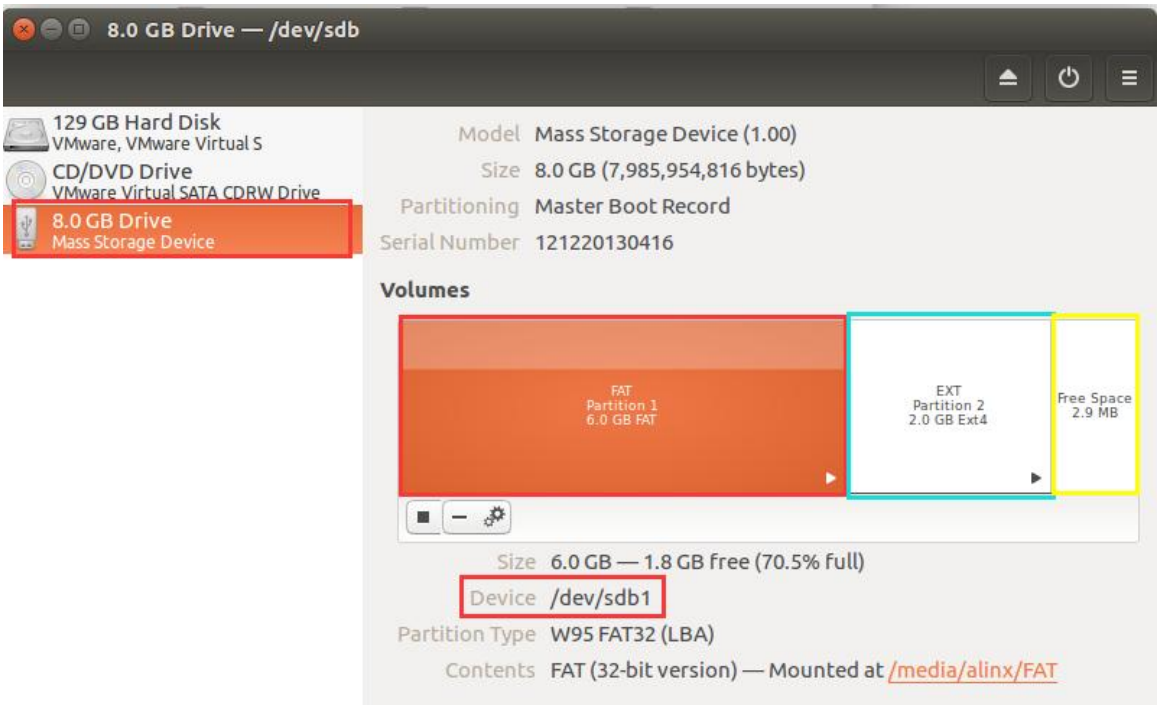
8.2.1 SD 卡修改分区

1. 把开发板的 sd 卡插入读卡器，然后插入电脑的 USB 口。如果读卡器没有连接到虚拟机 Linux 中，参考下图的方法。

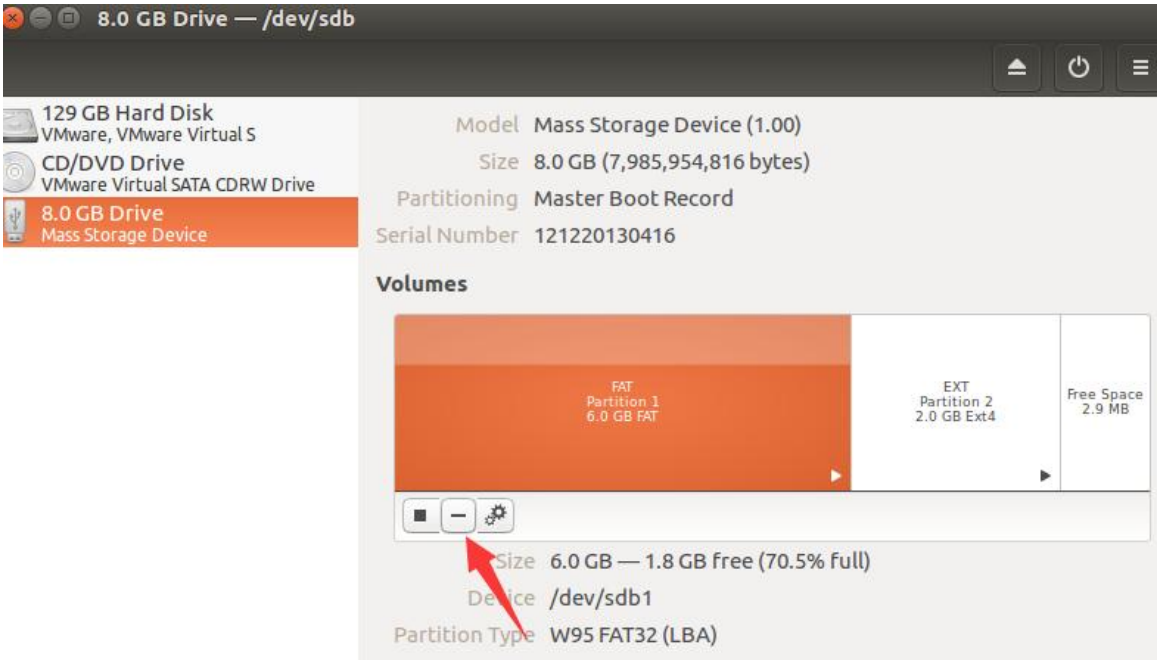


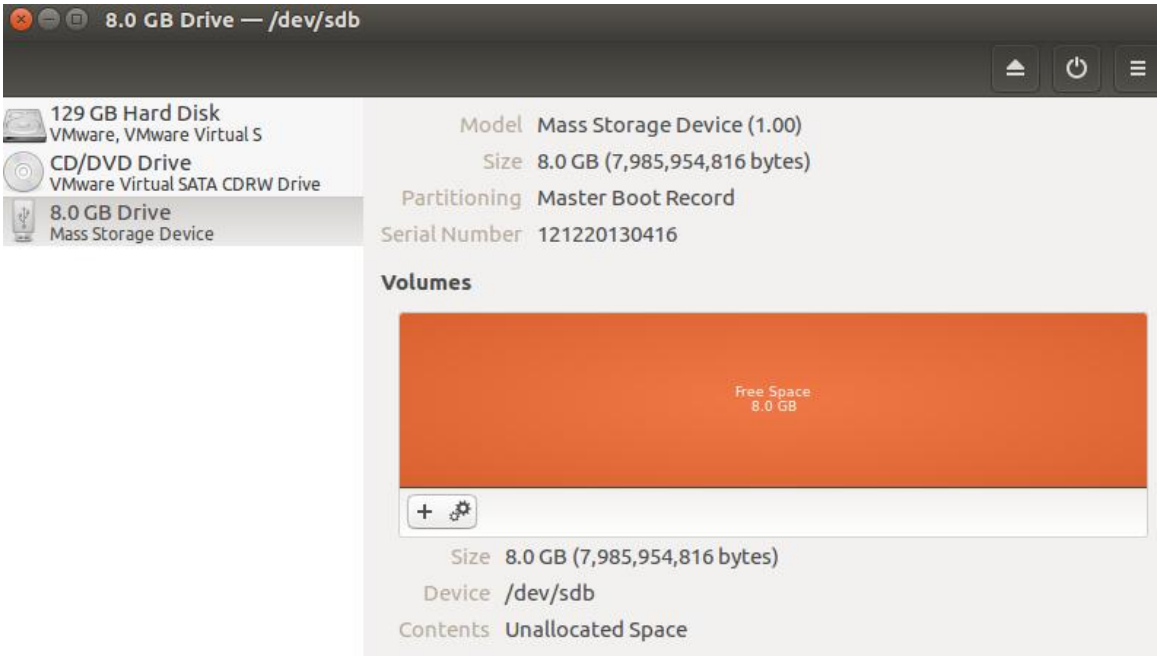
2. 在 ubuntu 的搜索路径中，输入 disk，会出现 Disks 的图标。

3. 鼠标点击 Disks 图标，出现"Disks"的对话框，这里的 SD 卡已经分为 2 个分区，一个名为 FAT，一个名为 EXT，但是不管原先 SD 卡分区是什么情况，这里都要重新分区。

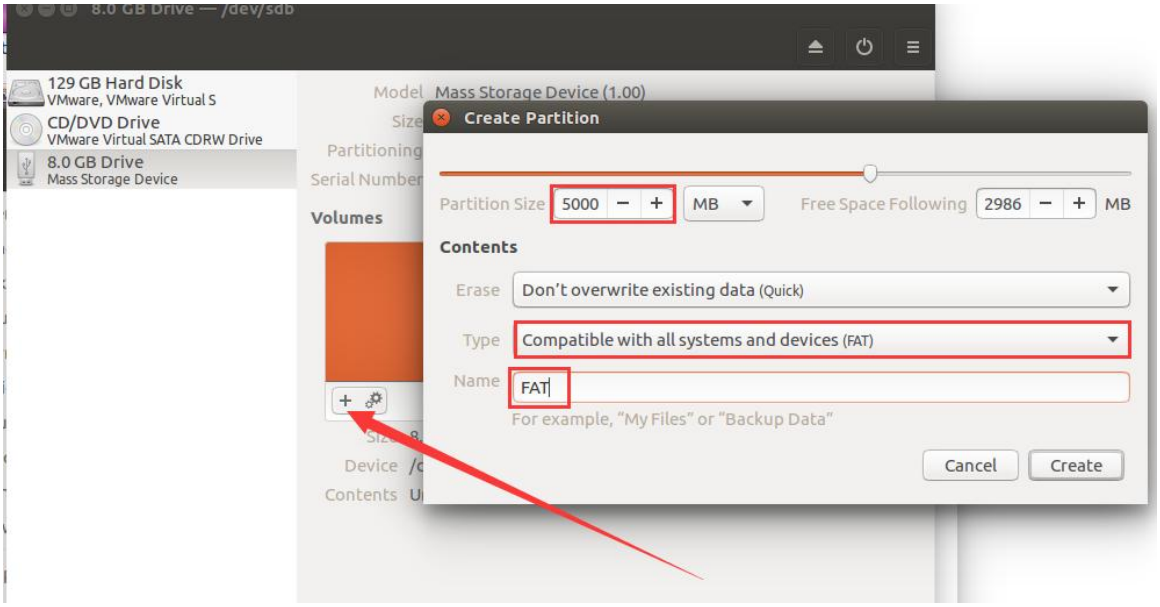


4. 选择每个分区下面的删除分区图标，删除所有分区。

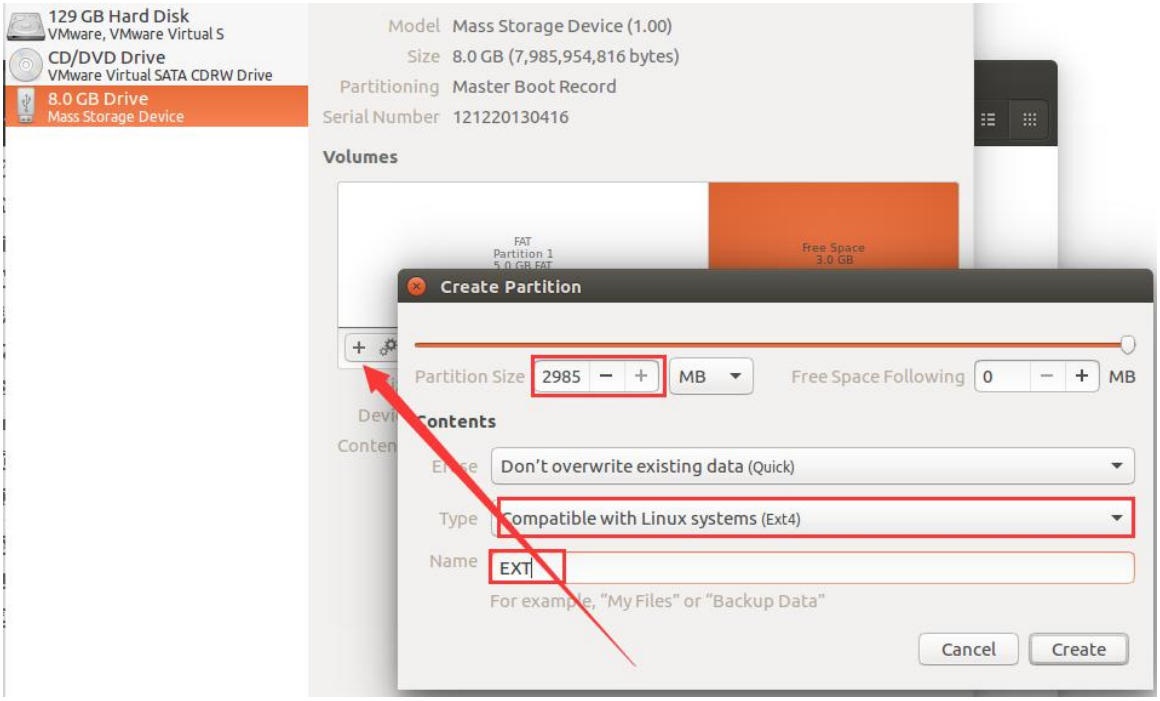




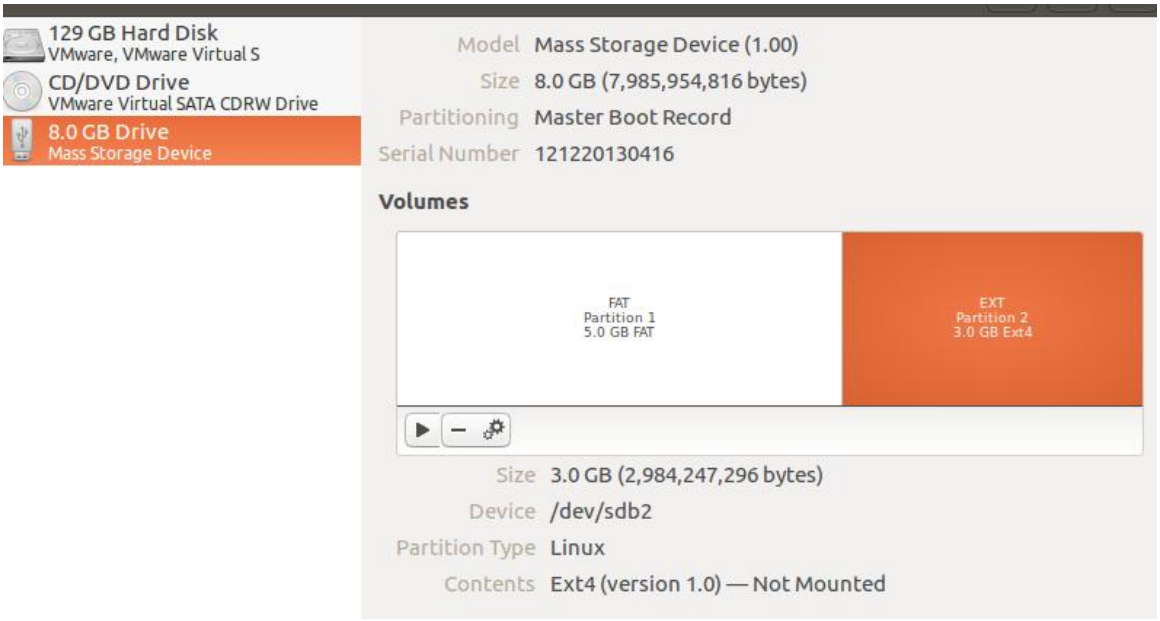
5. 点击添加分区的图标，添加第一个分区，本实验填写 5000MB，格式为 FAT，用于存放 ZYNQ 的启动文件 BOOT.bin 和内核文件、设备树，名称为 FAT。



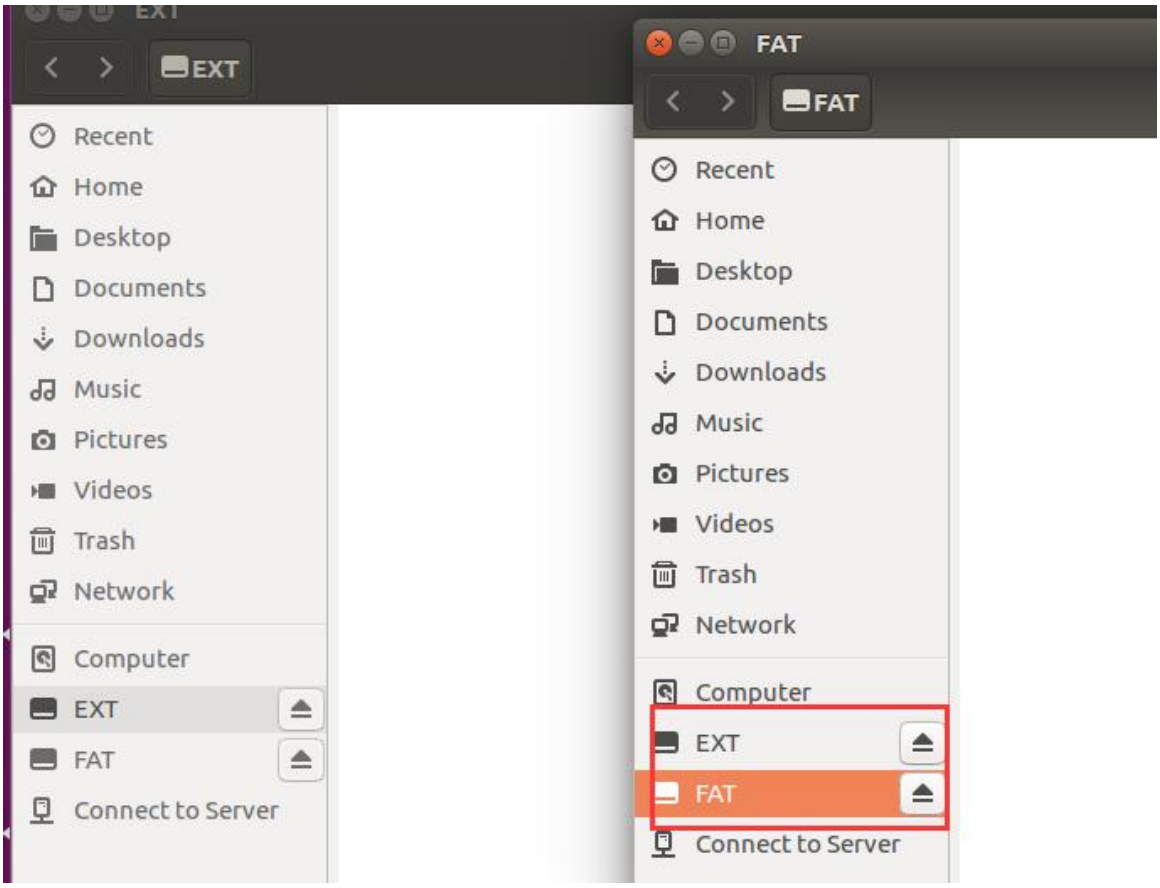
6. 创建第二个分区，用于存放根文件系统，格式为 EXT4，名称为 EXT。



7. 创建分区完成后重新插入 sd 卡。



8. 系统会自动挂载分区，并弹出窗口



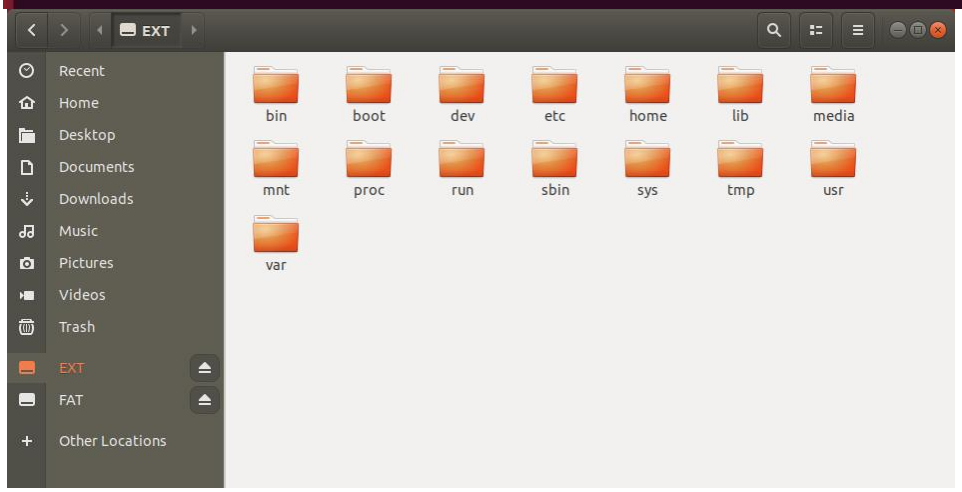
8.2.2 制作启动 linux 的 SD 卡

1) 输入下面的解压缩命令，解压根文件系统到 SD 卡 EXT4 分区。请注意解压命令格式，必须和教程一致，“/media/alinx/EXT”为当前虚拟机环境中 SD 卡 EXT 分区的路径，如果没有完

全按照教程执行可能会有不同，请根据自己实际情况修改，petalinux 工程路径“<petalinux path>”是需要你自己补全你 images 前面的全部路径。

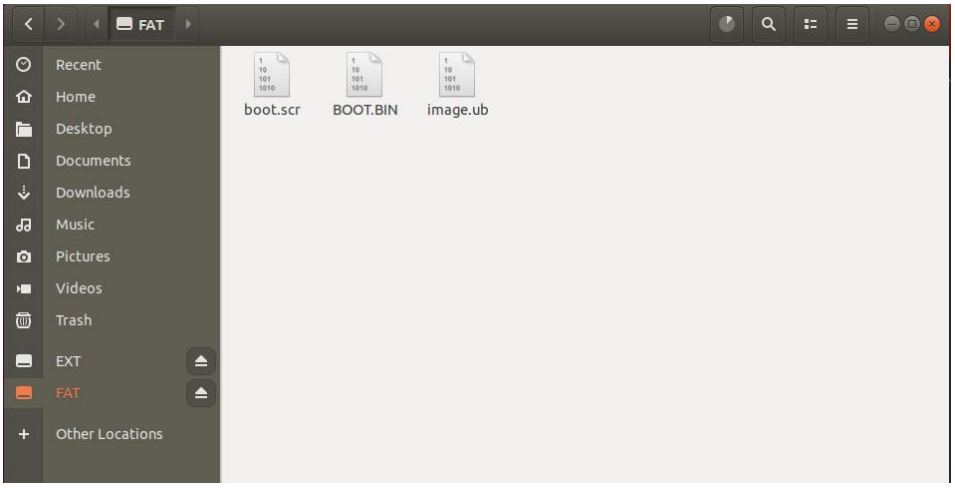
```
sudo tar -zxvf <petalinux path>/images/linux/rootfs.tar.gz -C /media/alinx/EXT
```

```
bash: petalinux: No such file or directory
alinx@ubuntu:~$ sudo tar -zxvf /home/alinx/peta_prj/linuxPsBase/petalinux/ima
ges/linux/rootfs.tar.gz -C /media/alinx/EXT
```

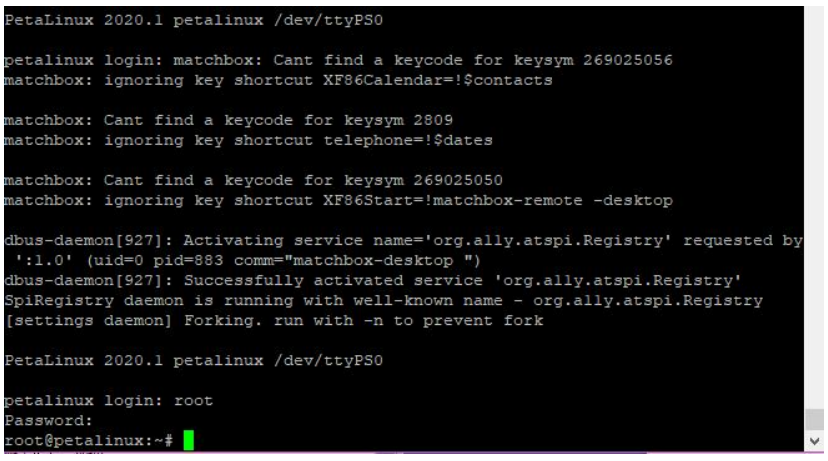


2) 在命令窗口输入"sudo sync"同步缓存数据，同步可能需要比较长的时间。

3) 把 BOOT.bin、boot.scr 和 image.ub 复制到 sd 的 FAT 分区中。



4) 设置开发板为 sd 模式启动，启动开发板。此时的系统就是重启不会丢失应用和数据的系统了。



第九章 Vitis 加速基本平台创建

注意：本教程需要熟练使用 vivado 和 petalinux，同时推荐学习 OpenCL、HLS 后再尝试 Vitis 加速。

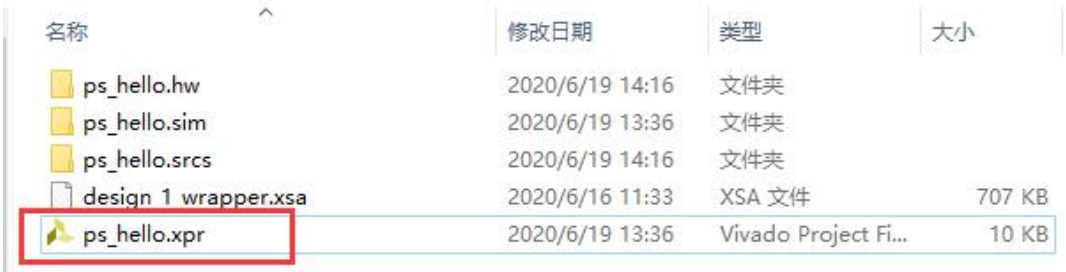
本章参考 xilinx 官方例程：https://github.com/Xilinx/Vitis-Tutorials/blob/2020.1/Vitis_Platform_Creation/Introduction/02-Edge-AI-ZCU104/step1.md#start-of-content。

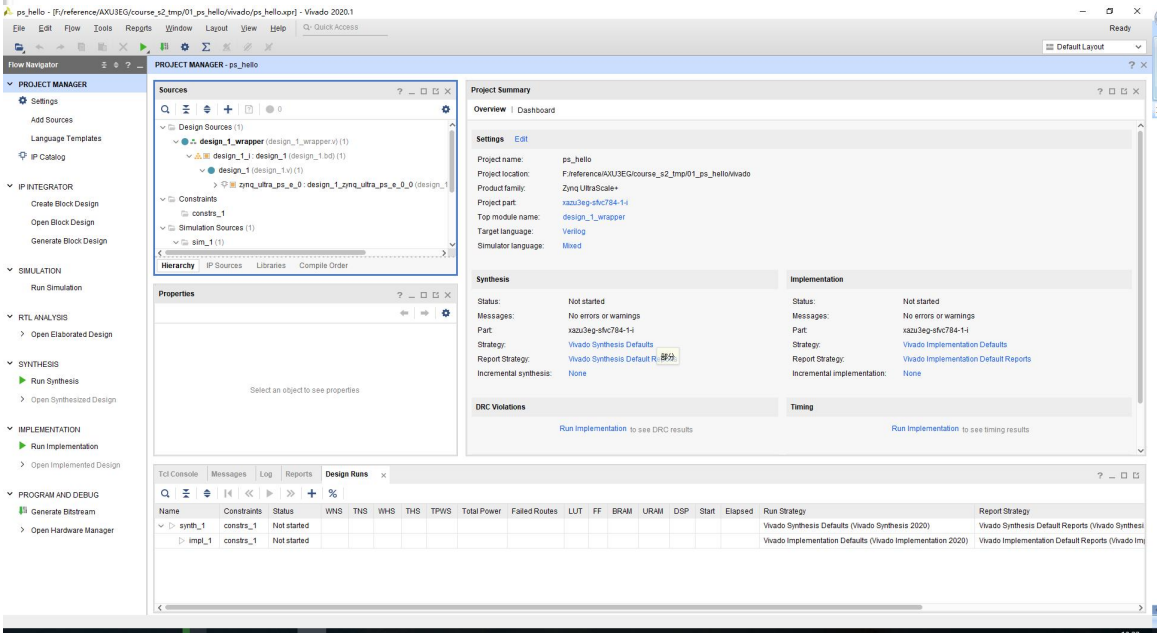
本章主要介绍如何定制使用 PL 端 FPGA 加速的软硬件平台，需要在 Vivado 工程里特殊处理才能用来加速。加速需要了解 OpenCL 和 HLS。

9.1 Vivado 硬件平台

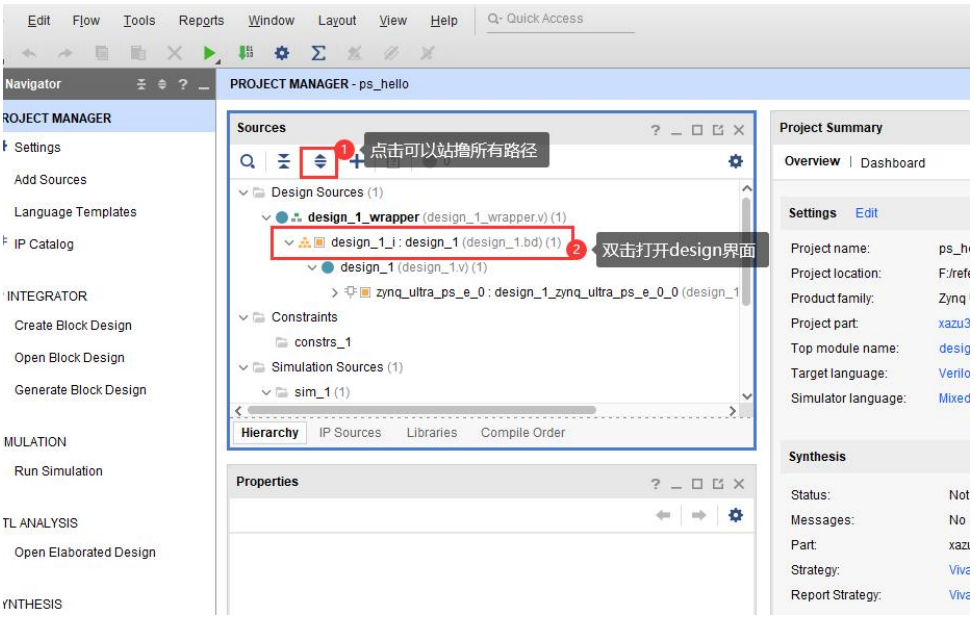
vivado 工程我们在名为“ps_hello”的 vivado 工程（使用“ALINX_ZYNQ_MPSoC 开发平台 Vitis 应用教程”教程对应源码包中的或者本教程源码包中的都可以）基础上修改。

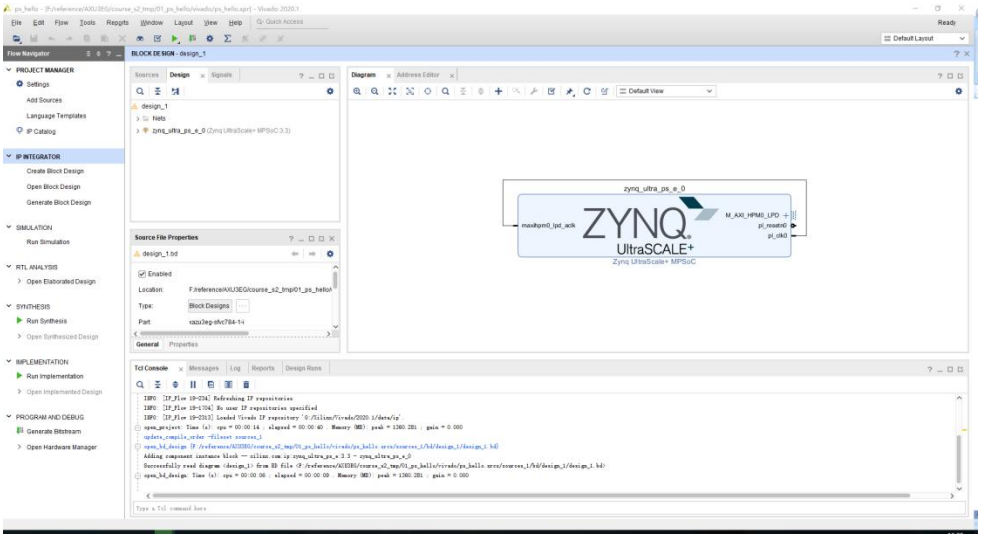
1. 打开 vivado 工程



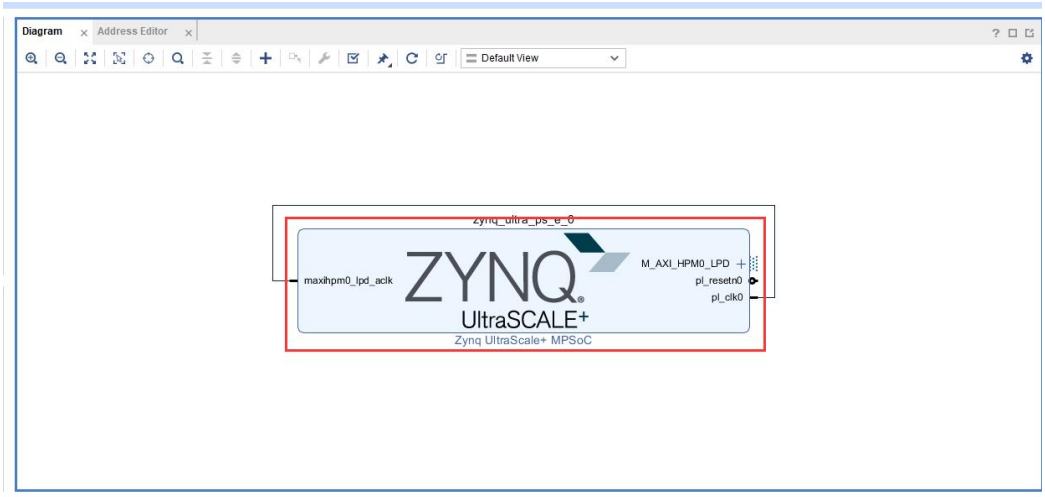


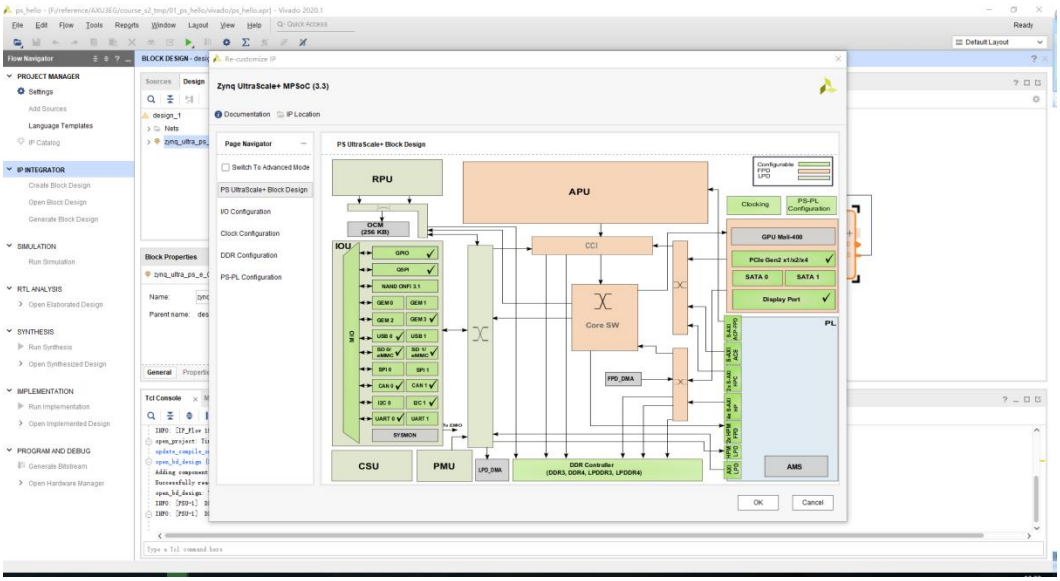
2. 展开 Source 窗口中的所有路径，然后双击“design_1_i:design_1(design_1.bd)”标签打开 design 界面



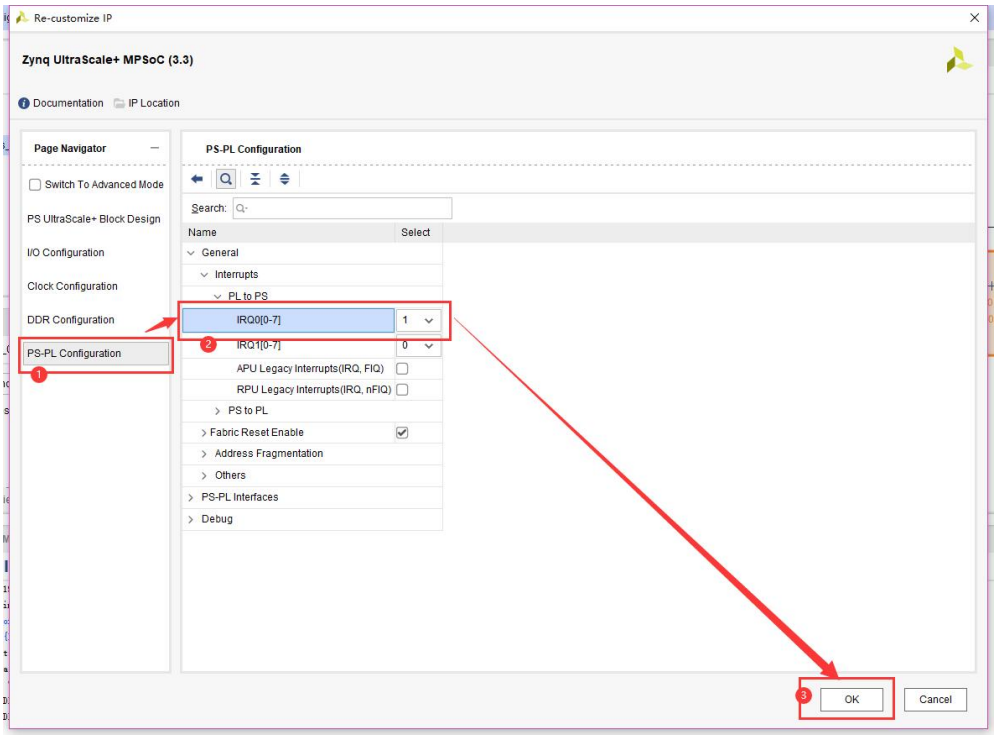


3. 双击“Diagram”界面中的“zynq_ultra_ps_e_0”,打开 ps 端的界面

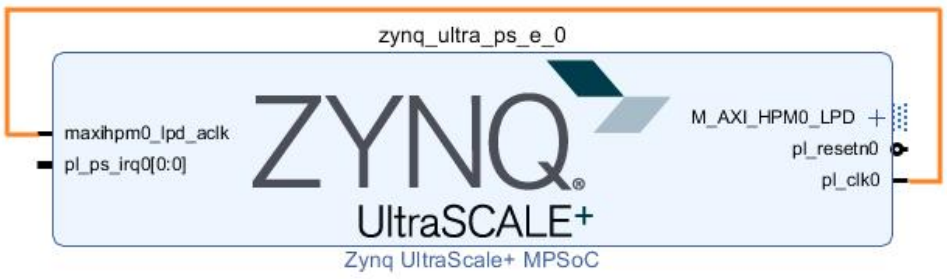




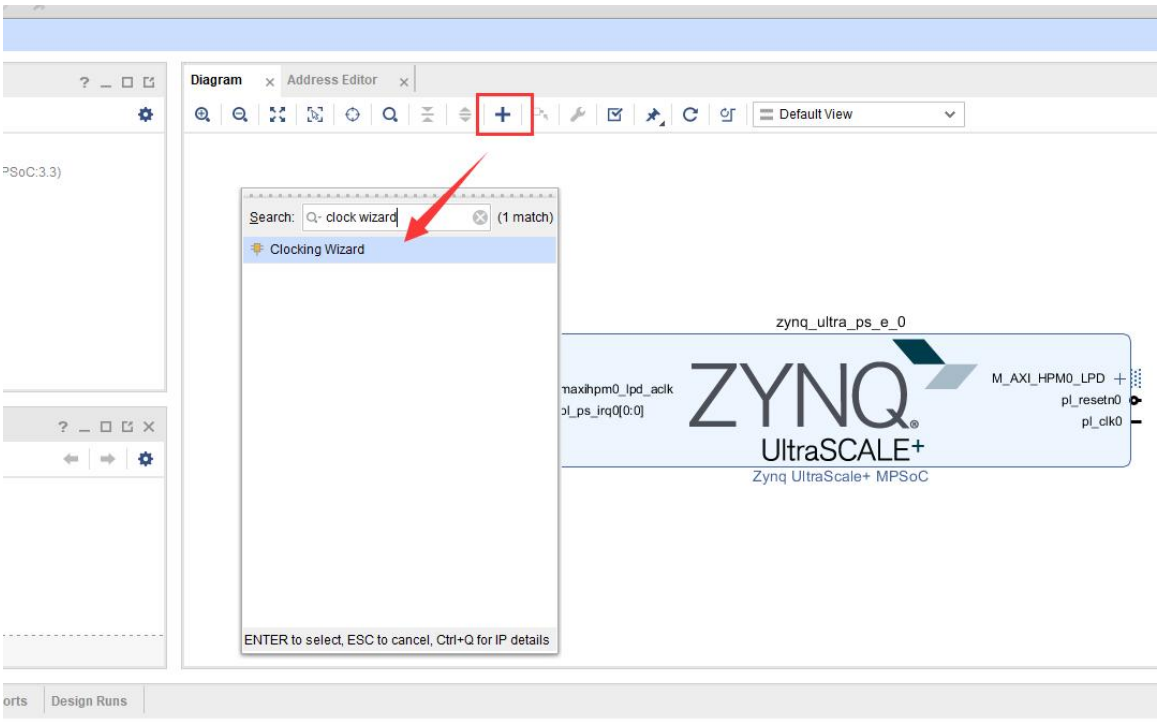
4. 选择“PS-PL Configuration”标签，把“General > Interrupt > PL to PS > IRQ0[0-7]”标签后的值改成 1，然后点 OK



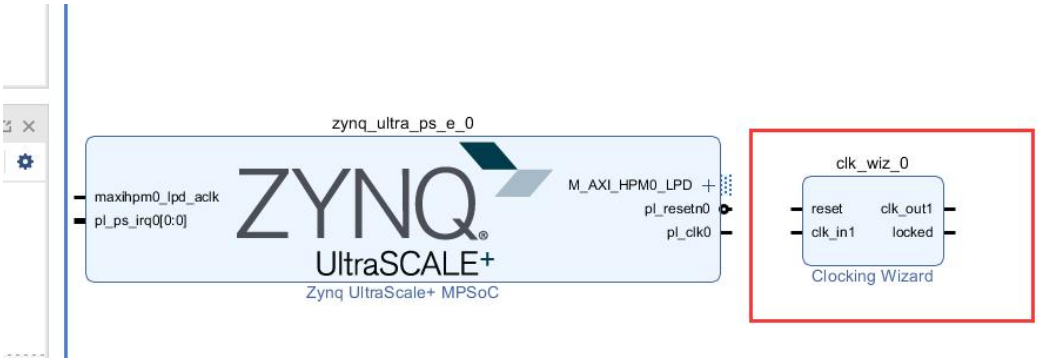
5. 把下图中的连线先删掉，选中连线然后按键盘上的“Del”键



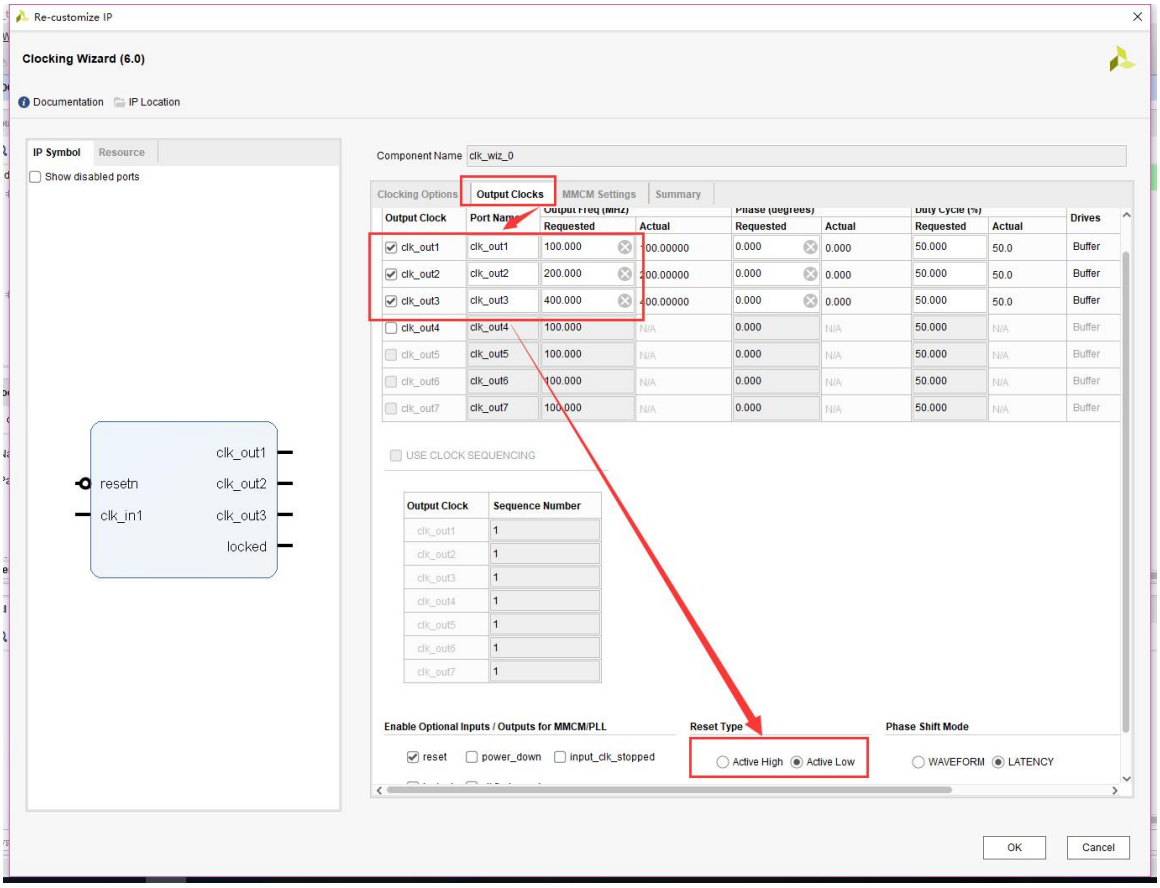
6. 添加一个时钟 ip



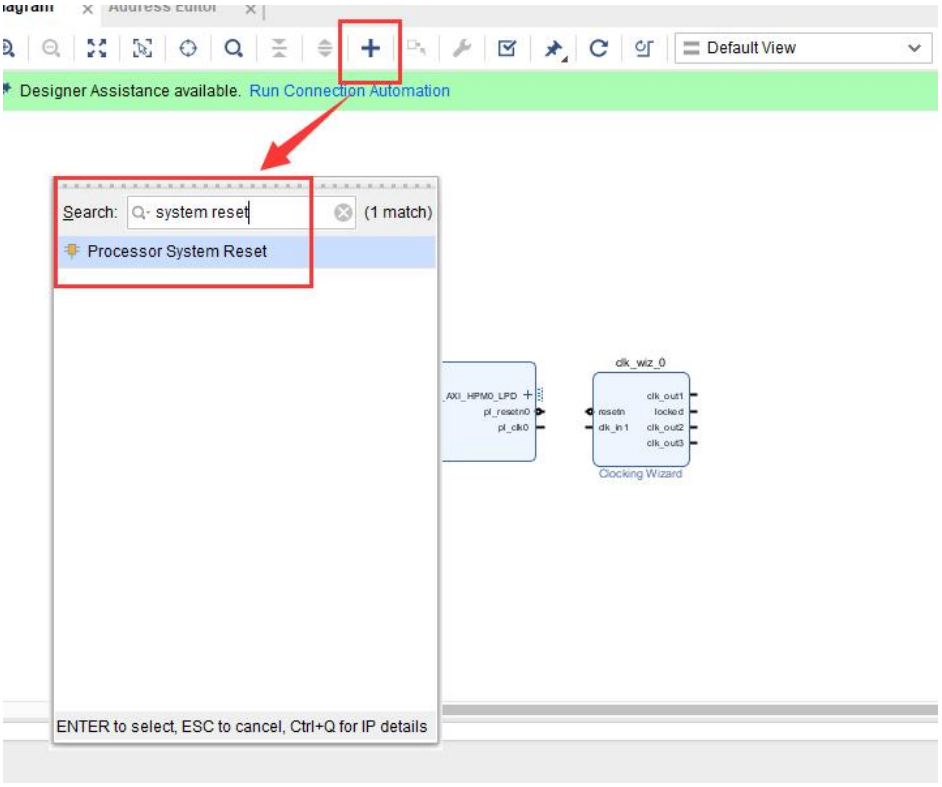
然后双击这个 ip，打开 ip 的配置界面

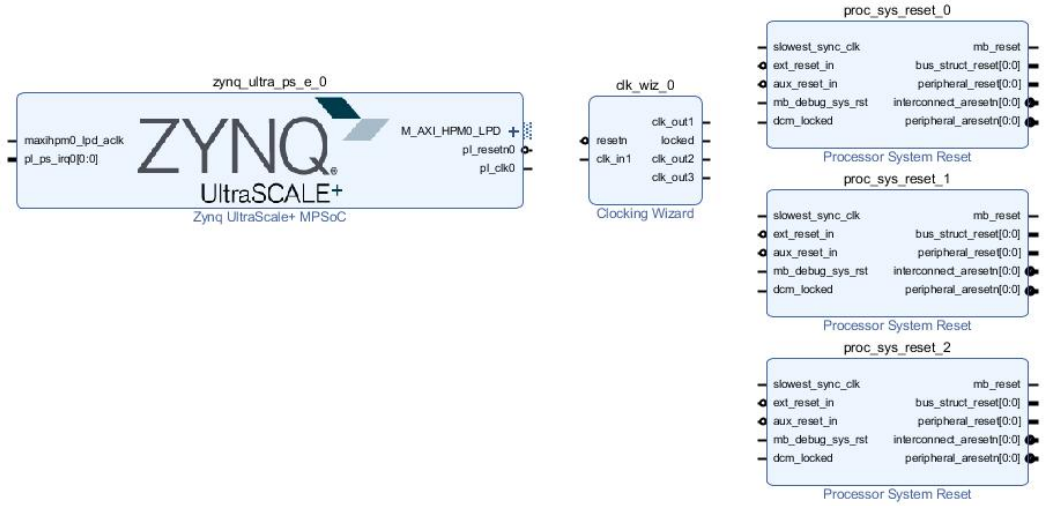


按照下图配置好，然后点击 ok 按钮

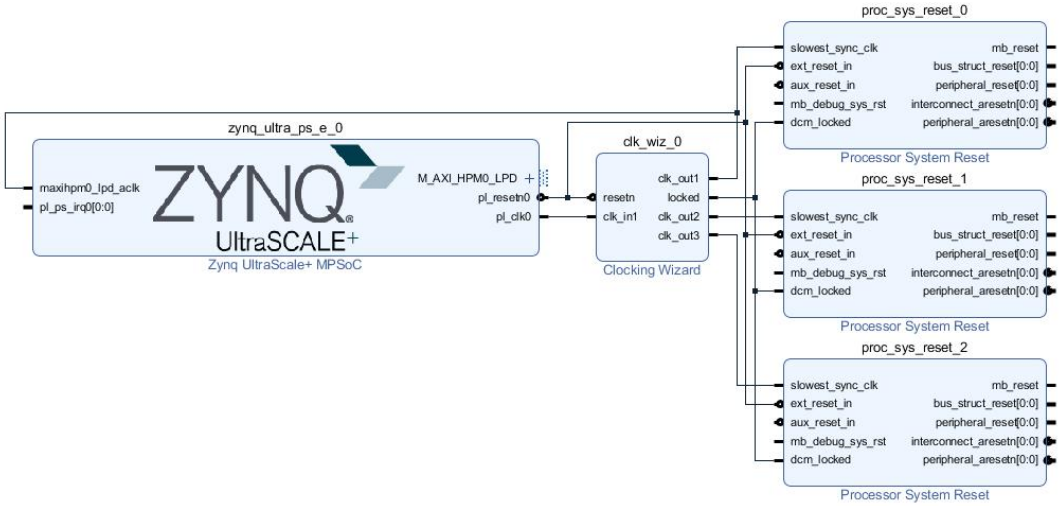


7. 添加三个“Processor System Reset” ip

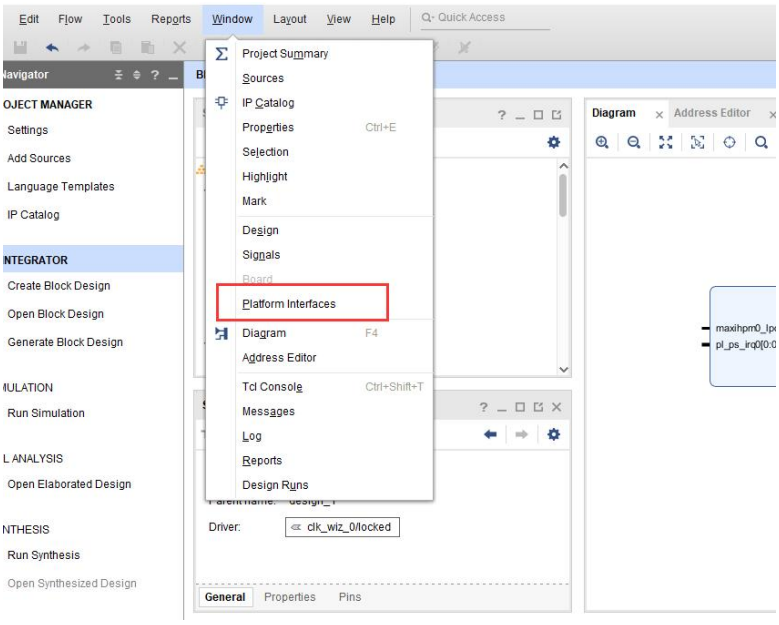




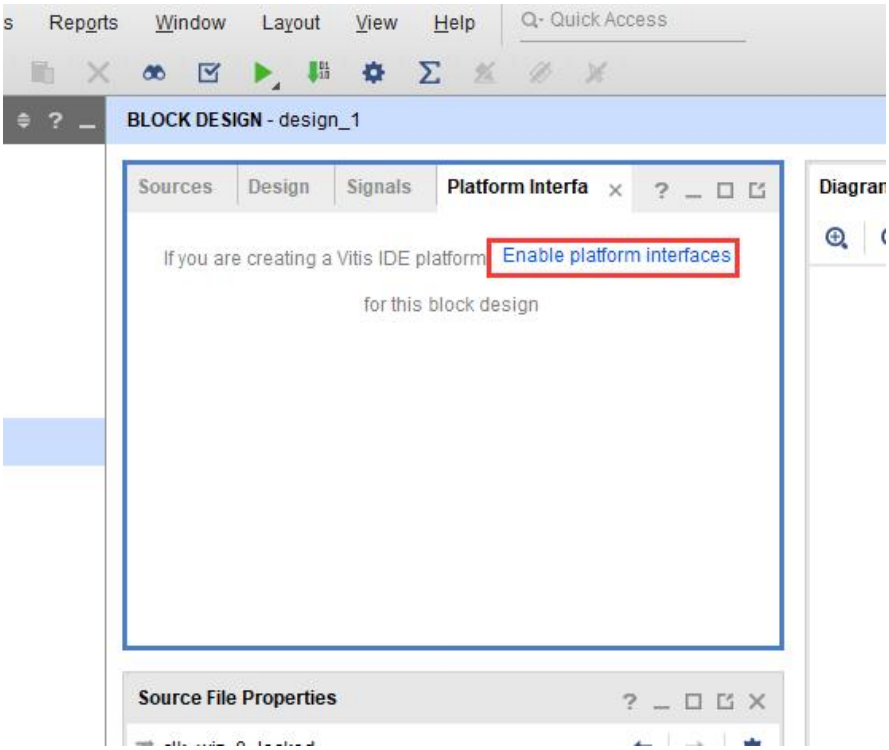
8. 按下图连线



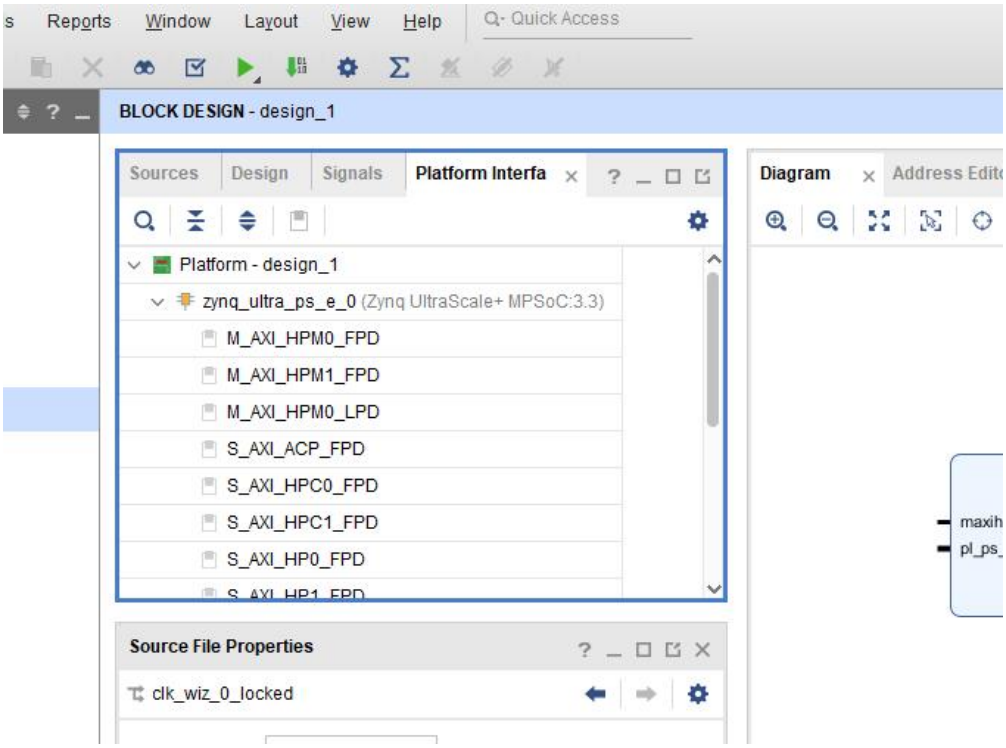
9. 点击菜单栏上的“Window”按钮，选择“Platform Interfaces”选项



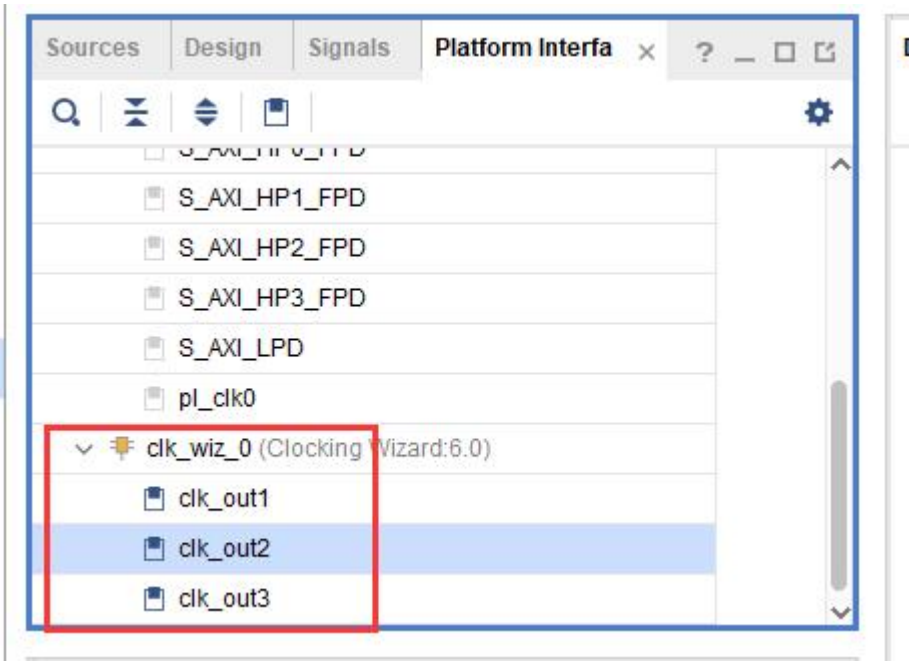
然后左上角的窗口会变成下图的状态，点击里面的蓝色字样



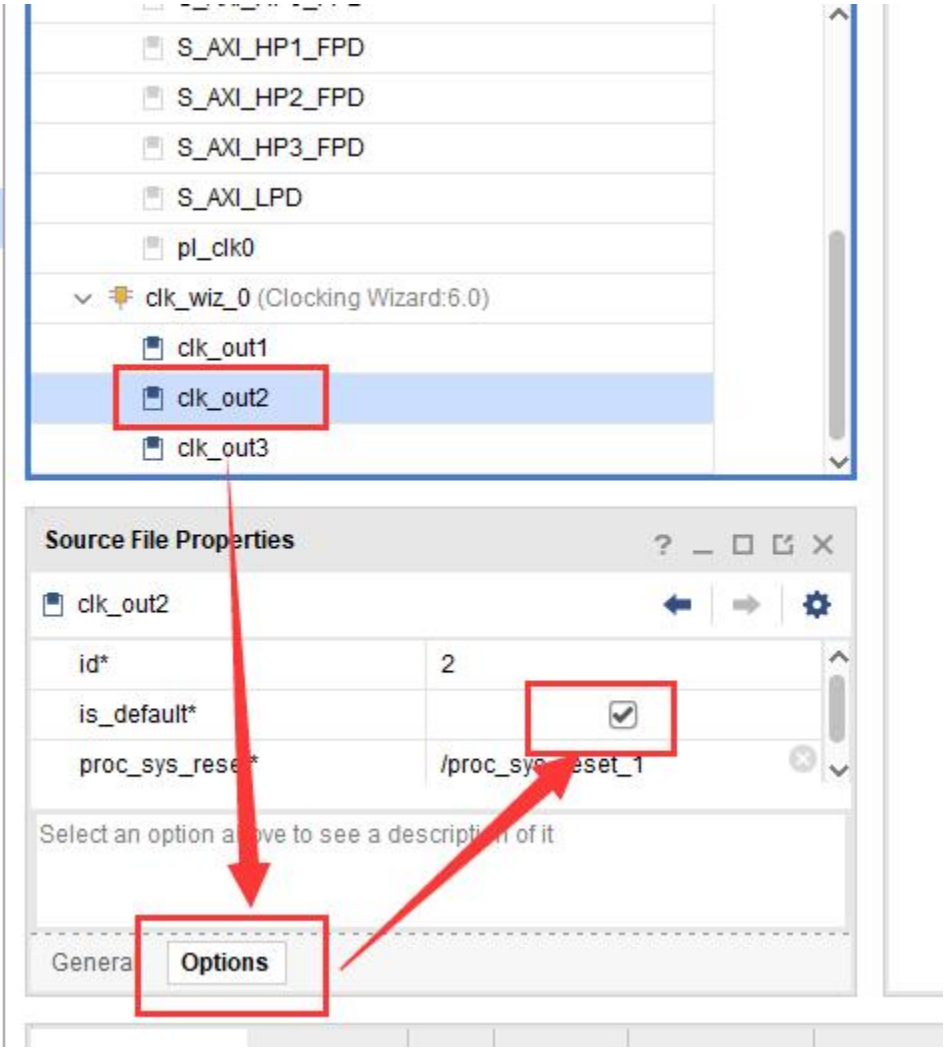
之后这个窗口会变成下图的状态



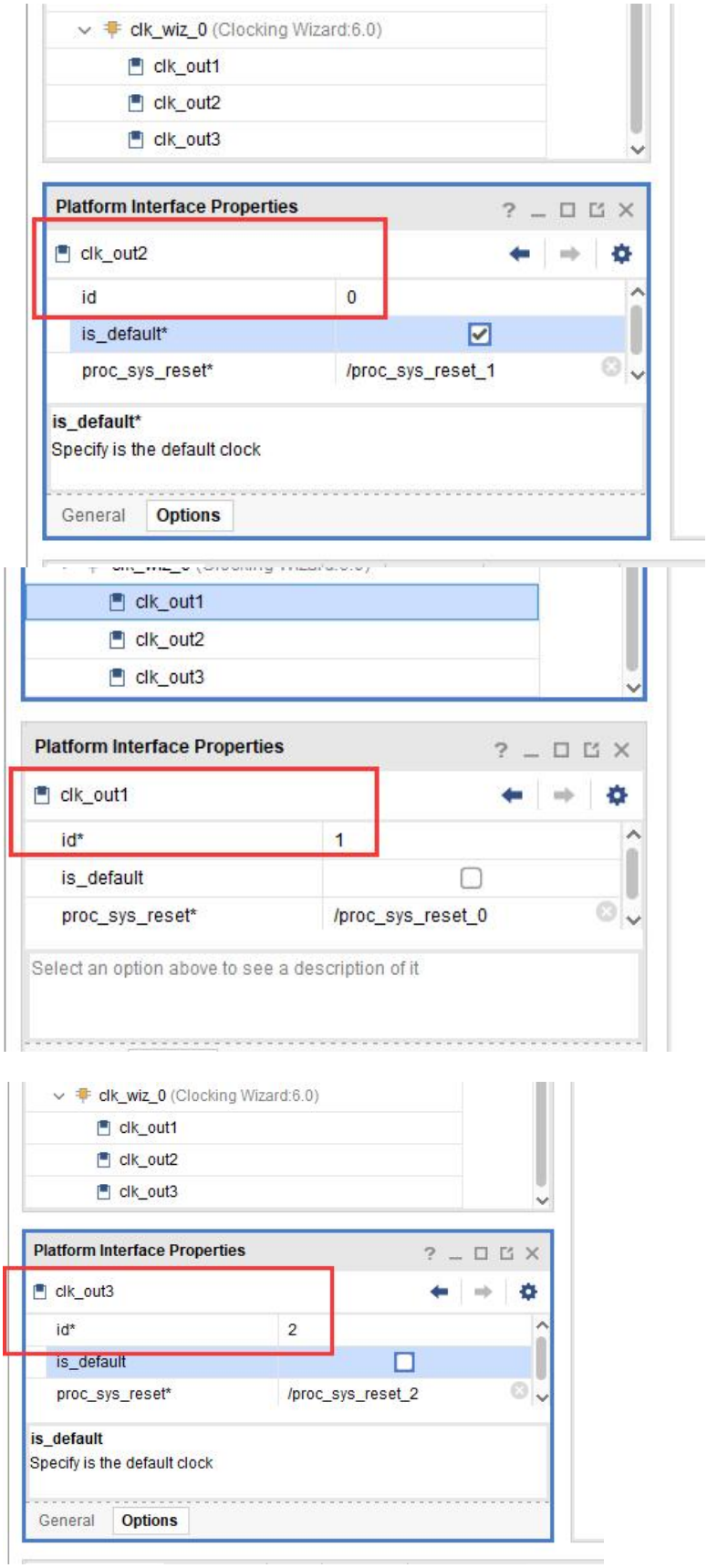
往下翻找到“clk_wiz_0”,然后双击里面的“clk_out1”、“clk_out2”、“clk_out3”使能这些时钟，前面的图标会从灰色变成蓝色



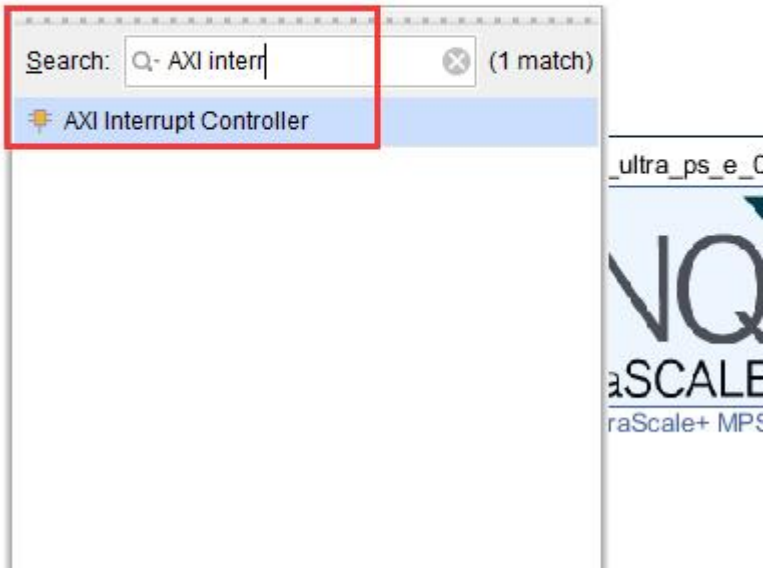
然后点击“clk_out2”，在下面的“Source File Properties”窗口中把“clk_out2”设置成默认时钟，如下图



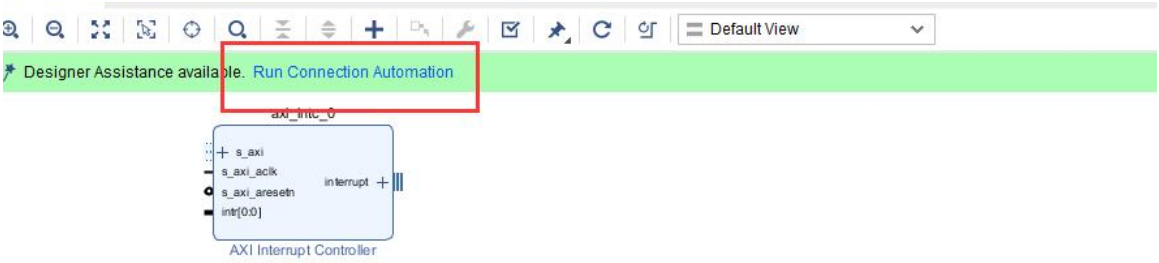
按照下图修改三个时钟的 id，“clk_out2”id 设置为 0，“clk_out1”id 设置为 1，“clk_out3”id 设置为 2。注意改好 id 数字之后一定要按回车确认。



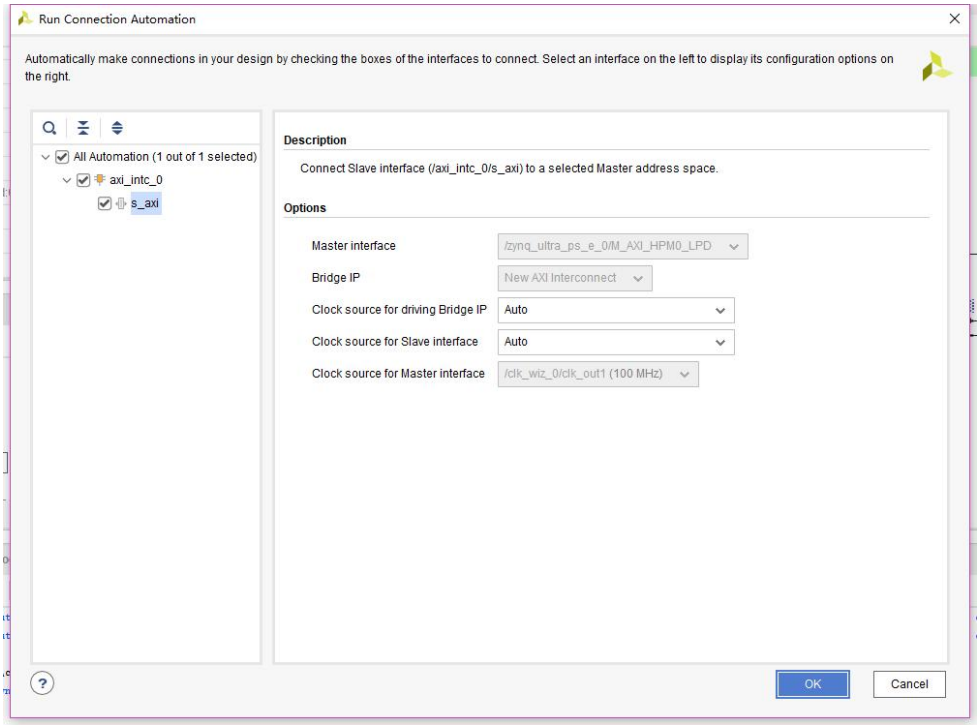
10. 添加一个中断控制器 IP



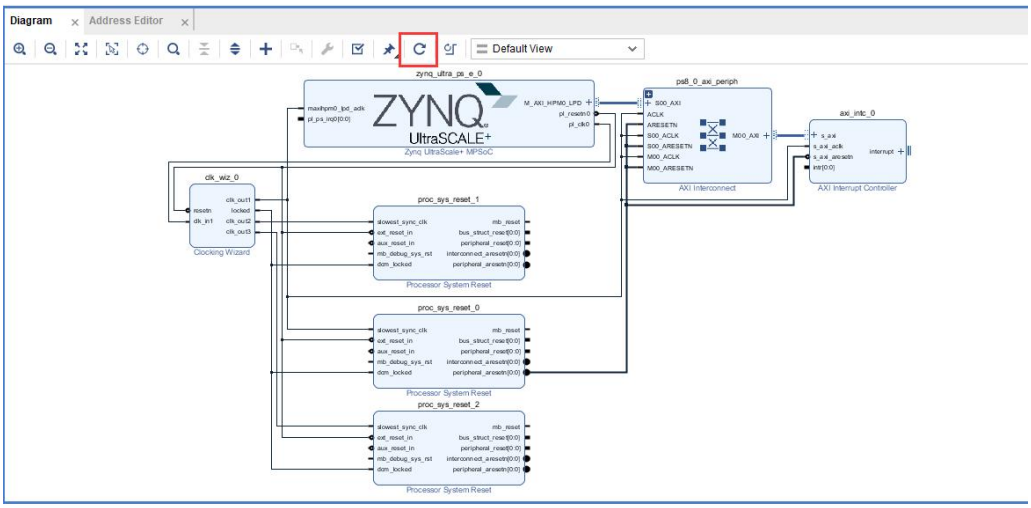
然后点击“Run Connection Automation”，自动布线



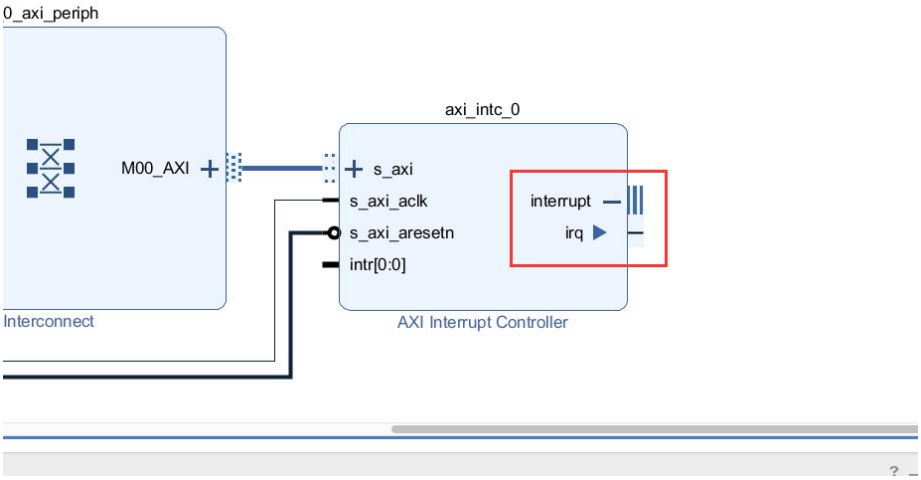
弹出这个界面直接点 ok



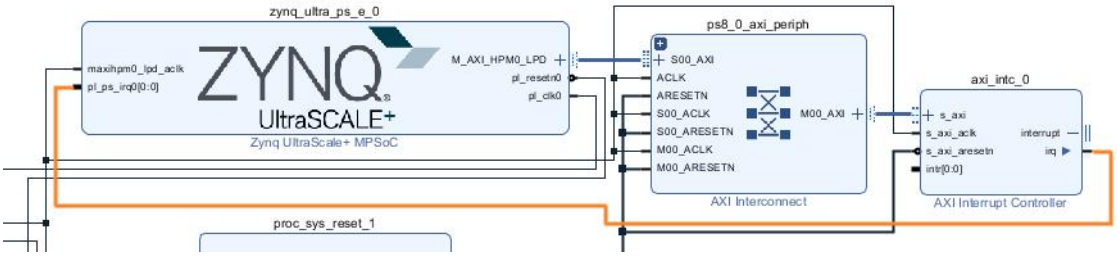
然后点一下整理布局的按钮，变成如下图的状态



展开终端控制器 IP 的“interrupt”脚

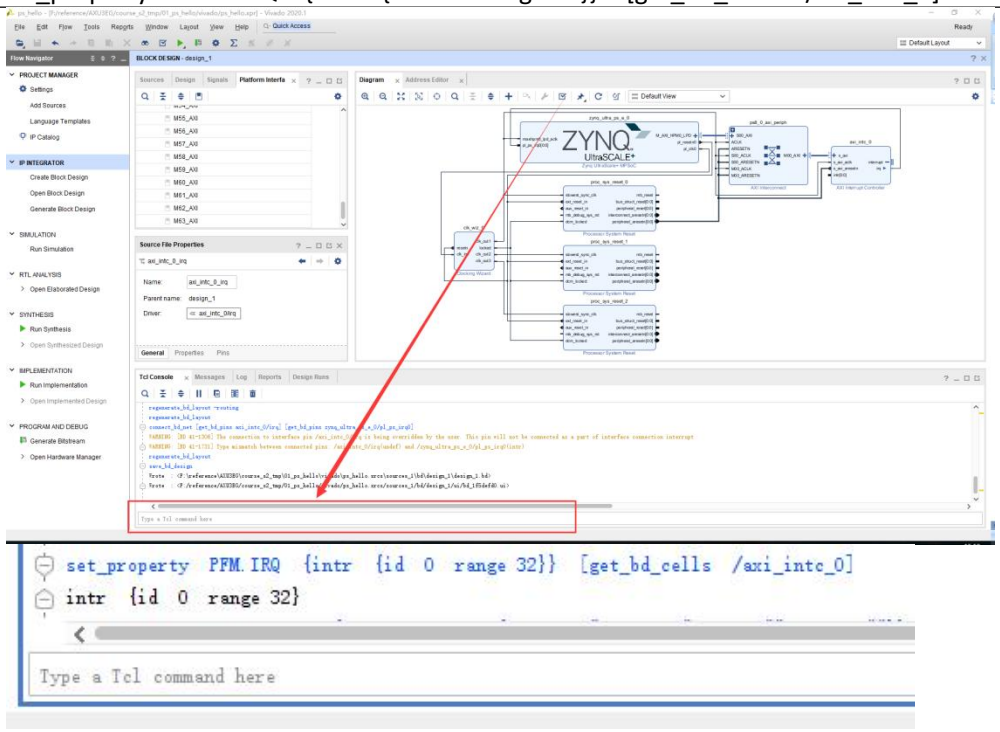


把展开出来的“irq”按下图连接，然后再整理一下布局

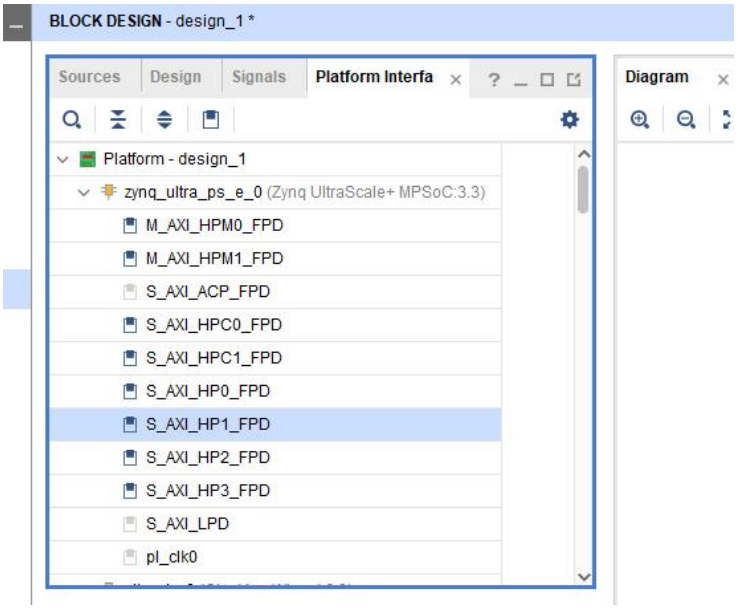


11. 在下方 Tcl 窗口中输入这条命令

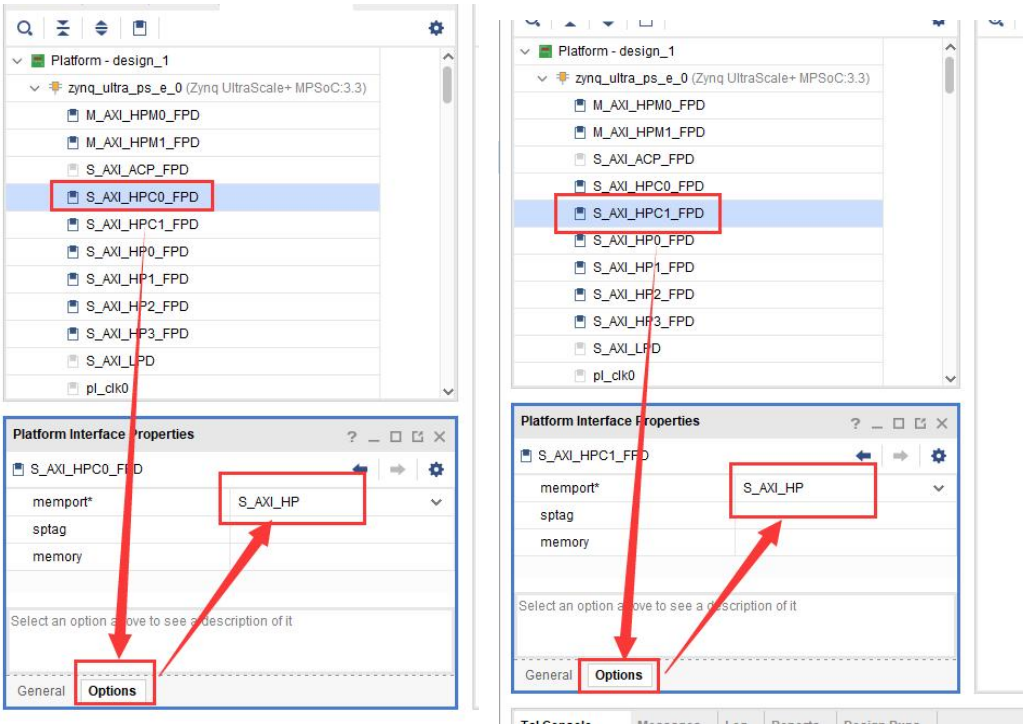
```
set_property PFM.IRQ {intr {id 0 range 32}} [get_bd_cells /axi_intc_0]
```



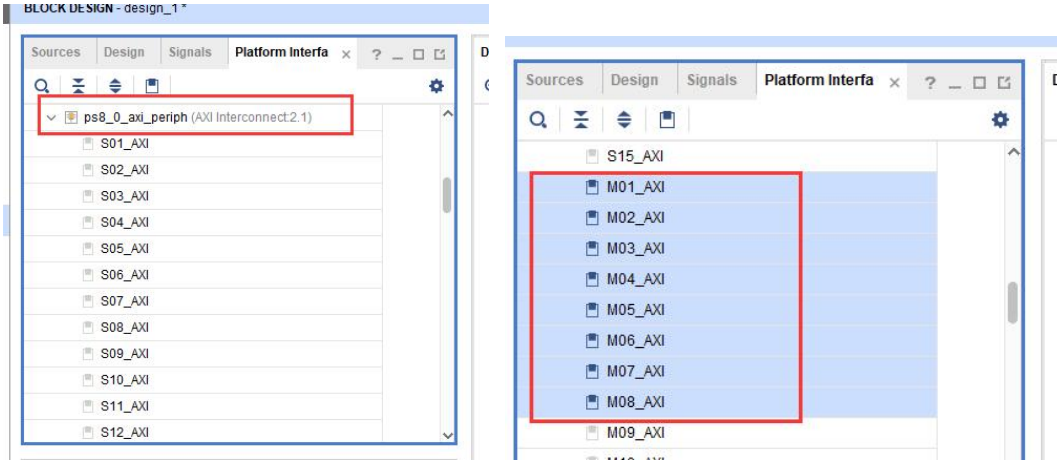
12. 然后回到“Platform”窗口中，按下图使能端口



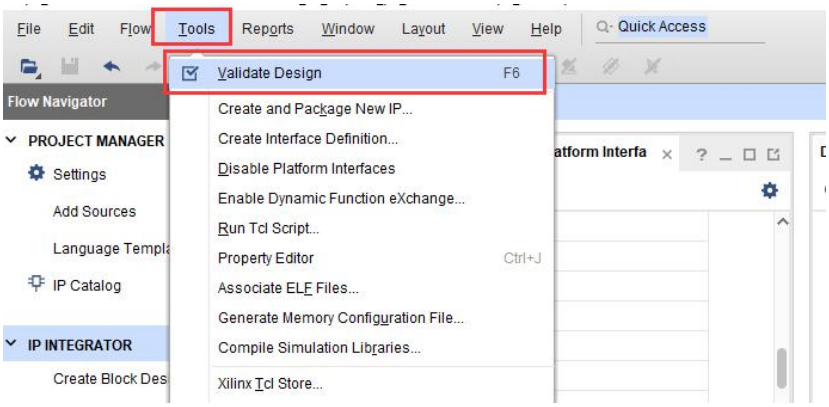
13. 修改“S_AXI_HPC0_FPD”和“S_AXI_HPC1_FPD”两个端口的“memport”为“S_AXI_HP”



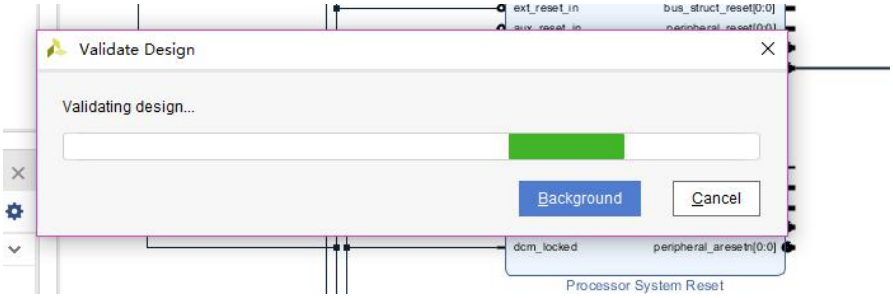
14. 使能“ps8_0_axi_periph(AXI Interconnect2.1)”中的“M01_AXI”到“M08_AXI”，然后按“Crtl + S”保存一下修改



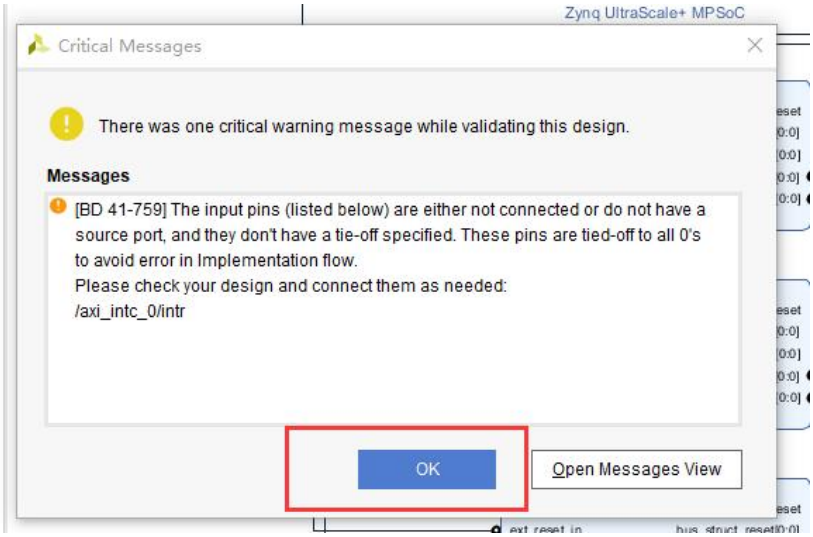
15. 点击菜单栏中的“Tools”，选择第一个选项“Validate Design”



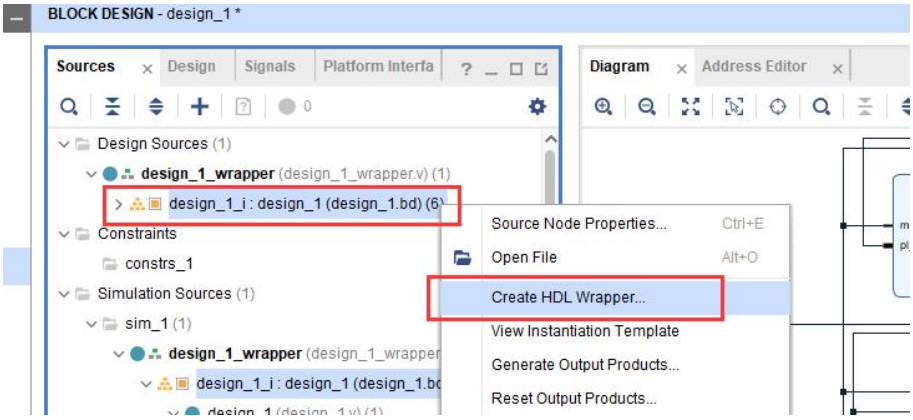
等待完成



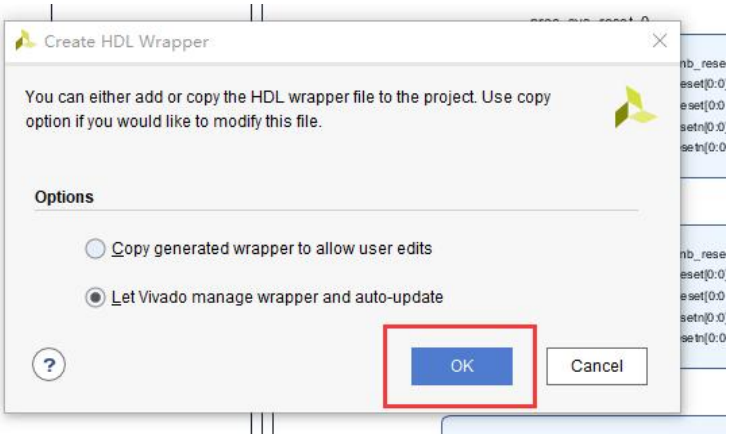
弹出下图的界面，有一个警告不用管，点击 OK



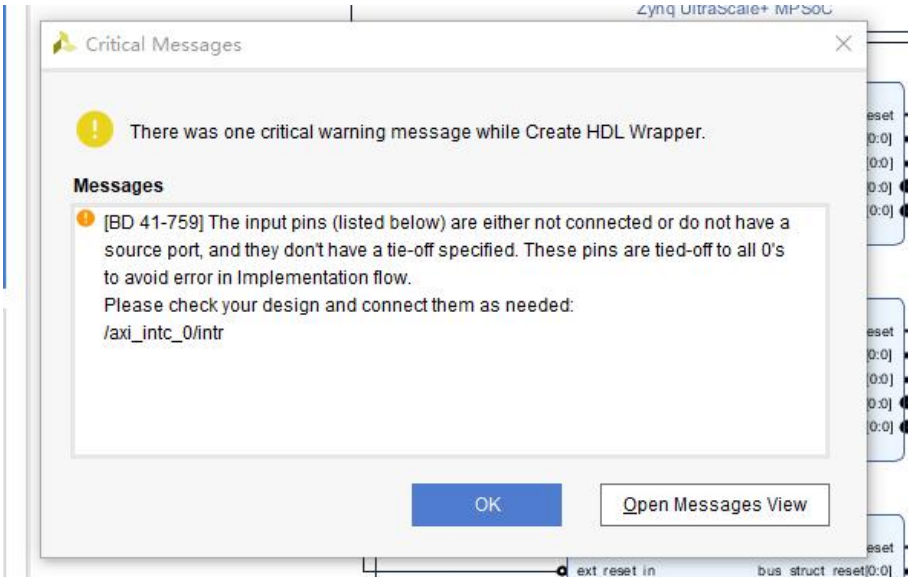
16. 在“Source”窗口中，右击“design_1_i:design_1(design_1.bd)”，选择“Create HDL Wrapper”选项



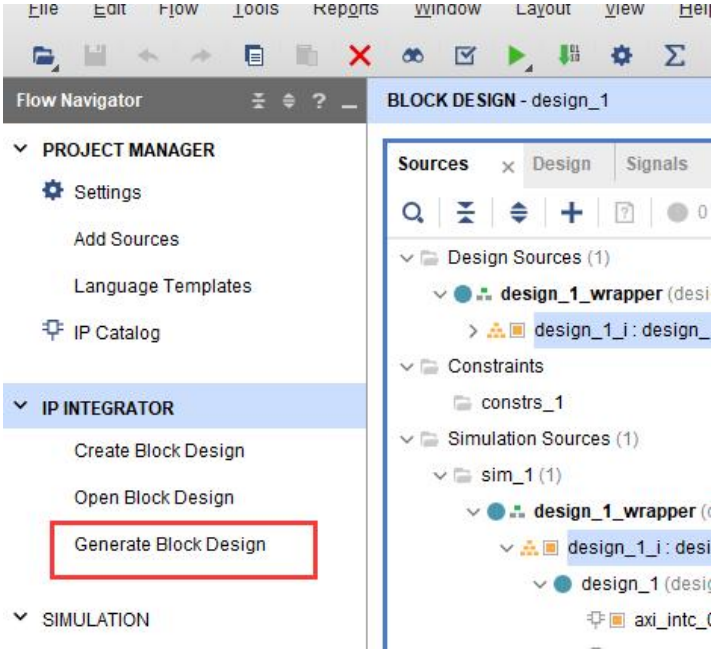
弹出下面的界面直接点 OK



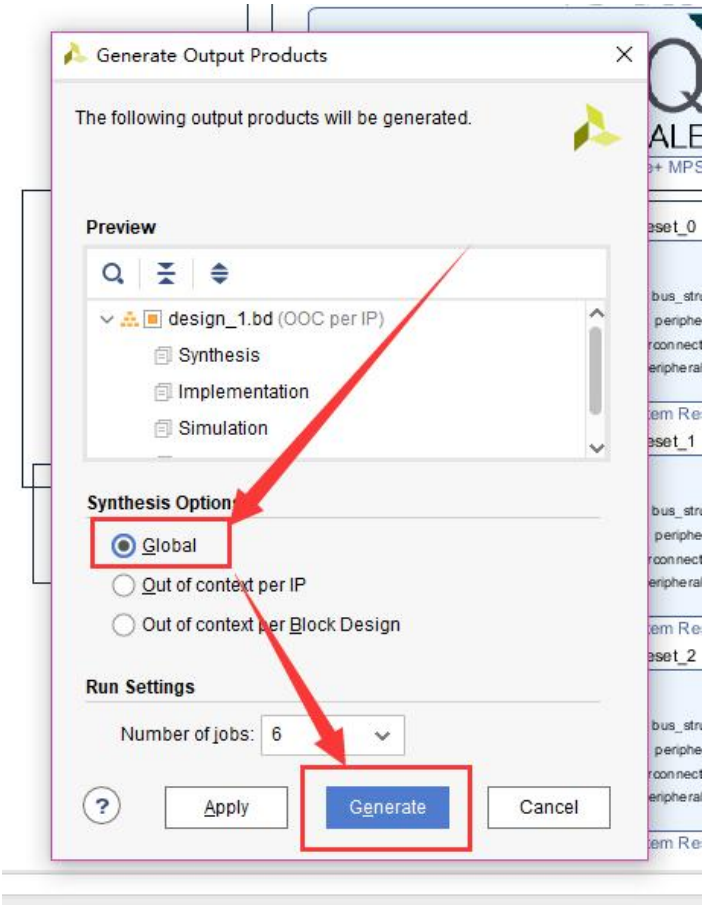
然后会弹出下面的弹窗，同样有这个警告不用管，直接点 OK



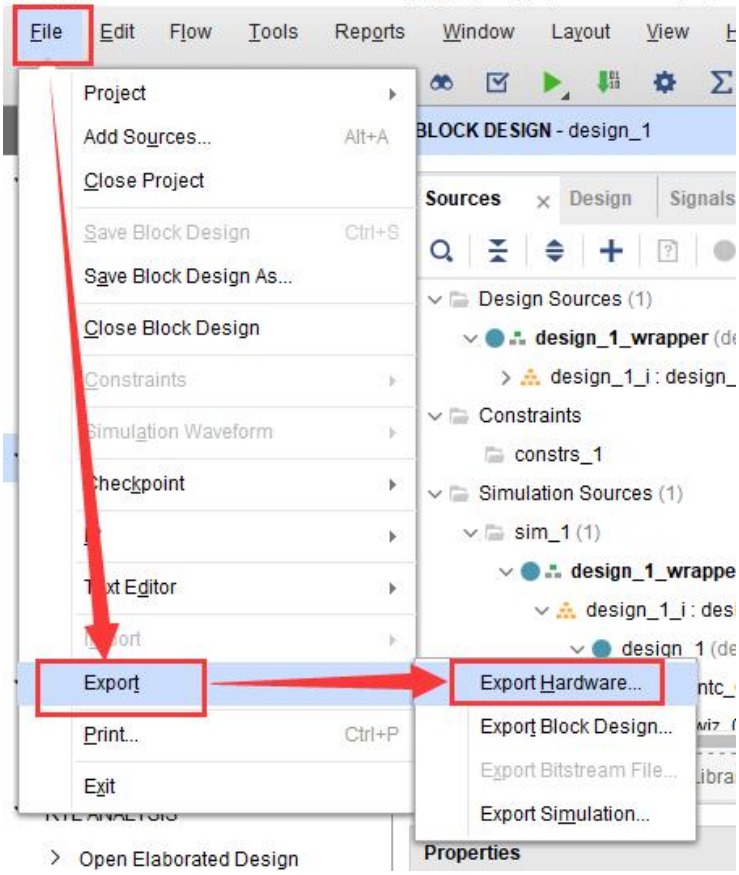
17. 点击左边菜单栏中的“Generate Block Design”



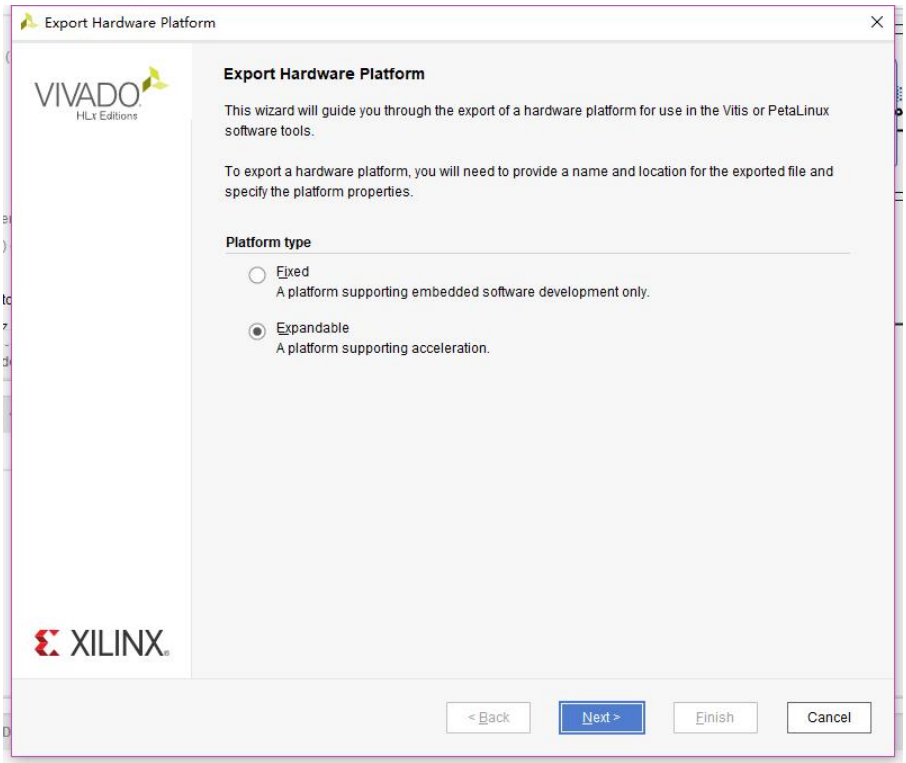
然后弹出下图的弹窗，线选择“Global”，然后点击“Generate”，完成后的弹窗，仍然直接点 OK，不用管警告。

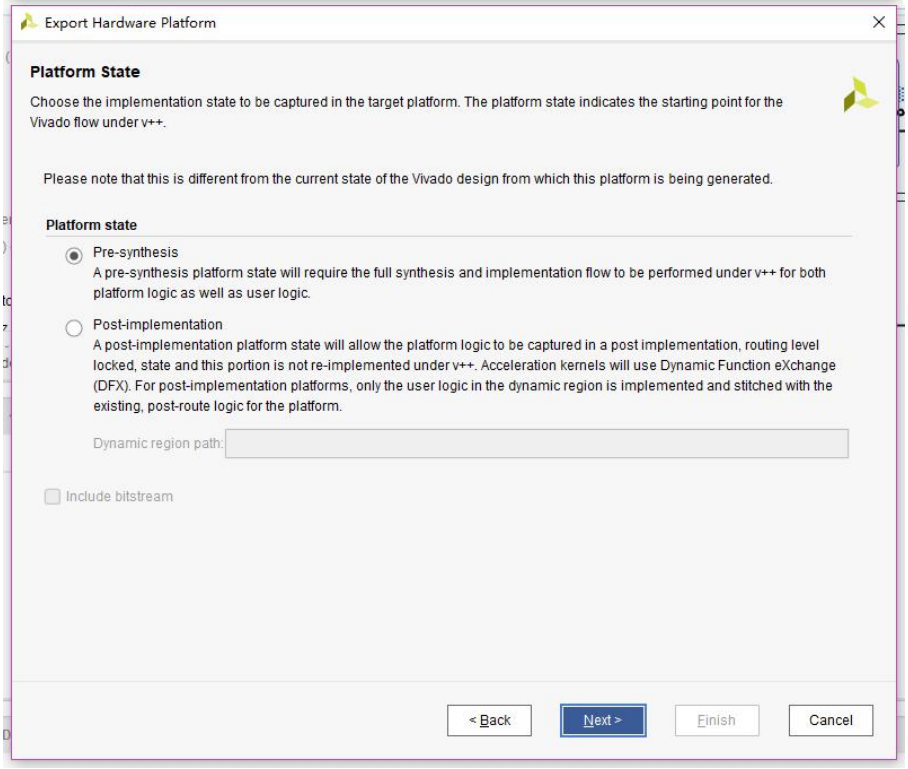


18. 点击菜单栏中的“File > Export > Exoprt Hardware”导出硬件描述文件

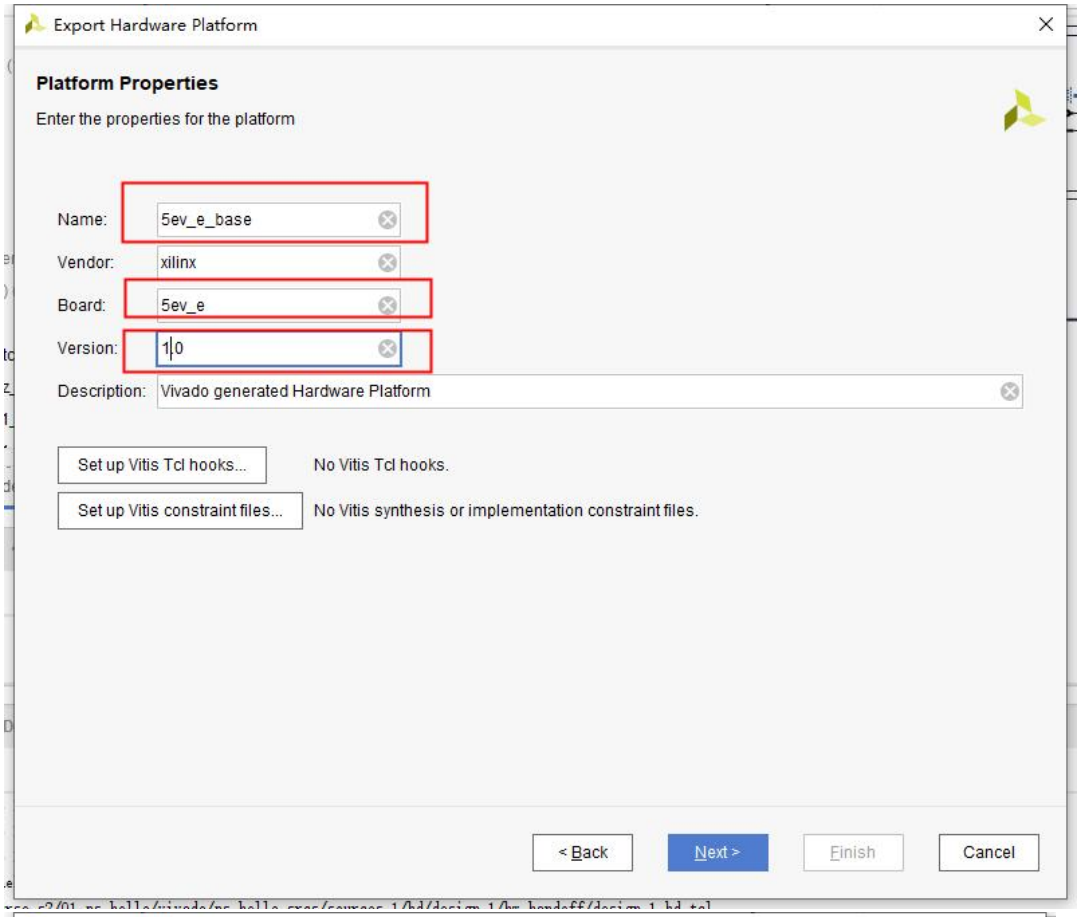


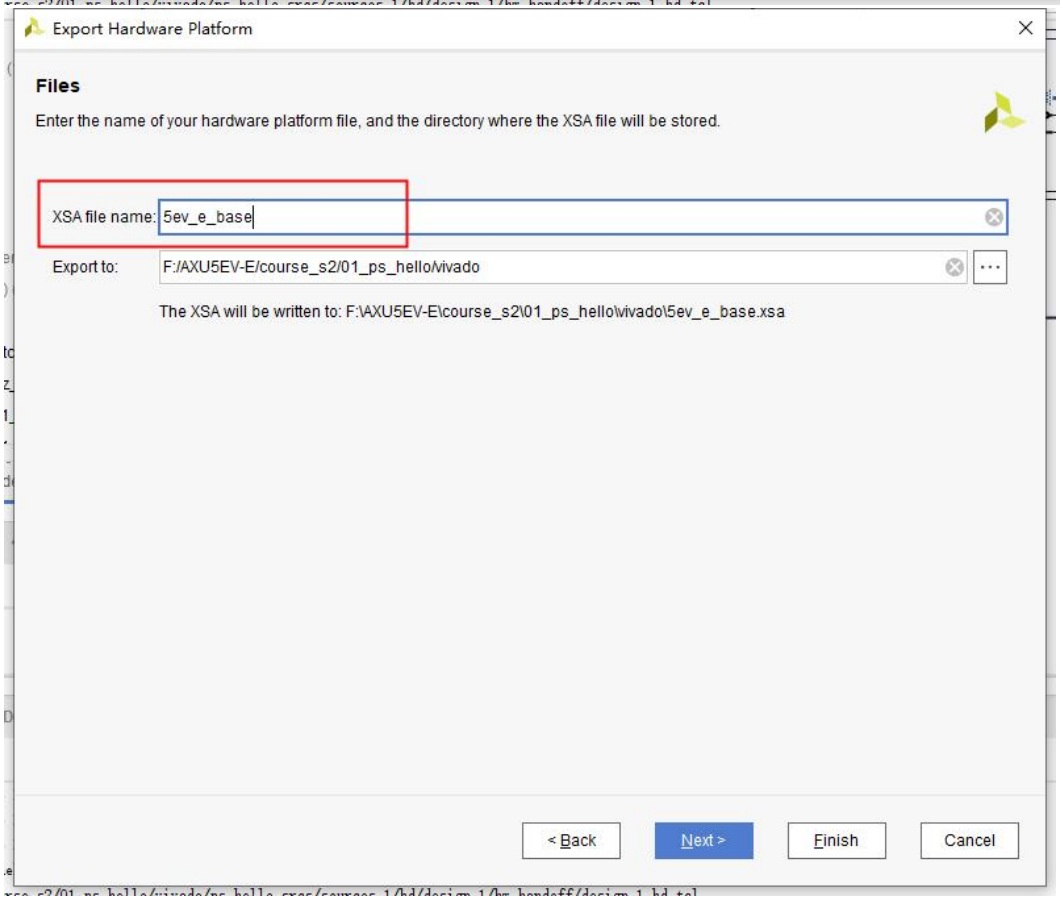
弹出下面的界面直接点“Next”



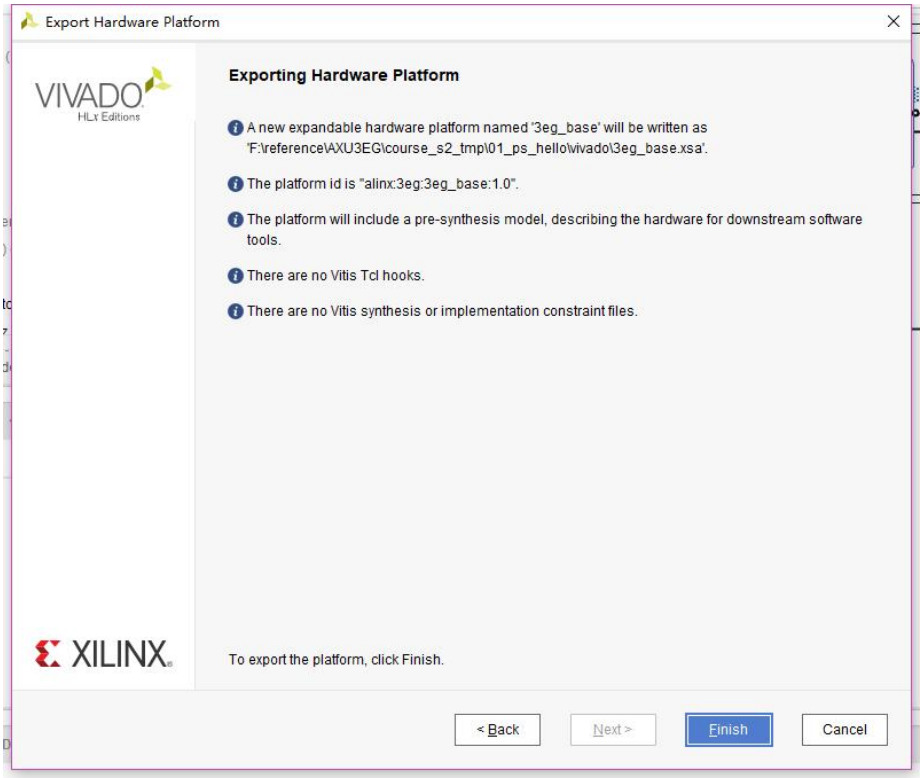


接下来的界面按下图修改后点击“Next”


Export Hardware Platform
Platform Properties
Enter the properties for the platform
Name: 5ev_e_base
Vendor: xilinx
Board: 5ev_e
Version: 1.0
Description: Vivado generated Hardware Platform
Set up Vitis Tcl hooks...
No Vitis Tcl hooks.
Set up Vitis constraint files...
No Vitis synthesis or implementation constraint files.
< Back
Next >
Finish
Cancel


Export Hardware Platform
Files
Enter the name of your hardware platform file, and the directory where the XSA file will be stored.
XSA file name: 5ev_e_base
Export to: F:/AXU5EV-E/course_s2/01_ps_hello/vivado
The XSA will be written to: F:\AXU5EV-E\course_s2\01_ps_hello\vivado\5ev_e_base.xsa
< Back
Next >
Finish
Cancel

最后点 finish

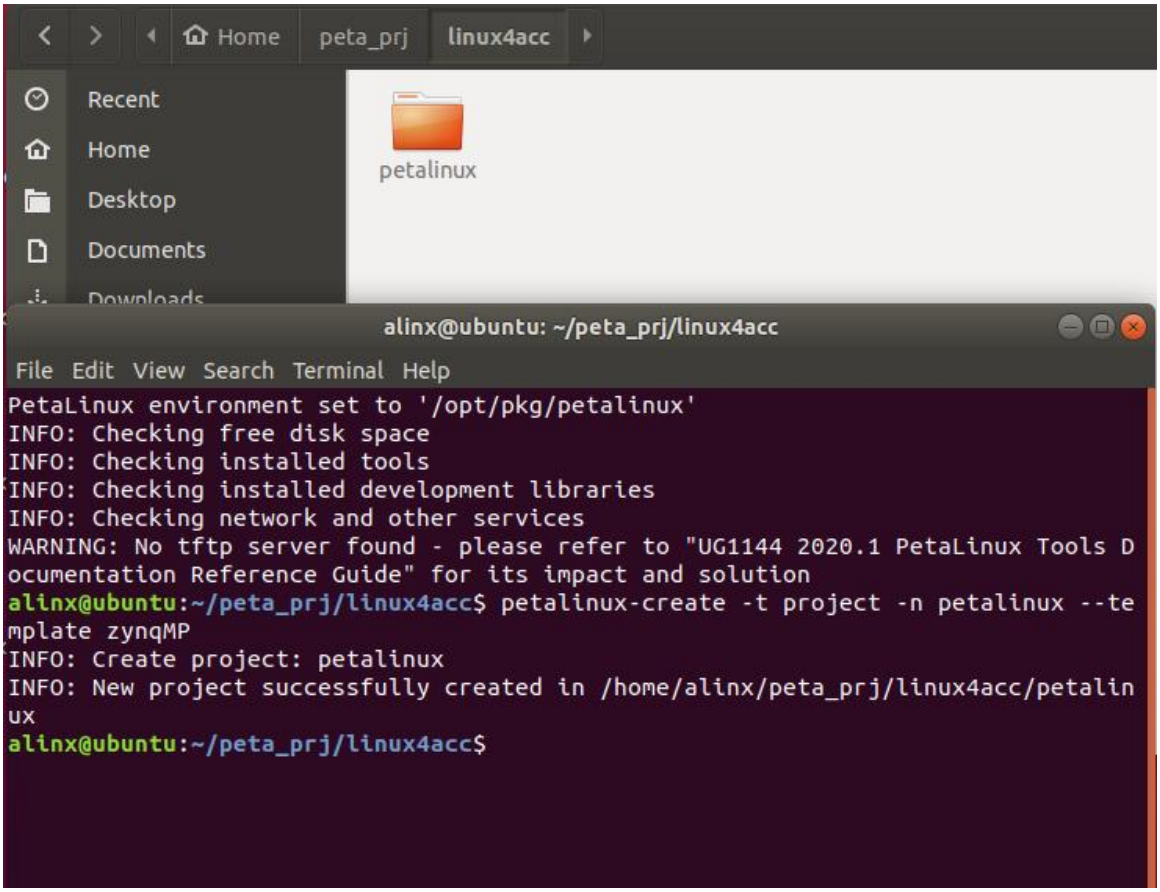


19. 导出完成之后在 vivado 工程路径中可以找到名为“5ev_e_base.xsa”的 xsa 文件，此时可以退出 Vivado 工程了。

9.2 petalinux 定制系统

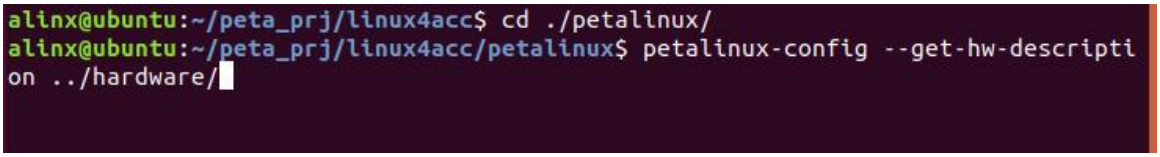
1. 新建一个路径“/home/alinx/peta_prj/linux4acc”，在这个路径中新建一个 petalinux 工程。

```
petalinux-create -t project -n petalinux --template zynqMP
```

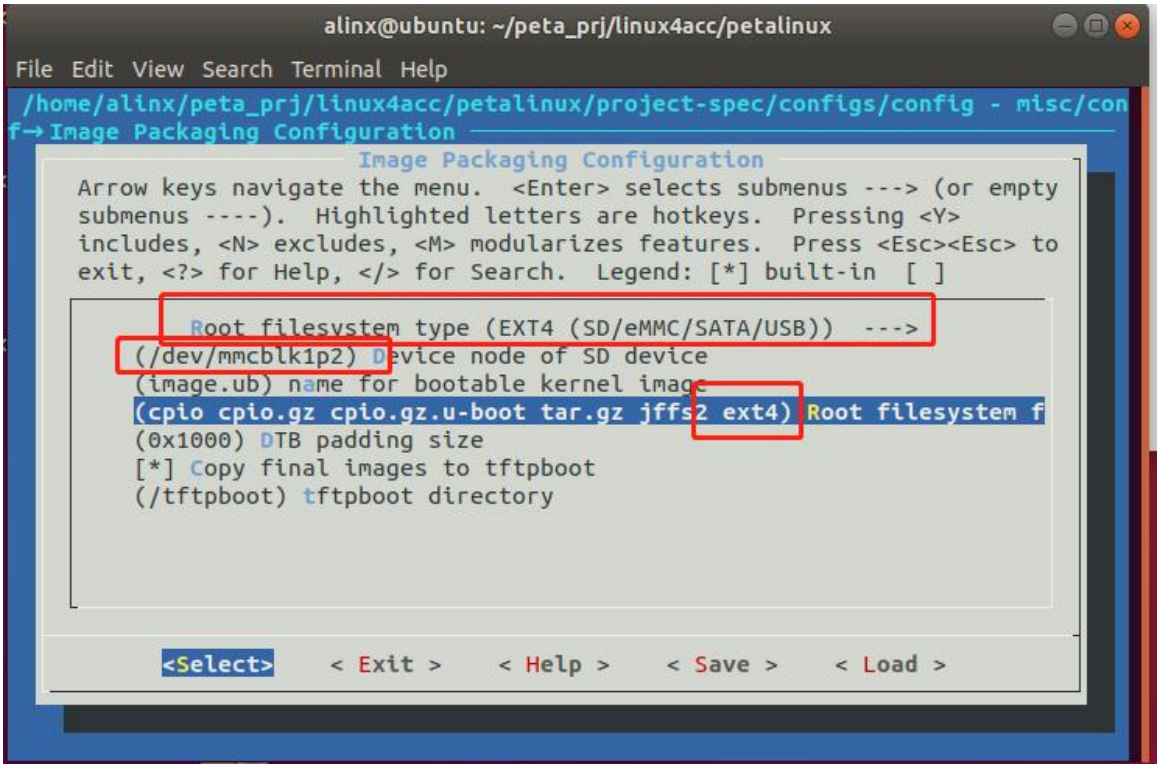


2. 在“/home/alinx/peta_prj/linux4acc”路径中新建一个“hardware”文件夹，把“9.1”节中制作的“5ev_e_base.xsa”拷贝过来。

3. 回到终端窗口，cd 进入 petalinux 工程路径，设置硬件描述文件。

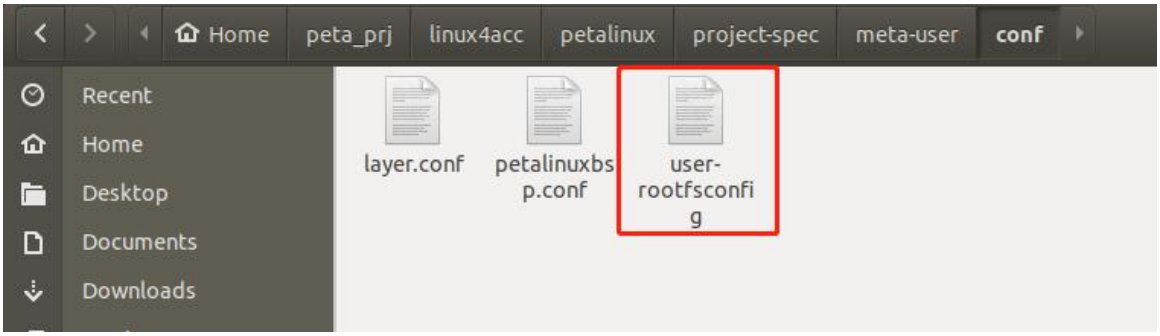


弹出配置界面后，设置成从 SD 卡启动，如下图。



有需要的话设置离线编译，参考第一章，这里就不重复操作了。然后保存配置退出配置界面。

4. 进入到 petalinux 工程的“project-spec/meta-user/conf”路径中，打开“user-rootfsconfig”文件。



修改成如下内容。

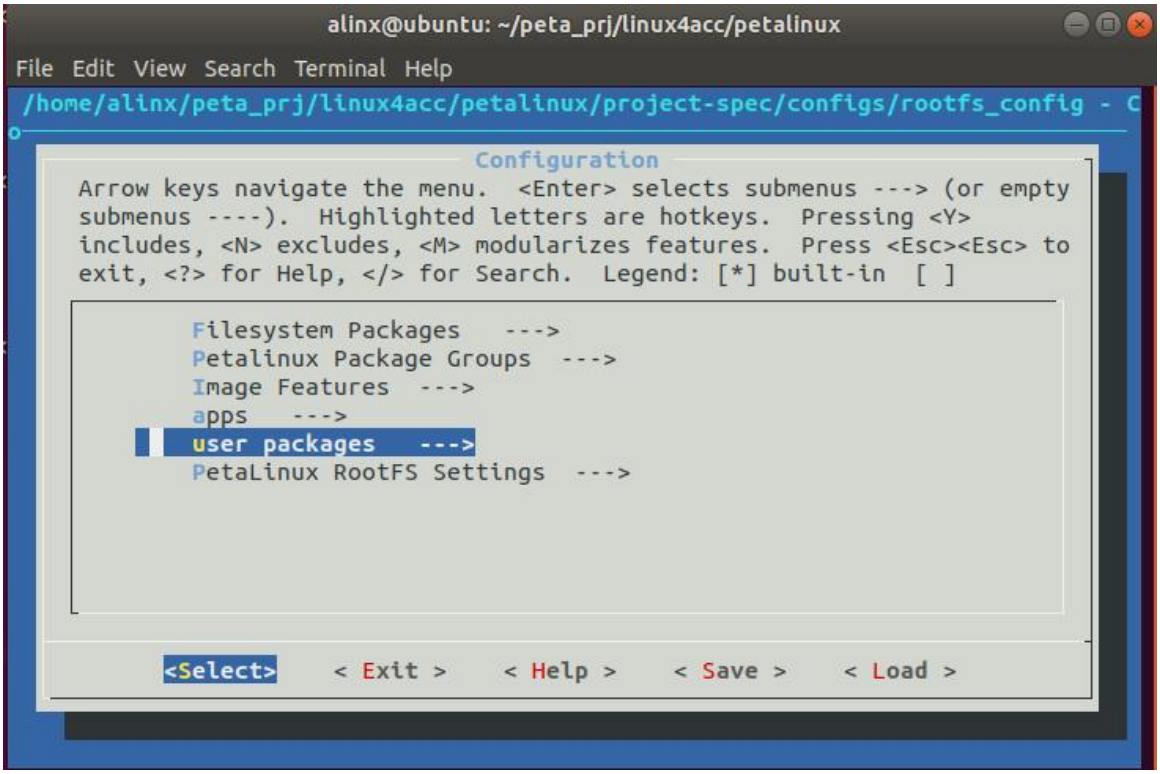
```
#Note: Mention Each package in individual line
#These packages will get added into rootfs menu entry

CONFIG_gpio-demo
CONFIG_peekpoke

#Packages for base XRT support:
CONFIG_packagegroup-petalinux-xrt
CONFIG_xrt-dev

#Packages for easy system management
CONFIG_dnf
CONFIG_e2fsprogs-resize2fs
CONFIG_parted
```

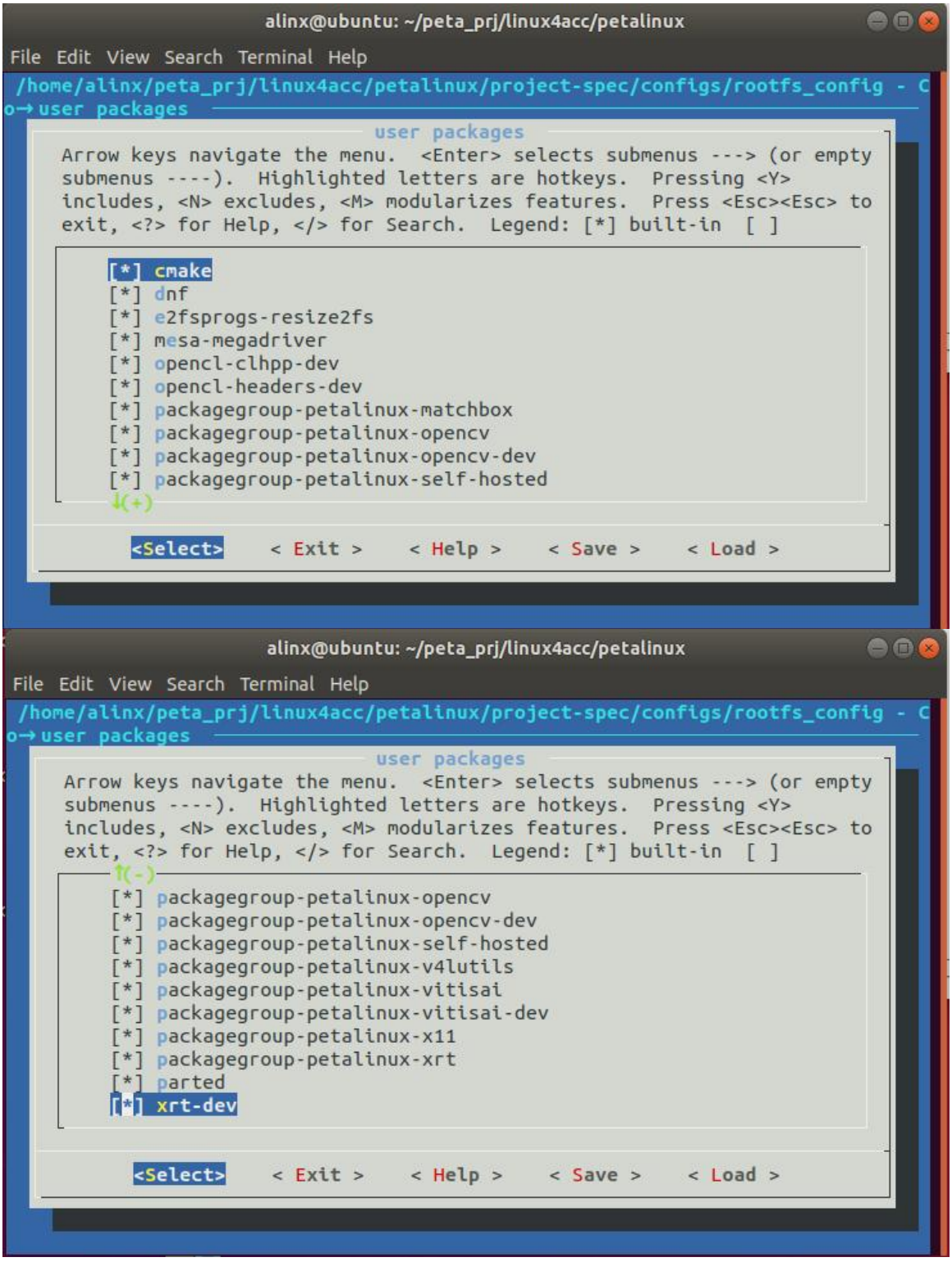
```
#Packages for Vitis-AI dependencies support:
CONFIG_packagegroup-petalinux-vitisai

#Packages for natively building Vitis AI applications on target board:
CONFIG_packagegroup-petalinux-self-hosted
CONFIG_cmake
CONFIG_packagegroup-petalinux-vitisai-dev
#CONFIG_xrt-dev
CONFIG_opencl-clhpp-dev
CONFIG_opencl-headers-dev
CONFIG_packagegroup-petalinux-opencv
CONFIG_packagegroup-petalinux-opencv-dev

#Packages for running Vitis-AI demo applications with GUI#
CONFIG_mesa-megadriver
CONFIG_packagegroup-petalinux-x11
CONFIG_packagegroup-petalinux-v4lutils
CONFIG_packagegroup-petalinux-matchbox
```

5. 在控制台输入下面的命令配置根文件系统。

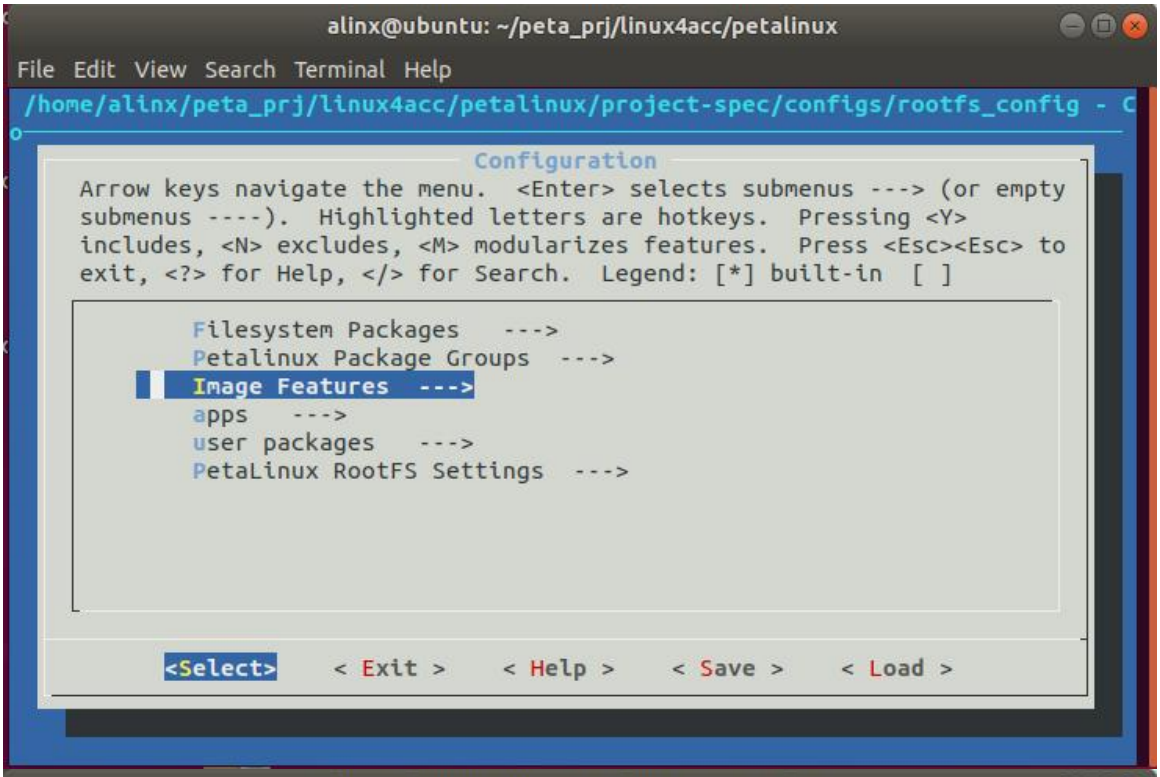
```
petalinux-config    -c    rootfs
```

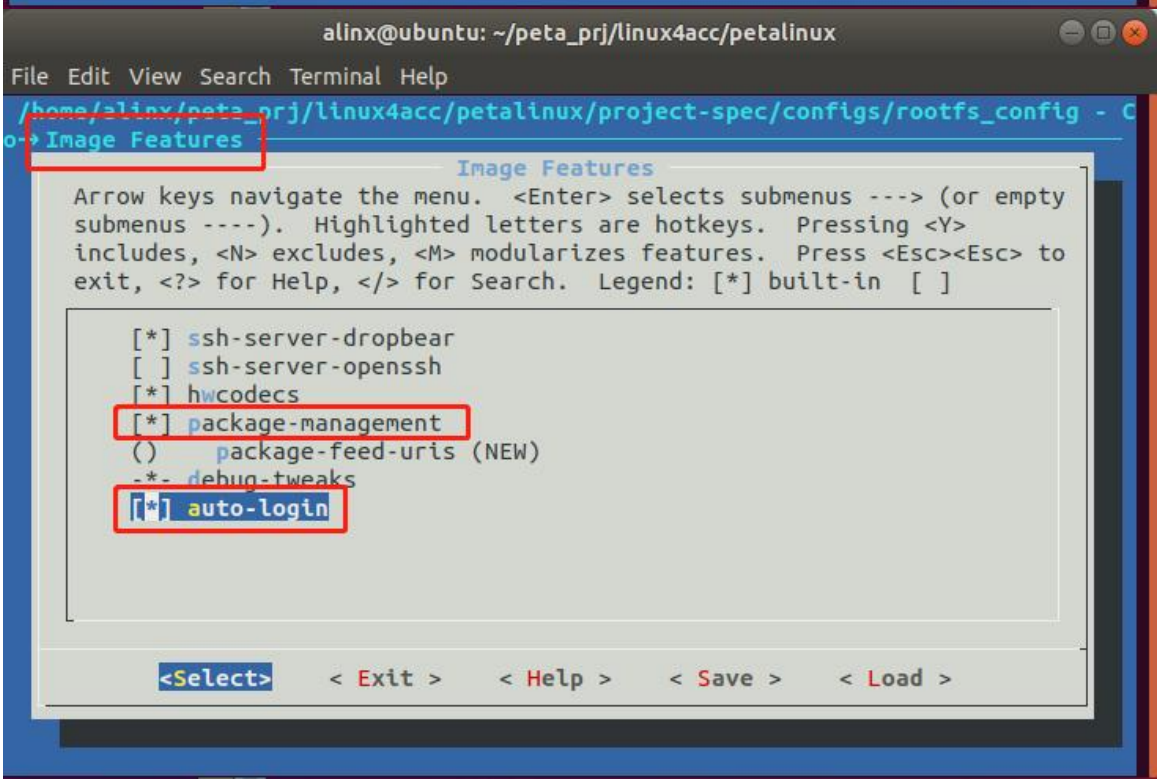
把“user packages --->”中的所有选项全部选上。



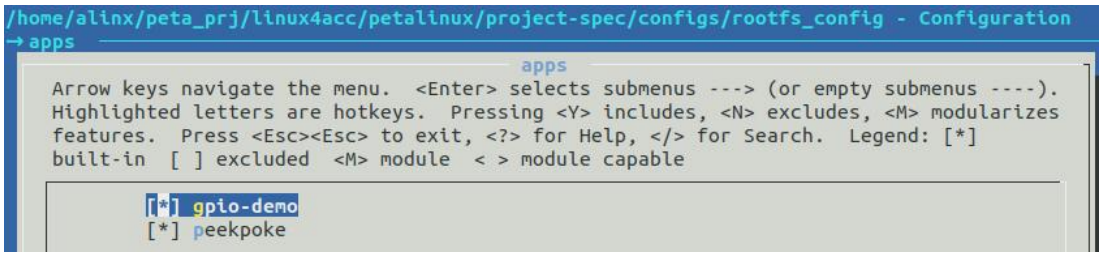


6. “Image Features --->”中配置如下图


alinx@ubuntu: ~/peta_prj/linux4acc/petalinux
File Edit View Search Terminal Help
/home/alinx/peta_prj/linux4acc/petalinux/project-spec/configs/rootfs_config - C
Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
Filesystem Packages --->
Petalinux Package Groups --->
Image Features --->
apps --->
user packages --->
PetaLinux RootFS Settings --->
<Select> < Exit > < Help > < Save > < Load >


alinx@ubuntu: ~/peta_prj/linux4acc/petalinux
File Edit View Search Terminal Help
Image Features
Image Features
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
[*] ssh-server-dropbear
[ ] ssh-server-openssh
[*] hwcodecs
[*] package-management
() package-feed-uris (NEW)
-*- debug-tweaks
[*] auto-login
<Select> < Exit > < Help > < Save > < Load >

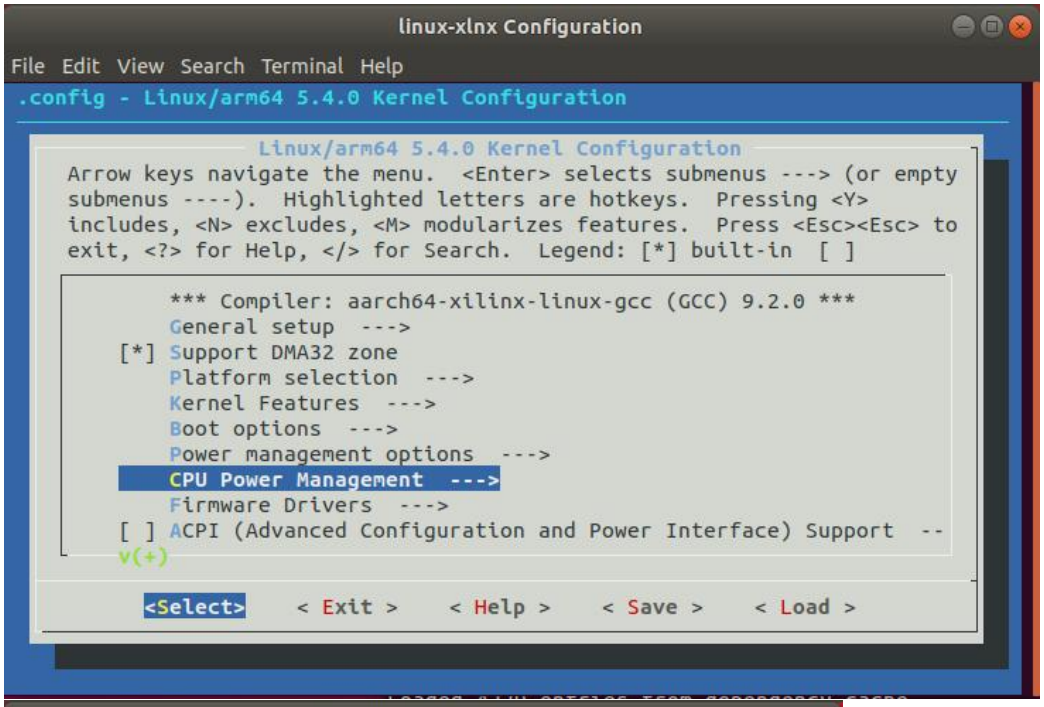
7. 在 apps 里将两个选项选上，然后保存退出

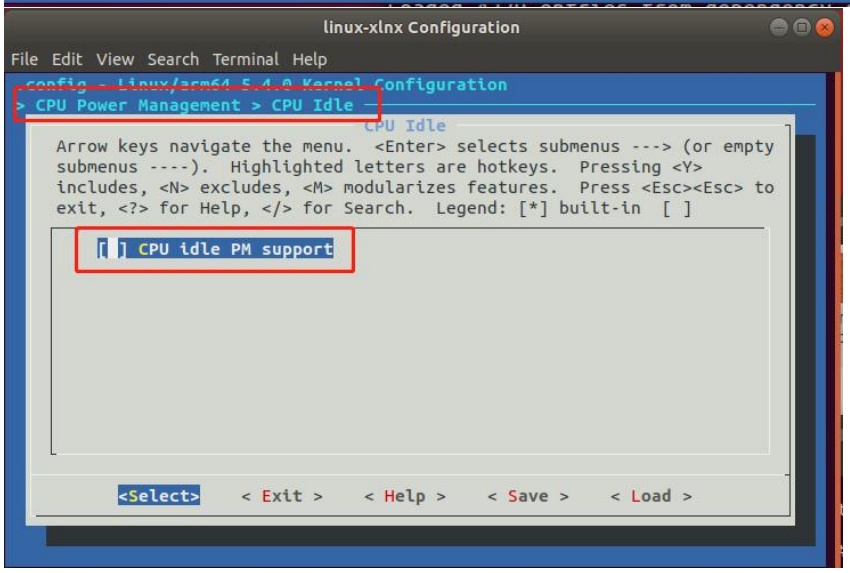


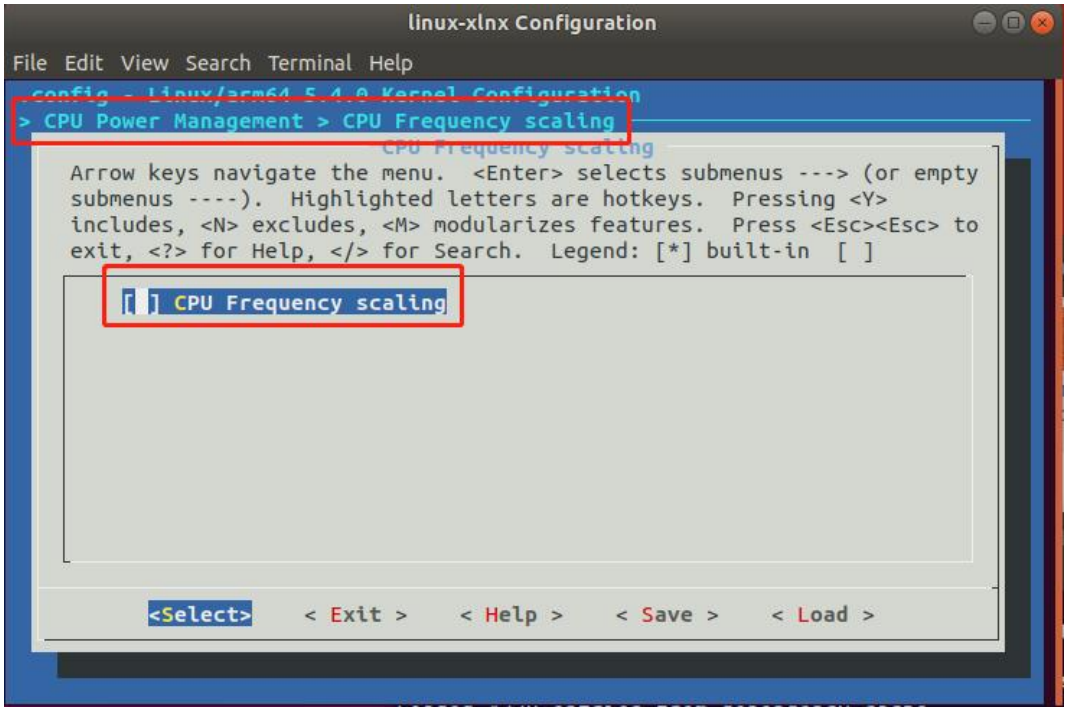
8. 在控制台输入下面的命令配置内核，可能需要等待一段时间，配置界面才会弹出。

```
petalinux-config -c kernel
```

把“CPU Power Management --->”中的选项全部去掉，如下图。

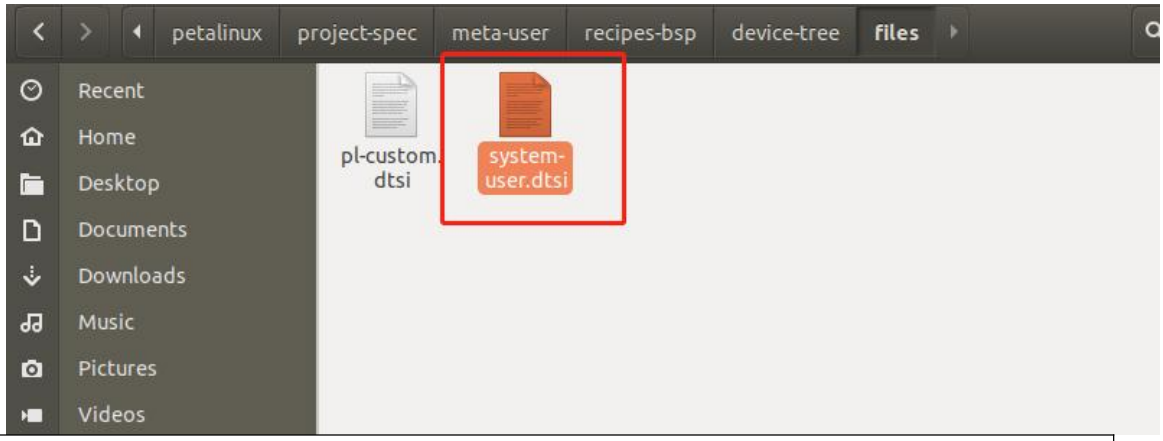






然后保存退出配置界面。

9. 打开 petalinux 工程中的路径“project-spec/meta-user/recipes-bsp/device-tree/files”，修改里面的 “system-user.dtsi”文件，修改设备树，内容如下

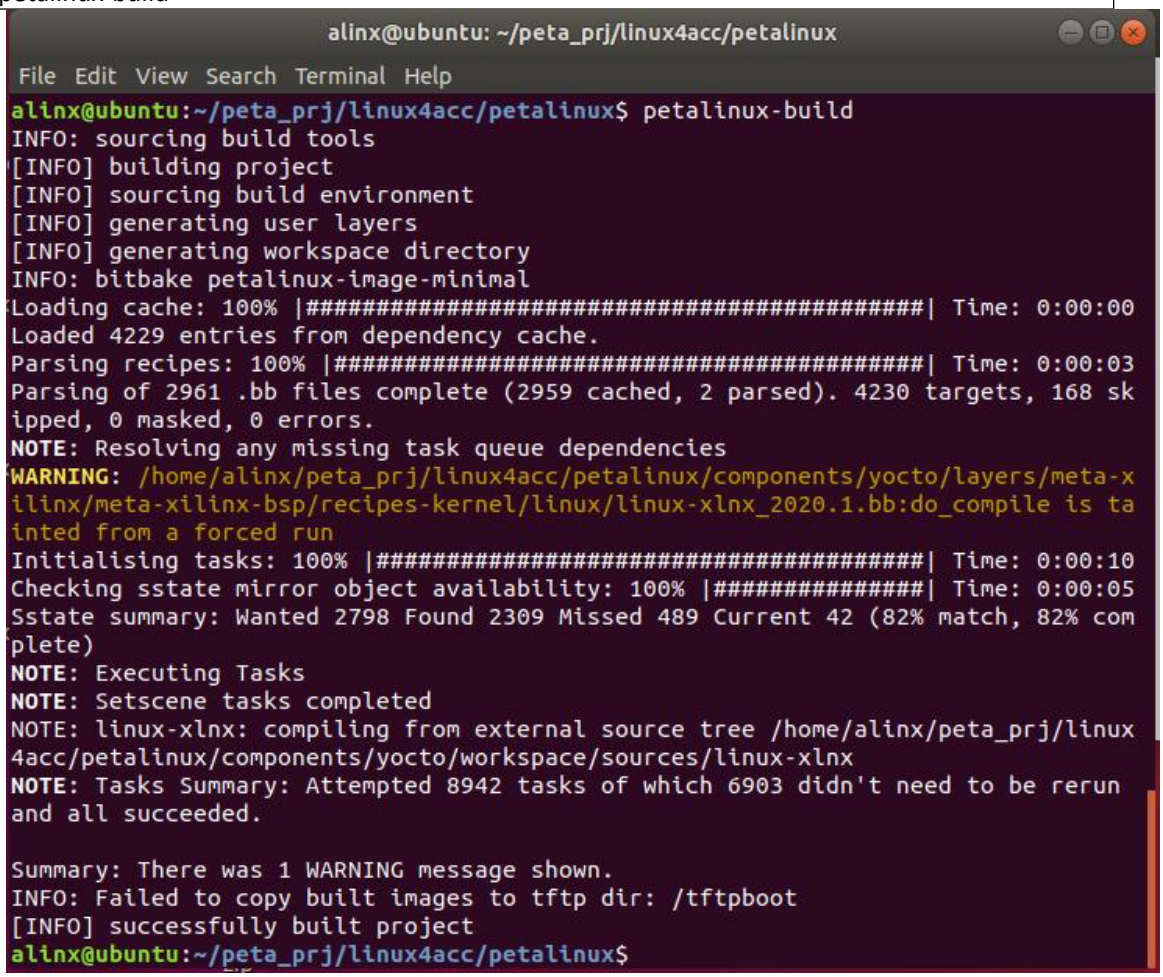



```
/include/ "system-conf.dtsi"
/ {
	chosen {
		bootargs = "earlycon console=ttyPS0,115200 clk_ignore_unused
root=/dev/mmcblk1p2 rw rootwait cma=512M";
	};
};

&amba {
	zyxclmm_drm {
		compatible = "xlnx,zocl";
		status = "okay";
		interrupt-parent = <&axi_intc_0>;
		interrupts = <0  4>, <1  4>, <2  4>, <3  4>,
			<4  4>, <5  4>, <6  4>, <7  4>,
```

```
                        <8   4>, <9   4>, <10 4>, <11 4>,
                        <12 4>, <13 4>, <14 4>, <15 4>,
                        <16 4>, <17 4>, <18 4>, <19 4>,
                        <20 4>, <21 4>, <22 4>, <23 4>,
                        <24 4>, <25 4>, <26 4>, <27 4>,
                        <28 4>, <29 4>, <30 4>, <31 4>;
    };
};

&axi_intc_0 {
        xlnx,kind-of-intr = <0x0>;
        xlnx,num-intr-inputs = <0x20>;
        interrupt-parent = <&gic>;
        interrupts = <0 89 4>;
};

&sdhci1 {
        no-1-8-v;
        disable-wp;
};
```

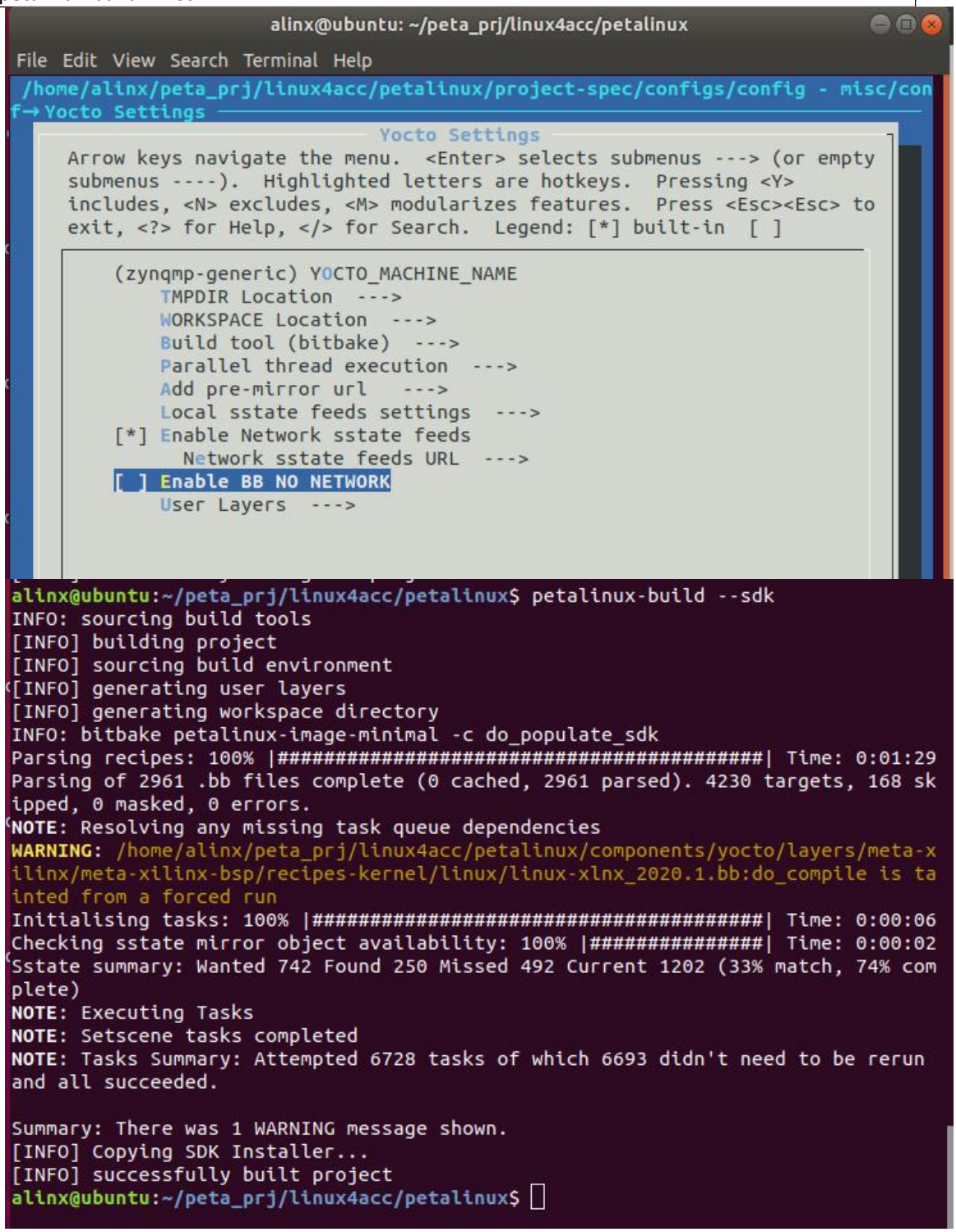
10. 使用下面的命令编译 petalinux 工程

```
petalinux-build
```

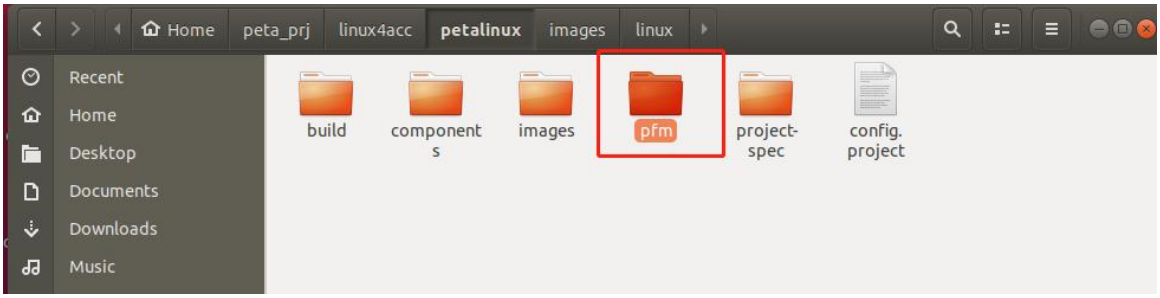


11. 构建 SDK，注意需要允许 petalinux 工程在线编译，和“QT 和 OPENCV 交叉编译开发环境”一章中构建 sdk 的方法相同。也可以使用我们编译好的，在本章节对应源码包的对应章节目录中，注意和“QT 和 OPENCV 交叉编译开发环境”一节中的 sdk 不是同一个。

```
petalinux-build  --sdk
```

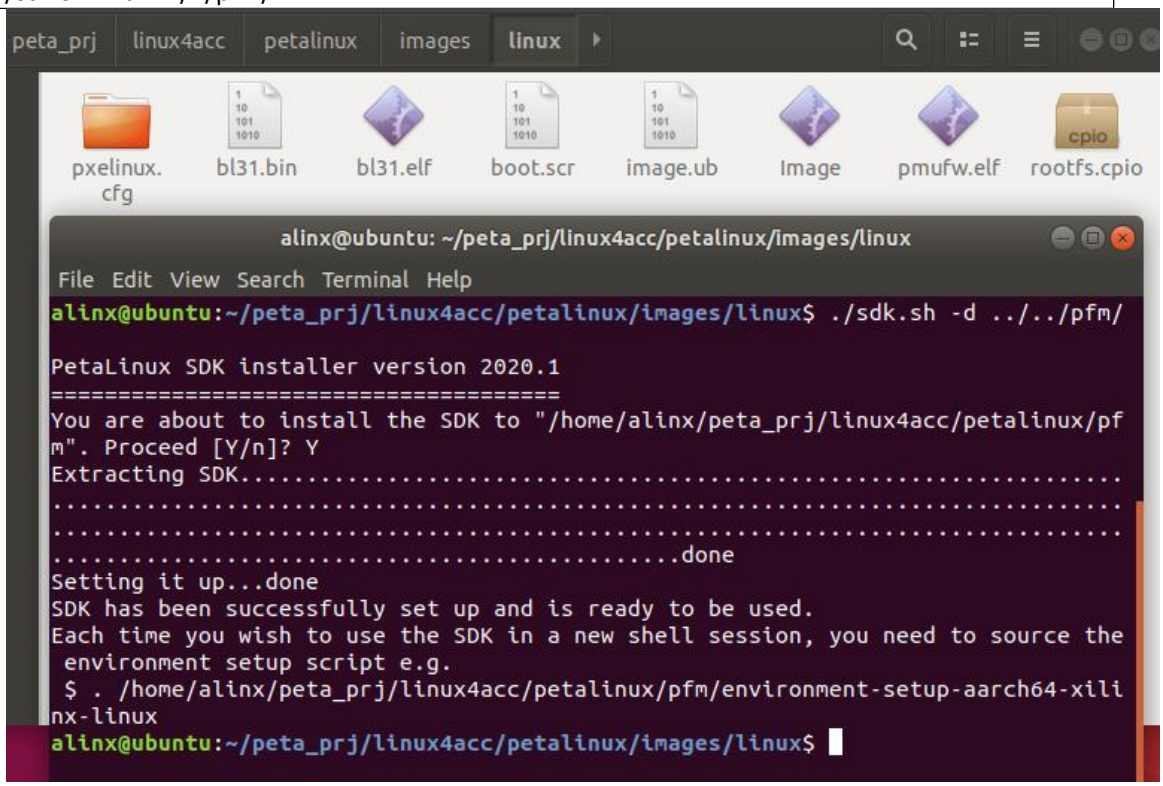


12. 在 petalinux 工程的目录中新建一个 pfm 文件夹，我们把刚才构建的 sdk 安装到这个文件夹中。

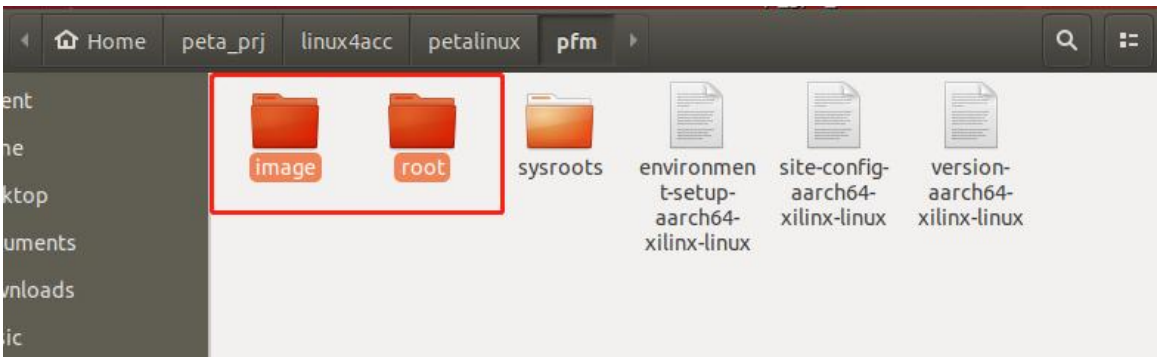


在 petalinux 工程的“images/linux”路径中打开终端，输入下面的命令来安装 sdk。

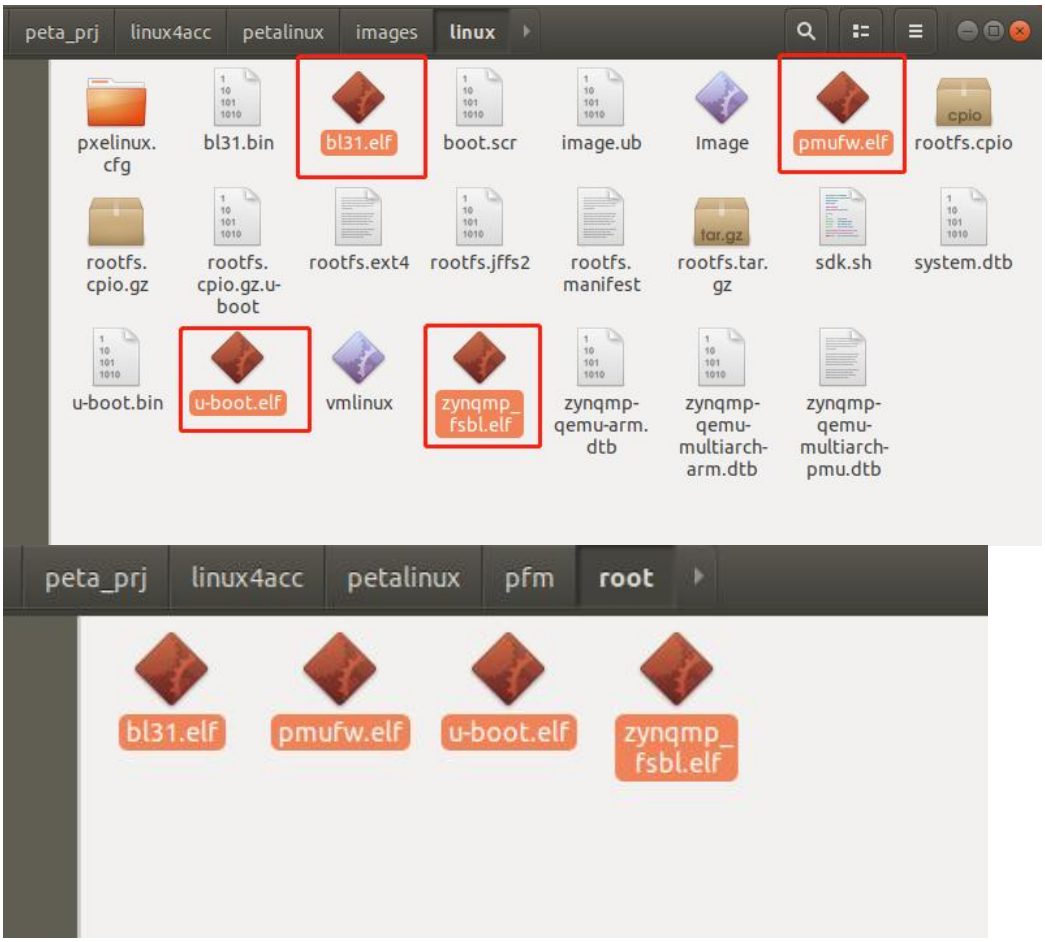
```
./sdk.sh    -d    ../../pfm/
```



13. 在 pfm 文件夹中新建两个文件夹“image”和“root”，如下图。

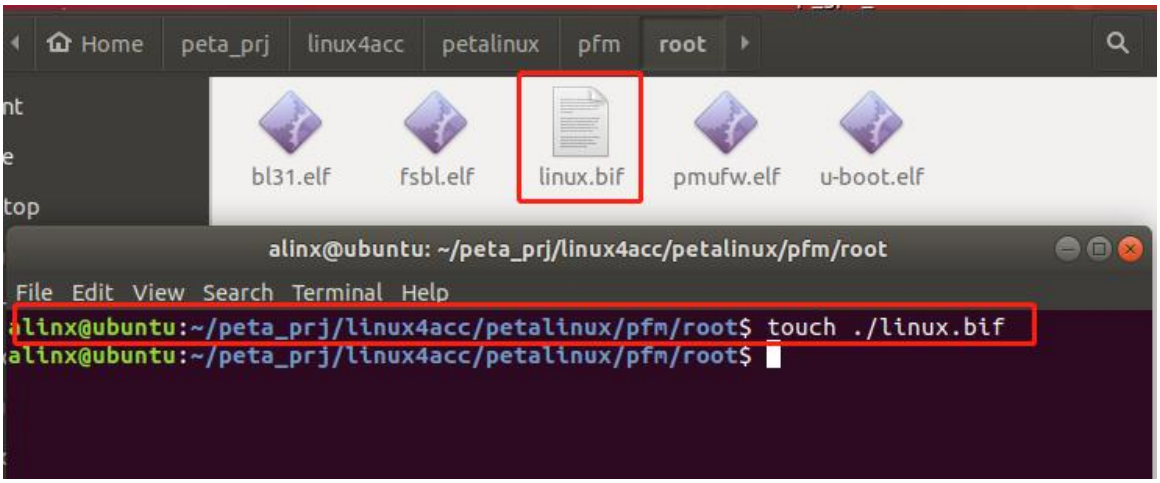


把 petalinux 工程“images/linux”路径中的四个.elf 文件拷贝到这里的 root 文件夹中。



重命名其中的“zynqmp_fsbl.elf”为“fsbl.elf”。

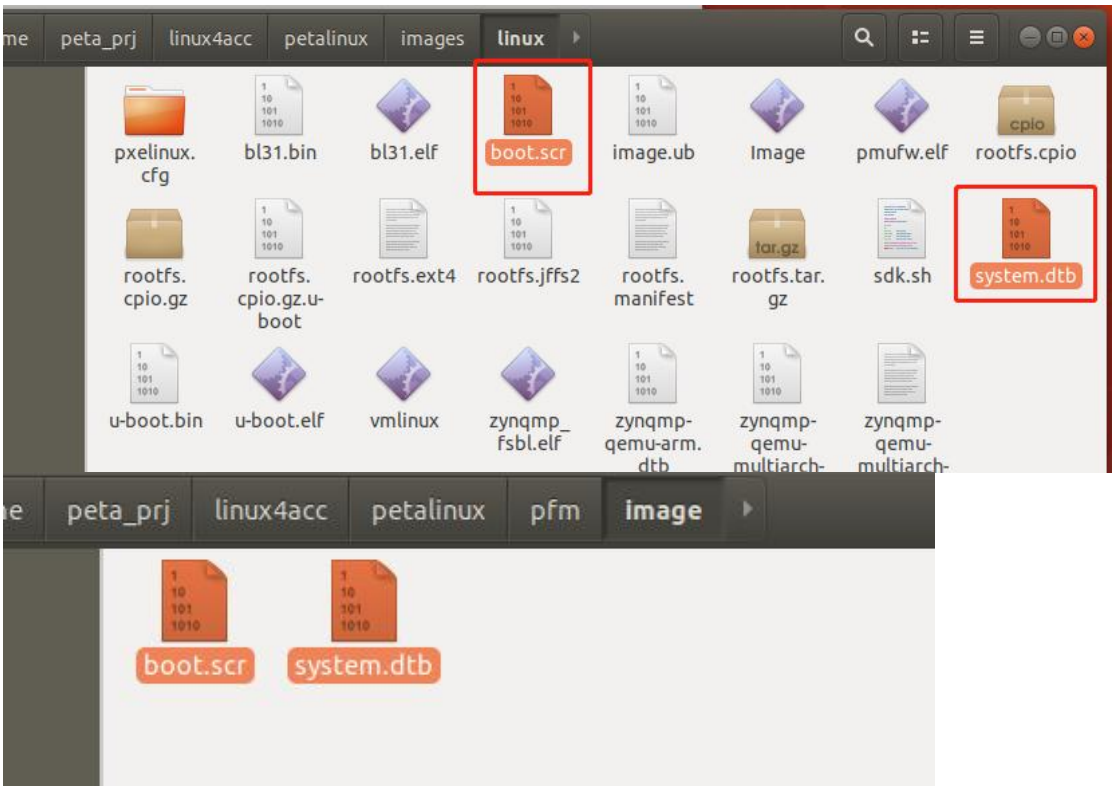
然后在 root 文件夹中新建一个“linux.bif”文件。



打开这个文件，输入下面的内容。

```
   /* linux */
the_ROM_image:
{
        [fsbl_config] a53_x64
        [bootloader] <fsbl.elf>
        [pmufw_image] <pmufw.elf>
        [destination_device=pl] <bitstream>
        [destination_cpu=a53-0, exception_level=el-3, trustzone] <bl31.elf>
        [destination_cpu=a53-0, exception_level=el-2] <u-boot.elf>
}
```

14. 把 petalinux 工程“images/linux”路径中“boot.scr”和“system.dtb”拷贝到 image 文件夹中。

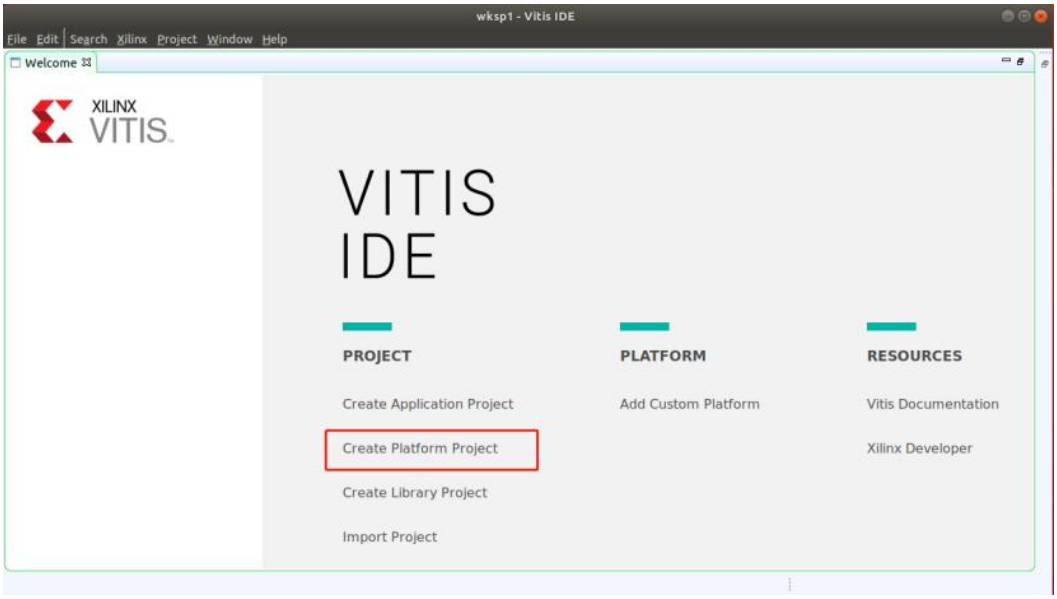


9.3 Vitis 创建加速平台

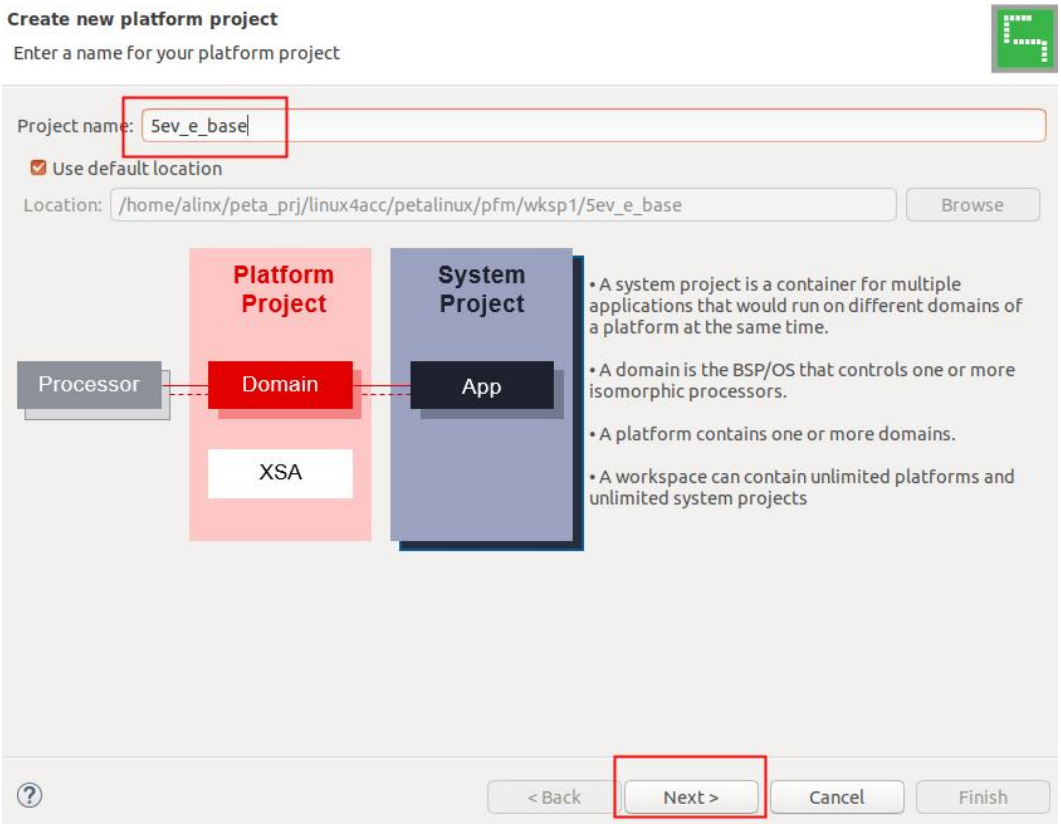
1. 在 pfm 目录中打开一个终端，输入下面的命令，新建一个 vitis 工程。

```
source /tools/Xilinx/Vitis/2020.1/settings64.sh && vitis -workspace wksp1
```

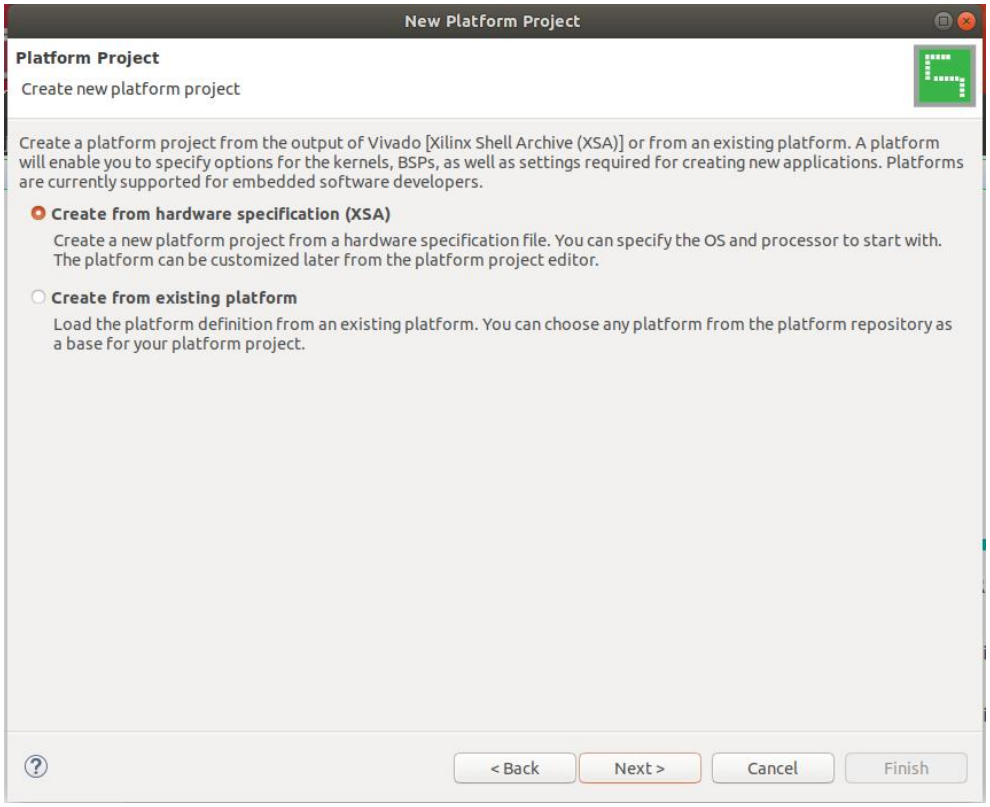
2. vitis 界面打开后选择欢迎页面中的“Create Platform Project”



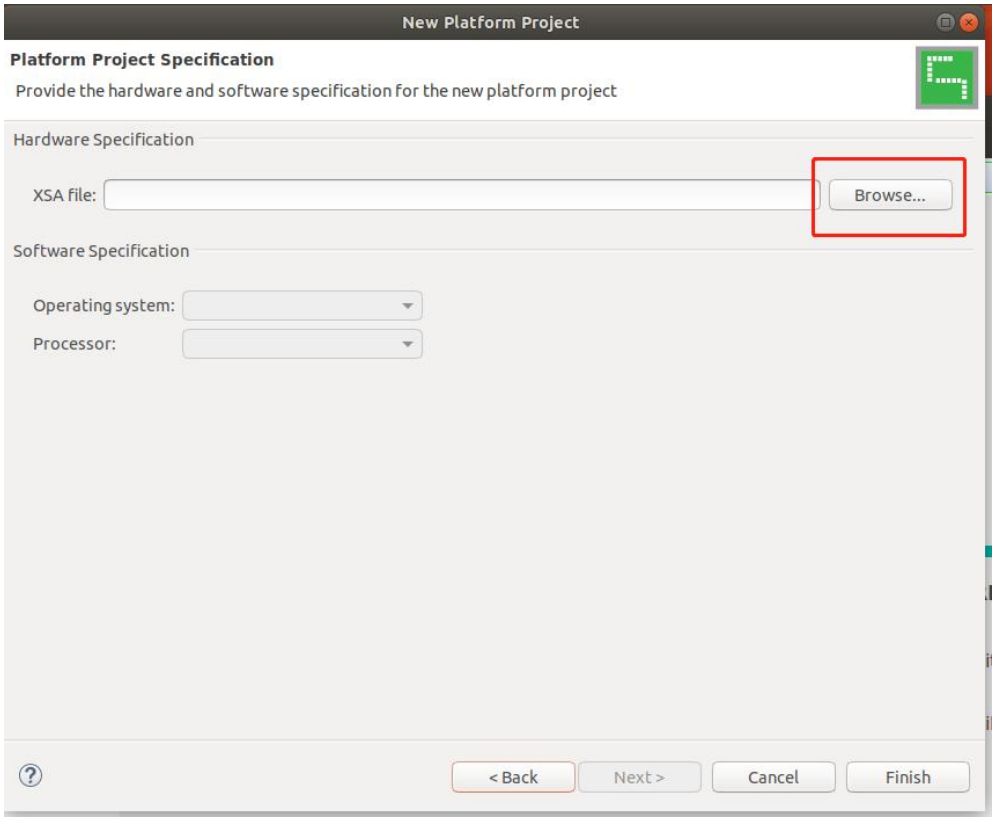
弹出的窗口中“Project name”填“5ev_e_base”,然后点 next。



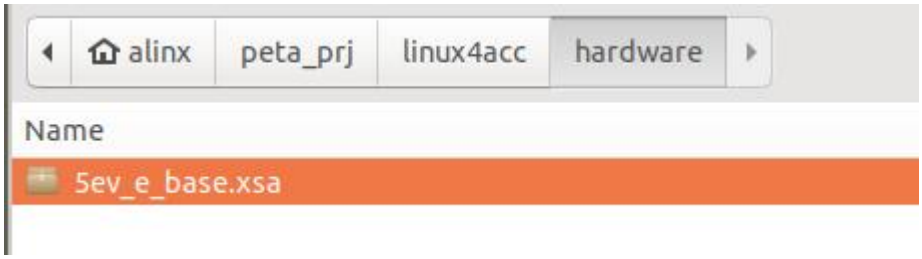
接下来的界面选择第一个选项，然后点 next。



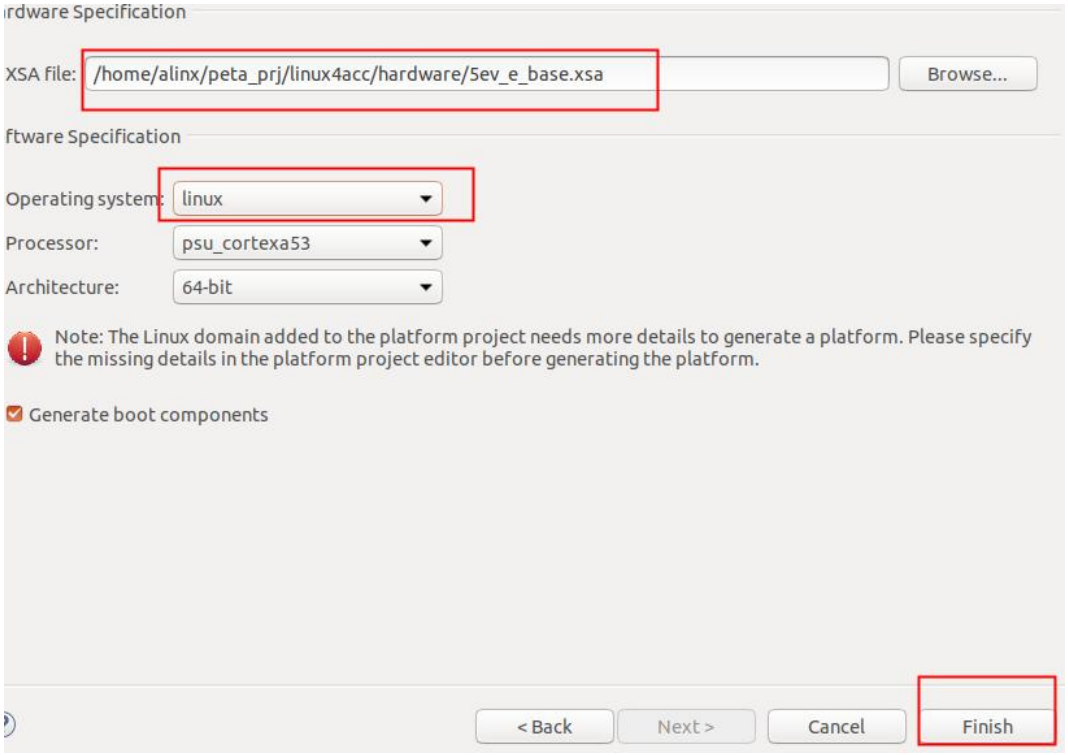
下一个界面，点击“Browse”按钮选择 xsa 文件。



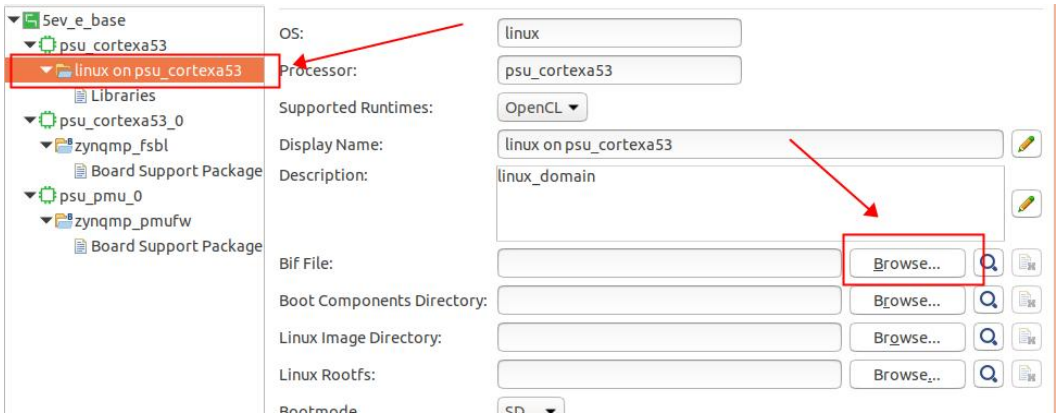
选择这一章 vivado 工程中得到的“5ev_e_base”。



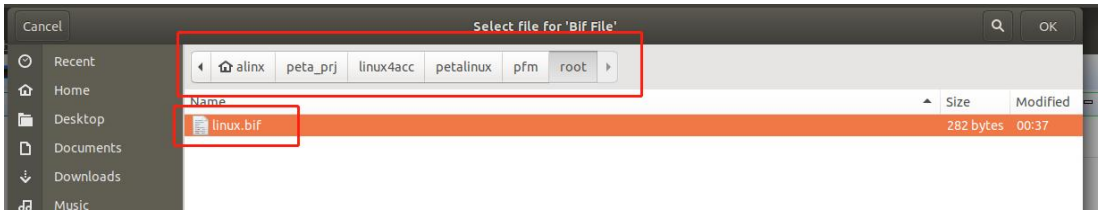
然后界面会多出一些选项，我们把其中的“Operating system”改成“linux”，点击 Finish 按钮。



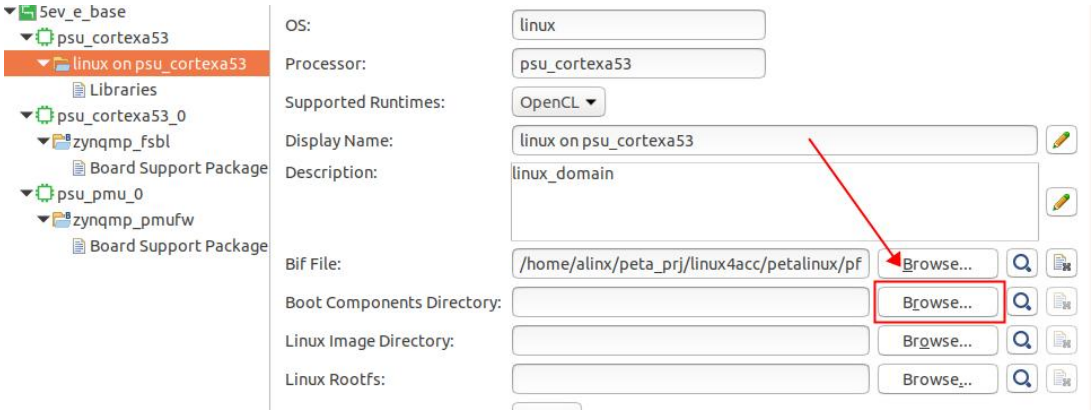
3. 之后会打开 vitis 工程，按下图顺序点击窗口中的内容，选择“Bif file”。



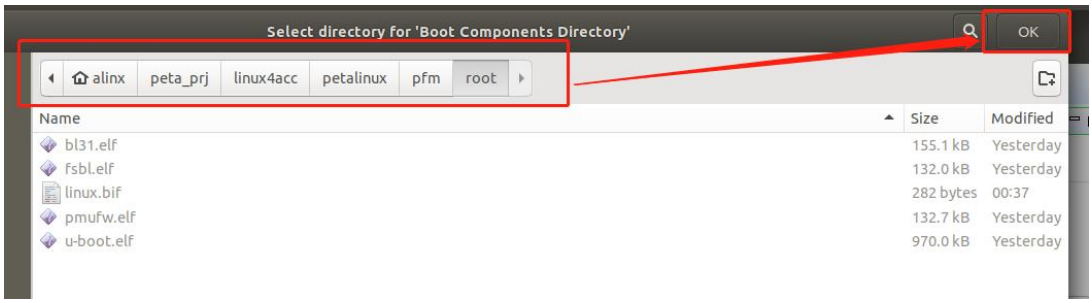
选择刚才新建的“linux.bif”。



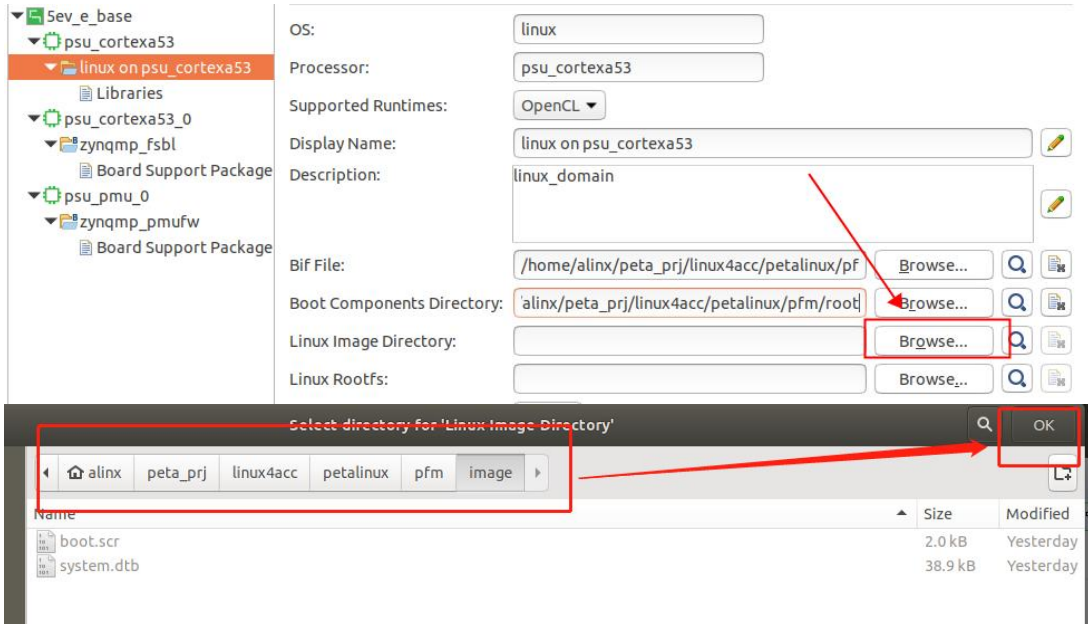
还在这个窗口中，选择“Boot Components Directory”的路径。



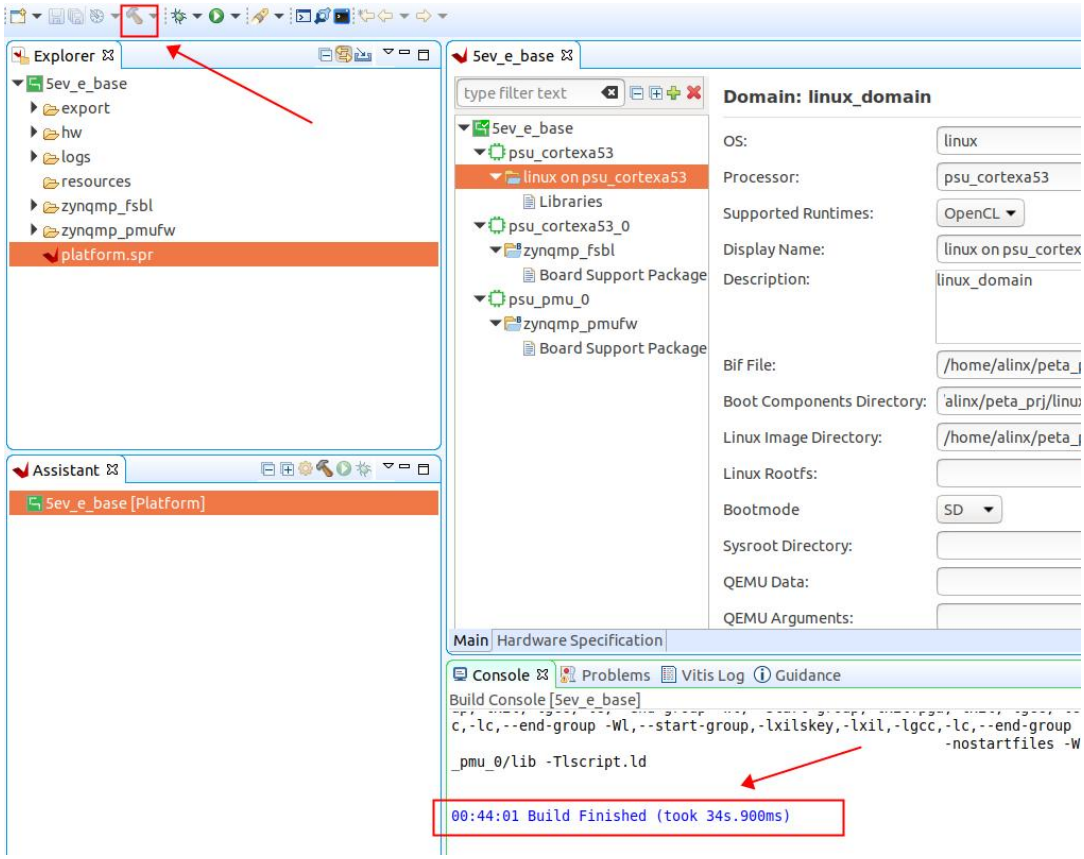
就选择之前新建的 root 路径即可。



然后选择 image 路径。

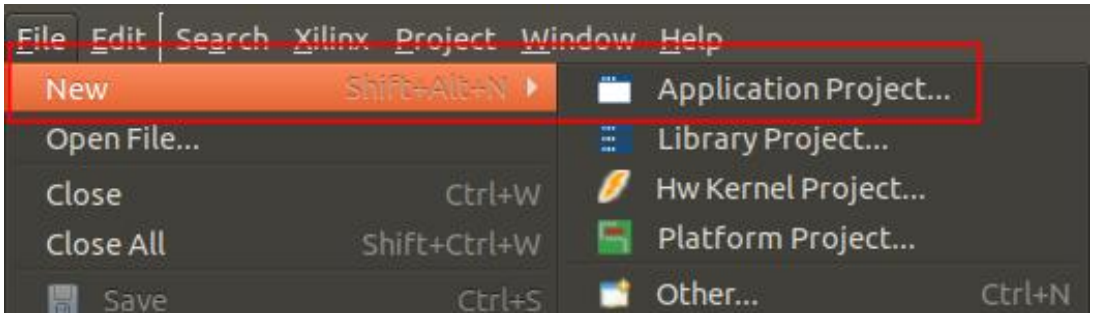


4. 点击 vitis 左上角的“小锤子”图标编译工程。

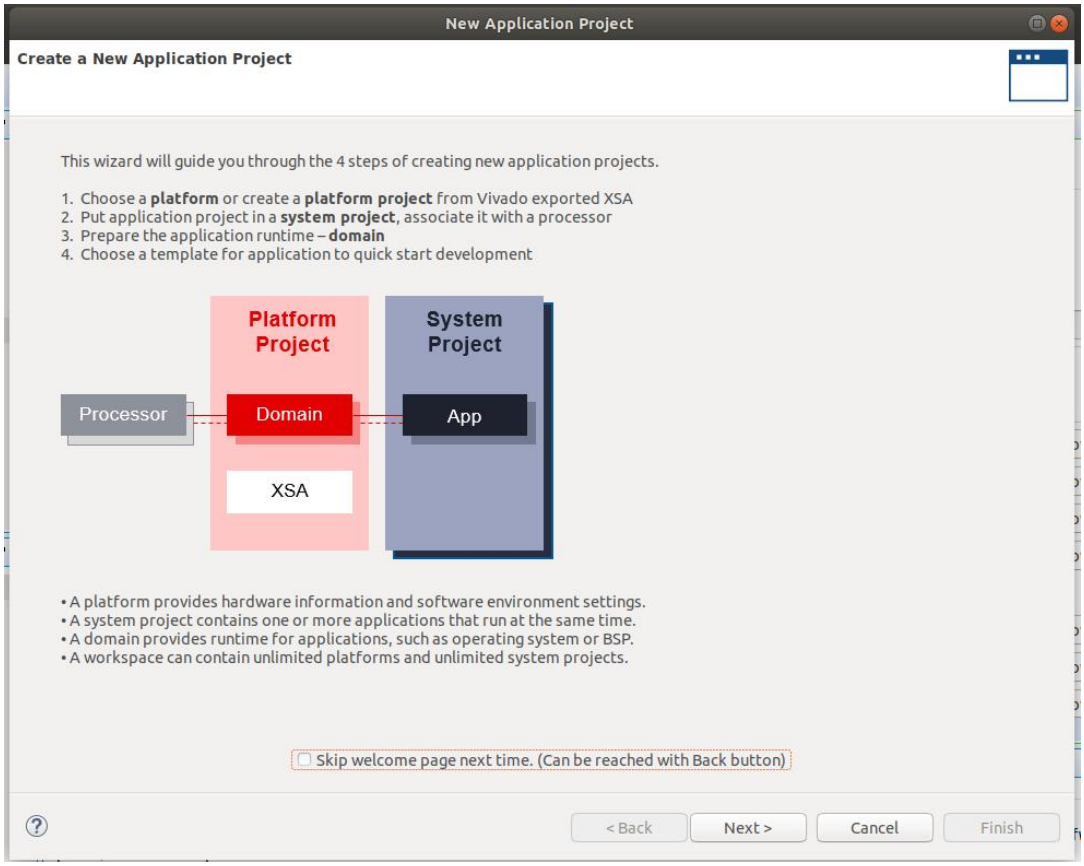


9.4 创建应用程序测试加速平台

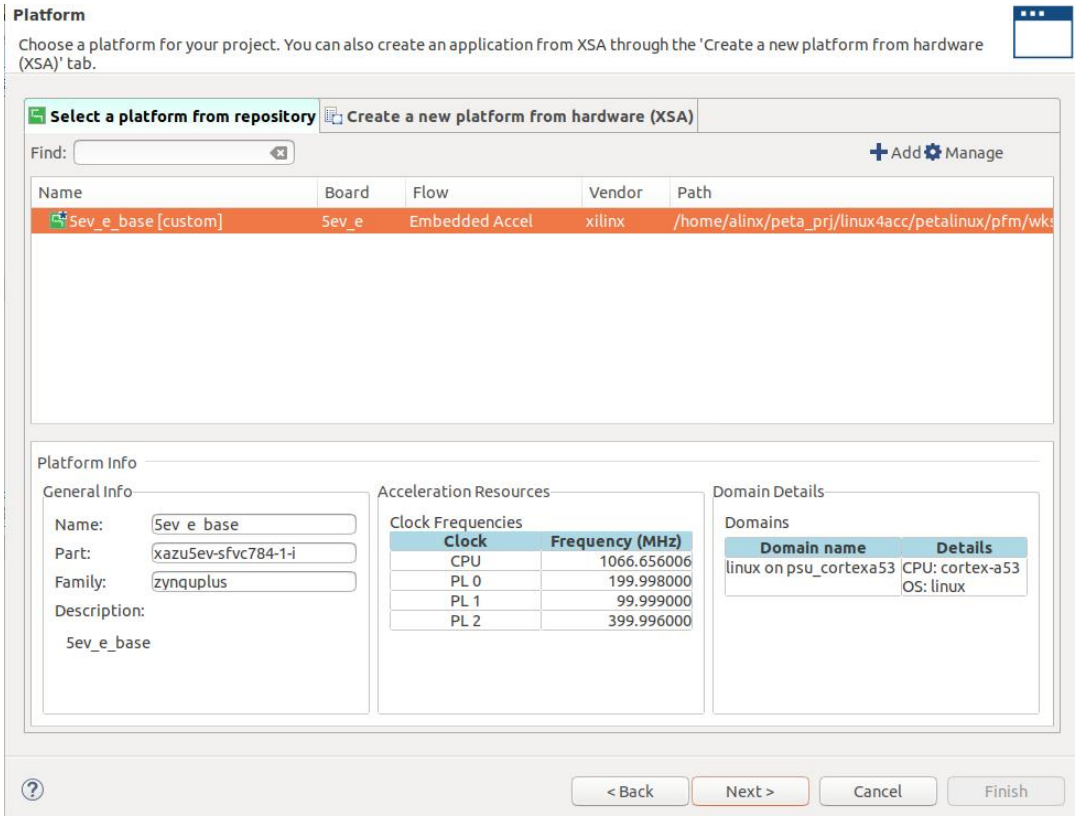
1. 编译成功后，选择菜单栏中的“File > New > Application Project…”,创建一个应用程序测试加速平台。



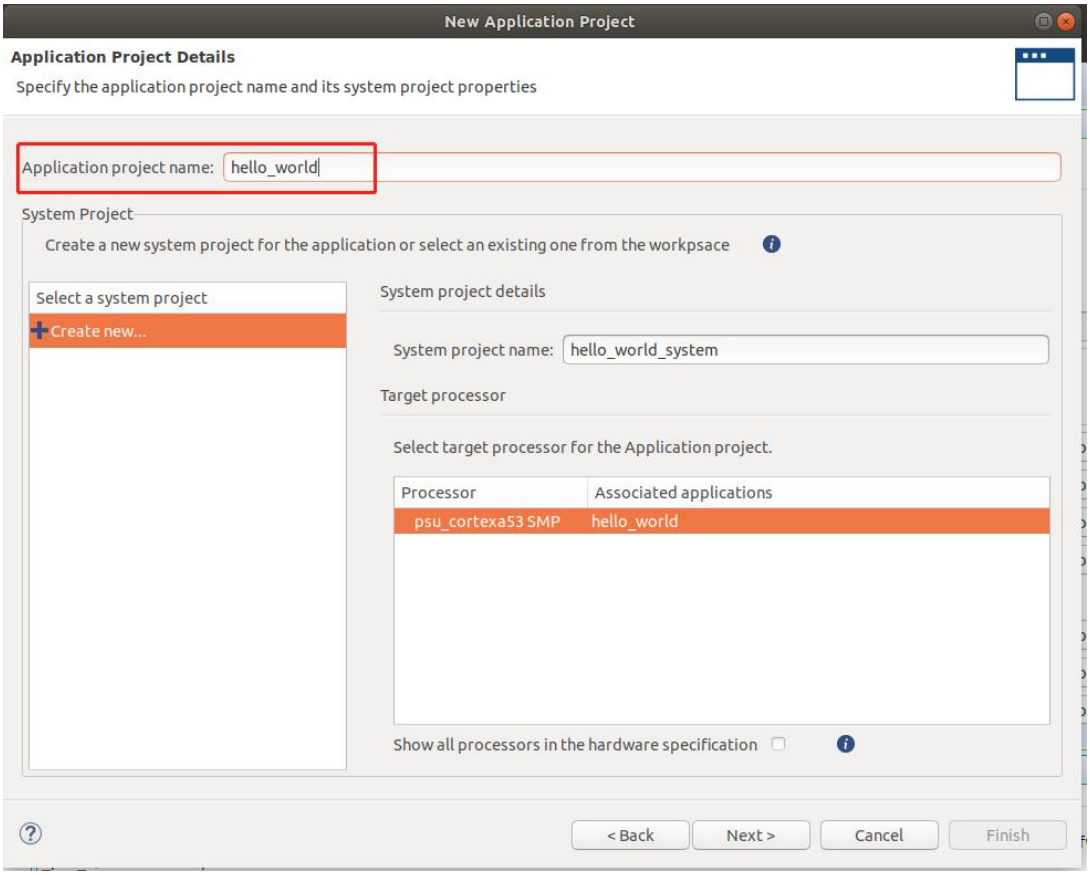
弹出下图的界面直接点 next。



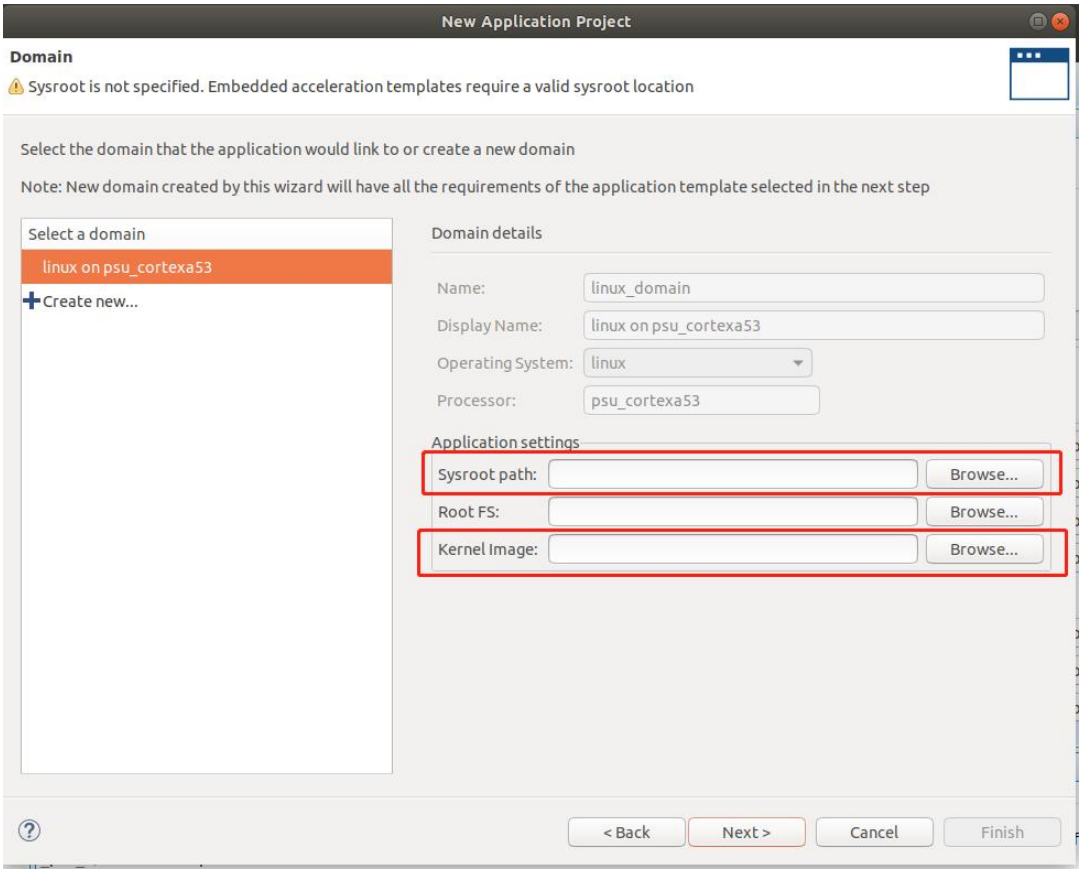
2. 下一个界面也不用修改，直接 next，这里的“5ev_e_base[custom]”也就是刚才编译出来的加速平台。

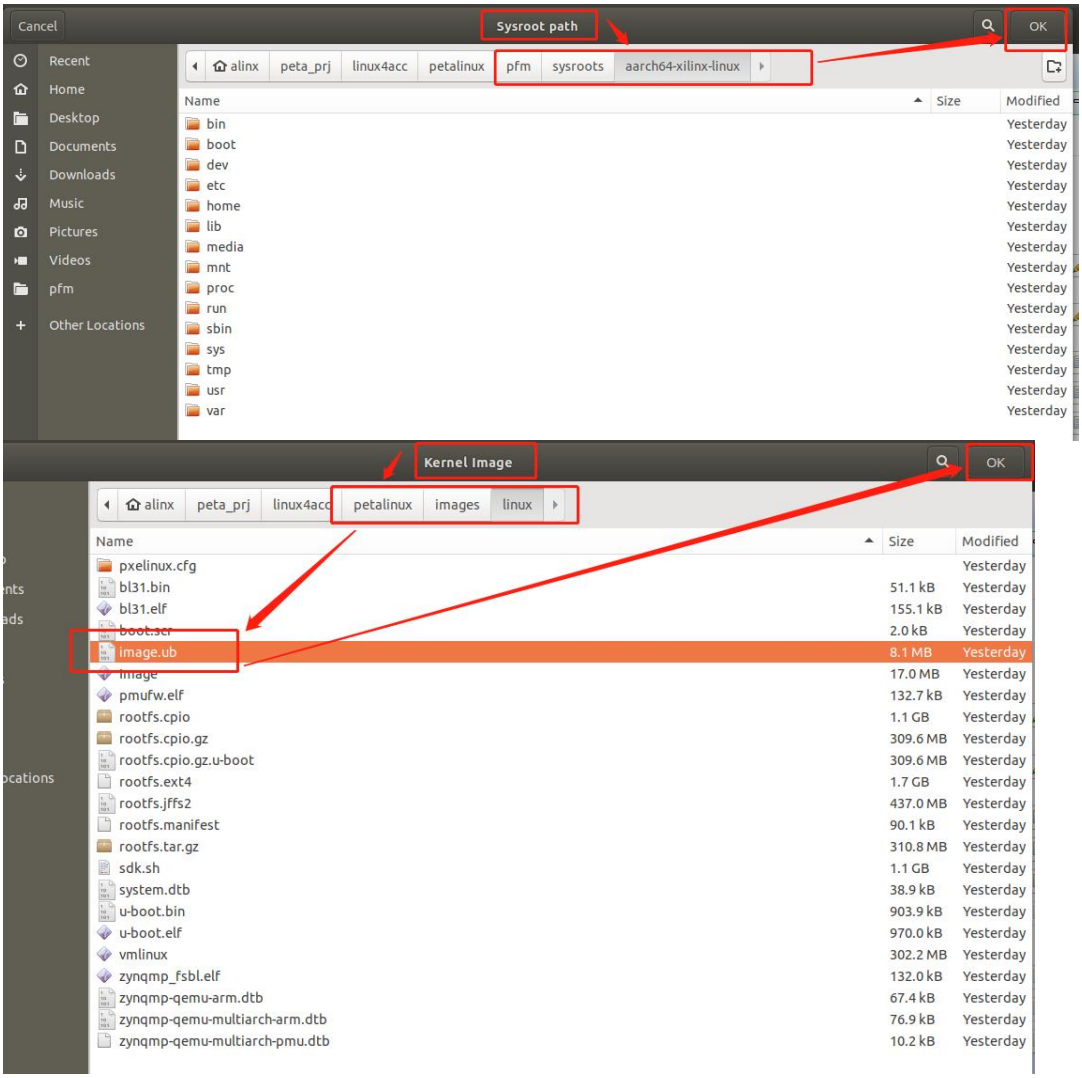


3. 下一个界面输入工程名称“hello_world”，然后 next。



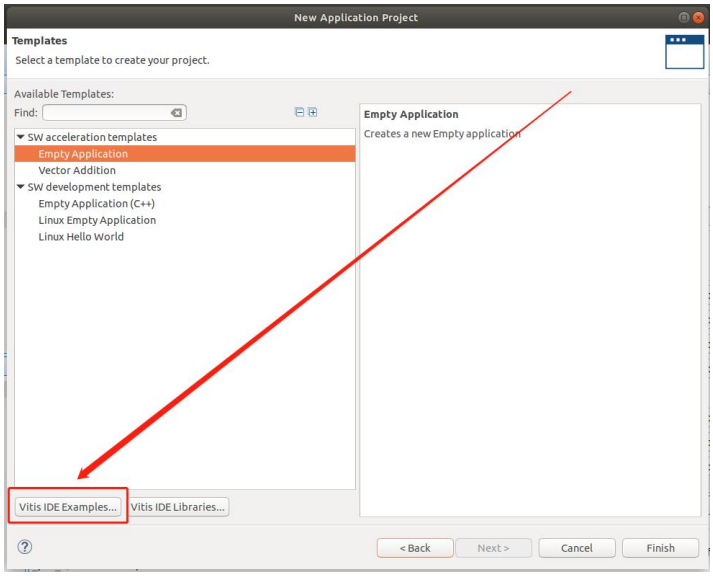
4. 下一个界面我们需要选择一下“Sysroot path”和“Kernel image”两个路径，如下图。

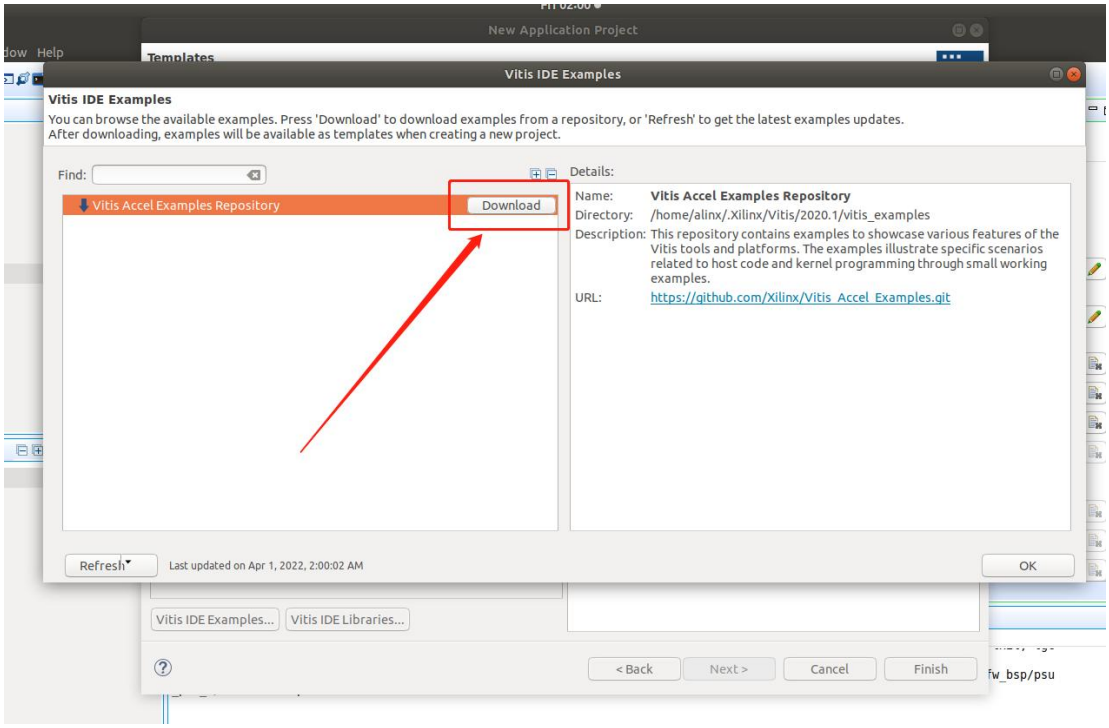




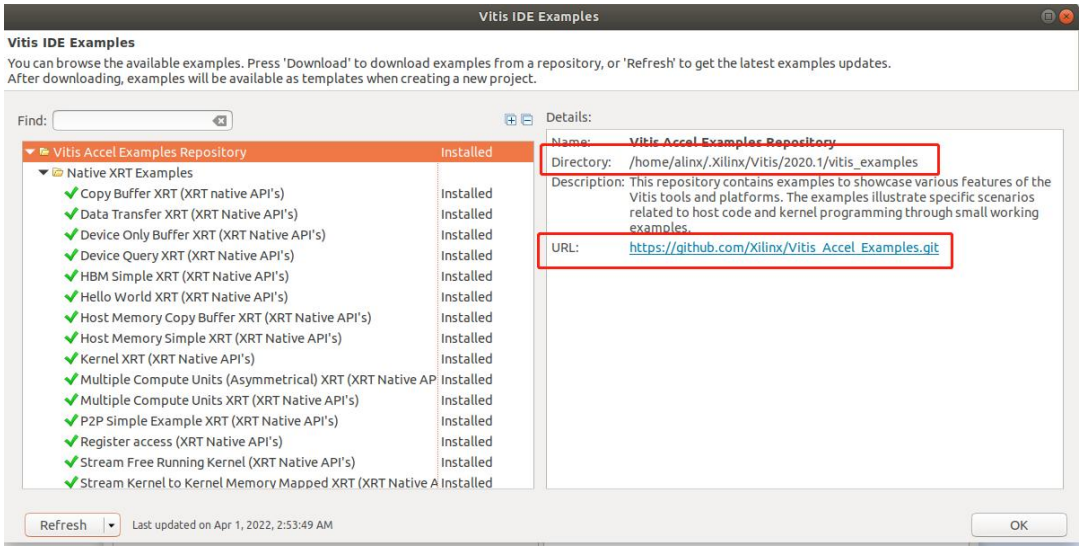
设置好之后点击 next。

5. 然后我们需要选择一个工程模板，但是这里没有我们需要的模板，点击界面左下角的“Vitis IDE Examples”按钮，然后点击 download 下载。

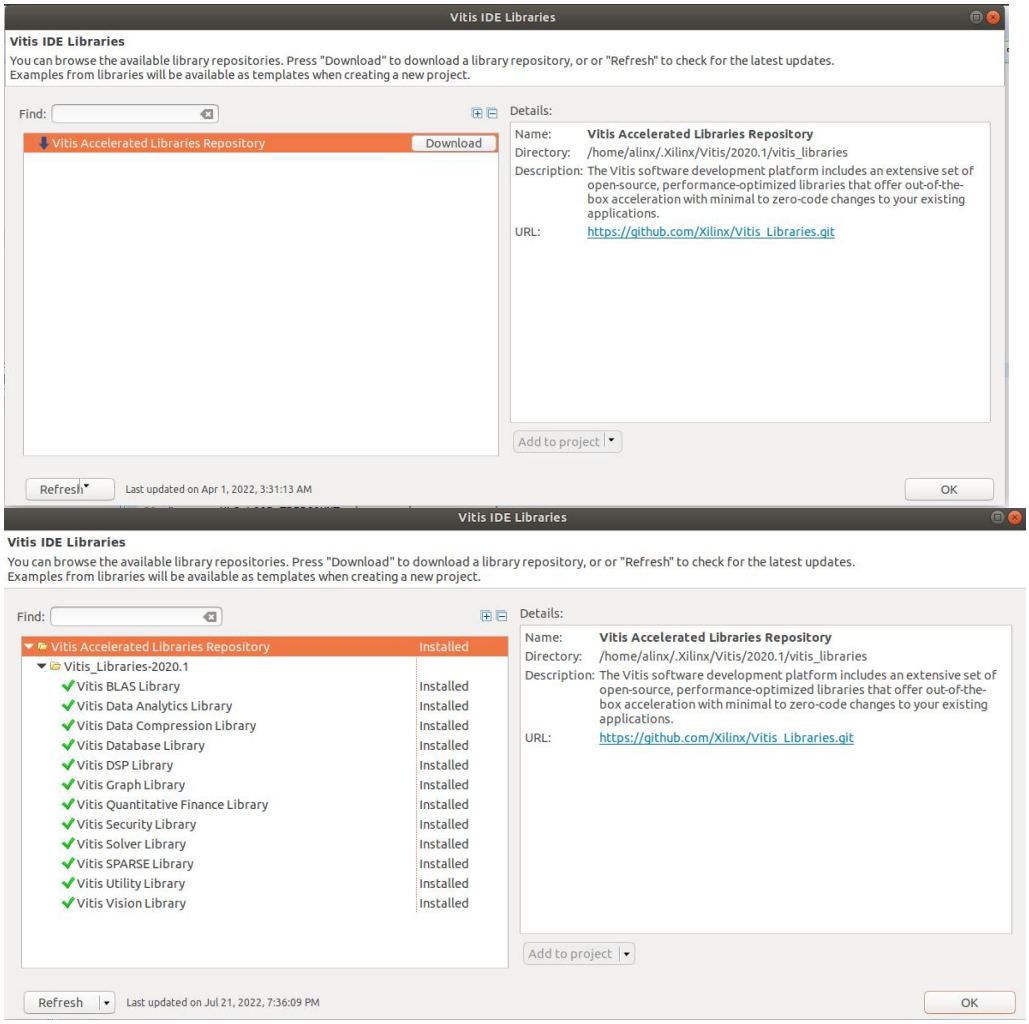




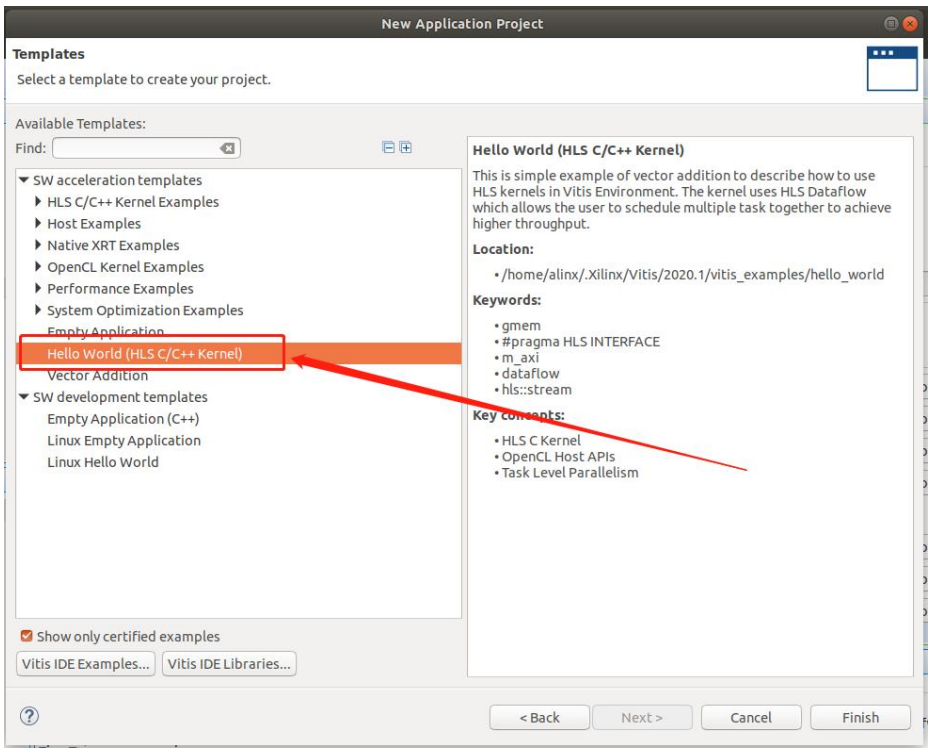
6. 下载完成后会多出如下图所示的内容，点击 ok。如果在 vitis 软件中下载失败，也可以去旁边给出的链接中手动下载，然后放到给出的路径中。我们也把下载好的官方例程包放到了本章节对应源码包的对应章节文件夹中。如果要自己下载的话，一定要注意版本 2020.1，如果不熟悉 github 使用的话建议直接使用我们提供的包。



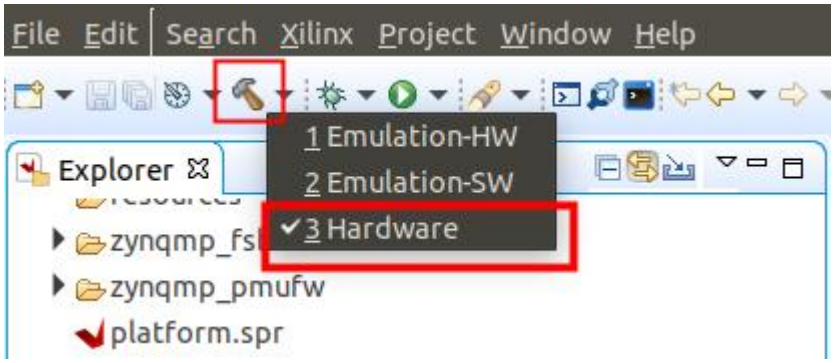
7. 同样左下角的“Vitis IDE Libraries...”按钮中的内容也需要下载。



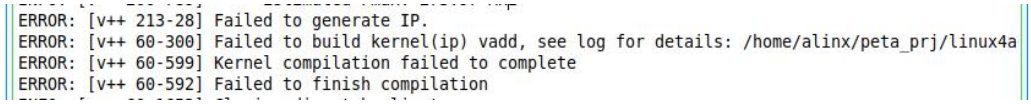
8. 然后在选择例程的界面中，也多了一些例程，我们选择“Hello World(HLS C/C++ kernel)”，然后点“Finish”。



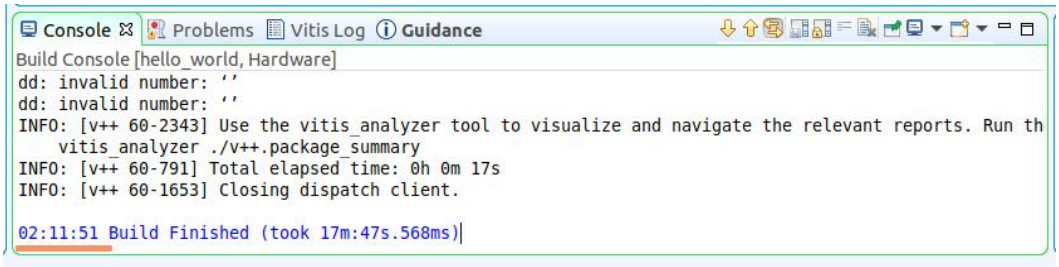
9. 选择小锤子旁边下拉菜单中的“hardware”，编译应用程序。这里需要注意，虚拟机内存如果不够会编译失败，本教程中给虚拟机分配了 16G 的内存可以编译成功。



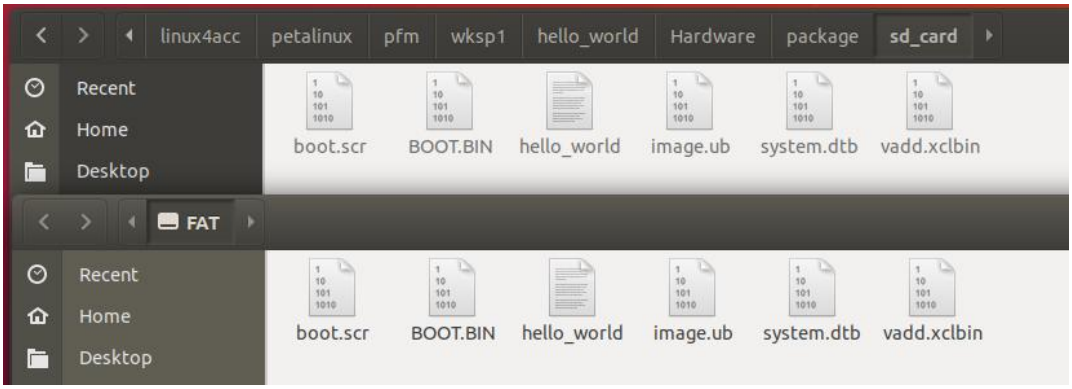
如果出现下面的错误，则需要打一下 vitis 软件的补丁(在本章对应源码包中名为“y2k22_patch-1.2.zip”的压缩包)，安装方法参考补丁压缩包中的“README”。



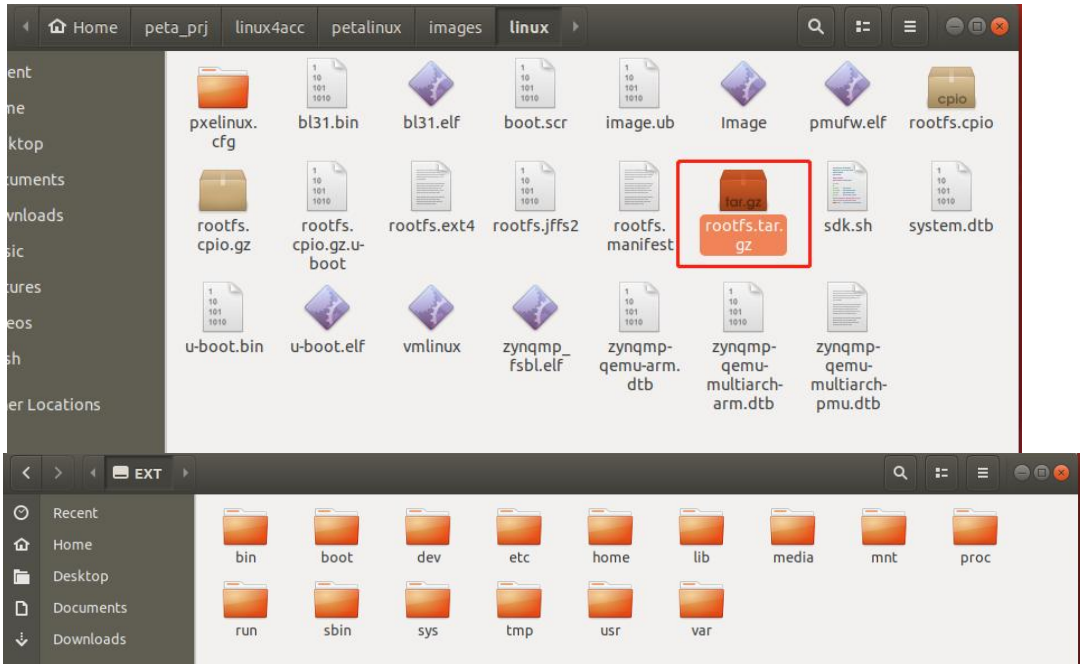
编译成功 log 如下图。



10. 编译成功后，到下图中的路径，把里面所有的文件都拷贝到 SD 的 FAT 分区中。



11. 把 petalinux 工程中“images/linux”路径中的“rootfs.tar.gz”按照“SD 卡根文件系统”一章中的方法，解压到 SD 卡的 EXT 分区中。



9.5 运行程序测试加速

1. 使用上一步中制作好的 SD 卡启动开发板。系统启动后，输入下面的命令进入 SD 卡的第一个分区。

```
cd  /media/sd-mmcblk1p1
```

```
root@petalinux:~# cd /media/sd-mmcblk1p1
root@petalinux:/media/sd-mmcblk1p1# ls
BOOT.BIN  boot.scr  hello_world  image.ub  system.dtb  vadd.xclbin
root@petalinux:/media/sd-mmcblk1p1#
```

2. 在运行测试程序之前，先用下面的命令设置环境变量。

```
export  XILINX_XRT=/usr
```

```
root@petalinux:/media/sd-mmcblk1p1# export XILINX_XRT=/usr
root@petalinux:/media/sd-mmcblk1p1#
```

3. 使用下面的命令运行应用程序。

```
./hello_world  vadd.xclbin
```

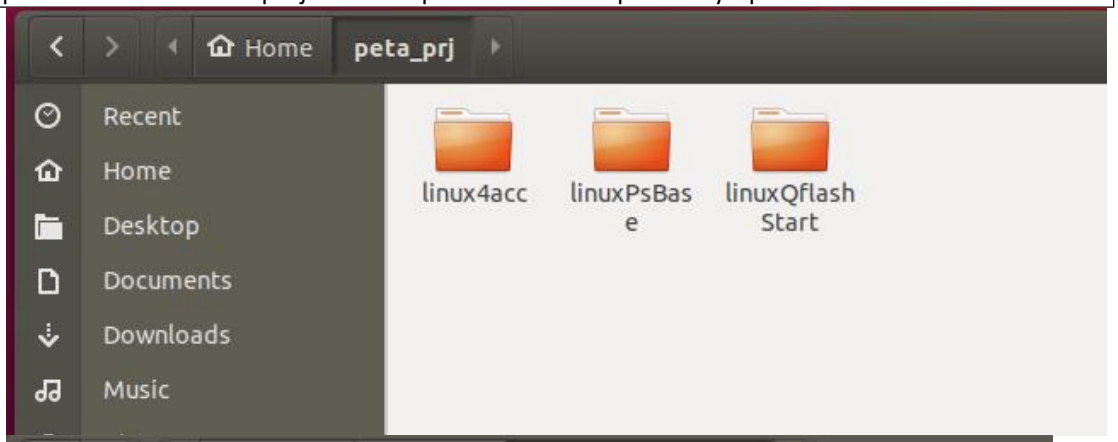
如果串口中打印了“TEST PASSED”就说明加速平台创建成功了。

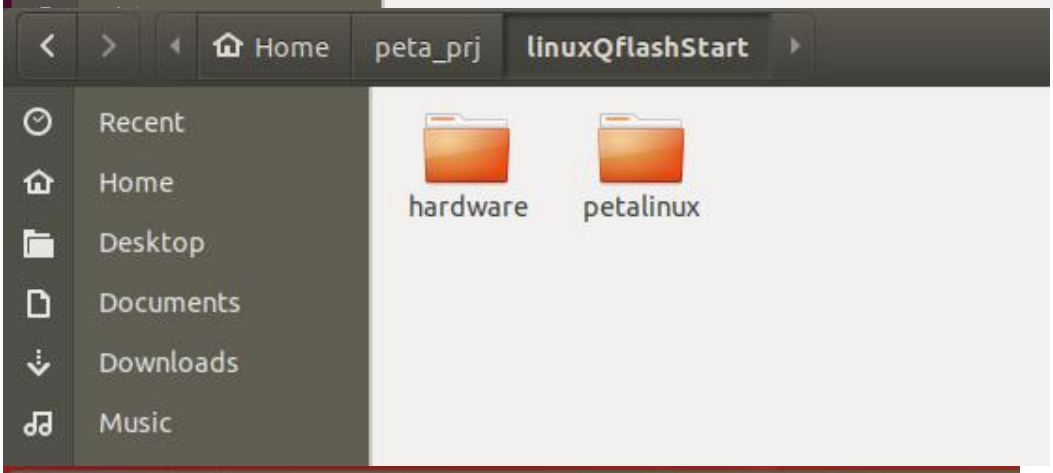
```
[  434.947545] [drm] now xclbin can be changed
[  434.955278] [drm] <- Release xclbin 57D5DA29-775D-407D-9F56-E21FA4ABB14E, to ref
=0
[  434.959601] [drm] -> Hold xclbin 57D5DA29-775D-407D-9F56-E21FA4ABB14E, from ref=
0
Device[0]: program successful!
TEST PASSED
[  434.967177] [drm] <- Hold xclbin 57D5DA29-775D-407D-9F56-E21FA4ABB14E, to ref=1
[  434.982963] [drm] -> Release xclbin 57D5DA29-775D-407D-9F56-E21FA4ABB14E, from r
ef=1
[  434.990275] [drm] now xclbin can be changed
[  434.998011] [drm] <- Release xclbin 57D5DA29-775D-407D-9F56-E21FA4ABB14E, to ref
=0
[  435.005736] [drm] Pid 1045 closed device
root@petalinux:/media/sd-mmcblk1p1#
```

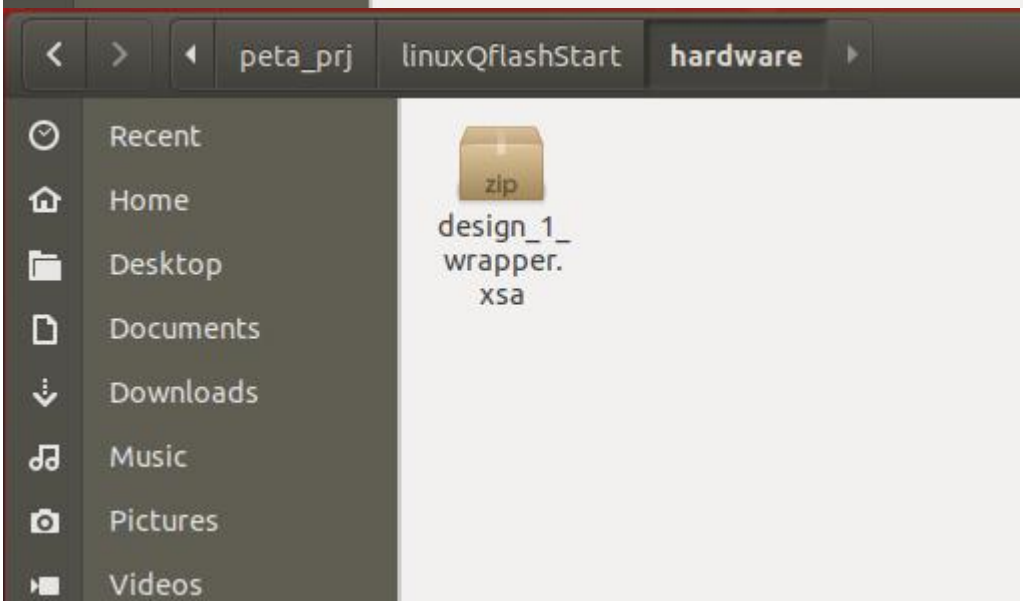
第十章 从 QSPI Flash 启动 Linux

1. 在“/home/alinx/peta_prj”路径中新建一个文件夹“linuxQflashStart”，在这个文件夹中打开终端，用下面的命令新建一个 petalinux 工程，并建立一个 hardware 文件夹存放硬件描述文件（仍然使用名为 ps_hello 的 vivado 工程）。

```
petalinux-create -t project -n petalinux --template zynqMP
```





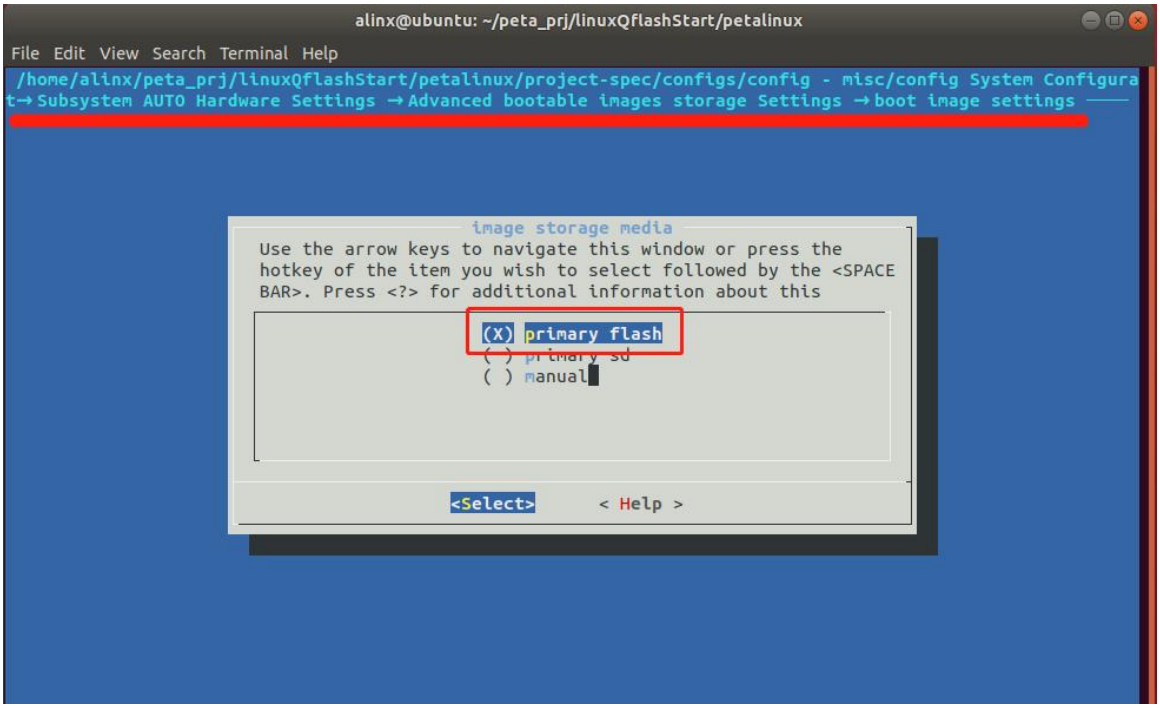


2. 终端中进入 petalinux 文件夹，用下面的命令配置 petalinux 工程的硬件描述文件，打开的配置界面不要配置任何东西直接退出。

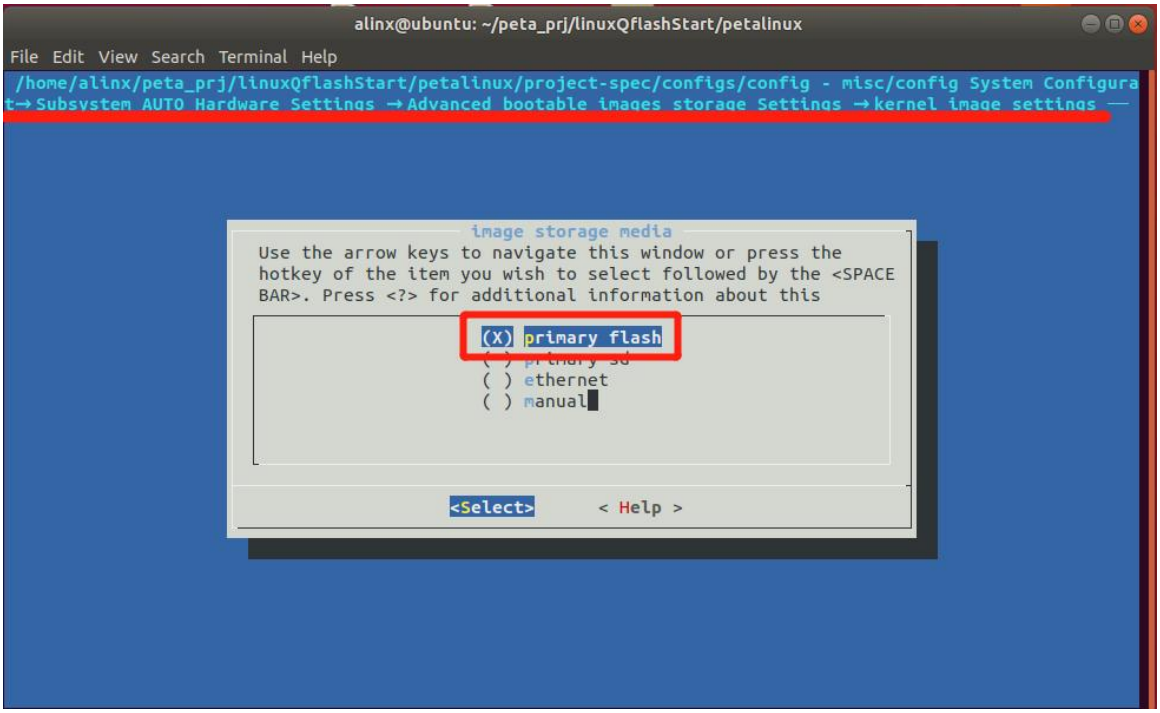
```
petalinux-config    --get-hw-description ../hardware/
```

```
alinx@ubuntu:~/peta_prj/linuxQflashStart$ petalinux-create -t project -n petalinux --template zynqMP
INFO: Create project: petalinux
INFO: New project successfully created in /home/alinx/peta_prj/linuxQflashStart/petalinux
alinx@ubuntu:~/peta_prj/linuxQflashStart$ cd petalinux/
alinx@ubuntu:~/peta_prj/linuxQflashStart/petalinux$ petalinux-config  --get-hw-description ../hardware/
INFO: sourcing build tools
INFO: Getting hardware description...
INFO: Rename design_1_wrapper.xsa to system.xsa
[INFO] generating Kconfig for project
[INFO] menuconfig project
configuration written to /home/alinx/peta_prj/linuxQflashStart/petalinux/project-spec/configs/config

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

[INFO] extracting yocto SDK to components/yocto
[INFO] sourcing build environment
[INFO] generating kconfig for Rootfs
[INFO] silentconfig rootfs
[INFO] generating plnxtool conf
[INFO] generating user layers
[INFO] generating workspace directory
alinx@ubuntu:~/peta_prj/linuxQflashStart/petalinux$
```

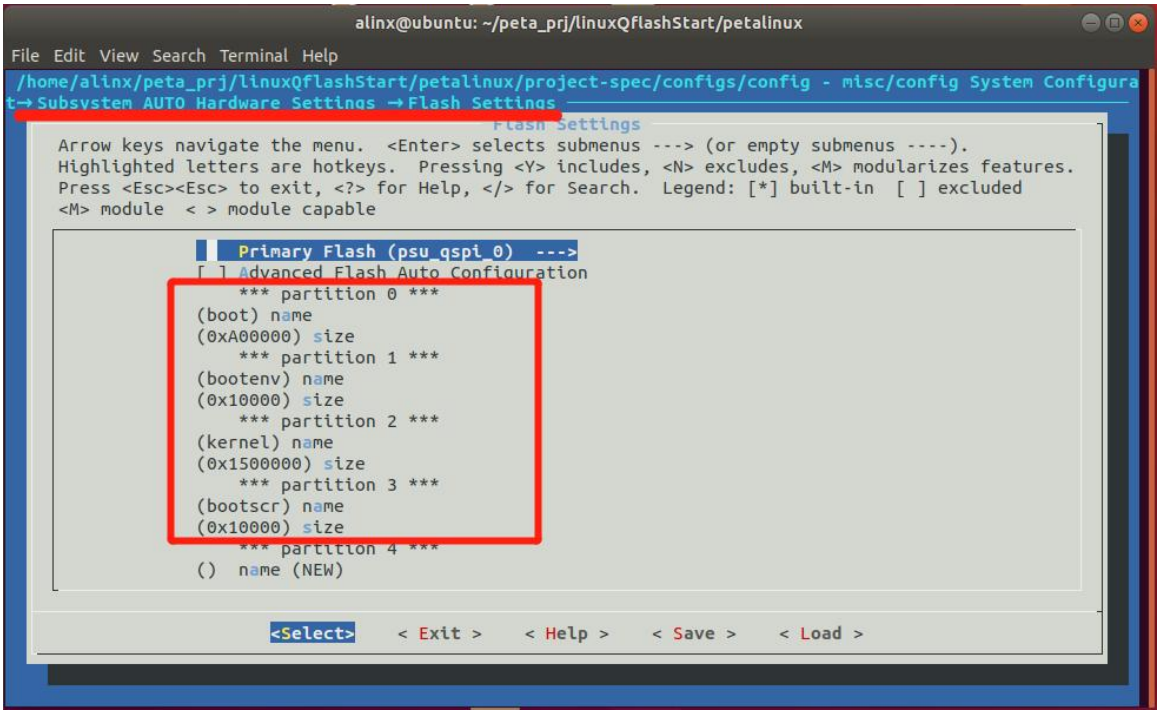
3. 使用 petalinux-config 命令配置 petalinux 工程。在 Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings --->boot image settings ---> image storage media 界面中选择 primary flash。



4. 在 Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings ---> kernel image settings ---> image storage media 选项中选择 primary flash。

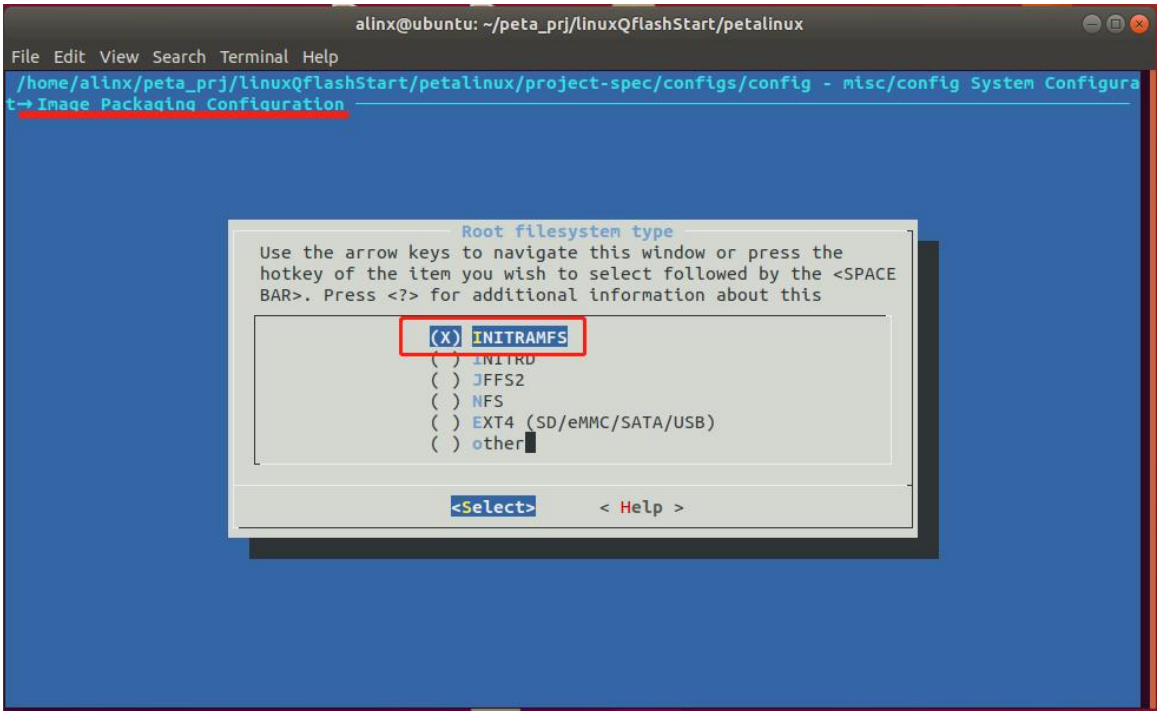


5. Subsystem AUTO Hardware Settings ---> Flash Settings 中，按下图修改 QSPI flash 的分区大小。

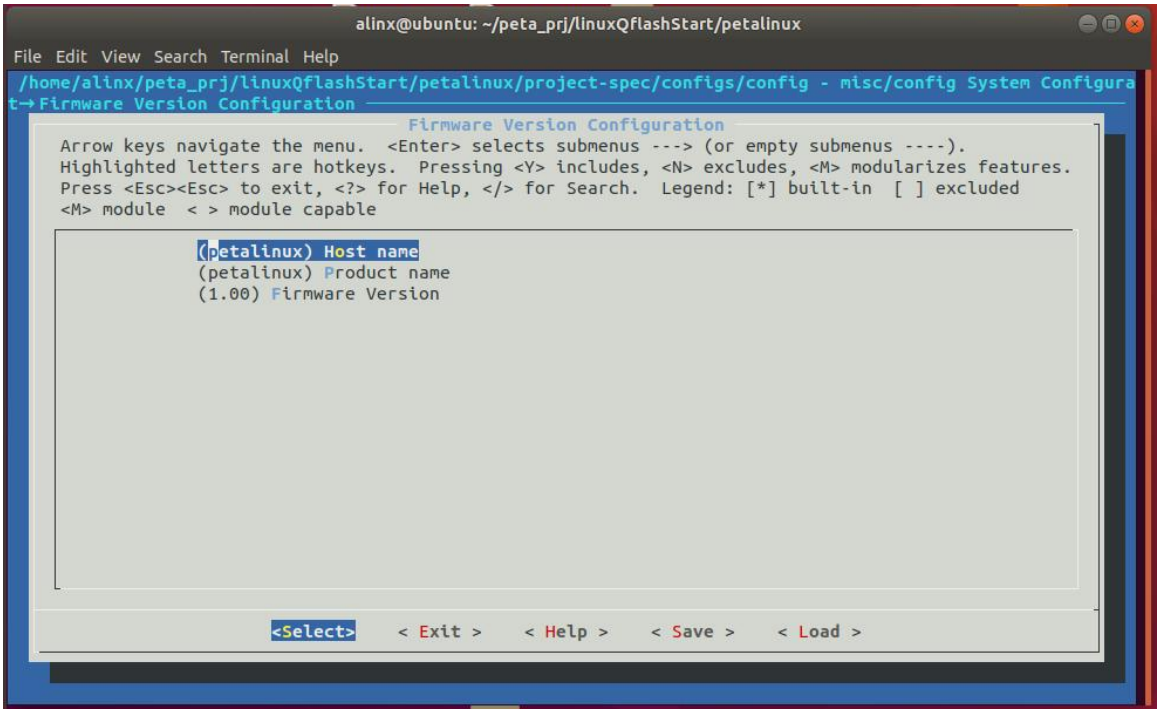


- boot 分区中存放描述 FPGA 的.bit 文件、u-boot、fsbl、设备树.dtb 等文件。boot 分区的大小需要大于这些文件大小总和，需要根据实际情况修改。
- bootenv 分区存放 u-boot 的环境变量，根据实际情况修改。
- kernel 分区存放 image.ub，分区大于 image.ub 即可，需要根据实际情况修改。
- bootscr 分区存放 boot.scr，分区大于 boot.scr 即可，需要根据实际情况修改。
- 另外需要注意，所有打包文件的大小总和，不能超过 flash 的大小。

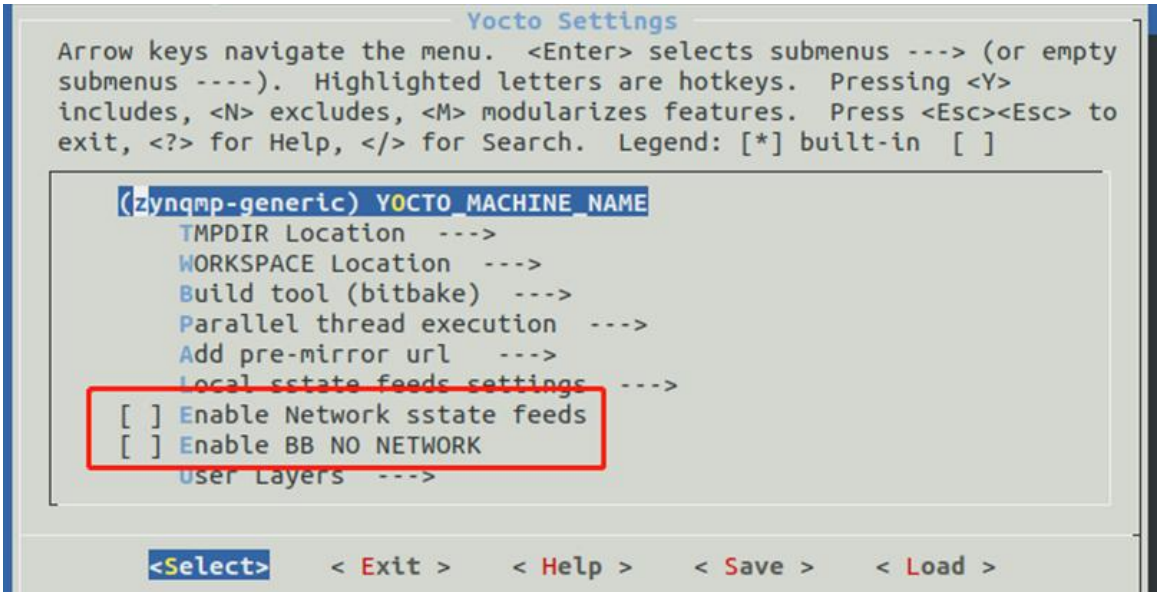
6. 在 Image Packaging Configuration ---> Root filesystem type 选择 INITRAMFS，使用 RAM 类型的根文件系统，这样就可以方便打包烧写到 QSPI Flash。



7. 在 Firmware Version Configuration ---> 中可以修改 Host name 等信息，这里按需修改，教程中保持原状。



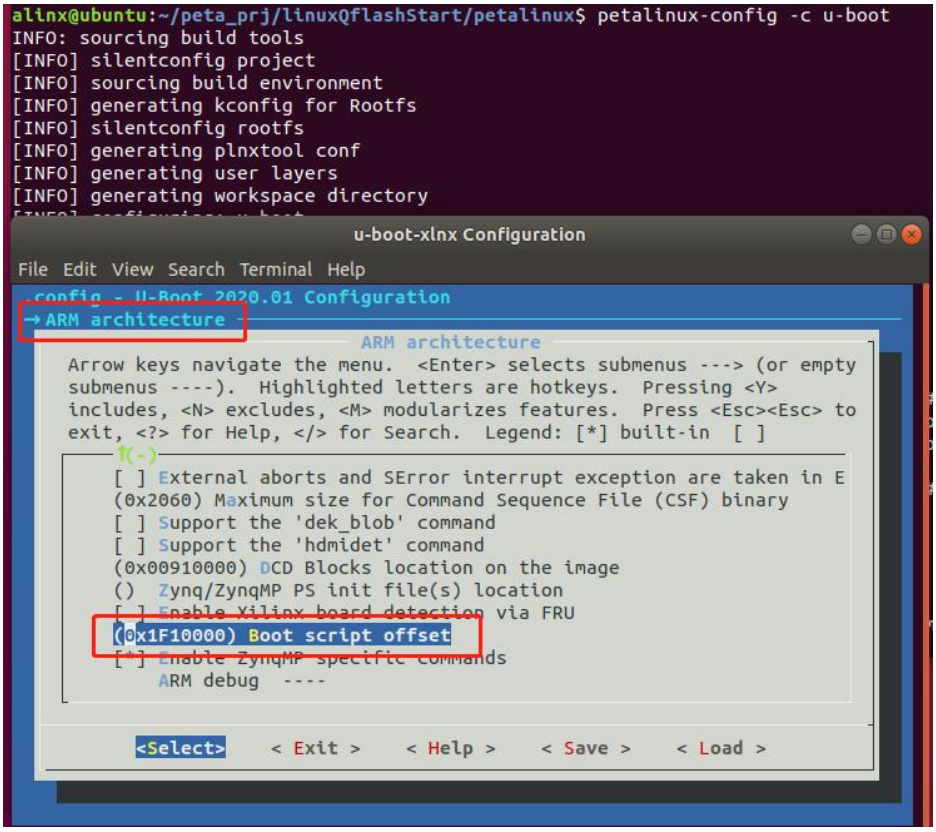
8. 按需设置离线编译。“Yocto Settings”界面配置如下。保存后退出 petalinux-config 的配置界面。



9. 使用 petalinux-config -c u-boot 命令配置 u-boot。配置界面打开后，进入 ARM architecture ---> Boot script offset 界面，设置 bootscr 分区的偏移地址。

偏移地址 = boot 大小 + bootenv 大小 + kernel 大小。

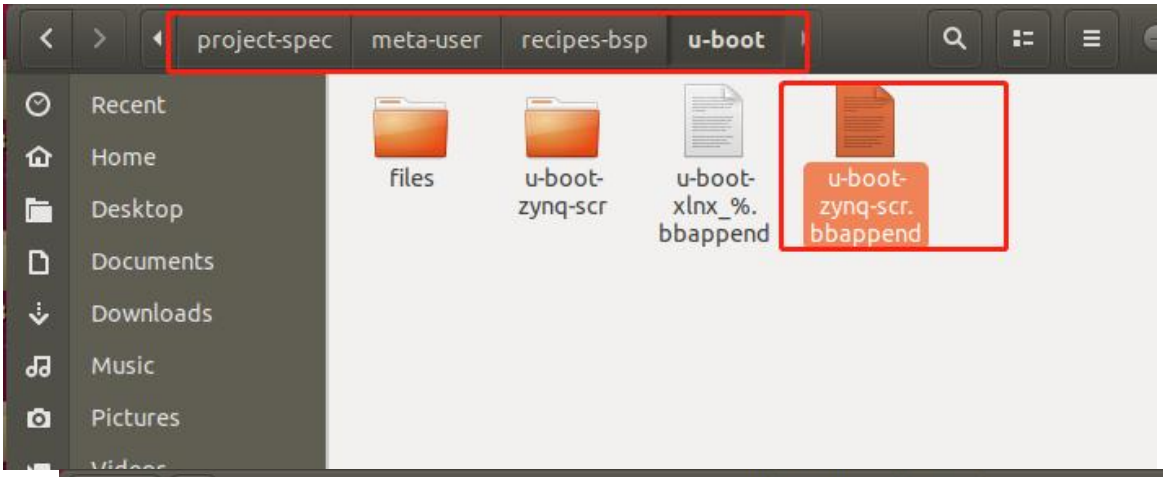
0x1F10000 = 0xA00000 + 0x10000 + 0x1500000。



设置完成后保存退出配置界面。

10. 打开 petalinux 工程路径下的 project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr.bbappend。修改 kernel 偏移地址

和 kernel 大小，与 petalinux-config 命令的“Subsystem AUTO Hardware Settings ---> Flash Settings”中的设置对应。



Open

u-boot-zynq-scr.bbappend

~/peta_prj/linuxQflashStart/petalinux...ject-spec/meta-user/recipes-bsp/u-boot

```
## Below offsets and sizes are based on 128MB QSPI Memory for zynqmp/versal
## For zynqMP
## Load boot.scr at 0x3F80000 -> 62MB of QSPI/NAND Memory
QSPI_KERNEL_OFFSET = "0xA10000"
QSPI_KERNEL_OFFSET_zynqmpdr = "0x3F00000"
QSPI_RAMDISK_OFFSET = "0x4000000"
QSPI_RAMDISK_OFFSET_zynqmpdr = "0x5D00000"

NAND_KERNEL_OFFSET_zynqmp = "0x4100000"
NAND_RAMDISK_OFFSET_zynqmp = "0x7800000"

QSPI_KERNEL_SIZE_zynqmp = "0x1D00000"
QSPI_RAMDISK_SIZE = "0x4000000"
QSPI_RAMDISK_SIZE_zynqmpdr = "0x1D00000"

## For versal
## Load boot.scr at 0x7F80000 -> 127MB of QSPI/NAND Memory
QSPI_KERNEL_OFFSET_versal = "0xF00000"
QSPI_RAMDISK_OFFSET_versal = "0x2E00000"

NAND_KERNEL_OFFSET_versal = "0x4100000"
NAND_RAMDISK_OFFSET_versal = "0x8200000"

QSPI_KERNEL_SIZE_versal = "0x1D00000"
QSPI_RAMDISK_SIZE_versal = "0x4000000"

QSPI_KERNEL_IMAGE_zynq = "image.ub"
QSPI_KERNEL_IMAGE_zynqmp = "image.ub"
QSPI_KERNEL_IMAGE_versal = "image.ub"

NAND_KERNEL_IMAGE = "image.ub"

FIT_IMAGE_LOAD_ADDRESS = "${@append_baseaddr(d,"0x10000000")}"

QSPI_FIT_IMAGE_LOAD_ADDRESS = "${@append_baseaddr(d,"0x10000000")}"
QSPI_FIT_IMAGE_SIZE = "0x1500000"
QSPI_FIT_IMAGE_SIZE_zynqmpdr = "0x3F00000"
QSPI_FIT_IMAGE_SIZE_zynq = "0xF00000"
```

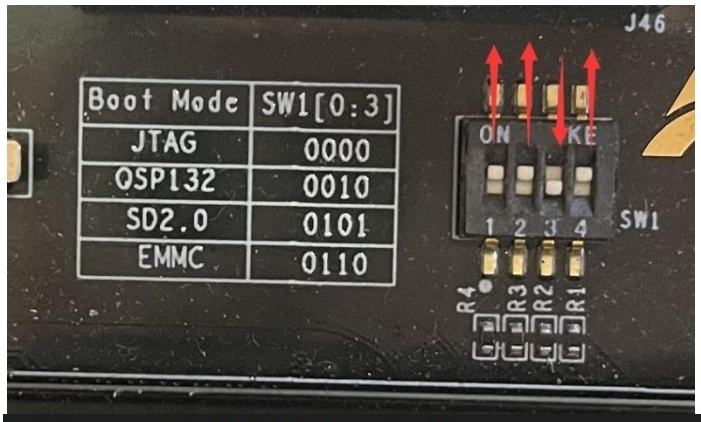
11. 保存后，使用 petalinux-build 命令编译工程。
12. 编译成功后，使用下面命令合成 BOOT.bin，使用--kernel 选项把内核编译进 BOOT.bin。使用--add 选项把 boot.scr 编译进内核，注意 boot.scr 后需要通过--offset 选项设置偏移地址，与步骤 8 中的设置对应。因为 vivado 工程中没有 pl 端所以参数中没有—fpga。

```
petalinux-package --boot --fsbl ./images/linux/zynqmp_fsbl.elf --u-boot --kernel --add ./images/linux/boot.scr --offset 0x01F10000 --force
```

```
alinx@ubuntu:~/peta_prj/linuxQflashStart/petalinux$ petalinux-package --boot --fsbl ./images/linux/zynqmp_f
sbl.elf  --u-boot --kernel --add ./images/linux/boot.scr --offset 0x01F10000  --force
INFO: sourcing build tools
INFO: Getting system flash information...
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/zynqmp_fsbl.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/pmufw.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/bl31.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/system.dtb"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/image.ub"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashStart/petalinux/images/linux/boot.scr"
INFO: Generating zynqmp binary package BOOT.BIN...


****** Xilinx Bootgen v2020.1
  **** Build date : May 26 2020-14:07:15
    ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.


[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
alinx@ubuntu:~/peta_prj/linuxQflashStart/petalinux$
```

13. 参考 vitis 教程中的方法把 BOOT.bin 烧录到开发板中。

14. 把开发板上的拨码开关设置为 QSPI 启动后再给开发板上电。




```
haveged: haveged: fills: 0, generated: 0

[   17.437589] random: crng init done
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCm93TD3Rwx6gG+j/DPrTuzu/8t7A5dfHiZLacCbLNs
fKQvMqWOmrhT+CN4KKCks9uMX65Bc7Zmm488swcaYfGLBuU46WBgPKODoK0yNDYxtZhtgp5T9Z2Y8yIl
id1w+CLkYV5syNtl86kklQ7cJNpijBnG2EdbB5GXugVtQNLXBbzQ6+xD32t1yy3oDguzJqAuhvXhFSmI
ujX+mffJOzOLfZcEWDzSXZ6m+uuqmnWUY83fSvJhQWecTltNS174p3PlsgdbggM5PPDjXmKynUoabT3z
RkEozQJF3+qpcjOqWBWJvlK9u5CC6CKSCFN2on1ZA2Egeha1ysYnPynzvOSN root@petalinux
Fingerprint: sha1!! 57:e4:6c:1a:71:e2:9e:1f:96:44:85:ba:d8:8e:b4:3a:4c:41:5c:02
dropbear.
Starting internet superserver: inetd.
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2020.1 petalinux /dev/ttyPS0

petalinux login:
```

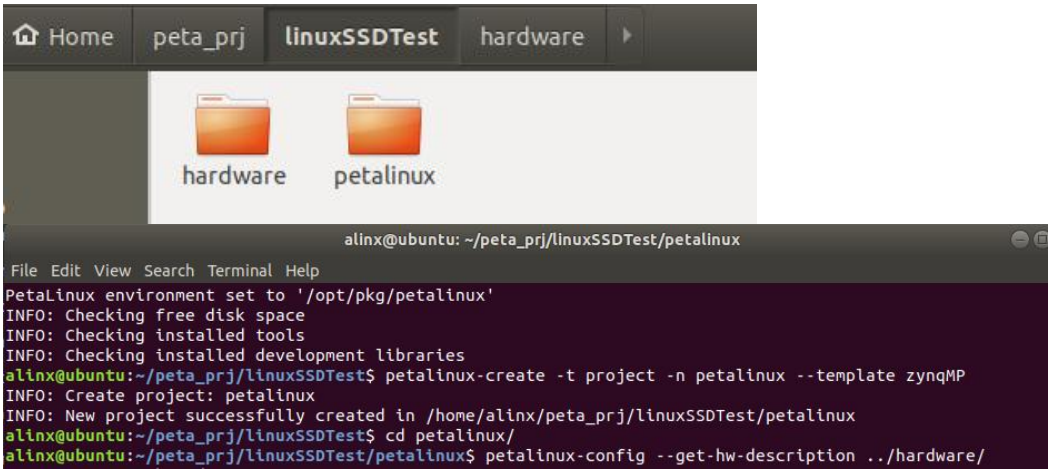
第十一章 Linux 下的 NVMe SSD 操作

注意：在使用 SSD 时，请勿触摸 ssd 表面，以免静电损坏器件导致 SSD 无法正常使用。

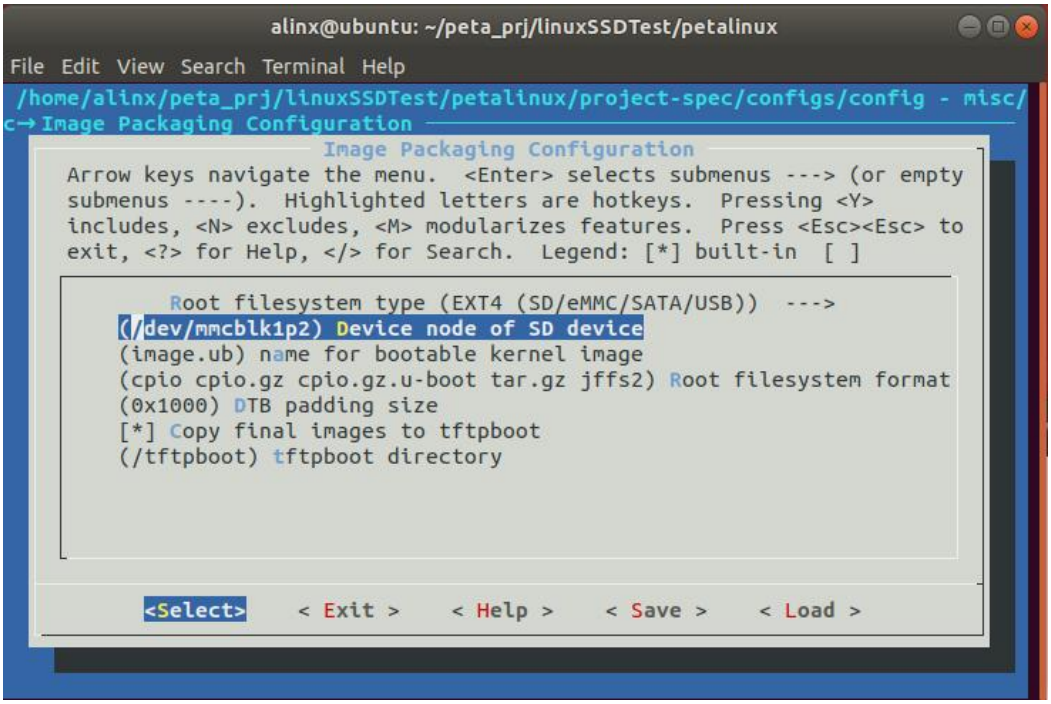
11.1 petalinux 配置

11.1.1 新建 petalinux 工程

使用第一章中使用的 xsa 文件(也就是“Vitis 应用教程”源码包中的“ps_hello”工程)来配置 petalinux 工程。



设置成从 SD 卡挂载根文件系统(具体方法参考本教程“第八章 SD 卡根文件系统”)。



修改设备树。

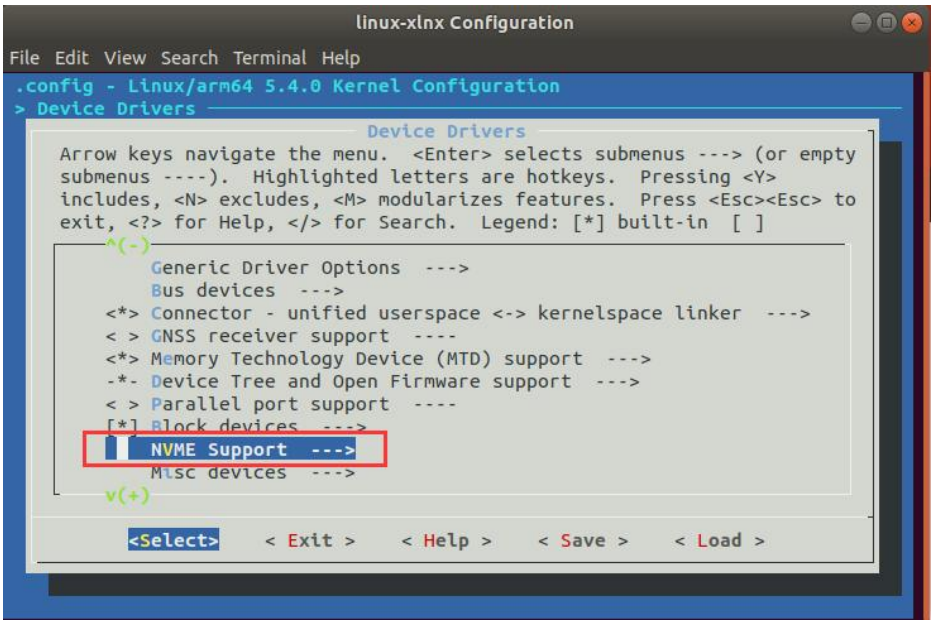
```
/include/ "system-conf.dtsi"
/ {
};

/* SD */
&sdhci1 {
	disable-wp;
	no-1-8-v;
};

/* USB */
&dwc3_0 {
	status = "okay";
	dr_mode = "host";
};
```

11.1.2 配置 Linux 内核

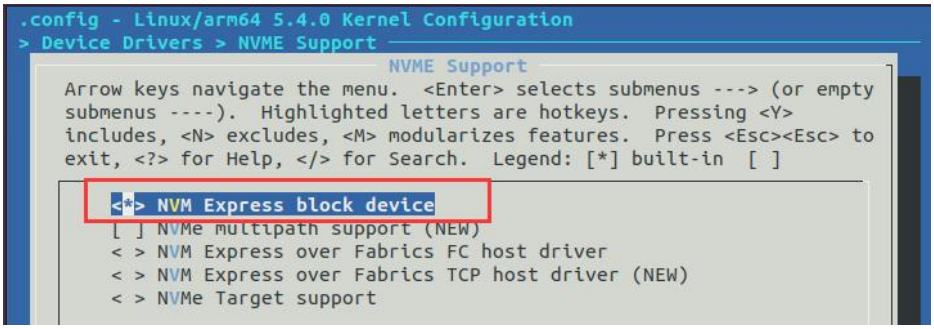
使用 M.2 接口的 SSD，需要内核加载 PCIe 和 NVME 驱动。PCIe 驱动是默认加载的，还需要配置 NVME 驱动。在 petalinux 工程目录下输入下面的命令打开内核配置界面。

```
petalinux-config    -c    kernel
```

在 Device Drivers 中选择 NVME Support。



给 NVM Express block device 选项打上*号。保存退出配置界面。

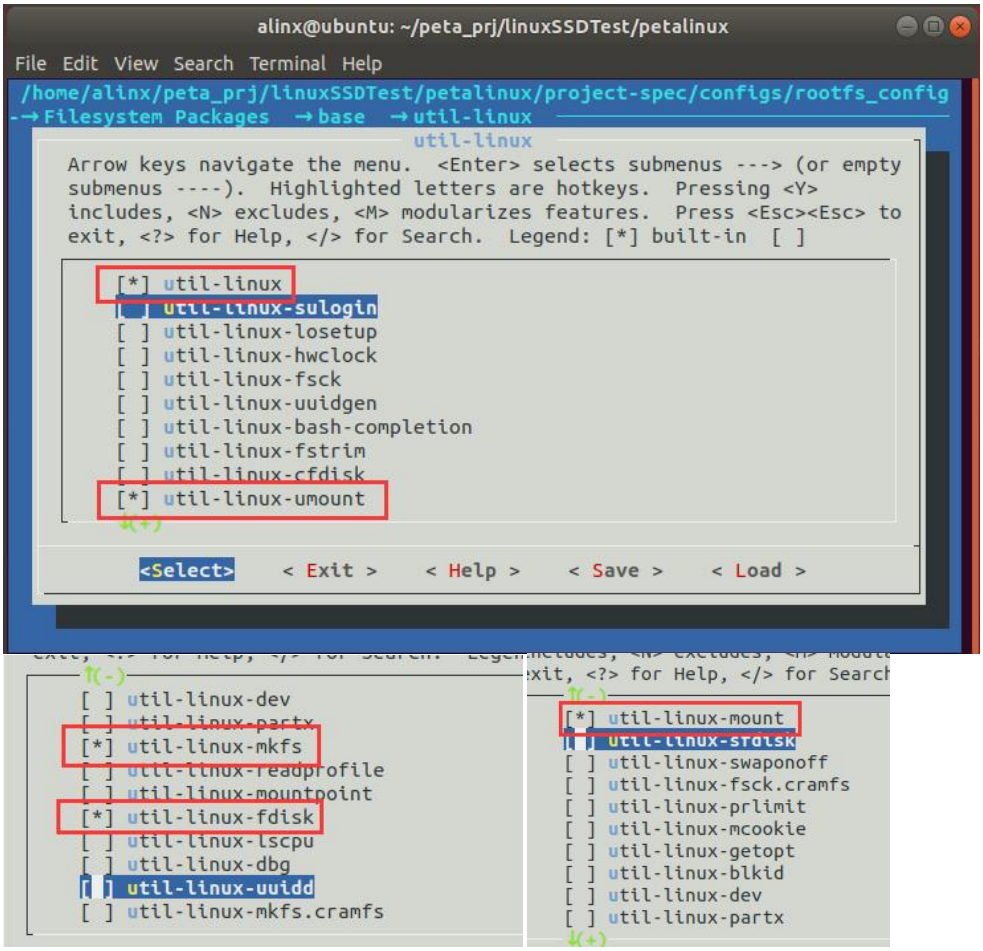


11.1.3 配置根文件系统工具

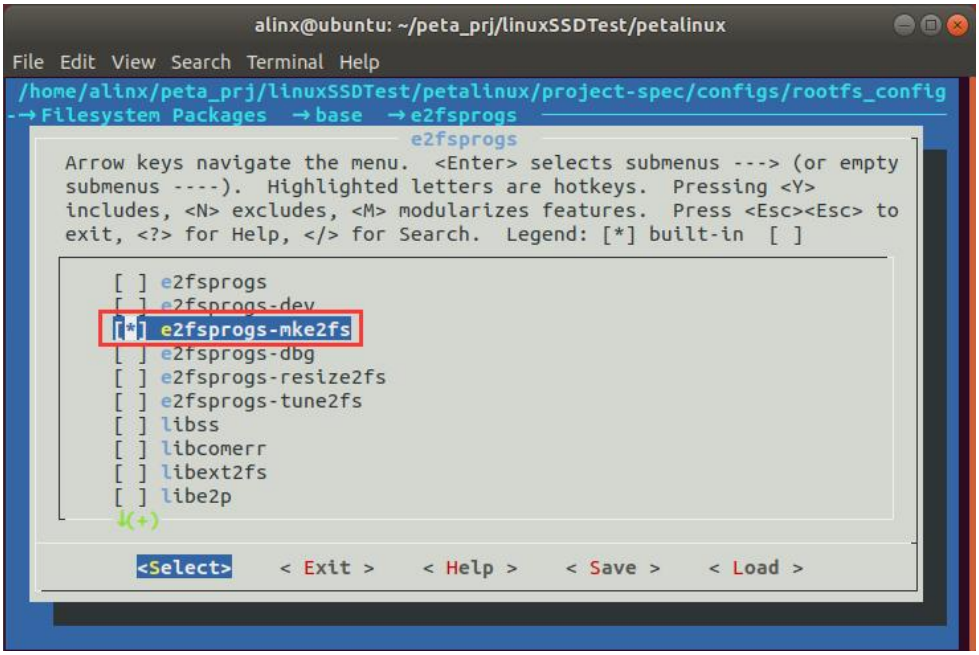
使用 SSD 还需要用到文件系统制作工具，使用下面的命令来配置文件系统，在文件系统中选择号文件系统制作工具。

```
petalinux-config -c rootfs
```

进入 Filesystem Packages → base → util-linux，按下图中选择选项。



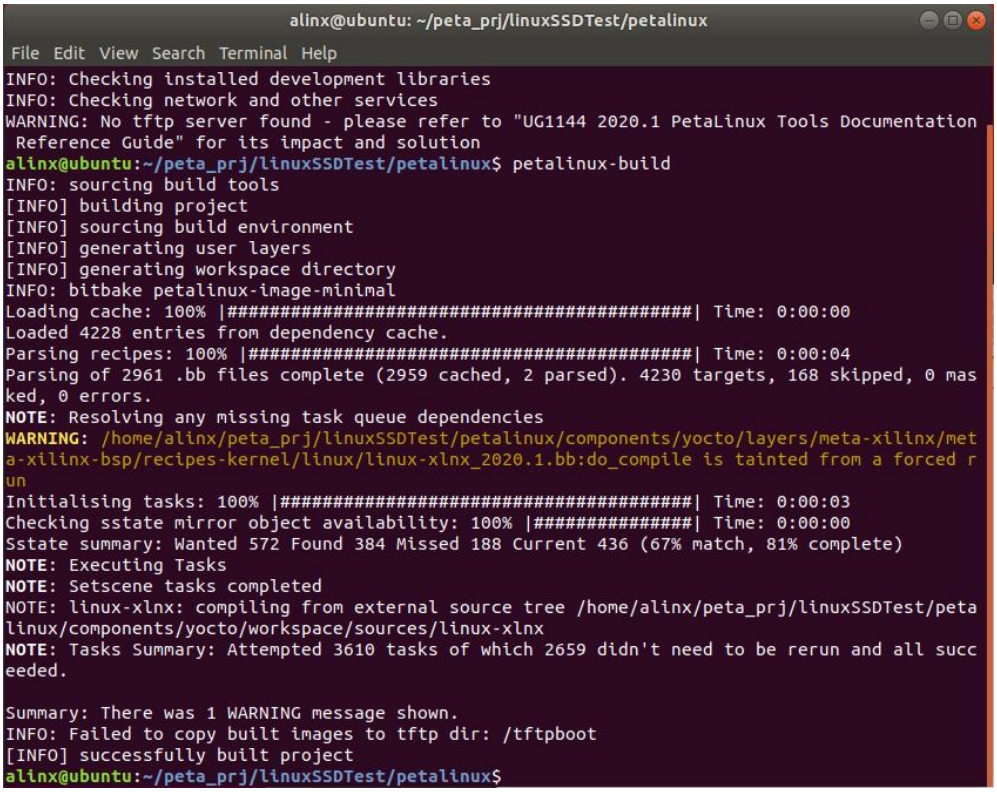
进入 Filesystem Packages → base → e2fsprogs，选上 e2fsprogs-mke2fs。



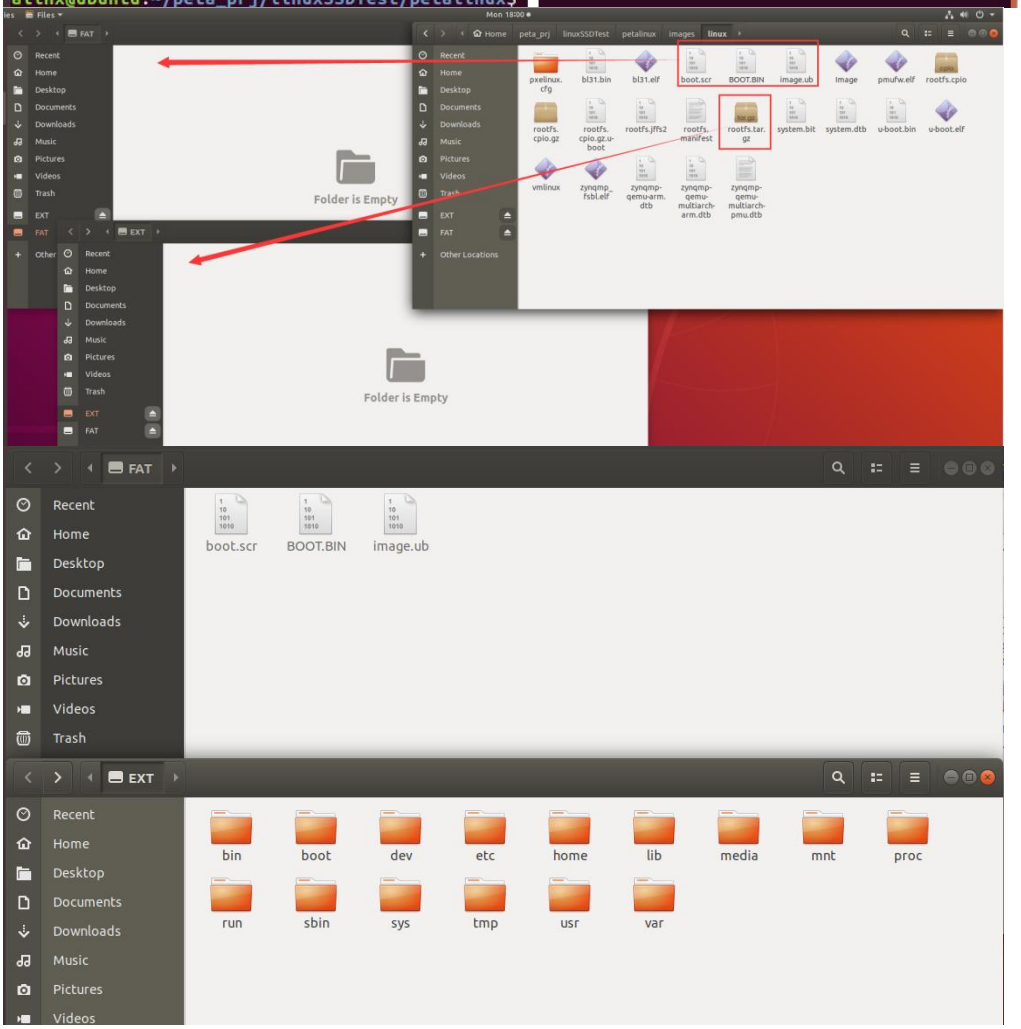
保存退出配置界面。

11.1.4.

编译 petalinux 工程，然后打包 BOOT.bin。把 petalinux 工程 image/linux 路径中的 image.ub、BOOT.bin 和 boot.scr 拷贝到 SD 卡的 FAT 分区中。把 rootfs.tar.gz 解压到 SD 卡的 EXT 分区中，具体操作可以参考本教程的“第八章 SD 卡根文件系统”。



```
alinx@ubuntu:~/peta_prj/linuxSSDTest/petalinux$ petalinux-package --boot --u-boot --fsbl --f
pga --force
INFO: sourcing build tools
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/images/linux/zynqmp_fsb
l.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/images/linux/pmufw.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/project-spec/hw-descrip
tion/top.bit"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/images/linux/bl31.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/images/linux/system.dtb
"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxSSDTest/petalinux/images/linux/u-boot.elf
"
INFO: Generating zynqmp binary package BOOT.BIN...


****** Xilinx Bootgen v2020.1
  **** Build date : May 26 2020-14:07:15
    ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.


[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
alinx@ubuntu:~/peta_prj/linuxSSDTest/petalinux$
```



11.2 设置分区

1. 把刚才制作的 SD 卡插上开发板，然后插上 SSD 并启动板卡，运行下面的命令查看 SSD 设备是否正常加载。

```
ls  /dev/nvme*
```

```
root@petalinux:~# ls /dev/nvme*
/dev/nvme0     /dev/nvme0n1
```

2. 操作/dev/nvme0n1 给 SSD 设置分区。

```
fdisk /dev/nvme0n1
```

```
root@petalinux:~# fdisk /dev/nvme0n1

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help):
```

输入 n 创建新的分区，输入 p 创建一个主分区，分区号、起始扇区和结束扇区这里实验中都保持默认，直接按回车。

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1):
First sector (2048-500118191, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-500118191, default 500118191):

Created a new partition 1 of type 'Linux' and of size 238.5 GiB.

Command (m for help):
```

3. 输入 wq，保存设置并退出。

```
Command (m for help): wq
```

4. 使用下面的命令制作分区文件系统(格式化分区)。

```
mkfs  -t  ext4  /dev/nvme0n1p1
```

```
root@petalinux:~# mkfs -t ext4 /dev/nvme0n1p1
mke2fs 1.45.3 (14-Jul-2019)
Discarding device blocks: done
Creating filesystem with 62514518 4k blocks and 15630336 inodes
Filesystem UUID: 6d822d51-1b67-4c09-89d8-4c44f87e820d
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000, 7962624, 11239424, 20480000, 23887872

Allocating group tables: done
Writing inode tables: done
Creating journal (262144 blocks): done
Writing superblocks and filesystem accounting information: done

root@petalinux:~#
```

5. 新建一个文件夹，使用 mount 命令挂载分区。

```
mkdir  -p  /run/media/nvme0n1p1
mount  /dev/nvme0n1p1  /run/media/nvme0n1p1
```

```
root@petalinux:~# mkdir -p  /run/media/nvme0n1p1
root@petalinux:~# mount /dev/nvme0n1p1 /run/media/nvme0n1p1
[  457.532964] EXT4-fs (nvme0n1p1): mounted filesystem with ordered data mode. Opts: (null)
root@petalinux:~#
```

11.3 读写测速

SSD 格式化以及挂载之后就可以使用 time 和 dd 命令来读写测速了。(新的 linux 系统中需要用 time 命令打印时间自己算速度。)

1. cd 进入挂载的路径。

```
cd   /run/media/nvme0n1p1
```

```
root@petalinux:~# cd /run/media/nvme0n1p1/
root@petalinux:/run/media/nvme0n1p1#
```

2. 顺序写测试

```
time   dd   if=/dev/zero   of=./testfile1   bs=100k   count=1310
```

```
root@petalinux:/run/media/nvme0n1p1# time dd  if=/dev/zero  of=./testfile1  bs=100k  count=1310
1310+0 records in
1310+0 records out

real    0m0.632s
user    0m0.000s
sys     0m0.622s
root@petalinux:/run/media/nvme0n1p1#
```

写速度为（1310 * 100) / 1024 / 0.632 = 202.42 M/s

3. 顺序读测试，测试之前注意清理缓存。

```
echo 3 > /proc/sys/vm/drop_caches
time   dd   if=./testfile1   of=/dev/null   bs=100k
```

```
root@petalinux:/run/media/nvme0n1p1# echo 3 > /proc/sys/vm/drop_caches
[ 1947.920011] sh (628): drop_caches: 3
root@petalinux:/run/media/nvme0n1p1#
root@petalinux:/run/media/nvme0n1p1#
root@petalinux:/run/media/nvme0n1p1# time  dd  if=./testfile1  of=/dev/null  bs=100k
1310+0 records in
1310+0 records out

real    0m0.378s
user    0m0.000s
sys     0m0.182s
root@petalinux:/run/media/nvme0n1p1#
```

读速度为（1310 * 100) / 1024 / 0.378 = 338.43 M/s

4. 使用 dd 命令拷贝文件，测试读写速度。

```
echo 3 > /proc/sys/vm/drop_caches
time   dd   if=./testfile1   of=./testfile2   bs=100k
```

```
root@petalinux:/run/media/nvme0n1p1# echo 3 > /proc/sys/vm/drop_caches
[ 2148.095223] sh (628): drop_caches: 3
root@petalinux:/run/media/nvme0n1p1# time  dd  if=./testfile1  of=./testfile2  bs=100k
1310+0 records in
1310+0 records out

real    0m0.719s
user    0m0.004s
sys     0m0.671s
root@petalinux:/run/media/nvme0n1p1#
```

读写速度为（1310 * 100) / 1024 / 0.719 = 177.2 M/s

第十二章 可远程调试的 Hello World

12.1 GDB 简介

GDB, the GNU Project debugger，允许您查看程序在执行时“内部”发生了什么，或者一个程序在崩溃时正在做什么。主要功能如下：

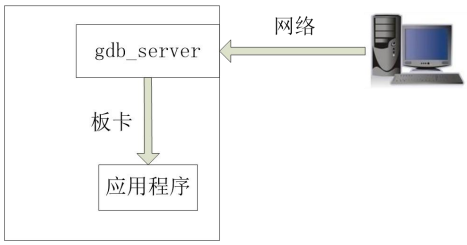
① 启动程序，指定任何可能影响其行为的内容，如环境变量，运行参数等

② 使程序在指定条件下停止运行

③ 检查程序停止时发生了什么，如寄存器状态，变量值等

④ 手动修改程序运行中的各种值，这样可以更方便的验证程序功能

Linux 下的 GNU 调试工具是 gdb、gdbserver。其中 gdb 和 gdbserver 可完成对目标板上 Linux 下应用程序的远程调试。 gdbserver 是一个很小的应用程序，运行于目标板上，可监控被调试进程的运行，可通过网络与上位机上的 gdb 通信。开发者可以通过上位机的 gdb 输入命令，控制目标板上进程的运行，查看内存和寄存器的内容



12.2 QtCreator 简介

Qt Creator 是一个跨平台的集成开发环境（IDE），允许开发人员跨桌面、移动和嵌入式平台创建应用程序。

我们通常说的 Qt 库，则是我们开发桌面等应用而调用的库，它将会被链接进我们的运行程序。而 Qt Creator 是一个工具，帮助我们开发应用程序的工具。所以，两者可以认为是完全不一样的东西。

12.3 实验目标

① 认识 Qt Creator 工程

② QtCreator 配置 GDB 连接板卡

③ 将可运行的目标程序搬移至板卡

④ 给运行程序传递参数

12.4 Qt Creator 工程

Qt Creator 的工程文件是 xxx.pro，文件中”QT=”指定需要用到的 QT 库组件，如果为空，则表示不调用 QT 库。

12.5 获取板卡的 IP 地址

(1) 将我们的开发板，装好有 linux 系统的 sd 卡并上电

(2) 查看 ip 地址

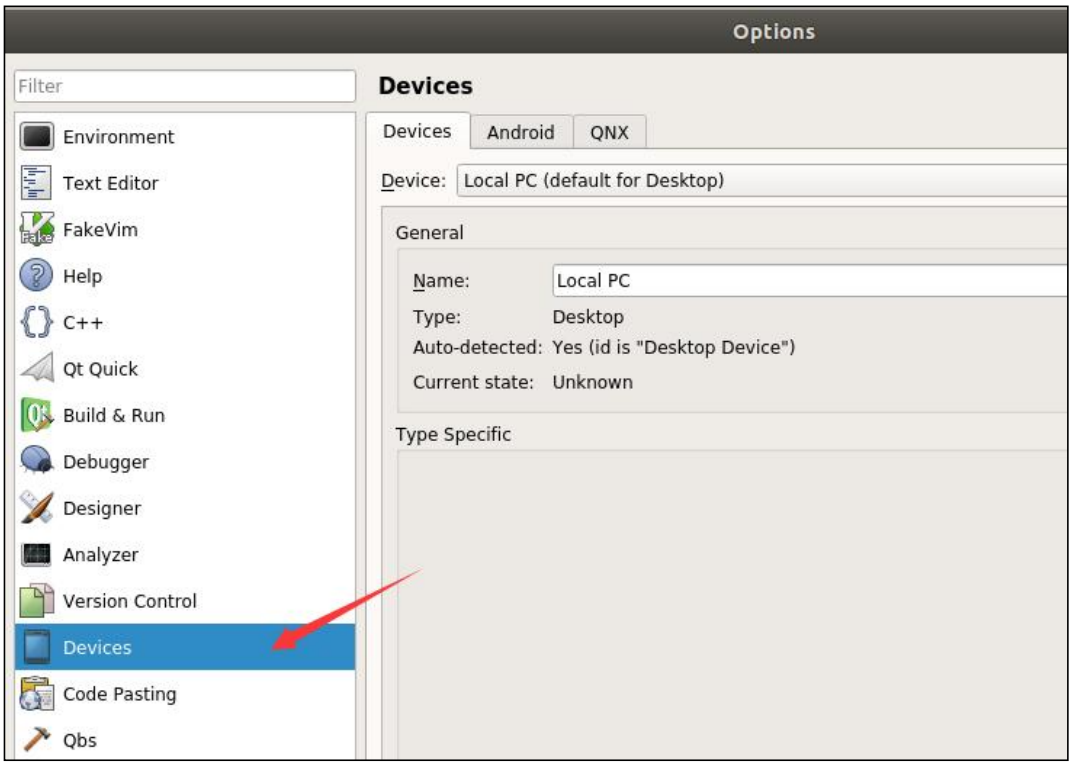
可通过“ifconfig”命令查看板卡的 ip 地址，系统默认启用 DHCP，如果本地不支持 DHCP 服务，则查询结果如下图：

```
root@petalinux:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:22:01
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:29
```

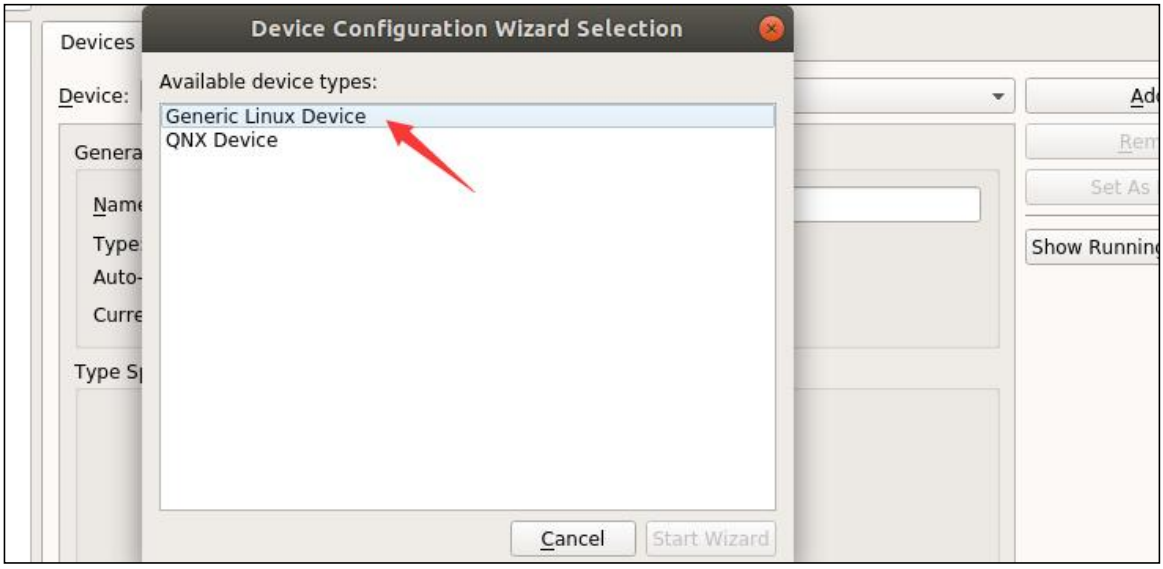
此时需要手动分配一个 ip 地址，可使用命令“iconfig eth0 192.168.1.45 netmask 255.255.255.0”。这里我们设置板卡的 ip 地址为：192.168.1.45

12.6 QtCreator 配置 GDB 连接板卡

(1) 打开 QtCreator 的工具设置项(Tools-->Options)



(2) 点击”Add...”按钮，选择“Generic Linux Device”



(3) 参数设置如下

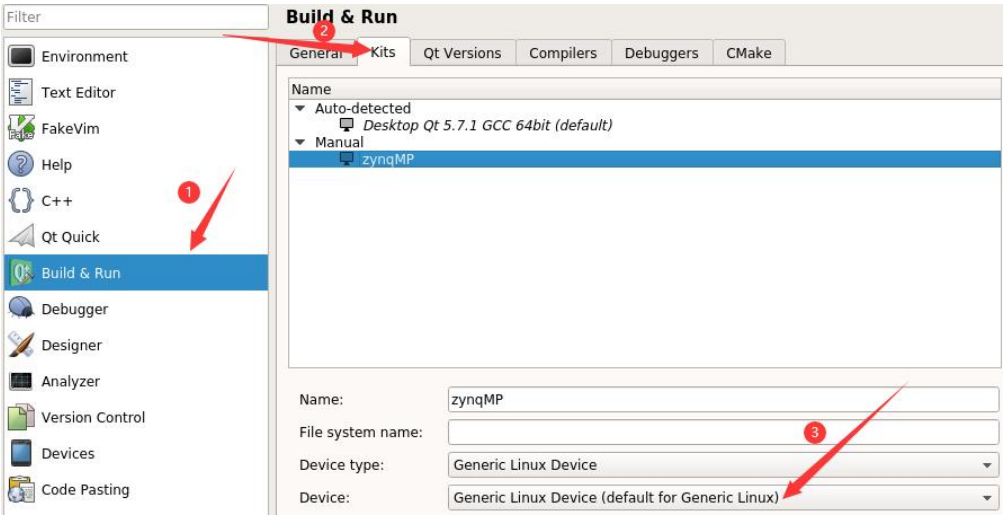
其中密码默认是 root，需要填写

(4) 点击 “Next” 及 “Finish”，后面出现的测试中，成功后的界面如下

如果没有成功，则需要检测网络连接

(5) 回到”Device”的设置项，点击“Apply”，完成此项设置

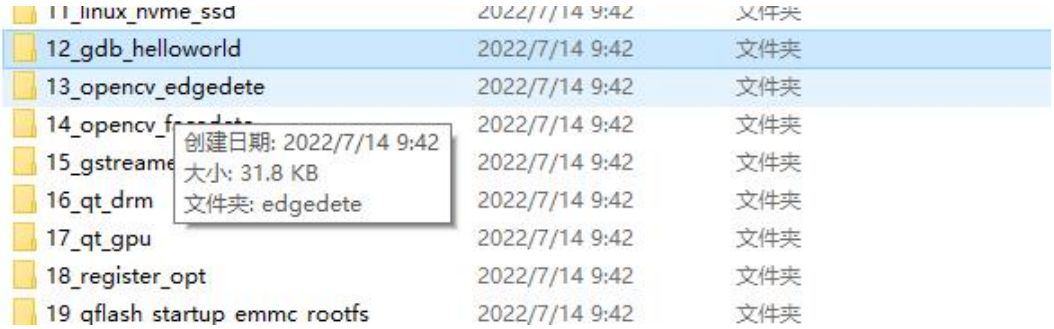
(6) 切换设置项及配置如下

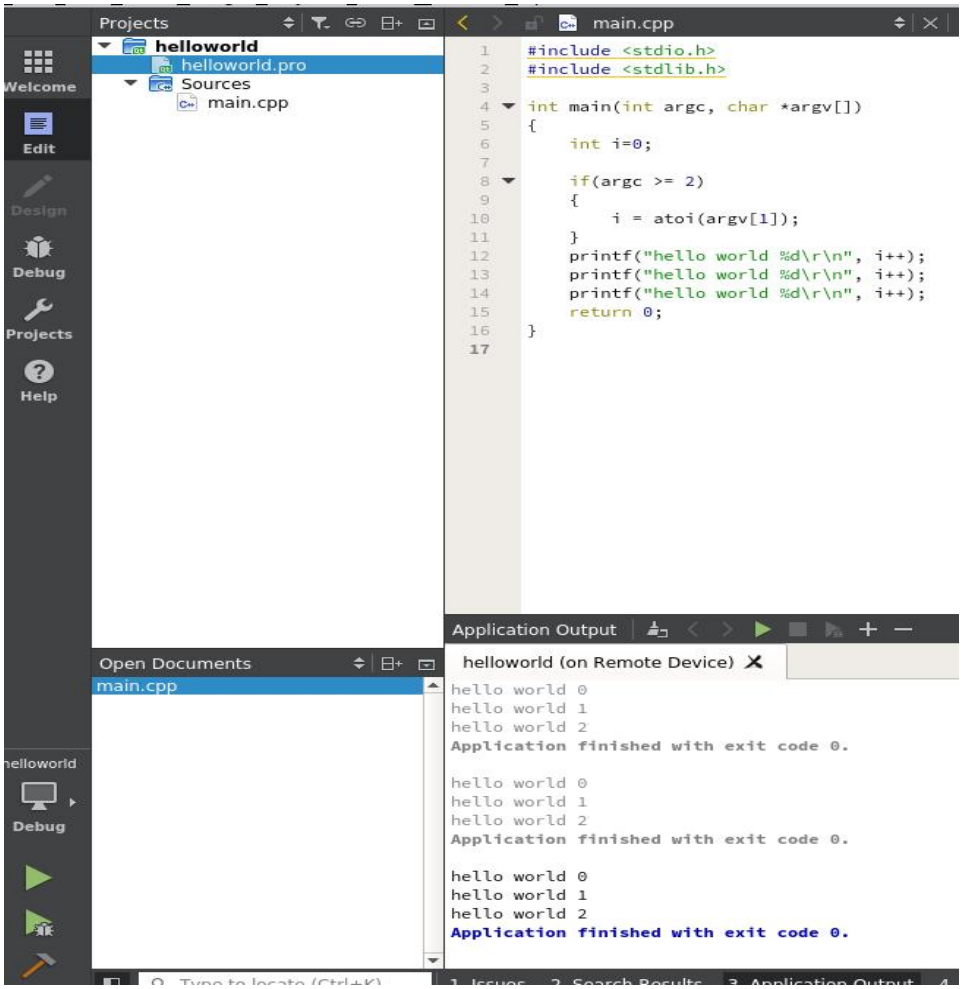


(7) 点击”OK”，完成设置

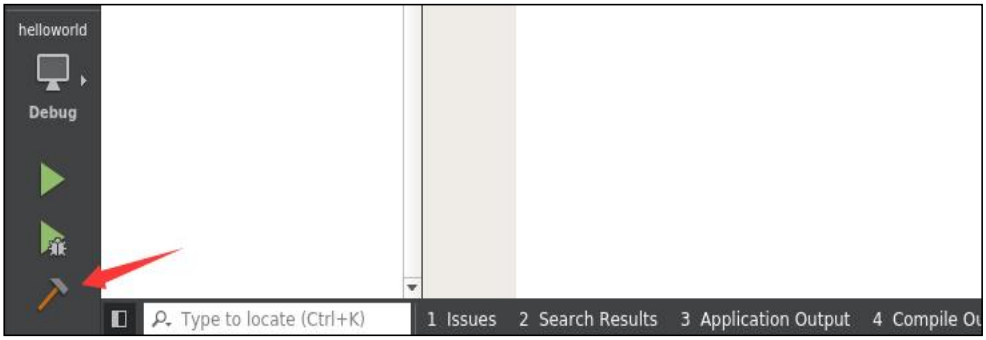
12.7 运行 Hello World

(1) 打开 helloworld 工程，在提供的第十二章节源码包里面，将这个文件夹拖进虚拟机，然后运行里面的 pro 工程，然后点里面的 Sources 下的 main.cpp 编译，完成后点上面的.pro

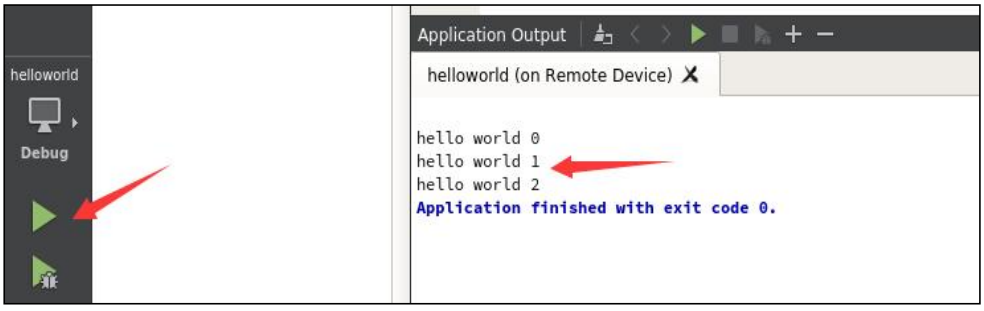




(2) 编译应用程序



(3) 点击 QtCreator 的运行按钮



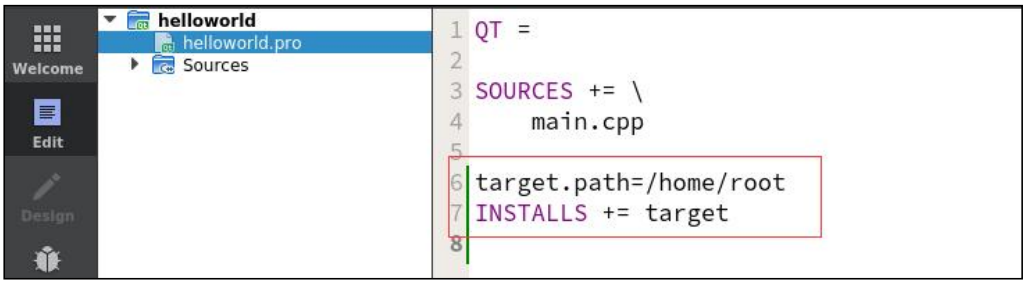
我们可以看到，右边的运行打印信息，说明程序已经运行并退出。

① 如果提示如下错误，则需要检查板卡是否上电并进入 linux 系统，网络是否正常

② 若提示如下错误，则需要清除编译结果，并重新编译

(4) 此时我们查看板卡的用户目录”/home/root”，发现多出一个文件 helloworld

(5) 我们打开工程的 pro 文件

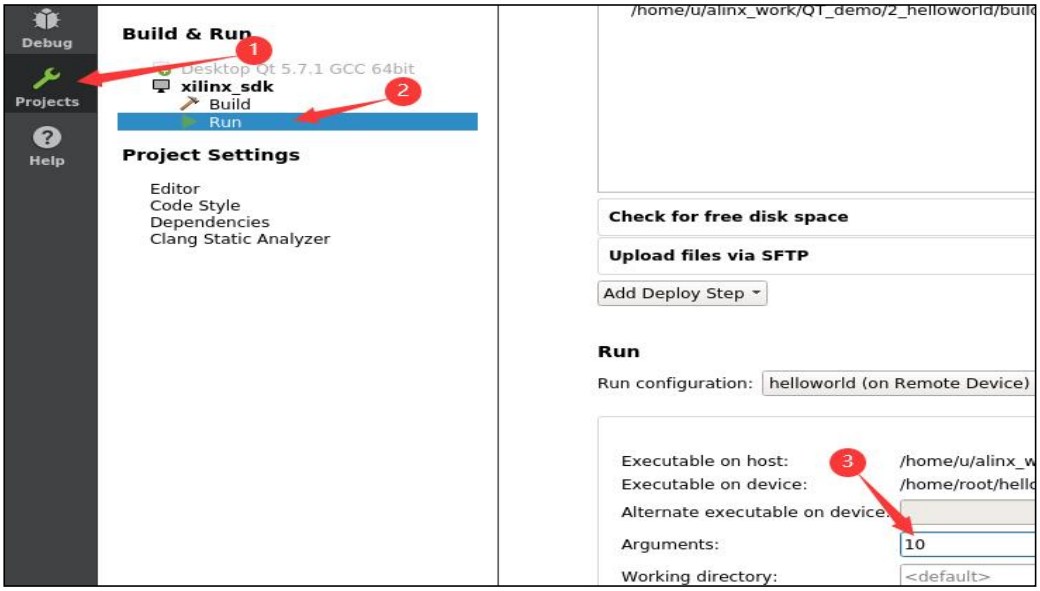


红色框中，是运行时，程序自动加载的目录，这就解释了为什么会在开发板系统”/home/root”目录下出现文件 helloworld

12.8 运行参数设置

有时候，我们运行程序，需要在后面加上一些运行参数，下面我们介绍如何在远程调试时，为运行程序加上运行参数

(1) 打开如下设置项

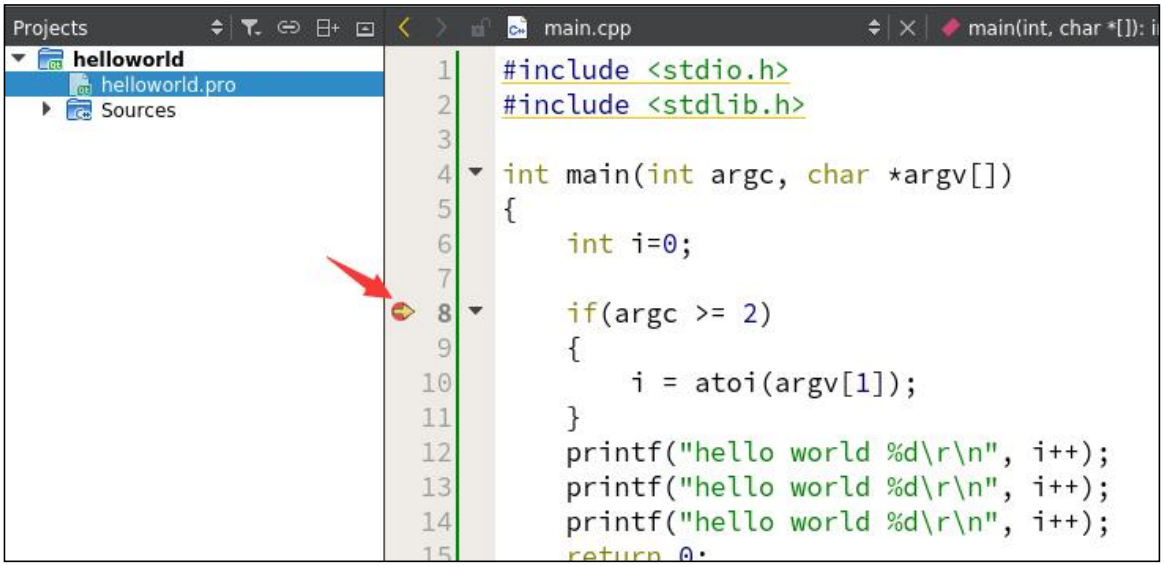


此处参数设置为 10

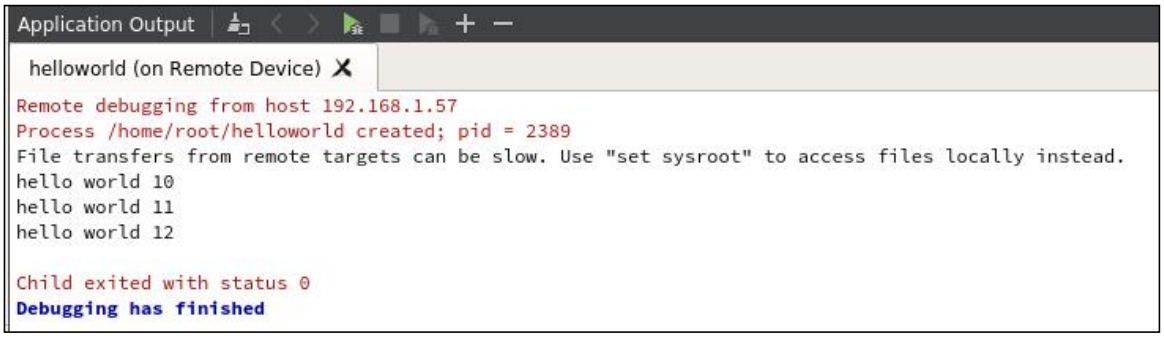
(2) 在 main 函数 if 位置设置一个断点

(3) 点击 debug 调试按钮或快捷键 F5

(4) 我们看到，程序已经停止在我们设置的断点处



(5) 按 F5 继续运行，我们看到打印信息如下

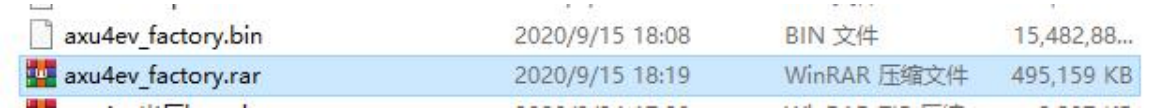


我们看到，”hello world”后面的开始数字已经变为 10，表示我们设置的参数已经起作用

第十三章 OpenCV 之边缘检测

注意：从本章到第十八章都需要用出厂系统，具体步骤如下：

找到此文件夹解压，再解压这个文件：



接着参考下面的 PDF 文档制作出厂系统。

第一次上电会有配置，执行下列语句，再对开发板 Reset，就会更快的进入系统。

```
rm -rf ./*
```

13.1 OpenCV 简介

OpenCV（开源计算机视觉库）是一个开源的计算机视觉和机器学习软件库。OpenCV 的建立是为了为计算机视觉应用提供一个通用的基础设施，并加速机器感知在商业产品中的应用。作为一个 BSD 许可的产品，OpenCV 使企业可以很容易地利用和修改代码。

该库有 2500 多种优化算法，其中包括一整套经典和最先进的计算机视觉和机器学习算法。这些算法可用于检测和识别人脸、识别物体、对视频中的人类行为进行分类、跟踪摄像机运动、跟踪运动物体、提取物体的三维模型、从立体摄像机生成三维点云、将图像缝合在一起以生成整个场景的高分辨率图像，从图像库中查找相似的图像，从使用闪光灯拍摄的图像中移除红眼，跟踪眼球运动，识别景物并建立标记以覆盖增强现实等。OpenCV 拥有超过 47000 个用户社区，估计下载量超过 1800 万。广泛应用于公司、研究团体和政府机构。

Xilinx 的 reVision 的很多实现，就是基于 opencv 的接口，其中的硬件实现是 xfopencv。这将有助于我们实现相应算法的硬件(FPGA)加速的实现。

13.2 边缘检测简介

边缘检测是图像处理和计算机视觉中的基本问题，边缘检测的目的是标识数字图像中亮度变化明显的点。图像属性中的显著变化通常反映了属性的重要事件和变化。 这些包括深度上的不连续、表面方向不连续、物质属性变化和场景照明变化。 边缘检测是图像处理和计算机视觉中，尤其是特征提取中的一个研究领域。

图像边缘信息主要集中在高频段，通常说图像锐化或检测边缘，实质就是高频滤波。我们知道微分运算是求信号的变化率，具有加强高频分量的作用。在空域运算中来说，对图像的锐化就是计算微分。由于数字图像的离散信号，微分运算就变成计算差分或梯度。图像处理中有多种边缘检测（梯度）算子，常用的包括普通一阶差分，Robert 算子（交叉差分），Sobel 算子

等等，是基于寻找梯度强度。拉普拉斯算子（二阶差分）是基于过零点检测。通过计算梯度，设置阈值，得到边缘图像。

边缘检测一般步骤有：

①滤波：边缘检测算法主要是基于图像强度的一阶和二阶导数，但是导数对于噪声很敏感，因此需要采用滤波器来改善与噪声有关的边缘检测器的性能

②增强：增强边缘的基础是确定图像各点邻域强度的变化值。增强算法可以将灰度点邻域强度值有显著变化的点凸显出来

③检测：邻域中有很多的点的梯度值较大，但是在特定的应用中，这些点并不是要找的边缘点，需要取舍

13.3 实验目标

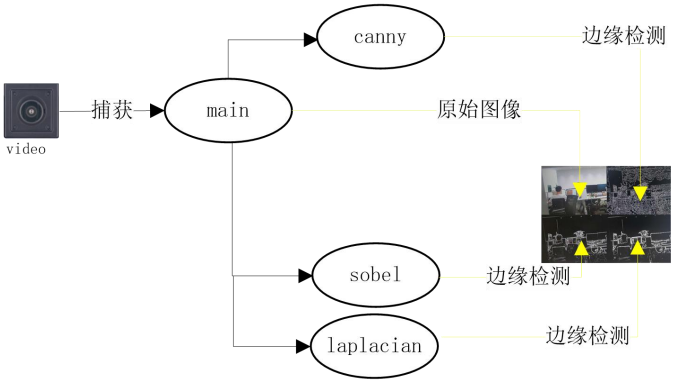
① 使用 opencv 来捕获 usb 摄像头图像

② canny、sobel、laplacian 算法的边缘检测

③ 使用 opencv 来图像显示

④ 响应 opencv 窗口关闭操作

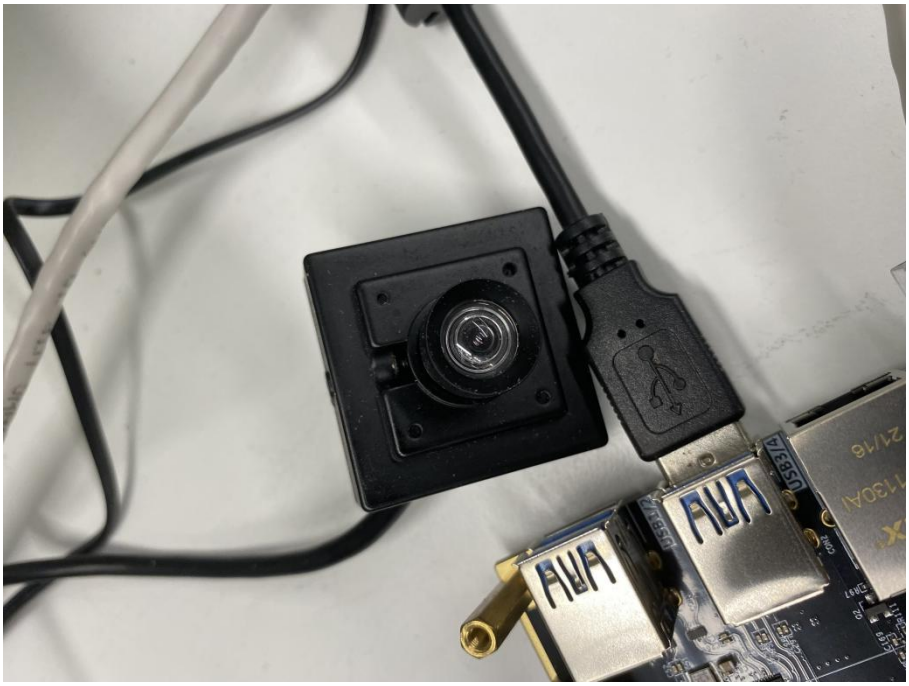
13.4 软件流程图



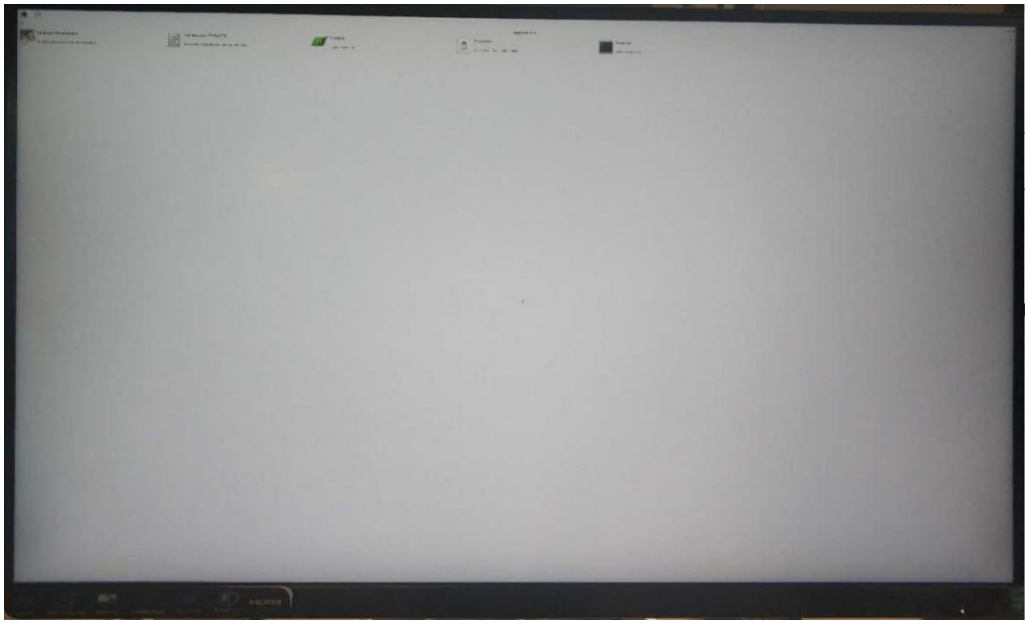
13.5 运行准备

(1) dp 接口连接至显示器

(2) 插好 usb 摄像头



(3) 开发板上电，等待系统启动完成后，dp 显示器可以正确显示开机桌面

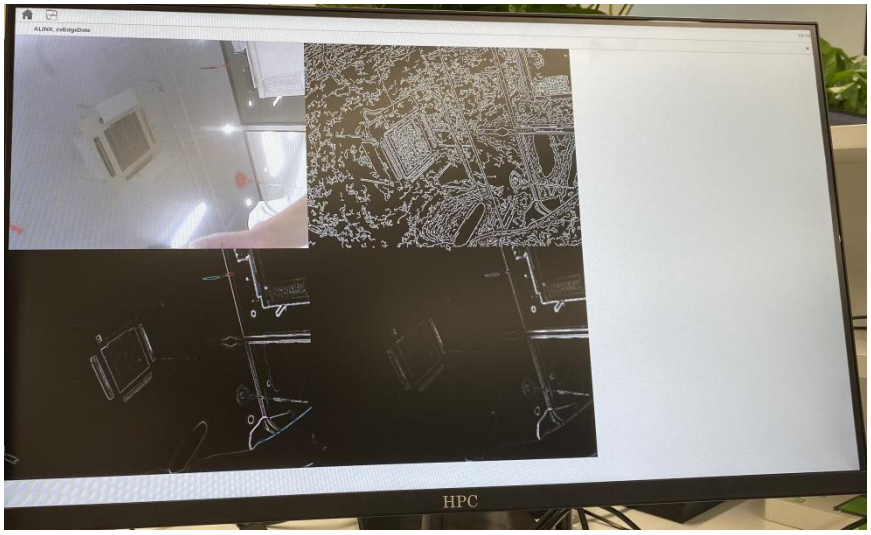


(4) 在终端下，可以查看到 usb 摄像头设备（查看之后，记住自己的摄像头设备号）

```
root@petalinux:~# ls /dev/video*
/dev/video0  /dev/video1
root@petalinux:~#
```

13.6 程序运行

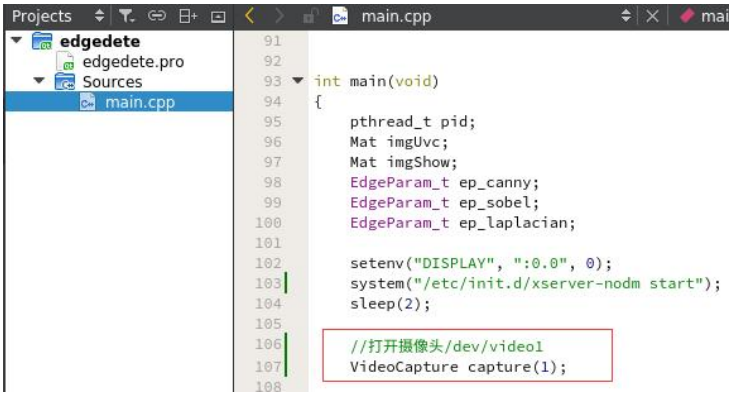
(1) 可以参照上一章例程，打开 13_opencv_edgedete 工程，使用 QtCreator 直接运行应用程序，运行效果如下(如果运行出错，请看第二点)



(2) 如果报以下错误，则修改：

```
sh: /etc/init.d/xserver-nodm_xx: No such file or directory

(edgedete:790): GStreamer-CRITICAL **: 08:50:09.809: gst_element_get_state: assertion 'GST_IS_ELEMENT (element)' failed
User requested stop. Shutting down...
Timeout waiting for remote process to finish.
```

将摄像头设备号改为自己的设备号



13.7 代码解析

(1) 设置环境变量

在终端中输入如下命令，效果等同于代码中的 setenv 调用。

```
export DISPLAY=:0.0
```

(2) 启动桌面

```
/etc/init.d/xserver-nodm start
```

(3) 显示器分辨率调整

如上述实验，使用一台 4K 显示器，所以开机桌面上的图标很小，为了更好的显示效果，在应用程序中，通过系统调用，将显示输出分辨率设置为 1080P。命令如下：

```
xrandr --output DP-1 --mode 1920x1080
```

(4) 显示器输出管理

系统默认使用输出电源管理，即一段时间后，显示器黑屏。这时，我们可以使用命令，取消该功能，命令如下：

```
xset s 0 0
xset dpms 0 0 0
```

(5) 多线程操作

本实验使用三种不同算法完成边缘检测，三种算法被分配到三个线程任务，可以充分利用 ARM 的四核 A53

(6) usb 摄像头捕获

摄像头分辨率被设置为 640x480，默认打开的摄像头设备为：/dev/video0，因为这里算法运算有点慢，显示的图像会有一点延迟。

(7) canny 边缘检测

John F. Canny 于 1986 年开发出来的一个多级边缘检测算法，被很多人认为是边缘检测的*最优算法*，最优边缘检测的三个主要评价标准是:

- 低错误率: 标识出尽可能多的实际边缘，同时尽可能的减少噪声产生的误报。
- 高定位性: 标识出的边缘要与图像中的实际边缘尽可能接近。

- 最小响应：图像中的边缘只能标识一次。

cv：：canny 函数参数如下：

InputArray	image,	输入图像
OutputArray	edges	输出边缘图像
double	threshold1	阈值 1
double	Threshold2	阈值 2
int	apertureSize=3	Sobel 算子大小
bool	L2gradient=false	是否采用更精确的方式计算图像梯度

(8) sobel 边缘检测

这是一种较为常用的边缘检测方法，Soble 算子的功能集合了高斯平滑和微分求导，又被称为一阶微分算子，求导算子，在水平和垂直两个方向上求导，得到的是图像在 X 方向与 Y 方向梯度图像。缺点：比较敏感，容易受影响，要通过高斯模糊（平滑）来降噪，边缘定位精度不够高。

cv：：sobel 函数参数如下：

InputArray	src,	输入图像
OutputArray	dst	输出边缘图像
int	ddepth	输出图像深度
int	dx	导数 x 的阶数.
int	dy	导数 y 的阶数
int	ksize = 3	Sobel 内核的大小
double	scale = 1	计算的导数值的可选比例因子
double	delta = 0	在将结果存储到 dst 之前添加到结果中的可选增量值。
Int	borderType=BORDER_DEFAULT	像素外推法

(9) Laplacian 边缘检测

Laplacce 算子是 n 维欧几里德空间中的一个二阶微分算子，在只关心边缘的位置而不考虑其周围的象素灰度差值时比较合适。Laplace 算子对孤立象素的响应要比对边缘或线的响应要更强烈，因此只适用于无噪声图象。存在噪声情况下，使用 Laplacian 算子检测边缘之前需要先进行低通滤波。所以，通常的分割算法都是把 Laplacian 算子和平滑算子结合起来生成一个新的模板。 Laplace 算子和 Sobel 算子一样，属于空间锐化滤波操作。

cv：：Laplacce 函数参数如下：

InputArray	src,	输入图像
OutputArray	dst	输出边缘图像
int	ddepth	输出图像深度
int	ksize = 1	用于计算二阶导数内核的大小

double	scale = 1	用于计算 Laplacce 的可选比例因子
double	delta = 0	在将结果存储到 dst 之前添加到结果中的可选增量值。
Int	borderType=BORDER_DEFAULT	像素外推法

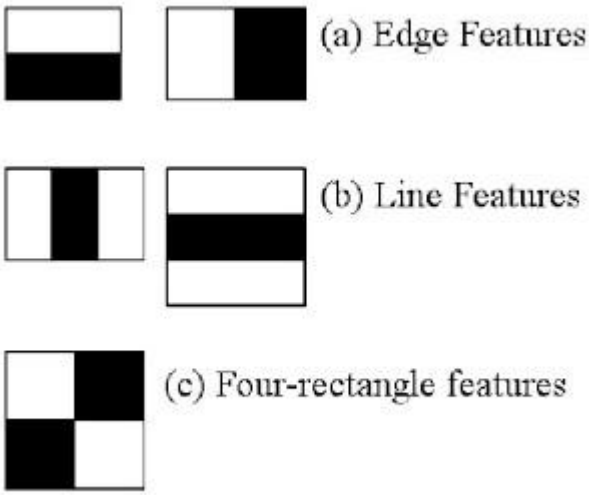
第十四章 OpenCV+Qt 之人脸检测

14.1 OpenCV 的级联分类器

分类器，是判断某个事物是否属于某种分类的器件。 级联分类器： 可以理解为将 N 个单类的分类器串联起来。如果一个事物能属于这一系列串联起来的的所有分类器，则最终结果就判定为是，若有一项不符，则判定为否。比如人脸，它有很多属性，我们将每个属性做一成个分类器，如果一个模型符合了我们定义的人脸的所有属性，则我们人为这个模型就是一个人脸。那么这些属性是指什么呢？ 比如人脸需要有两条眉毛，两只眼睛，一个鼻子，一张嘴，一个大概 U 形状的下巴或者是轮廓等等。

基于 Haar 特征的级联分类器的目标检测是 Paul Viola 和 Michael Jones 在 2001 年的论文《Rapid Object Detection using a Boosted Cascade of Simple Features》中提出的一种有效的目标检测方法。这是一种基于机器学习的方法，其中一个级联函数是从许多正面和负面图像中训练出来的。然后，它被用于检测其他图像中的对象。

我们将在这里进行面部检测。该算法首先需要大量的正图像（人脸图像）和负图像（无人脸图像）来训练分类器。然后我们需要从中提取特征。



Haar 特征分为三类：边缘特征、线性特征、中心特征和对角线特征，组合成特征模板。特征模板内有白色和黑色两种矩形，并定义该模板的特征值为白色矩形像素和减去黑色矩形像素和。Haar 特征值反映了图像的灰度变化情况。例如：脸部的一些特征能由矩形特征简单的描述，如：眼睛要比脸颊颜色要深，鼻梁两侧比鼻梁颜色要深，嘴巴比周围颜色要深等。但矩形特征只对一些简单的图形结构，如边缘、线段较敏感，所以只能描述特定走向（水平、垂直、对角）的结构。

通过改变特征模板的大小和位置，可在图像子窗口中穷举出大量的特征。上图的特征模板称为“特征原型”；特征原型在图像子窗口中扩展（平移伸缩）得到的特征称为“矩形特征”；矩形特征的值称为“特征值”。

矩形特征可位于图像任意位置，大小也可以任意改变，所以矩形特征值是矩形模版类别、

矩形位置和矩形大小这三个因素的函数。故类别、大小和位置的变化，使得很小的检测窗口含有非常多的矩形特征，如：在 24*24 像素大小的检测窗口内矩形特征数量可以达到 16 万个。这样就有两个问题需要解决了：（1）如何快速计算那么多的特征？（2）哪些矩形特征才是对分类器分类最有效的？

14.2 积分图

为了快速计算特征，这里引入了积分图，无论图像有多大，它都会将给定像素的计算减少到仅涉及四个像素的操作。积分图就是只遍历一次图像就可以求出图像中所有区域像素和的快速算法，大大的提高了图像特征值计算的效率。

积分图主要的思想是将图像从起点开始到各个点所形成的矩形区域像素之和作为一个数组的元素保存在内存中，当要计算某个区域的像素和时可以直接索引数组的元素，不用重新计算这个区域的像素和，从而加快了计算（这有个相应的称呼，叫做动态规划算法）。积分图能够在多种尺度下，使用相同的时间（常数时间）来计算不同的特征，因此大大提高了检测速度。

积分图是一种能够描述全局信息的矩阵表示方法。积分图的构造方式是位置（i,j）处的值 ii(i,j)是原图像(i,j)左上角方向所有像素的和：

$$\mathrm{ii}(i,j) = \sum_{k\le i\ l\le j} \mathrm{f}(k,l)$$

积分图构建算法

① 用 s(i,j)表示行方向的累加和，初始化 s(i,-1)=0;

② 用 ii(i,j)表示一个积分图像，初始化 ii(-1,i)=0；

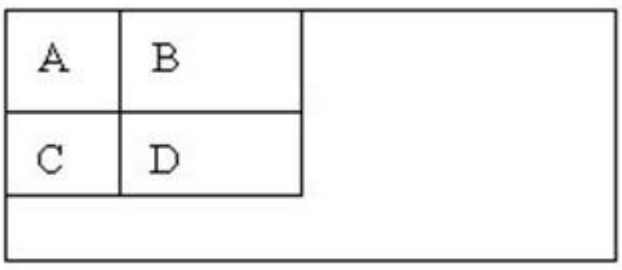
③ 逐行扫描图像，递归计算每个像素(i,j)行方向的累加和 s(i,j)和积分图像 ii(i,j)的值

s(i,j)=s(i,j-1)+f(i,j)

ii(i,j)=ii(i-1,j)+s(i,j)

④ 扫描图像一遍，当到达图像右下角像素时，积分图像 ii 就构造好了

积分图构造好之后，图像中任何矩阵区域的像素累加和都可以通过简单运算得到如图所示。



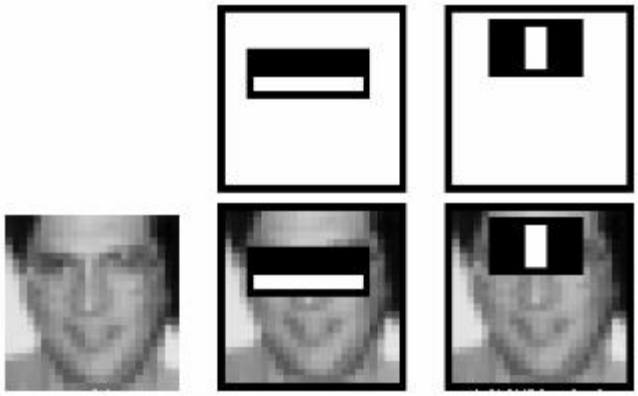
设 D 的四个顶点分别为α、β、γ、δ，则 D 的像素和可以表示为

Dsum = ii(α)+ii(β)-(ii(γ)+ii(δ));

而 Haar-like 特征值无非就是两个矩阵像素和的差，同样可以在常数时间内完成。所以矩形特征的特征值计算，只与此特征矩形的端点的积分图有关，所以不管此特征矩形的尺度变换如何，特征值的计算所消耗的时间都是常量。这样只要遍历图像一次，就可以求得所有子窗口的特征值。

14.3 选取特征

在我们计算的所有这些特征中，大多数都是无关的。例如，考虑下面的图片。最上面一排显示了两个很好的特性。选定的第一个特征似乎集中在眼睛区域通常比鼻子和脸颊区域暗的属性上。选择的第二个特征依赖于眼睛比鼻梁暗的属性。但同样的窗口应用于脸颊或任何其他地方都是没有作用的。那么我们如何从 160000 多个功能中选择最好的功能呢？这里可以通过 AdaBoost 算法来实现。



我们将每个特征应用于所有训练图像。对于每一个特征，它都会找到最佳的阈值，将人脸分为正反两类。显然，会有正确或错误的分类。我们选择错误率最小的特征，这意味着它们是最准确地分类人脸和非人脸图像的特征。（过程并不像这样简单。每幅图像在开始时都有一个相等的权重。每次分类后，错误分类图像的权重都会增加。然后做同样的过程。计算新的错误率。还有新的权重。该过程将继续进行，直到达到所需的精度或错误率或找到所需数量的特征）。

14.4 Qt 简介

Qt 是一个跨平台的 C++ 开发库，主要用来开发图形用户界面（Graphical User Interface，GUI）程序，当然也可以开发不带界面的命令行（Command User Interface，CUI）程序。Qt 还存在 Python、Ruby、Perl 等脚本语言的绑定， 也就是说可以使用脚本语言开发基于 Qt 的程序。Qt 支持的操作系统有很多，例如通用操作系统 Windows、Linux、Unix，智能手机系统 Android、iOS、WinPhone， 嵌入式系统 QNX、VxWorks 等等。Qt 虽然经常被当做一个 GUI 库，用来开发图形界面应用程序，但这并不是 Qt 的全部；Qt 除了可以绘制漂亮的界面（包括控件、布局、交互），还包含很多其它功能，比如多线程、访问数据库、图像处理、音频视频处理、网络通信、文件操作等，这些 Qt 都已经集成了。

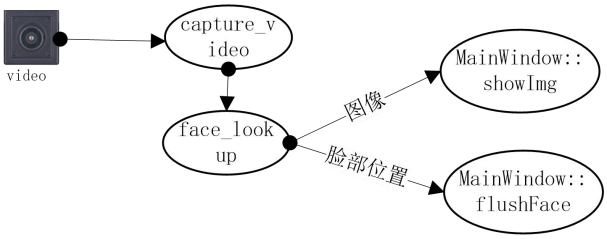
Qt 虽然也支持手机操作系统，但是由于 Android 本身已经有 Java 和 Kotlin，iOS 本身已经有 Objective-C 和 Swift，所以 Qt 在移动端的市场份额几乎可以忽略。

14.5 实验目标

① 调用 Qt 库显示视频图像与脸部框图

② opencv 库的 CascadeClassifier 类的使用

14.6 软件流程图



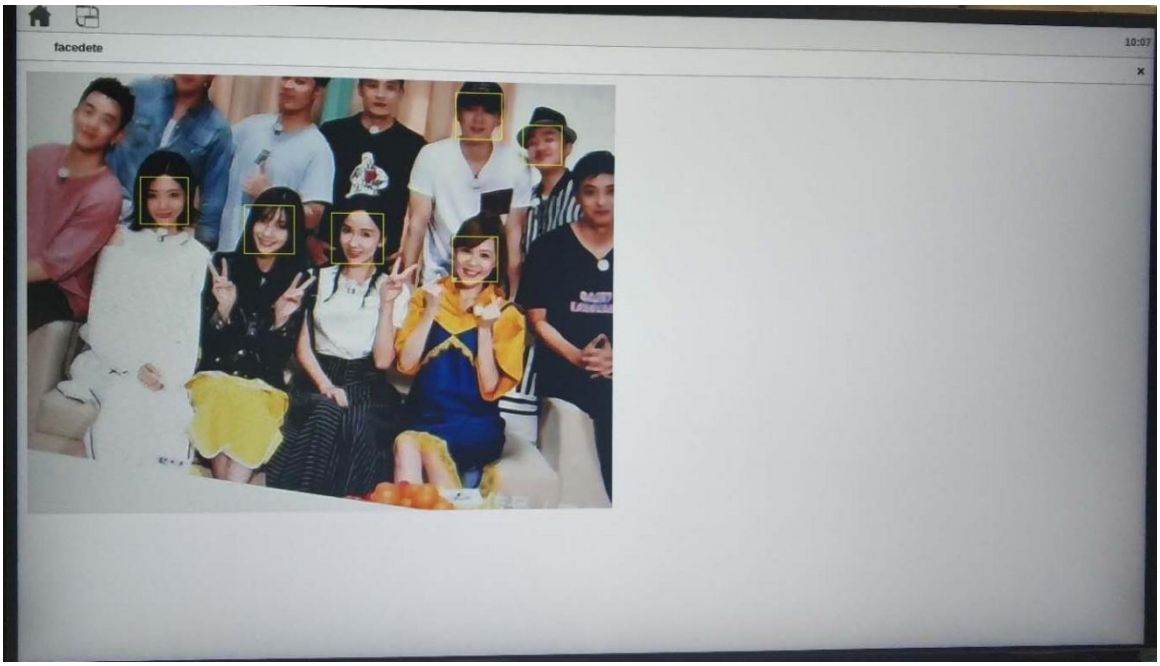
14.7 运行准备

(1) 按上章节做好准备工作

(2) 将 haar_train 文件夹复制到开发板/home/root 目录(使用 NFS 挂载到开发板上)

14.8 程序运行

(1) 运行应用程序，效果如下（如果遇到.pro 工程报错，先看第二点）：



(2) 如果运行.pro 有如下错误：

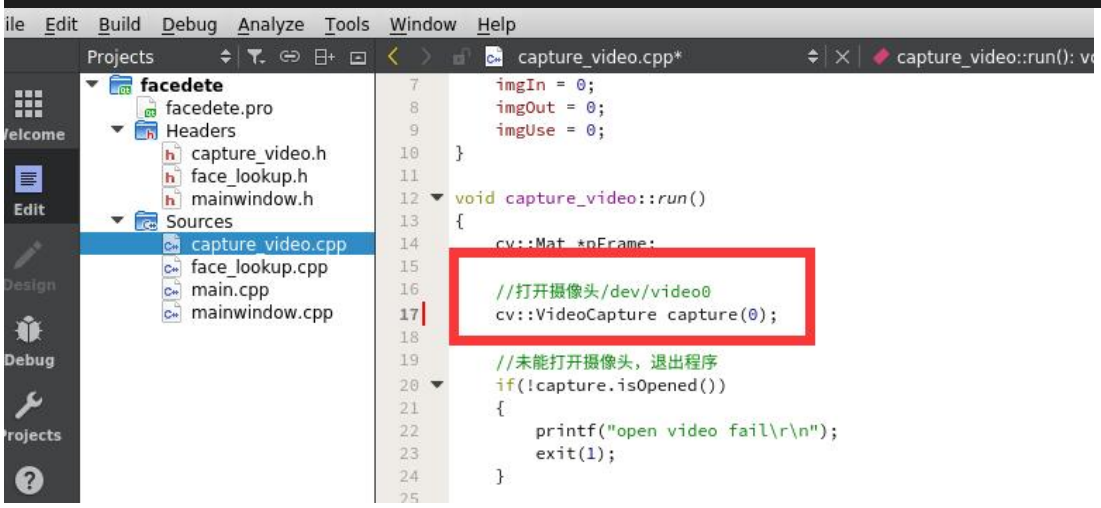
```
facedete (on Remote Device)
Please consult the The X.Org Foundation support
         at http://wiki.x.org
 for help.
(EE)
xrandr: cannot find mode 1280x720
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/var/volatile/tmp/runtime-root'
qt.qpa.xcb: QXcbConnection: XCB error: 146 (Unknown), sequence: 183, resource id: 0, major code: 139 (Unknown), minor code: 20

** (facedete:756): CRITICAL **: 08:25:51.430: gst_v4l2_object_destroy: assertion 'v4l2object != NULL' failed

** (facedete:756): CRITICAL **: 08:25:52.273: gst_v4l2_object_destroy: assertion 'v4l2object != NULL' failed
X connection to :0 broken (explicit kill or server shutdown).
```

那么请做如下修改：

1）这里先看 13.5 的第四点，查看自己摄像头的路径，如我的摄像头是 video1，当我拔掉摄像头时，显示为第二张图，此时修改第三张图红框内的“0”为 1，工程通过，能正常显示。

```
root@petalinux:~# ls /dev/video*
/dev/video0  /dev/video1  /dev/video2
```

```
root@petalinux:~# ls /dev/video*
/dev/video0  /dev/video1  /dev/video2
root@petalinux:~# [  217.897291] usb 1-1.3: USB disconnect, device number 4
[  217.902604] uvcvideo: Failed to resubmit video URB (-19).
[  217.908278] uvcvideo: Failed to resubmit video URB (-19).
[  217.913775] uvcvideo: Failed to resubmit video URB (-19).
[  217.919459] uvcvideo: Failed to resubmit video URB (-19).
[  217.925454] uvcvideo: Failed to resubmit video URB (-19).
[  217.931460] uvcvideo: Failed to resubmit video URB (-19).
```



14.9 代码解析

(1) 检测人脸主要是 face_lookup.cpp 文件中的 face_cascade.detectMultiScale，初始化时，调用 face_cascade.load，加载训练好的模型 haarcascade_frontalface_alt.xml。

(2) 在 MainWindow::flushFace 中，将根据上面检测结果，显示人脸框图。

(3) 因为训练模型的原因，这里检测，需要人脸不能是倾斜的或侧面的。

(4) .pro 文件

因为调用 Q 显示界面，在 facedete.pro 文件中，“QT=”后面添加了 core gui widgets 几个 Qt 组件

(5) Qt 的信号与槽

程序将 face_lookup 和 MainWindow 的信号与槽绑定，由 MainWindow 处理需要显示的图片和标志出脸的位置

(6) 与上面例程相同，显示效果比较卡顿。

第十五章 GStreamer 的摄像头显示

15.1 GStreamer 简介

GStreamer 是一个创建流媒体应用程序的框架。其基本设计思想来自于俄勒冈(Oregon)研究生学院有关视频管道的创意，同时也借鉴了 DirectShow 的设计思想。它的特点如下：

① 结构清晰，功能强大

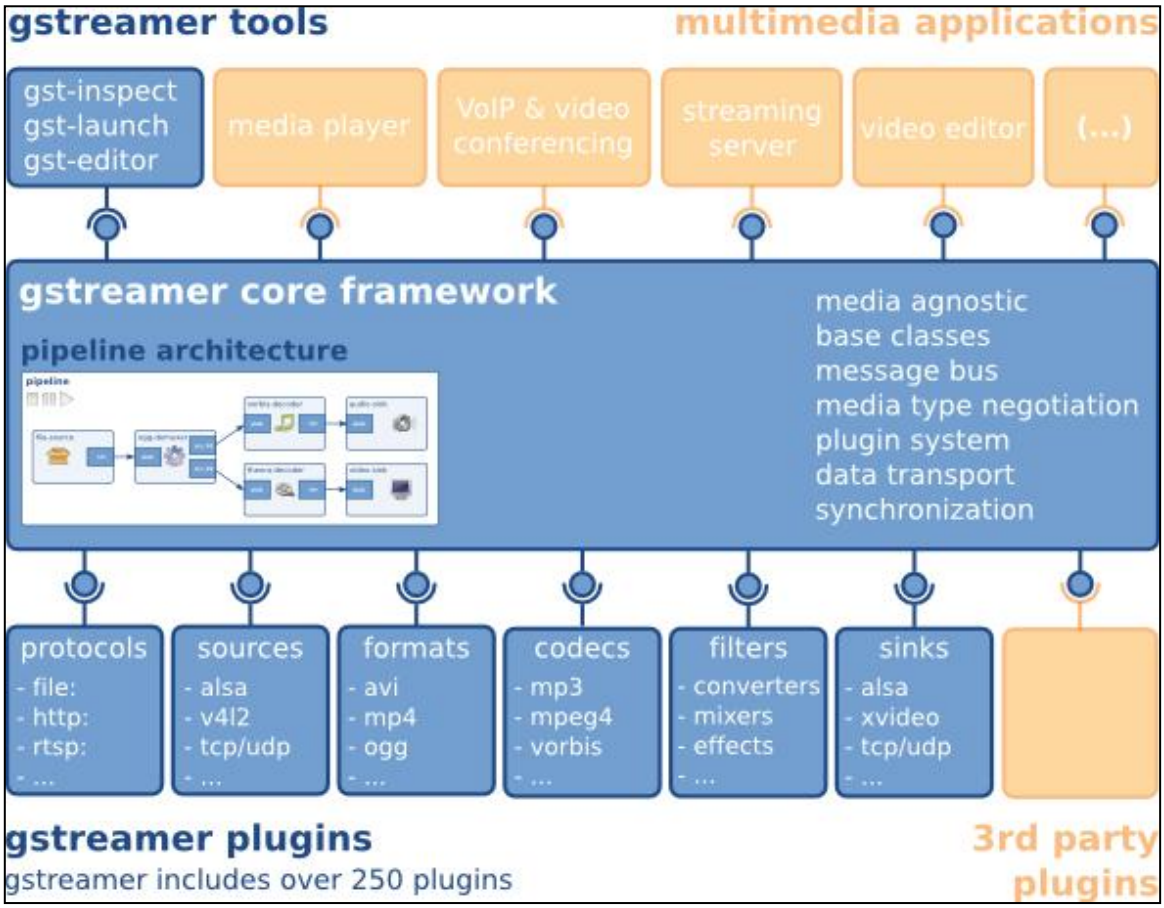
② 面向对象的编程思想

③ 灵活的可扩展功能

④ 高性能

⑤ 核心库与插件分离

GStreamer 的架构如下，它支持的应用范围从简单的 Ogg/Vorbis 回放、音频/视频流到复杂的音频（混合）和视频（非线性编辑）处理。应用程序可以透明地利用先进的编解码器和过滤技术。



15.2 GStreamer 基本概念

(1) 元件（element）

Elements 是组成管道的最基本的构件。可以将若干个元件(Elements)连接在一起,从而创建一个管道(pipeline)来完成一个特殊的任务,例如,媒体播放或者录像。Elements 根据功能分类：

① Source elements

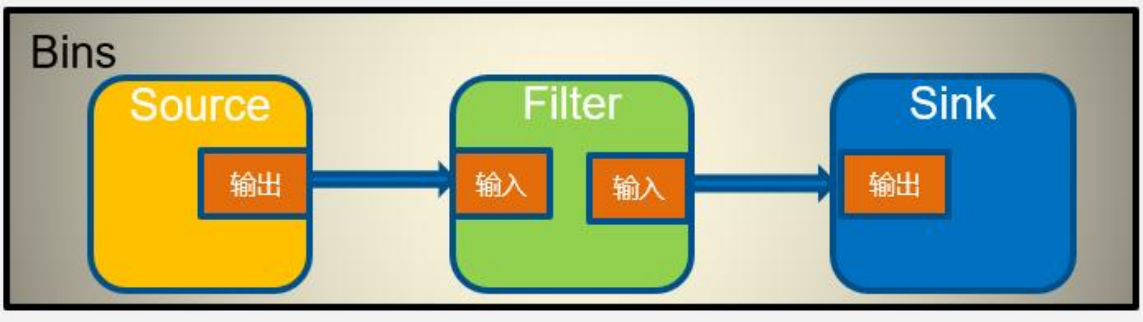
② Sink elements

③ Filter elements

应用框图如下：

(2) 箱柜（bin）和管道（Pipeline）

箱柜(Bins)是一个可以装载元件(element)的容器。管道(pipelines)是箱柜(Bins)的一个特殊的子类,管道(pipelines)可以操作包含在它自身内部的所有元件(element)。如下图



(3) 衬垫（Pad）

衬垫（Pads）相当于一个元件的接口，各个元件（element）通过这个接口进行连接，这样数据流就可以在这些元件中进行传输。Pads 分类：

① 永久型（always）

② 随机型（sometimes）

③ 请求型 （on request）

(4) 功能（Cap）

Caps 描述了能够通过衬垫或当前通过衬垫的数据流格式。通过 gst-inspect-1.0 命令可以查看元件所支持的媒体类型。

(5) 总线（Bus）

每一个管道默认包含一条总线，应用程序不需要再创建总线。应用程序在总线上设置

一个消息处理器。当主循环运行的时候，总线将会轮询这个消息处理器是否有新的消息，当消息被采集到后，总线将调用相应的回调函数来完成任务

(6) 缓冲区（Buffer）

缓冲区包含了创建的管道里的数据流。通常一个源元件会创建一个新的缓冲区，同时元件还将会把缓冲区的数据传递给下一个元件。当使用 GStreamer 底层构造来创建一个媒体管道的时候，不需要自己来处理缓冲区,元件将会自动处理这些缓冲区。缓冲区主要由以下组件构成：

① 指向某块内存的指针

② 内存的大小

③ 缓冲区的时间戳

④ 一个引用计数，指出了缓冲区所使用的元件数。没有元件可引用的时候，这个引用将用于销毁缓冲区

(7) 事件（Events）

事件包括管道里的控制信息，如寻找信息和流的终止信号。

15.3 实验目标

① gstreamer 工具使用

② gstreamer 编程

15.4 Gstreamer 常用工具

(1) gst-inspect-1.0

不带参数，它会列出所有可用的 element，也就是你所有可以使用的元素，带一个文件名，它会把这个文件作为 GStreamer 的一个插件，试着打开，然后列出内部所有的 element，带一个 GStreamer 的 element，会列出该 element 的所有信息。

打印插件列表及其信息，如：

打印支持的插件列表	gst-inspect-1.0 -a
打印插件 filesrc 信息	gst-inspect-1.0 filesrc

Pad Templates：这部分会列出所有的 Pad 的种类以及它们的 Caps。通过这些你可以确认是否可以和某一个 element 连接

(2) gst-discoverer

这个工具可以用来查看文件里面包含的 Caps，它是对 GstDiscoverer 对象的一个封装。接受从命令行输入一个 URI，然后打印出所有的信息。这个在查看媒体是如何编码如何复用时是很有用的，这样我们可以确定把什么 element 放到 pipeline 里面。

(3) gst-launch-1.0

这个工具可以创建一个 pipeline，初始化然后运行。它可以让你在正式写代码实现 pipeline 之前先快速测试一下，看看是否能工作。若需要查看 pipeline 里面一个 element

生成的 Caps，在 gst-launch 里面使用-v 参数即可，如下命令

```
gst-launch-1.0 -v v4l2src device=/dev/video0
```

从下图中可以看到，摄像头默认的视频格式为 YUY2，分辨率为 2592x1944

```
root@petalinux:~# gst-launch-1.0 -v v4l2src device=/dev/video0
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
/GstPipeline:pipeline0/GstV4l2Src:v4l2src0.GstPad:src: caps = video/x-raw, format=(string)YUY2, wid
th=(int)2592, height=(int)1944, pixel-aspect-ratio=(fraction)1/1, interlace-mode=(string)progressiv
e, framerate=(fraction)25/1, colorimetry=(string)bt709
ERROR: from element /GstPipeline:pipeline0/GstV4l2Src:v4l2src0: Intern[ 3714.942617] xhci-hcd xhci-
hcd.0.auto: ERROR unknown event type 37
al data stream error.
Additional debug info:
../../../../git/libs/gst/base/gstbasesrc.c(3055): gst_base_src_loop (): /GstPipeline:pipeline0/GstV
4l2Src:v4l2src0:
streaming stopped, reason not-linked (-1)
Execution ended after 0:00:00.435641850
Setting pipeline to PAUSED ...
Setting pipeline to READY ...
Setting pipeline to NULL ...
Freeing pipeline ...
root@petalinux:~#
```

15.5 gst-launch-1.0 摄像头显示

(1) 确认自己 usb 摄像头支持的视频格式

将 tools/uvc_fmt 拷贝至开发板下运行，可以扫描出支持的分辨率

```
root@petalinux:~# ./uvc_fmt 0
query /dev/video0 format
        1.YUYV 4:2:2
                1920x1080@60
                640x480@60
                800x600@60
                1024x768@60
                1600x1200@25
                2048x1536@25
                2592x1944@25
 root@petalinux:~#
```

(2) 在控制终端下，输入如下命令

```
gst-launch-1.0 -v v4l2src device=/dev/video0 !
video/x-raw,format=YUY2,width=1920,height=1080 ! kmssink bus-id=fd4a0000.zynqmp-display
fullscreen-overlay=1
```

注意：这里的分辨率与视频格式，需要自己的 usb 摄像头支持，可以根据自己摄像头支持的分辨率来设置，且对应的分辨率显示器也需要支持，当显示视频时，显示器的分辨率将被设置为与捕获的图像一致

(3) 运行效果

如果能够显示图像，我们可以看到图像将会非常流畅，但缺点就是，只能单画面显示图像，而且仅能显示摄像头与显示器都支持的分辨率。

15.6 编程运行程序

通过命令，我们可以快速的验证各种我们设想的方案，但如果想要更多的状态监控或用户控制，这就需要通过编程来实现。

本章的例程，是通过程序来实现上面的命令效果

(1) 需要注意的事项与上面的一样，需要修改 usb 摄像头支持的分辨率与格式

(2) 程序通过添加消息事件的回调函数 sink_message，监控与处理各种状态变化。

(3) 运行程序后，可以看到显示的图像与命令运行的效果一样，此时拔掉摄像头，程序打印的信息如下且程序退出运行。

```
get error: Could not read from resource.
play error
playing out
Application finished with exit code 0.
```

15.7 代码解析

(1) gst_parse_launch

该函数创建一个 pipelin，函数参数如下：

const gchar *	pipeline_description	描述管道
GError **	error	错误返回

pipeline_description 内容与前面讲的 gst-launch-1.0 验证的内容几乎相同。

(2) gst_bus_add_watch

添加总线的回调函数，处理必要的消息。

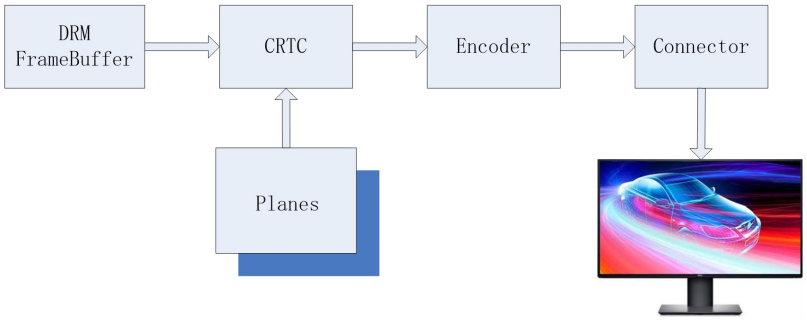
第十六章 Qt+DRM+Gstreamer 的摄像头显示

16.1 DRM 简介

DRM，英文全称 Direct Rendering Manager，即直接渲染管理器。它是为了解决多个程序对 Video Card 资源的协同使用问题而产生的。它向用户空间提供了一组 API，用于输出管理。

DRM 到现在已经涵盖了以前由用户空间程序处理的很多功能，比如 帧缓存区的管理和模式设置，内存共享对象和内存同步。其中一些拓展具有特定的名称，比如图形执行管理器 GEM 或者内核模式设置 KMS，这些都是属于 DRM 子系统。

DRM 显示，主要涉及以下 5 个模块



(1) DRM Framebuffer

在开发者看来，FrameBuffer 是一块内存，往内存中写入特定格式的数据就意味着向屏幕输出内容。所以说 FrameBuffer 就是一块画布。例如对于初始化为 16 位色的 FrameBuffer 来说， FrameBuffer 中的两个字节代表屏幕上一个点，从上到下，从左至右，屏幕位置与内存地址是顺序的线性关系。

(2) CRTC

CRTC 的任务是从 Framebuffer 中读出待显示的图像，并按照相应的格式输出给 Encoder。CRTC 虽然字面上意思为阴极射线显像管控制器，但 CRT 在普通显示设备中早已被淘汰，DRI 中 CRTC 主要承担的作用如下：

① 配置适合显示器的分辨率（kernel）并输出相应时序（hardware logic）

② 扫描 framebuffer 送显到一个或多个显示设备中

(3) Planes

随着软件技术的不断更新，对硬件的性能要求越来越高，在满足功能正常使用的前提下，对功耗的要求也越来越苛刻。本来 GPU 可以处理所有图形任务，但是由于它运行时的功耗实在太高，设计者们决定将一部分简单的任务交给 Display Controller 去处理（比如合成），而让 GPU 专注于绘图（即渲染）这一主要任务，减轻 GPU 的负担，从而达到降低功耗提升性能的目的。于是，Plane（硬件图层单元）就诞生了。Planes 主要是将多个图层进行叠加输出，有些硬件支持图层的平移、缩放、裁剪、alpha 通道等。

(4) Encoder

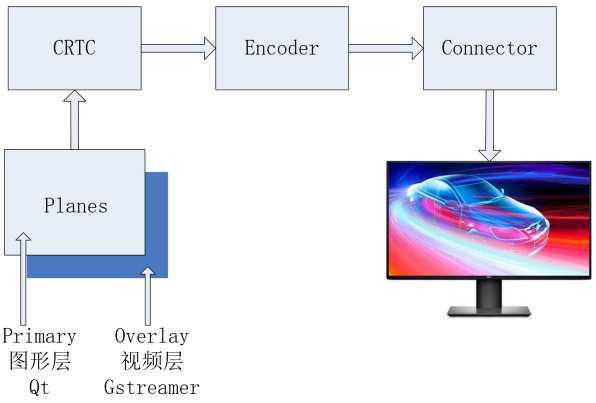
将一定格式的图像信号（如 RGB、YUV 等）编码成 connector 需要输出的信号。以 HDMI 为例，帧/行同步/显示内容都是通过 TMDS data 的串行总线输出的，那么并行的时序按照 HDMI 的标准编码为串行顺序则是 Encoder 的任务；

(5) Connector

Connector 其实就是和显示器连接的物理接口，常见的有 VGA/HDMI/DVI/DP 等，可以通过 connector 读出的显示器支持的参数；

16.2 实验介绍

MPSoc 的 DP 输出，支持两个图层，上面的图层(Primary)支持 alpha 通道。本章实验，使用 Qt 输出内容至 Primary 图层，Gstreamer 捕获视频数据至下面的图层(Overlay)。最终这两个图层由硬件混合输出。工作框图如下：



16.3 实验目标

① DRM 输出管理
② DRM 图层管理
③ Qt 与 GStreamer 分层显示

16.4 运行程序

(1) 需要注意，本实验需要 usb 摄像头支持 1080 分辨率(指的是裸流，如上面的 YUYV422 格式)

(2) 实验运行效果

可以看到在视频上面叠加了一个 Qt 界面

16.5 代码分析

(1) 检测显示器是否连接

在 main 函数中，调用 init_drm 判断初始化是否成功，返回-2，即没有支持的显示格式，说明没有连接显示器。

(2) 设置显示器输出分辨率

通过函数 drmModeSetCrtc，设置输出的分辨率及刷新率，这里没有设置 buffer，所以第三个参数设置为-1

(3) 设置图形层的透明属性

图形层支持全局的和单像素的 alpha 值设置。这里将其设置为单像素的，即将 g_alpha_en 设置为 0

(4) Qt 显示

Qt 默认显示在 Primary 层

(5) GStreamer 显示

通过 drmModeGetPlaneResources 可以获取 plane 的信息，包括 plane 的 id 号，这里获取到 Overlay 图层的 ID 为 35。通过代码可以看到，kmssink 的属性设置如下：kmssink sink-type=dp bus-id=fd4a0000.zynqmp-display fullscreen-overlay=false plane-id=35

第十七章 Qt+GPU 的摄像头显示

17.1 Mali GPU 简介

GPU 英文全称 Graphics Processing Unit，中文翻译为“图形处理器”。我们通常叫它显卡。GPU 的生产厂商非常多，最大的三家是英特尔、NVIDIA、AMD，他们主要应用于 PC 领域。

ARM CPU 占据了移动端 90%市场，但它的 Mali GPU 在移动端占有的市场只能算是小众的存在。Mali GPU 最早由挪威科技大学项目独立出来成立的 Falanx 公司开发，在 2006 年被 ARM 收购，成为 ARM 一个 GPU 事业部，并在 2007 年 mali 作为 arm 的一部分，发布了 mali-200 GPU。

Mali GPU 经历的三代架构如下：

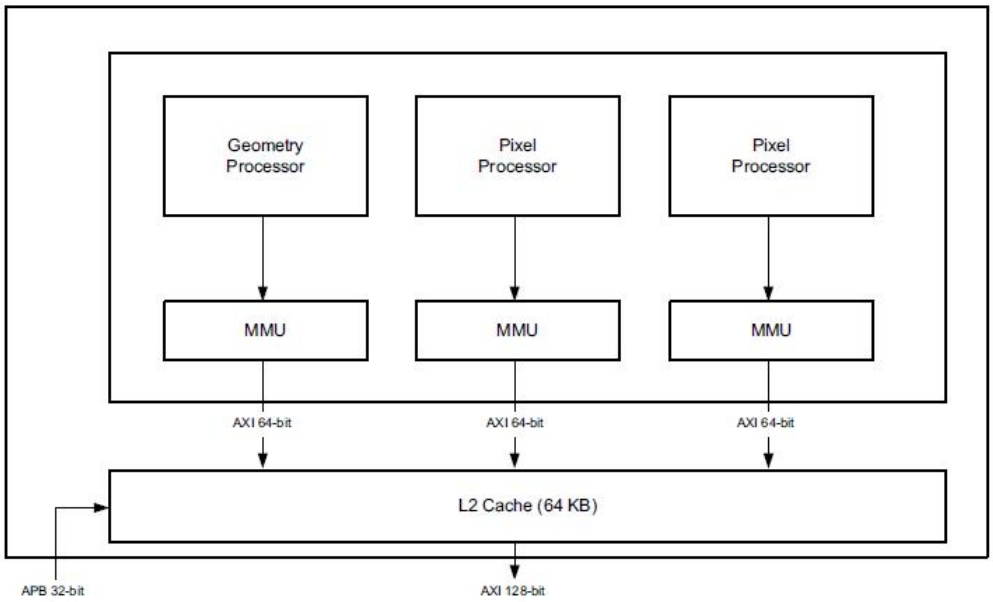
第一代 Utgard	第二代 Midgard	第三代 Bifrost
Mali-200/300/400/450/470	Mali-T600/T700/T800 系列	Mali-G3/G5/G7 系列

按照性能划分：

低功耗	中等	高性能
Mali-400/450/470 Mali-G31	Mali-G51/G52 Mali-T820/T830 Mali-T720	Mali-G71/72 Mali-T860/T880 Mali-T760

17.2 MPSoc GPU 简介

这里使用的 GPU 是一款基于 Mali™-400 MP2 硬件的 2D 和 3D 加速子系统。它的组成资源框图如下：



支持的功能如下：

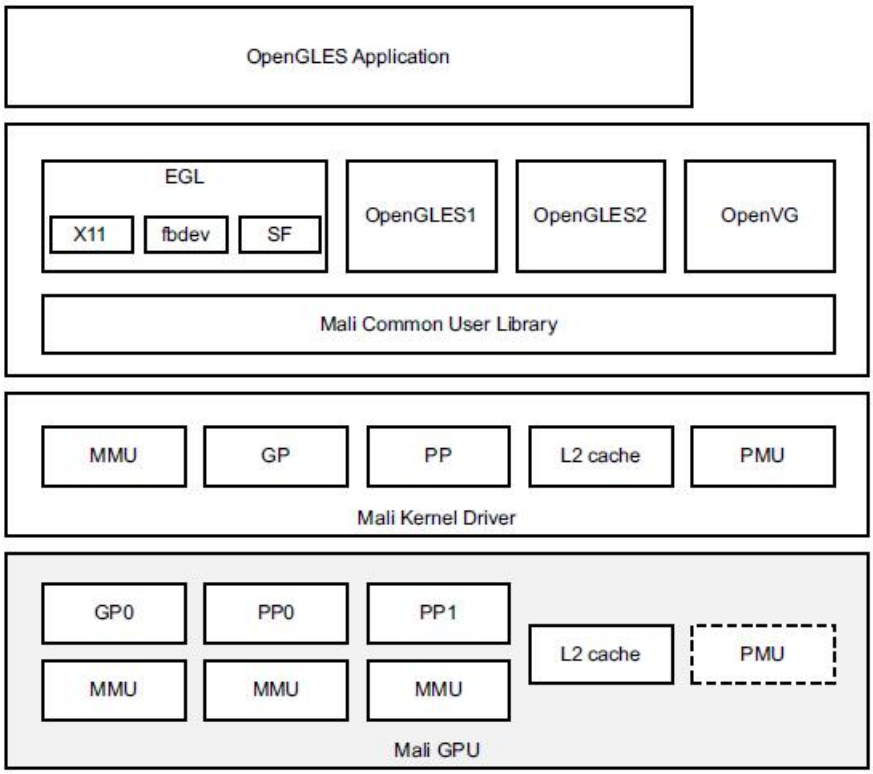
支持 OpenGL ES 1.1 and 2.0

支持 OpenVG 1.1

符合 IEEE 标准的 32 位浮点运算

支持纹理大小高达 4096 x 4096 像素

Linux 系统软件结构框图如下：



在 400M 的工作频率下，它所能达到的峰值性能如下：

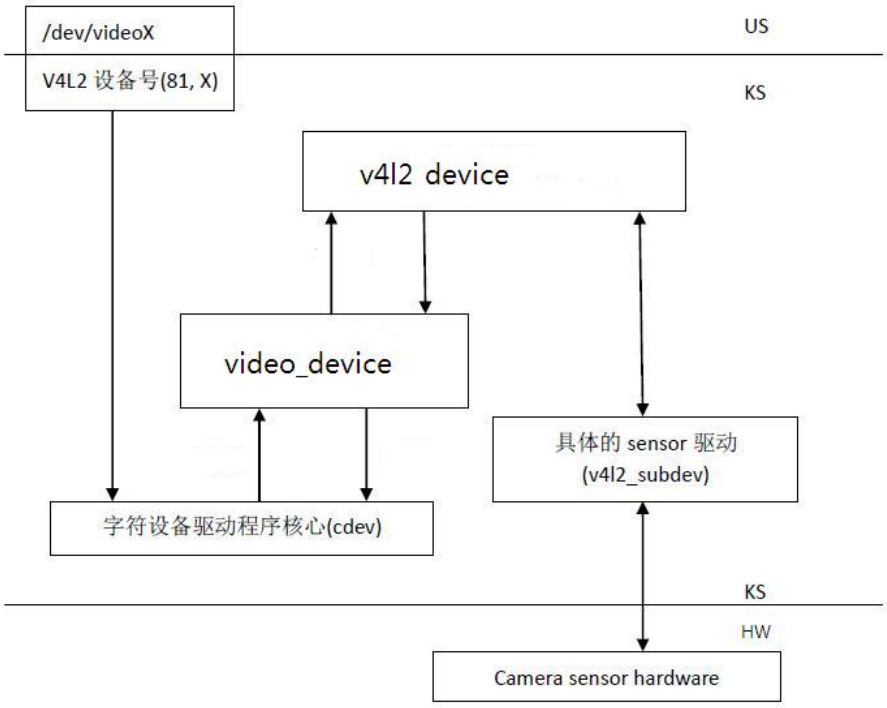
像素填充率：800 Mpixel/sec

顶点处理速度：40 Mvertex/sec

17.3 V4L2 简介

V4L2 的全称是 Video For Linux Two，是内核提供给应用程序访问音视频驱动的统一接口。它的一个最基本应用是从摄像头等设备上获取视频数据。设备名称为/dev/video,主设备号 81，子设备号 0~63.

在 Linux 系统中的结构图如下：



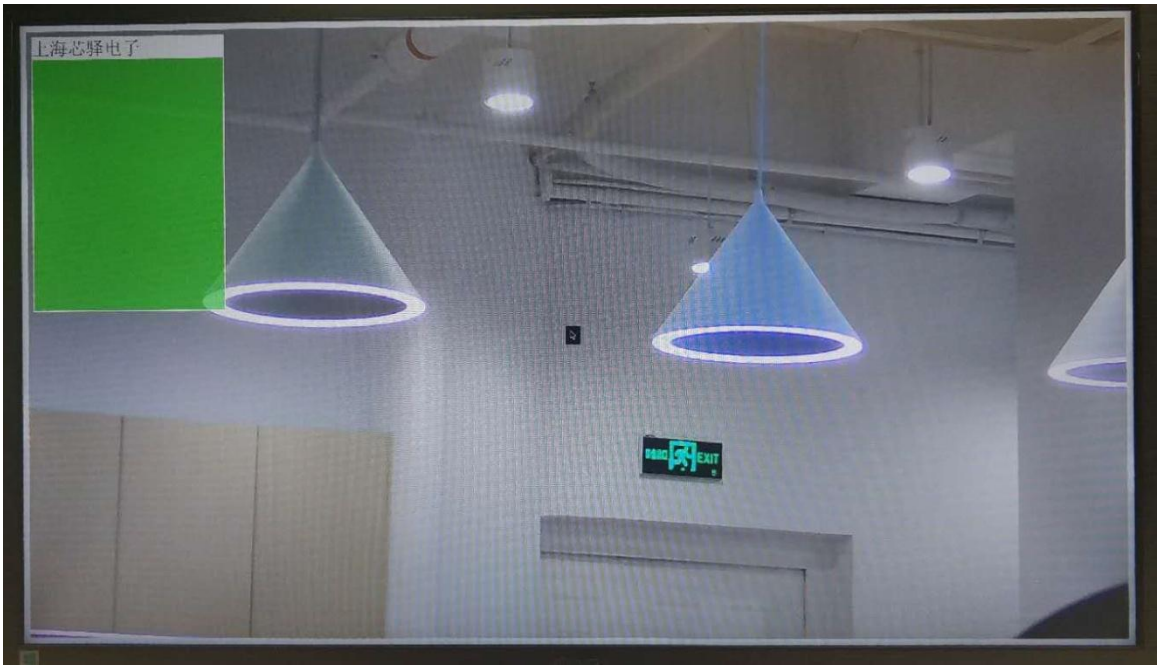
17.4 实验目标

① OpenGL 操作

② V4L2 操作

17.5 运行程序

程序运行效果如下，可以看到，与上个章节的显示效果相比，这里的图像并没有全屏显示，留了几个像素的白边



17.6 代码分析

显示处理部分主要封装在类 uOpenglYuv，这里我们仅演示了如何显示 YUYV422 格式的图像。

(1) YUYV422 如何获取一个像素

在字符串 fragmentStr 中:

纹理 Y 取出 Y 值 yuv.x = texture2D(tex_y, textureOut).x

纹理 UV 取出 U 值 yuv.y = texture2D(tex_uv, textureOut).y

纹理 UV 取出 V 值 yuv.z = texture2D(tex_uv, textureOut).w

其它的运算是将 yuv 转换成 rgb

(2) 图像刷新

在 MainWindow 中，接收到一帧图像后，调用 pOpenglYuv->update()，然后 uOpenglYuv 调用 paintGL，加载图像作为纹理并显示

(3) 摄像头捕获

这里摄像头捕获通过调用 v4l2 接口完成。

第十八章 linux 寄存器操作

18.1 linux 内存映射

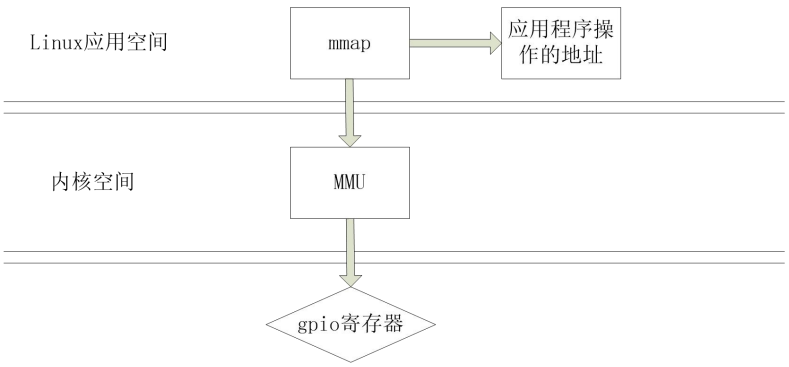
内存映射，就是将用户空间的一段内存区域映射到内核空间，映射成功后，用户对这段内存区域的修改可以直接反映到内核空间，同样，内核空间对这段区域的修改也直接反映用户空间。这对于内核空间与用户空间传输大量数据的话，效率是非常高的。

例如，从硬盘上将文件读入内存，都要经过文件系统进行数据拷贝，并且数据拷贝操作是由文件系统和硬件驱动实现的，理论上来说，拷贝数据的效率是一样的。但是通过内存映射的方法访问硬盘上的文件，效率要比 read 和 write 系统调用高，这是为什么呢？原因是 read()是系统调用，其中进行了数据拷贝，它首先将文件内容从硬盘拷贝到内核空间的一个缓冲区，然后再将这些数据拷贝到用户空间，在这个过程中，实际上完成了 两次数据拷贝 ； 而 mmap()也是系统调用， mmap()中没有进行数据拷贝，真正的数据拷贝是在缺页中断处理时进行的，由于 mmap()将文件直接映射到用户空间，所以中断处理函数根据这个映射关系，直接将文件从硬盘拷贝到用户空间，只进行了 一次数据拷贝 。因此，内存映射的效率要比 read/write 效率高。

18.2 实验简介

有时候，某个外设的功能已经在裸机下验证通过，如果该外设没有用到中断，这时，我们可以通过 linux 下映射寄存器的方法，将这个裸机下的程序直接移植到 linux 下使用，这样，可以省去开发驱动的事。

本例程移植了 vitis 下 gpio 的例程，驱动 ps_led，作每秒闪烁一次的动作。其它的操作，过程与之类似。



18.3 运行程序

运行应用程序 register_opt 后，可以看到，PS 的 led 灯每秒闪烁一次

18.4 代码分析

(1) 关于 gpio 的寄存器地址，这里都是从 vitisi 的 gpio 例程中抄出来的，后面我们的开发思路也应该是这样，先用 vitisi 的裸机程序验证。很多外设和 fpga 端的 ip，vitisi 会帮我们生成好操作的方法和操作的地址，这样就不需要我们再去找对应关系。

(2) 打开/dev/mem 的时候，使用选项 O_SYNC

向外部写入数据后，通常数据是写入 cache 缓冲。O_SYNC 将确保数据写入至外设才返回。需要注意，这里的 O_SYNC，仅影响写操作。

(3) msync 的调用

如果需要向外设一次写入比较多的数据，此时如果调用 O_SYNC，将会严重影响系统的性能，此时如果我们不使用 O_SYNC，而是在写完数据后，调用 msync，这样将会提升写的性能。

(4) 读操作一致性问题

如果需要读外设的数据，因为 cache 的存在，应用中取到的数据是 cache 中的数据，而不是外设的最新状态，此时读到的可能是一个错误的值。

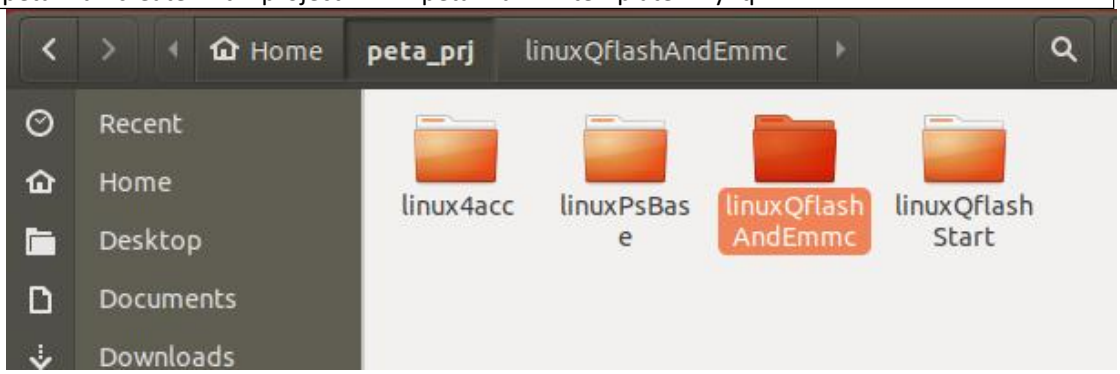
第十九章 从 QFlash 启动内核并从 EMMC 挂载根文件系统

我们的开发板不支持直接从 EMMC 启动，但可以使用从 QFlash 启动然后从 EMMC 加载内核并挂载根文件系统的方案。从 EMMC 加载内核以及挂载根文件系统和 SD 卡是类似的，但是 EMMC 不像 SD 卡可以直接在 PC 机上格式化并把系统复制进去。所以我们就需要借助其他方式启动一个系统，在这个系统中去制作 EMMC 分区，并把内核和根文件系统拷贝和解压到 EMMC 中。

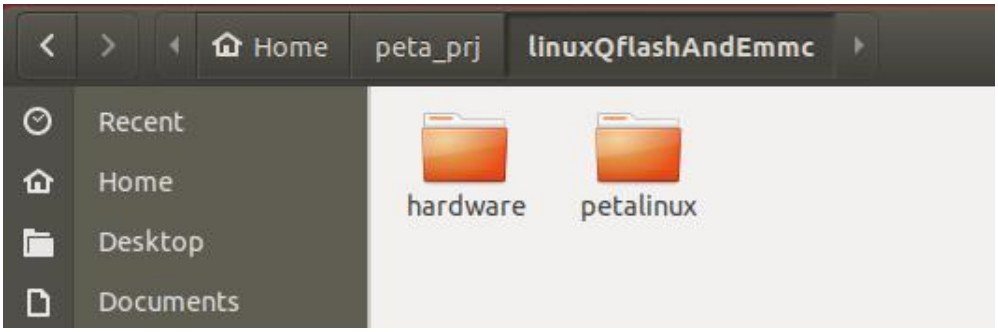
19.1 创建并配置 petalinux 工程

1. 在“/home/alinx/peta_prj”路径中新建一个文件夹“linuxQflashAndEmmc”。

```
petalinux-create -t project -n petalinux --template zynqMP
```



在这个文件夹中打开终端，用上面的命令新建一个 petalinux 工程，并建立一个 hardware 文件夹存放硬件描述文件（使用”Vitis 应用教程”中名为 ps_hello 的 vivado 工程编译出 xsa 文件）。



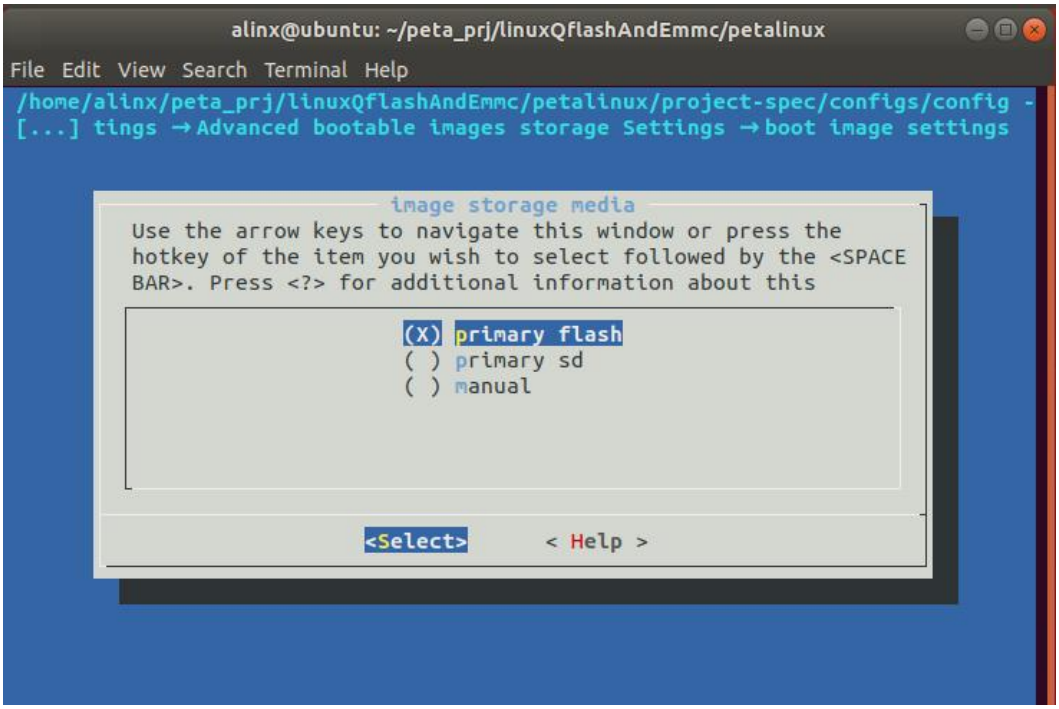
2. 终端中进入 petaLinux 工程文件夹，用下面的命令配置 petalinux 工程的硬件描述文件，打开的配置界面不要配置任何东西直接退出。

```
petalinux-config --get-hw-description ../hardware/
```

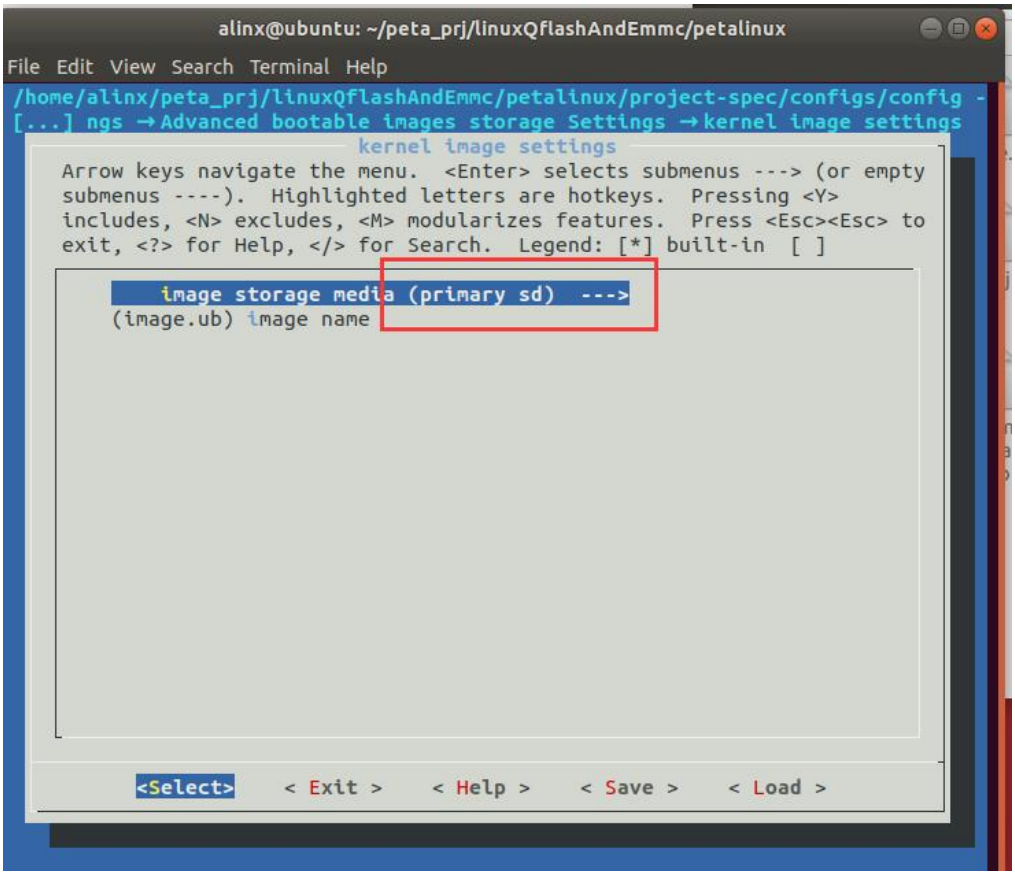
```
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc$ cd ./petalinux/
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc/petalinux$ petalinux-config  --get-hw
-description ../hardware/
INFO: sourcing build tools
INFO: Getting hardware description...
INFO: Rename design_1_wrapper.xsa to system.xsa
[INFO] generating Kconfig for project
[INFO] menuconfig project
configuration written to /home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/proje
ct-spec/configs/config

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

[INFO] extracting yocto SDK to components/yocto
[INFO] sourcing build environment
[INFO] generating kconfig for Rootfs
[INFO] silentconfig rootfs
[INFO] generating plnxtool conf
[INFO] generating user layers
[INFO] generating workspace directory
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc/petalinux$
```

3. 在 Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings --->boot image settings ---> image storage media 界面中选择 primary flash。

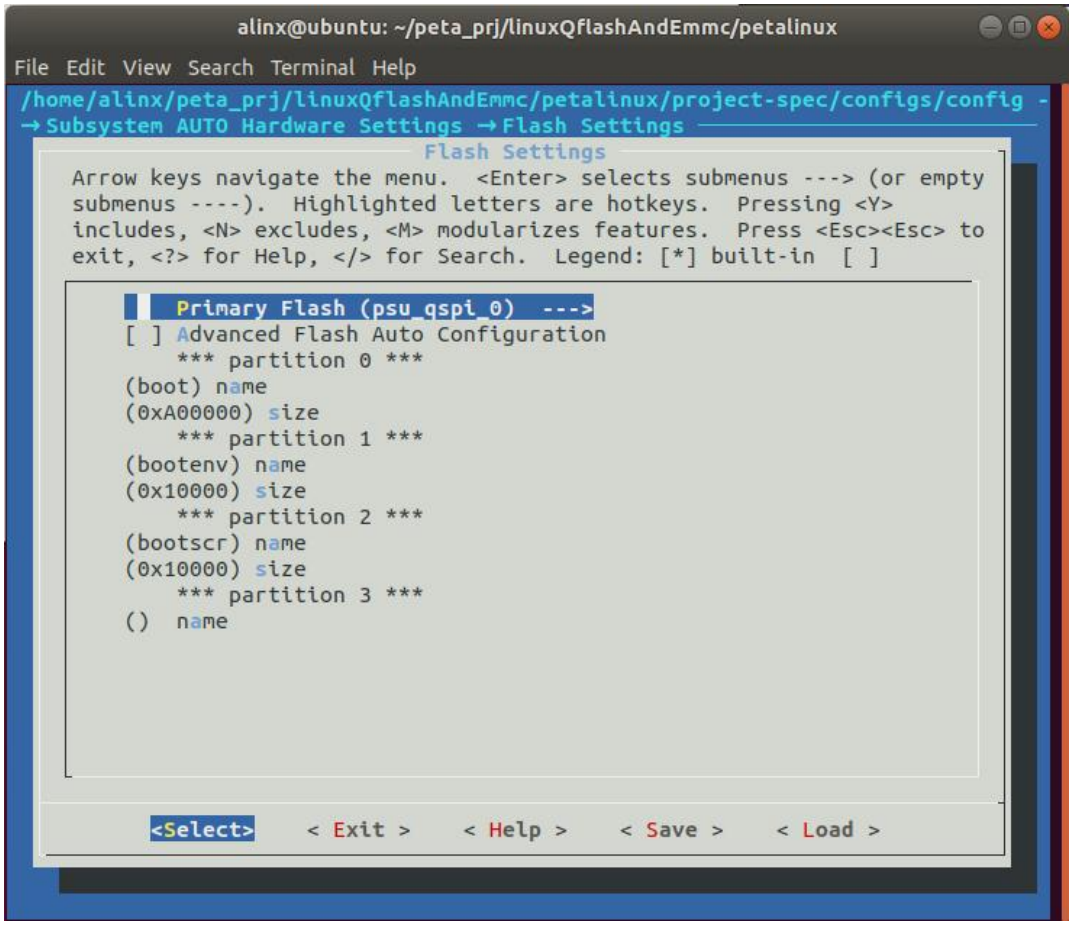
petalinux-config



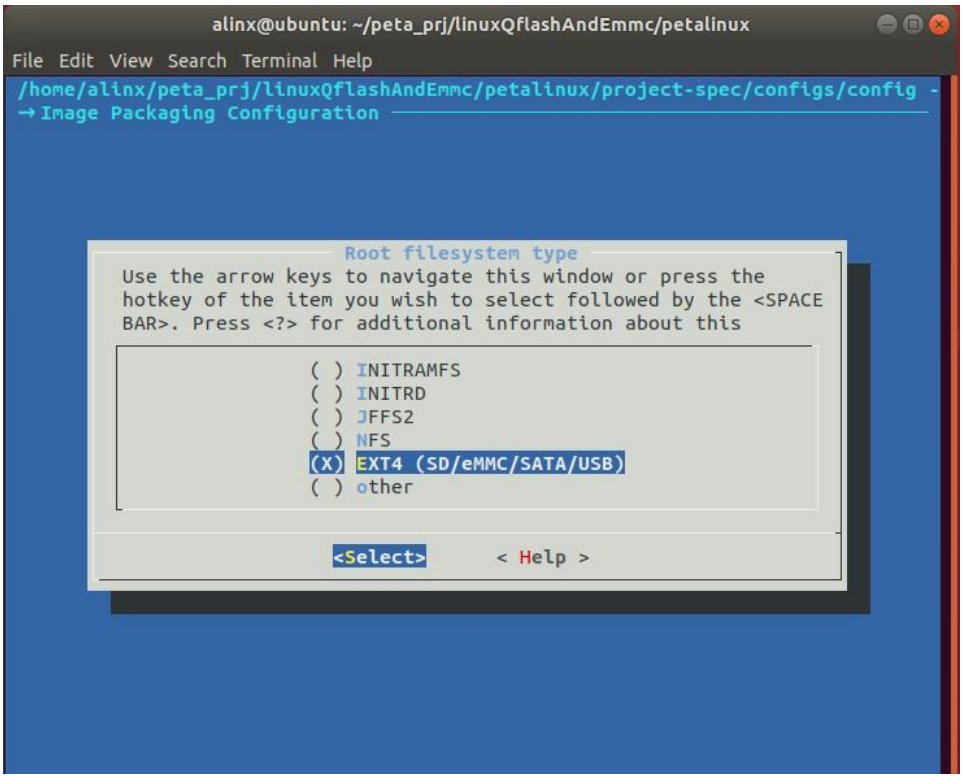
4. 在 Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings ---> kernel image settings ---> image storage media 选项中选择 primary sd。



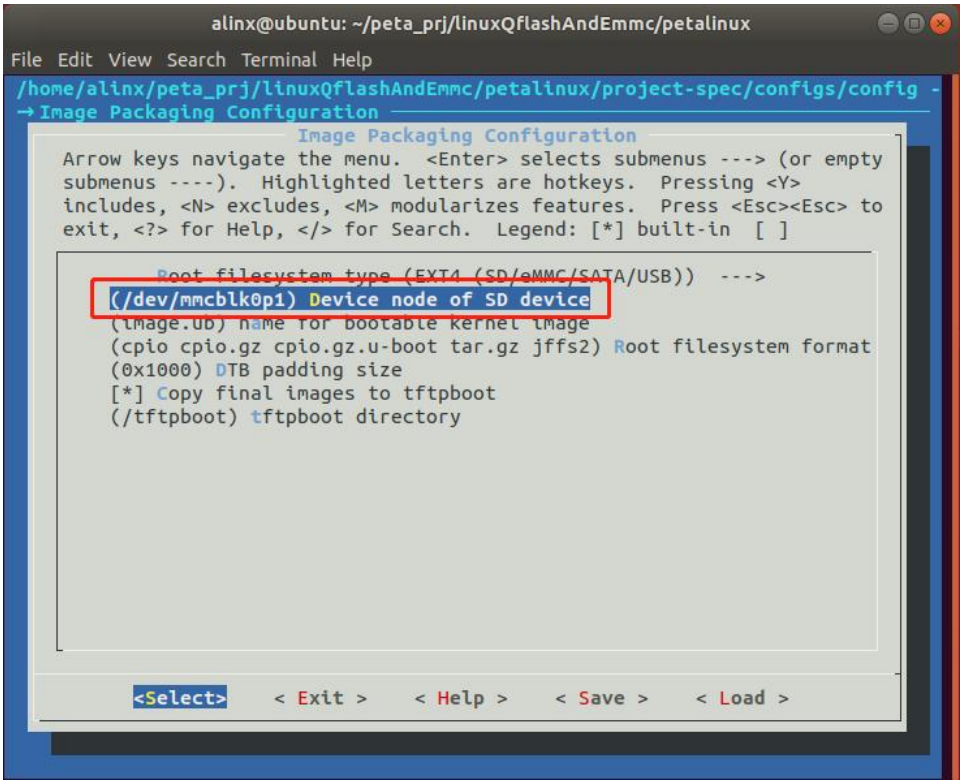
5. Subsystem AUTO Hardware Settings ---> Flash Settings 中，按下图修改 QSPI flash 的分区大小。



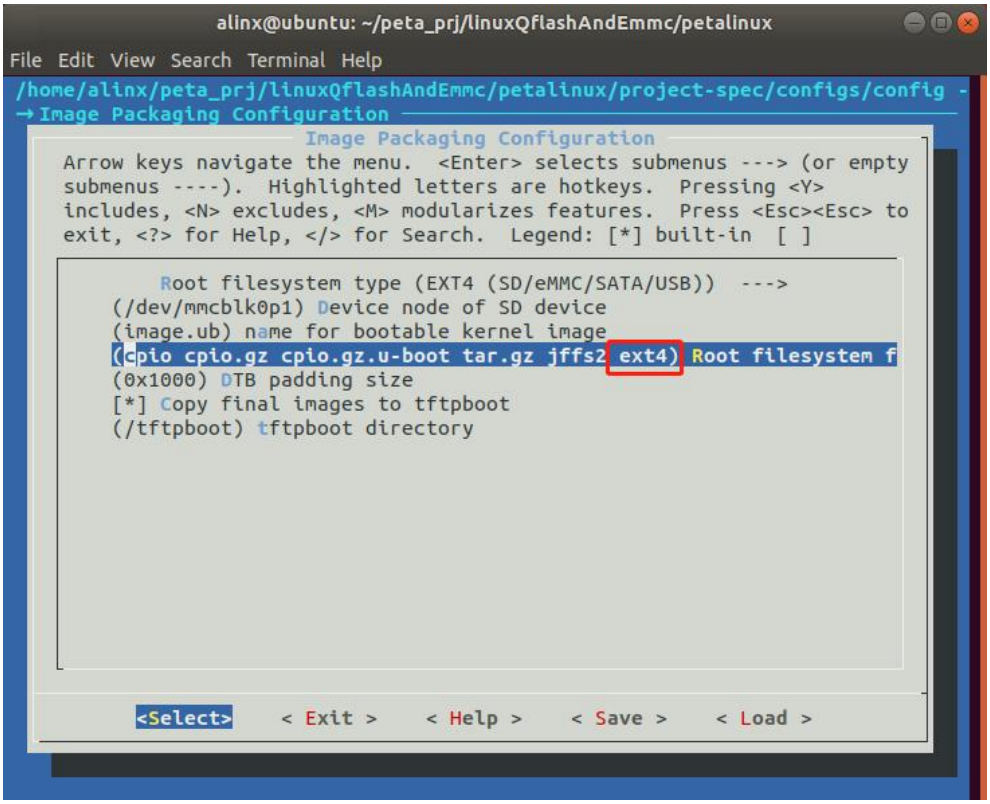
6. “Image Packaging Configuration ---> Root filesystem type”选择“EXT4”。



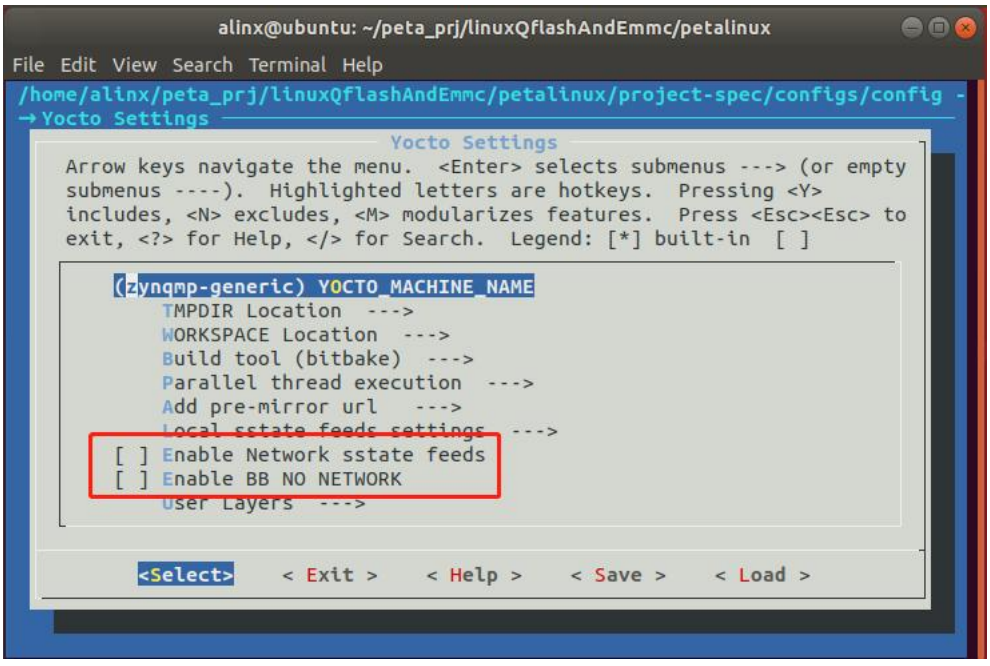
7. emmc 在教程的工程中是 mmc0，因为只要存放文件系统所以只要一个分区就可以了，所以“Device node of SD device”选项我们填写“/dev/mmcblk0p1”。



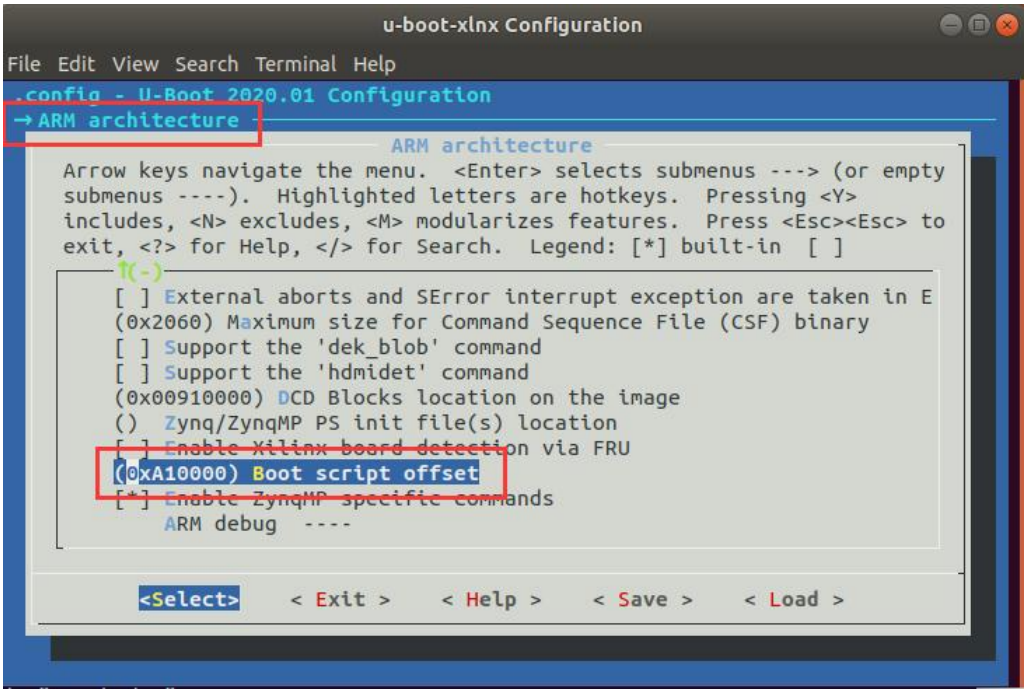
8. 在“Root filesystem formats”选项中添加一个“ext4”。



9. 按需配置离线编译。“Yocto Settings”界面配置如下。然后保存配置退出配置界面。

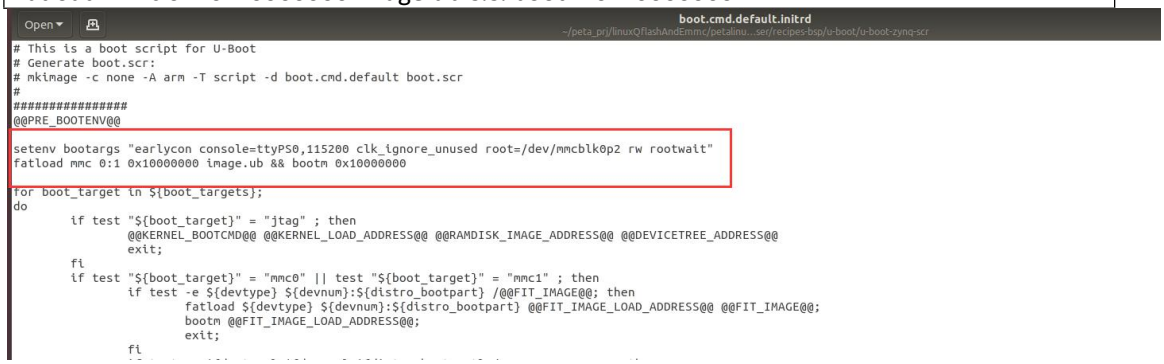


10. 使用 petalinux-config -c u-boot 命令配置 u-boot。配置界面打开后，进入 ARM architecture ---> Boot script offset 界面，设置 bootscr 分区的偏移地址为 0xA10000。



11. 打开 petalinux 工程“/project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr”路径中的“boot.cmd.default.initrd”文件，添加下面两行内容，如下图。

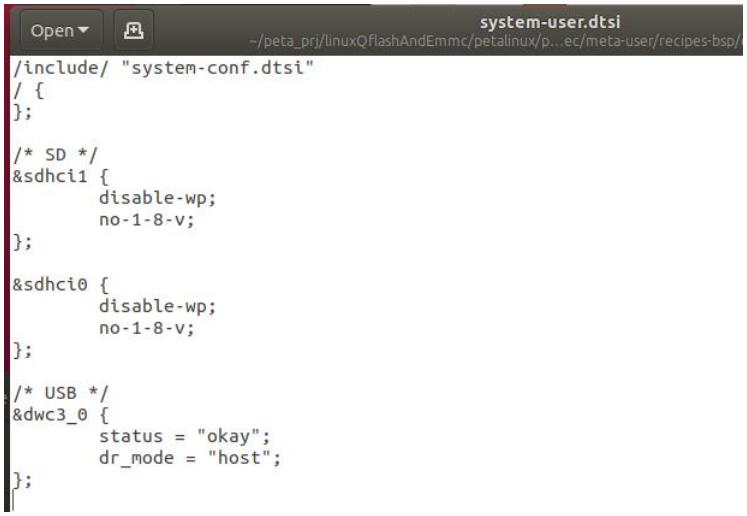
```
setenv bootargs "earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/mmcblk0p2 rw rootwait"
fatload mmc 0:1 0x10000000 image.ub && bootm 0x10000000
```



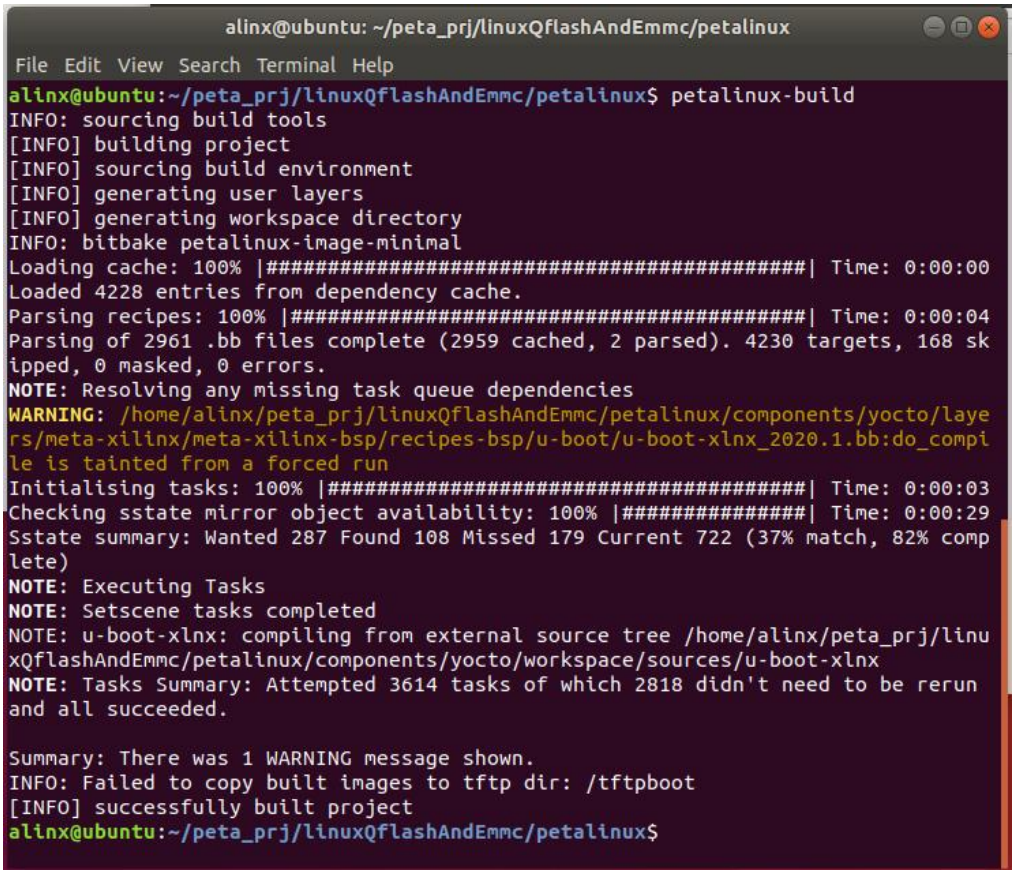
12. 修改设备树。

```
/include/ "system-conf.dtsi"
/ {
};

/* SD */
&sdhci1 {
	disable-wp;
	no-1-8-v;
};

&sdhci0 {
	disable-wp;
	no-1-8-v;
};
```

```
/* USB */
&dwc3_0 {
	status = "okay";
	dr_mode = "host";
};
```



13. 编译 petalinux 工程，编译完成后把 petalinux 工程 image/linux 路径中的 image.ub、rootfs.tar.gz 拷贝到 NFS 共享路径中。



14. 打包 BOOT.bin，需要包含 u-boot、fsbl 和 boot.scr。使用下面的命令

```
petalinux-package --boot --u-boot --add ./images/linux/boot.scr --offset 0xA10000 --load 0x20000000 --force
```

```
alinx@ubuntu: ~/peta_prj/linuxQflashAndEmmc/petalinux
File Edit View Search Terminal Help
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc/petalinux$
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc/petalinux$ petalinux-package --boot --u-boot --add ./images/linux/boot.scr --offset 0xA10000 --load 0x20000000 --force
INFO: sourcing build tools
INFO: Getting system flash information...
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/zynqmp_fsbl.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/pmufw.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/bl31.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/system.dtb"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/linuxQflashAndEmmc/petalinux/images/linux/boot.scr"
INFO: Generating zynqmp binary package BOOT.BIN...


****** Xilinx Bootgen v2020.1
  **** Build date : May 26 2020-14:07:15
    ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.


[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
alinx@ubuntu:~/peta_prj/linuxQflashAndEmmc/petalinux$
```

15. 把 BOOT.bin 烧录到开发板的 flash 中。烧录方法参考 Vitis 应用教程（烧录好了之后先不用管，接着做下面的）

19.2 设置 EMMC 分区制作根文件系统

用非 Qspi flash 的其他方式启动开发板，本实验中使用本教程“第十一章 Linux 下的 NVMe SSD 操作”中制作的系统来启动系统开发板（用第十一章工程的 BOOT.bin、image.ub 和 boot.scr 拷贝到 SD 卡的 FAT 分区中；把 rootfs.tar.gz 解压到 SD 卡的 EXT 分区中），设置 EMMC 分区。

1. 使用命令 fdisk /dev/mmcblk0 设置分区，如果已有分区，就使用命令 d 删除现有分区。

```
root@petalinux:~# fdisk /dev/mmcblk0

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/mmcblk0: 7.29 GiB, 7818182656 bytes, 15269888 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x2e086636

Device         Boot   Start      End Sectors Size Id Type
/dev/mmcblk0p1         2048  2099199 2097152   1G 83 Linux
/dev/mmcblk0p2      2099200 10487807 8388608   4G 83 Linux

Command (m for help): d
Partition number (1,2, default 2): 1

Partition 1 has been deleted.

Command (m for help): d
Selected partition 2
Partition 2 has been deleted.
```

2. 先输入 n 创建新分区存放内核，分区号和起始扇区保持默认设置直接按回车，结束扇区设置输入+1G。

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (1-4, default 1):
First sector (2048-15269887, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-15269887, default 15269887): +1G

Created a new partition 1 of type 'Linux' and of size 1 GiB.

Command (m for help):
```

3. 再输入 n 创建第二个分区放文件系统，所有设置都保持默认。

```
Command (m for help): n
Partition type
   p   primary (1 primary, 0 extended, 3 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (2-4, default 2):
First sector (2099200-15269887, default 2099200):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2099200-15269887, default 15269887):

Created a new partition 2 of type 'Linux' and of size 6.3 GiB.
Partition #2 contains a ext4 signature.

Do you want to remove the signature? [Y]es/[N]o: Y

The signature will be removed by a write command.

Command (m for help):
```

4. 然后输入 wq 保存退出分区设置。

5. 用下面的命令把 EMMC 的第一个分区格式化成 fat32，第二个分区格式化成 ext4。

```
mkfs.vfat   /dev/mmcblk0p1
mkfs.ext4   /dev/mmcblk0p2
```

```
root@petalinux:~# mkfs.vfat /dev/mmcblk0p1
root@petalinux:~# mkfs.ext4 /dev/mmcblk0p2
mke2fs 1.45.3 (14-Jul-2019)
Discarding device blocks: done
Creating filesystem with 1646336 4k blocks and 412080 inodes
Filesystem UUID: c344526f-31c9-4f0b-ad92-40960607231c
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

root@petalinux:~#
root@petalinux:~#
```

6. 如果分区设置好后系统没有自动挂载，就手动分别挂载 EMMC 的两个分区到 /media/sd-mmcblk0p1 和/media/sd-mmcblk0p2，如果这两个文件夹不存在就手动创建。

```
mkdir   /media/sd-mmcblk0p1
mkdir   /media/sd-mmcblk0p2
mount   /dev/mmcblk0p1   /media/sd-mmcblk0p1/
mount   /dev/mmcblk0p2   /media/sd-mmcblk0p2/
```

```
root@petalinux:~# mkdir /media/sd-mmcblk0p1
mkdir: can't create directory '/media/sd-mmcblk0p1': File exists
root@petalinux:~# mkdir /media/sd-mmcblk0p2
mkdir: can't create directory '/media/sd-mmcblk0p2': File exists
root@petalinux:/media# mount /dev/mmcblk0p1 /media/sd-mmcblk0p1/
root@petalinux:/media# mount /dev/mmcblk0p2 /media/sd-mmcblk0p2/
[  842.468406] EXT4-fs (mmcblk0p2): mounted filesystem with ordered data mode. Opts: (null)
root@petalinux:/media#
```

7. 挂载虚拟中的 nfs 共享路径(挂载共享路径的命令需要根据实际修改)。把共享路径中的 image.ub 拷贝到/media/sd-mmcblk0p1。

```
mount   -t   nfs   -o   nolock   192.168.0.252:/home/alinx/work   /mnt
cp   /mnt/image.ub   /media/sd-mmcblk0p1/
```

```
root@petalinux:/media# mount -t nfs -o nolock 192.168.0.252:/home/alinx/work /mnt
root@petalinux:/media# cp /mnt/image.ub /media/sd-mmcblk0p1/
root@petalinux:/media#
```

8. 用下面的命令解压共享路径中的 rootfs.tar.gz 到/media/sd-mmcblk0p2。

```
tar -zxvf /mnt/rootfs.tar.gz -C /media/sd-mmcblk0p2/
sync
```

```
root@petalinux:/media# tar -zxvf /mnt/rootfs.tar.gz -C /media/sd-mmcblk0p2/
./
./etc/
./etc/services
./etc/skel/
./etc/skel/.profile
./etc/skel/.bashrc
./etc/busybox.links
./etc/nsswitch.conf
./etc/mtab
./etc/syslog-startup.conf
./etc/motd
./etc/shells
./home/petalinux/
./home/petalinux/.profile
./home/petalinux/.bashrc
./home/root/
./tmp/
root@petalinux:/media# sync
root@petalinux:/media#
```

19.3 Qflash+EMMC 启动测试

把开发板设置成 Qspi flash 启动并上电。通过 PS_UART 串口可以看到系统成功启动。

```
Starting internet superserver: inetd.
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2020.1 petalinux ttyPS0

petalinux login:
```